

Sisteme de Operare

Prezentarea cursului

Cristian Vidraşcu

<http://www.info.uaic.ro/~vidrascu>

Cuprins

- Despre obiectivele disciplinei
- Resurse
- Prezentare generală – cursuri și laboratoare
- Examinare
- Feedback

Objective

Obiectivele generale urmărite:

1. Dobândirea de cunoștințe despre sistemele de operare, referitoare la tehnicile de proiectare și de implementare a acestora.
2. Deprinderea unor abilități de procesare paralelă și de utilizare a sistemului de operare UNIX/Linux.

Objective

Abilități dobândite:

- 1) Însușirea conceptelor de bază referitoare la funcționarea sistemelor de operare
- 2) Înțelegerea arhitecturii unui sistem de operare, cu principalele sale componente
- 3) Înțelegerea algoritmilor care sunt folosiți de un sistem de operare pentru administrarea resurselor
- 4) Utilizarea interfeței text oferită de sistemul de operare UNIX/Linux și a tehnicilor de procesare paralelă
- 5) Proiectarea de aplicații soft care să utilizeze serviciile oferite de un sistem de operare UNIX/Linux.

Resurse

- Pagina web a cursului Sisteme de Operare:

<http://www.info.uaic.ro/~vidrascu/SO/>

- Cărți:

- A.Tanenbaum, *Organizarea structurată a calculatoarelor*, Ed. Agora, Tg. Mureș, 1999
- F.M.Boian, *Sisteme de operare interactive*, Ed. Libris, Cluj-Napoca, 1994
- A.Silberschatz, P.Galvin, G.Gagne, *Operating Systems Concepts – 9th ed.*, Ed. John Wiley & Sons Inc., 2013
- A.Tanenbaum, *Modern Operating Systems – 4th ed.*, Ed. Prentice Hall International, 2014

- Documentații și resurse online

- <http://www.linux.org> , ș.a.

Prezentare generală

Cursul de Sisteme de Operare

Prezentare generală /1

- **Concepte de bază**

Istoric. Tipuri de sisteme de operare. Exemple.

- **Structura sistemelor de operare**

Componente. Servicii. Proiectarea și implementarea sistemelor. Nucleu.

- **Gestiunea proceselor**

Concepte. Concurență. Planificare.

- **Coordonarea proceselor**

Secțiuni critice. Sincronizări. Comunicații inter-procese. Interblocaj.

Prezentare generală /2

- **Administrarea memoriei**

Ierarhii. Alocare. Segmentare. Memorie virtuală. Paginare. Memorie partajată distribuită. Cache. Exemple.

- **Administrarea perifericelor**

Dispozitive de stocare. Sisteme de fișiere. Organizare și operații. Implementare. Protecția fișierelor. Politici.

- **Sisteme de operare distribuite**

Tipuri. Coordonare distribuită. Sisteme de fișiere distribuite. Exemple.

Prezentare generală

Laboratoarele de Sisteme de Operare

Prezentare generală /3

Sistemul de operare UNIX (Linux)

- **Utilizarea sistemului:**
 - Comenzi uzuale
 - Programare shell (bash)
- **Programare concurentă în C:**
 - Fișiere
 - Procese
 - Comunicații inter-procese (canale interne și externe, semnale, redirectări I/E, ș.a.)
 - Gestiunea resurselor (terminale, useri, ș.a.)

Câteva concepte:

- Programare secvențială (clasică):

= un program cu un singur flux de instrucțiuni în curs de execuție

vs.

- Programare **paralelă**, **concurentă** (uneori și **distribuită**):

= un program cu mai multe fluxuri de instrucțiuni în curs de execuție în “același” timp

Cele $N \geq 2$ fluxuri de instrucțiuni (*threads or sequential processes*) se execută în paralel și, în plus, ele **concurează** pentru utilizarea resurselor oferite de SO-ul acelu sistem de calcul.

Uneori, cele $N \geq 2$ fluxuri se pot executa într-o manieră **distribuită în spațiu**, i.e. pe sisteme de calcul diferite, interconectate prin rețea.

Cum se pot executa mai multe fluxuri de instrucțiuni în “același” timp?

- prin **parallelism aparent**: tehnica de **multiprogramare** (*multitasking*)

vs.

- prin **parallelism real**: tehnica de **multiprocesare** (*multiprocessing*)

Clasificare după UI (i.e. interfața cu utilizatorul):

- Program cu UI în mod text (prescurtat, *TUI*, or *CLI*):

= un program ce interacționează cu utilizatorul printr-o interfață în mod linie de comandă

→ paradigma de programare **imperativă** clasică (*program-driven programming*),
cu UI realizată prin instrucțiuni uzuale de intrare/ieșire (*input/output*), folosind doar tastatură (+ecran)

vs.

- Program cu UI grafică (prescurtat, *GUI*):

= un program ce interacționează cu utilizatorul printr-o interfață grafică

→ paradigma de programare **dirijată de evenimente** (*event-driven programming*),
cu UI realizată folosind diverse biblioteci / *framework*-uri, cu ierarhii formate din sute de clase

Interfața grafică poate fi de două feluri:

- *desktop metaphor*: GUI bazată pe interfețe WIMP (“*windows, icons, menus, pointer*”), folosind tastatură și *mouse*
- *multi-touch metaphor*: GUI bazată pe interfețe post-WIMP, pentru dispozitive cu ecran tactil (*touchscreen*)

Modul de examinare

I) Activitatea la laboratoare (Lab):

- 1) exerciții de laborator ExL: punctaj (maxim) 10p
- 2) două teste practice de rezolvat pe calculator în laborator
 - primul test TP1 : (aprox.) în săptămâna a 6-a
 - al doilea test TP2 : (aprox.) în săptămâna a 12-a

Aceste două teste practice **nu** se vor da și în restanțe!

(Notă: le veți putea reface în anul următor cu grupa specială de restanțieri)

- punctaj (maxim) : 25p pentru fiecare test practic

Punctajul total pentru laborator : $\text{Lab} = \text{ExL} + \text{TP1} + \text{TP2}$

Criteriul de promovare a laboratorului: $\text{Lab} \geq 20\text{p}$

Modul de examinare

II) Examenul scris (TS) :

- Se va susține la finalul semestrului [sau în restanțe] un test scris cu subiecte din partea de teorie
- Condiții de intrare în examen:
promovarea laboratorului și prezența la testul scris
- Punctajul (maxim) TS : 40p
- Criteriul de promovare a examenului: **$TS \geq 15p$**
- Bonus (pentru motivarea pregătirii practice):
dacă **$TP1 \geq 23p$** și **$TP2 \geq 22p$** , atunci NU mai este obligatoriu să susțineți testul scris, acesta fiind echivalat cu **$TS = (TP1+TP2)*4/5$**

Modul de examinare

III) Nota finală:

- Criteriul de promovare a disciplinei:

să fie promovate laboratorul și examenul scris

- Punctajul final (PF):

$$PF = TS + Lab$$

Notă: punctajul final se calculează numai pentru cei care îndeplinesc criteriul de promovare a disciplinei

- Nota finală:

se calculează pe baza PF aplicând “Gauss”, în conformitate cu procentajele adoptate de FII (5% - 10, 10% - 9, 20% - 8, 30% - 7, 25% - 6, 10% - 5)

Feedback

- Discuții (în clasă/la consultații)
- Pentru orice comentarii, observații, probleme referitoare la S.O. (curs/laboratoare)
 - email la vidrascu@info.uaic.ro
 - contactați echipa S.O. (C.Vidrașcu, I.Leahu, R.Benchea, M.Leonte, A.Simion, D.Moroșanu, A.Munteanu)
- Grupul local LUG

Întrebări ?

Sisteme de Operare

Concepte de bază

Cristian Vidraşcu

<http://www.info.uaic.ro/~vidrascu>

Cuprins

- Ce este un sistem de operare ?
 - Definiții tradiționale
- Arhitectura și Proiectarea S.O.
 - Evoluția S.O.
 - Tipuri de S.O.
 - Exemple
 - Influențe în proiectarea S.O.

Ce este un S.O. ?

- **Sistem de calcul** =
hardware, S.O., programe de aplicații, utilizatori
- **Hardware:**
procesor(oare), memorie, dispozitive de I/E
- **Software** (programe de aplicații):
compilatoare, diverse unelte și utilitare, SGBD-uri,
suite office, jocuri, ș.a.
- **Utilizatori:**
umani, alte calculatoare

Ce este un S.O. ?

Definiții tradiționale:

- **Administrator de resurse**

S.O.-ul gestionează dispozitivele fizice (hardware-ul)

- Mediu de tip **Mașină abstractă**

S.O.-ul definește un set de resurse logice (abstracte) și de operații asupra acelor resurse (i.e., o interfață de utilizare a acelor resurse)

- Permite **partajarea** resurselor

S.O.-ul controlează interacțiunile între diferiți utilizatori

Ce este un S.O. ?

Definiții tradiționale (continuare):

- Apariția **principiilor de proiectare a sistemelor**
- Are **rol susținător** – de a oferi servicii pentru îndeplinirea sarcinilor utilizatorilor, și nu de a fi un produs final
- Face distincție între codul protejat (privilegiat) – **nucleul** – și codul utilizator (neprivilegiat)

(intrarea în modul privilegiat se face via apeluri de sistem, întreruperi, sau alte mecanisme specializate)

S.O. este un **program de control** (monitor)

Ce este un S.O. ?

Comentarii:

- Nu există o definiție generală universal acceptată a ceea ce face parte dintr-un S.O. și ceea ce nu face parte din el
- Nu există un S.O. universal care să ruleze pe toate tipurile de sisteme de calcul existente în lume

Ce este un S.O. ?

Detalii

Administrator de resurse pentru dispozitivele fizice (hardware-ul) ale calculatorului:

- CPU (procesor/procesoare)
- Memoria principală (memoria RAM)
- Memoria secundară (HDD-uri, unități de bandă, ...)
- Dispozitive de rețea
- Periferice de intrare (tastatură, mouse, scanner, ...)
- Periferice de ieșire (monitor, imprimantă, difuzoare, ...)

Ce este un S.O. ?

Detalii

Administrator de resurse pentru dispozitivele fizice ale calculatorului (cont.):

- Asigură funcționarea lor simultană (concurență, paralelism)
- Asigură partajarea lor între task-uri
- Performanța relativă, capacitatea, costul lor sunt într-o continuă schimbare

Ce este un S.O. ?

Detalii

Mediu de tip **Mașină abstractă**:

- Procese sau thread-uri – `fork()`, `wait()`, `exec()`
- Spațiu de adresare – `malloc()`, `free()`
- Fișiere – `open()`, `read()`, `write()`, `close()`
- Mesaje – `send()`, `receive()`
- ș.a.

Arhitectura & Proiectarea S.O.

Istoricul evoluției S.O.:

- Fără nici un S.O.
 - programatorul = operator, administrator și utilizator obișnuit
- Monitor rezident
 - batch job-uri = secvențe de job-uri (fără interacțiune utilizator)
 - operatorul $\neq$ utilizatorul obișnuit
 - planificarea automată a ordinii în secvență a job-urilor (primul S.O. rudimentar): **monitorul rezident**
 - Monitor = încărcător S.O., planificator job-uri, interpretor I/E

Arhitectura & Proiectarea S.O.

Istoricul evoluției S.O. (continuare):

- Sisteme seriale (sisteme cu prelucrare de batch job-uri)

Sistemul de calcul IBM 1401, alcătuit din unitatea de procesare 1401, cititorul de cartele perforate 1402 și imprimanta 1403.



- Programarea cu cartele perforate



Arhitectura & Proiectarea S.O.

Istoricul evoluției S.O. (continuare):

- S.O. timpurii
 - *scheme de rezervare a resurselor*
 - soft suplimentar pentru perifericele de I/E = *drivere de periferice*
 - S.O. are rol de *mediu de dezvoltare* (asambleare, încărcătoare, compilatoare, editoare de legături, biblioteci, ...)
 - *independența de periferice* (periferice I/E logice)
 - *Buffer și Spool* (=Simultaneous Peripheral Operation On-Line) (Atlas '61)
 - Mono-utilizator

Arhitectura & Proiectarea S.O.

Istoricul evoluției S.O. (continuare):

- Multi-tasking (multi-programare)
 - planificarea CPU
- Time-sharing (timp CPU partajat)
 - exemple: MIT CTSS '62, MULTICS '65, UNIX '69
 - S.O. interactiv:
procese, comunicație inter-procese, gestiunea resurselor (fișiere, directoare, ș.a.), multi-utilizator
- Multi-processing (multi-prelucrare)
- S.O. distribuite (pentru sisteme de calcul distribuite)
 - partajarea resurselor, fiabilitate, comunicație, multi-utilizator
- S.O. în timp real (pentru sisteme de calcul în timp real)
- S.O. mobile (pentru dispozitive mobile)

Arhitectura & Proiectarea S.O.

Clasificarea S.O.:

1) După numărul de task-uri executate “simultan”:

- **mono-tasking** : CP/M, DOS
- **multi-tasking** : UNIX, OS/2, Windows (9x/NT/2k/XP/...)

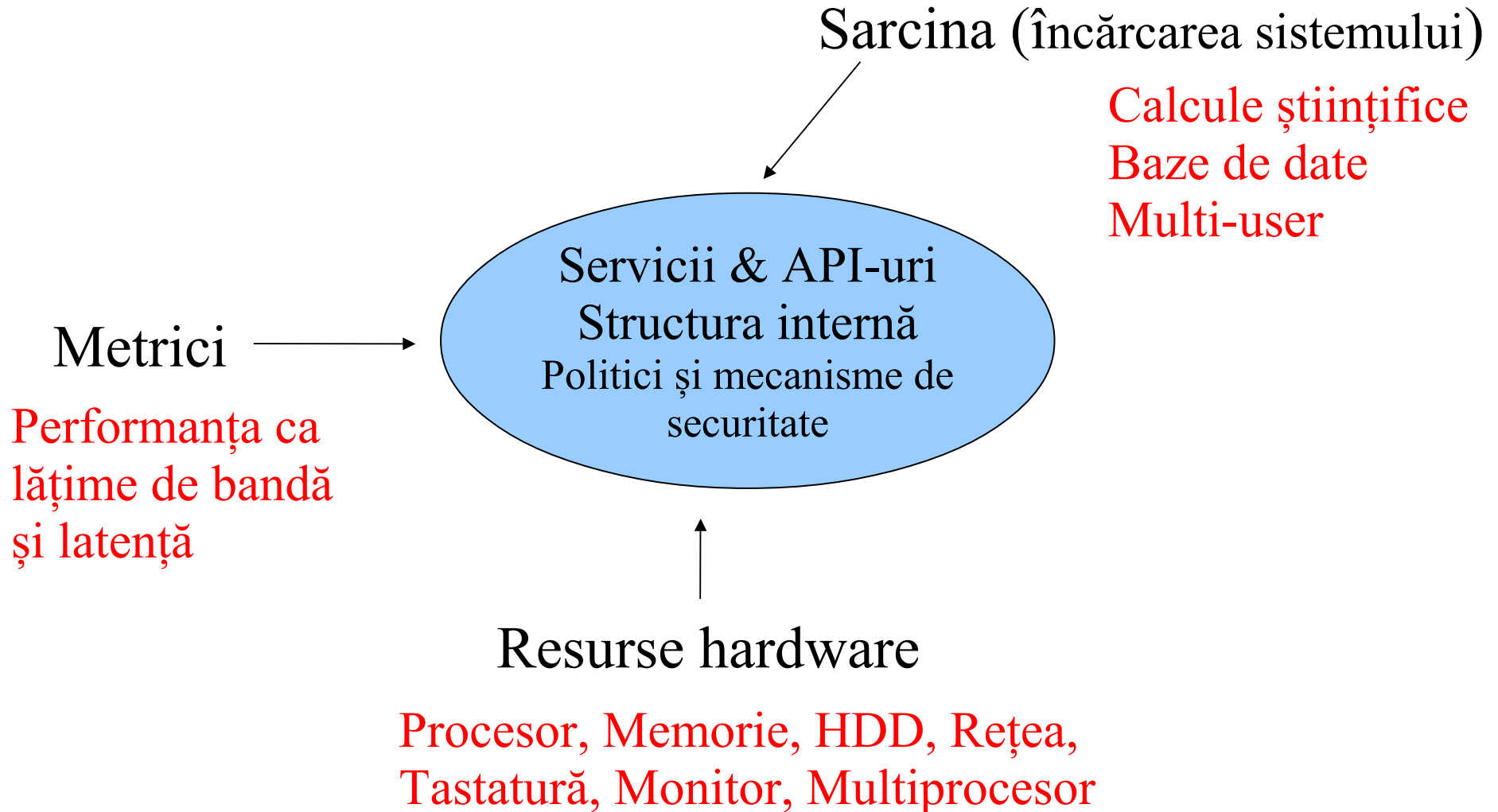
2) După numărul de utilizatori deserviți simultan:

a) **sisteme seriale** (fără interacțiune cu utilizatorul)

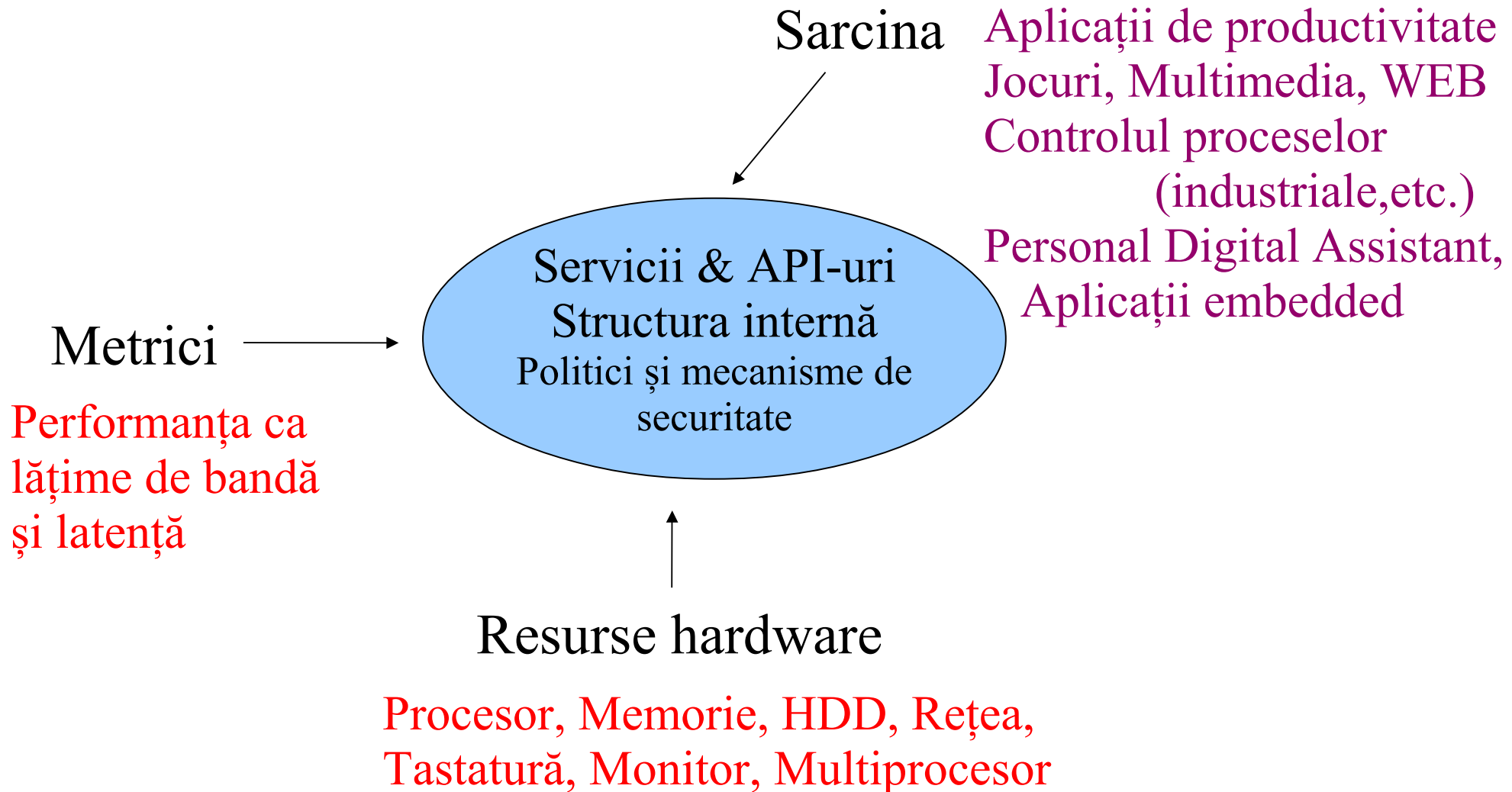
b) **sisteme interactive**:

- **mono-utilizator** : CP/M, DOS, Macintosh, OS/2, Windows desktop versions (3.x/9x/NT/2k/XP/Vista/Win7)
- **multi-utilizator** : UNIX, Mac OS X, Solaris, Windows server versions (NT/2k/2003/2008/2008R2)

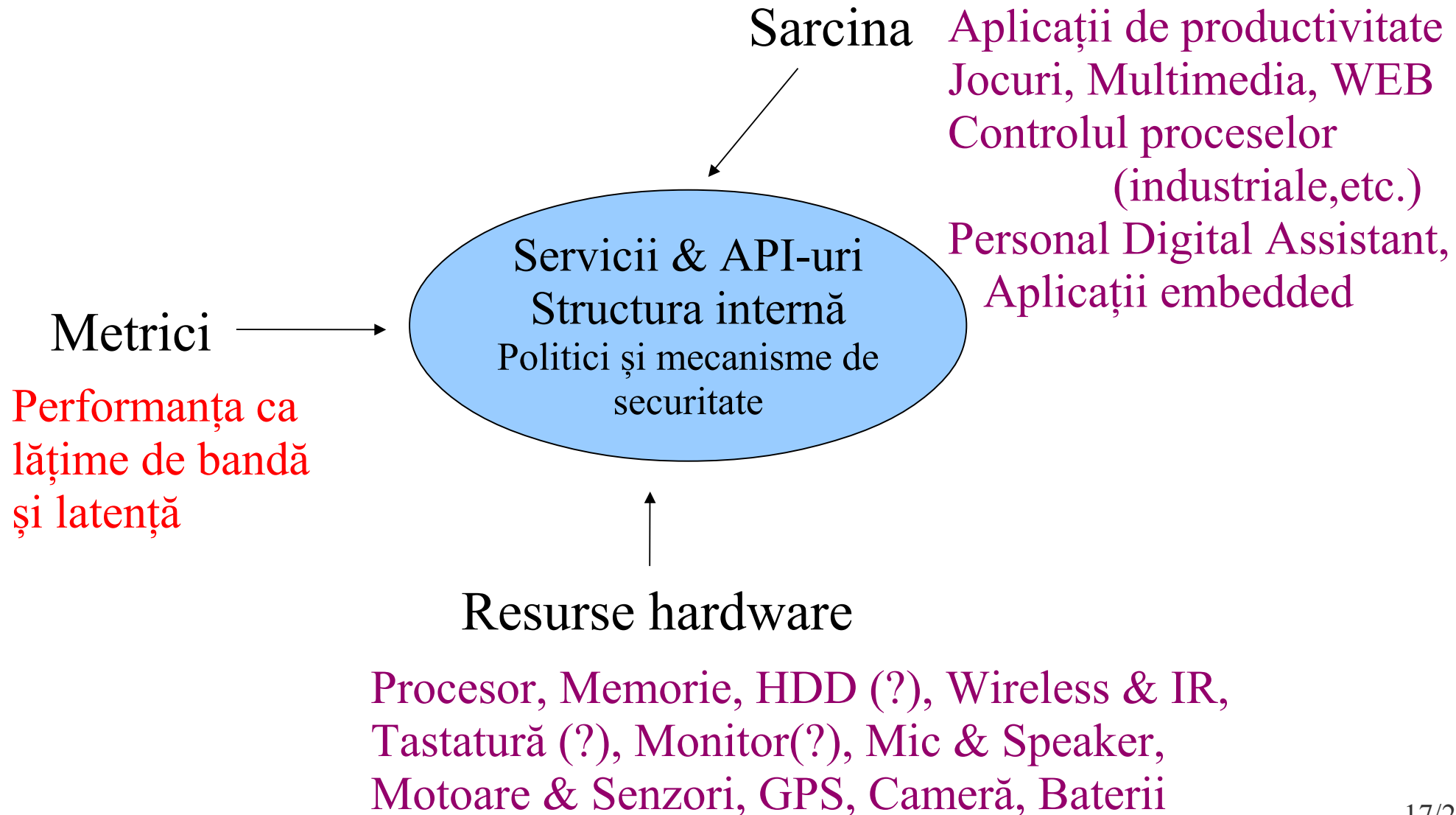
Proiectarea S.O. : influente /1



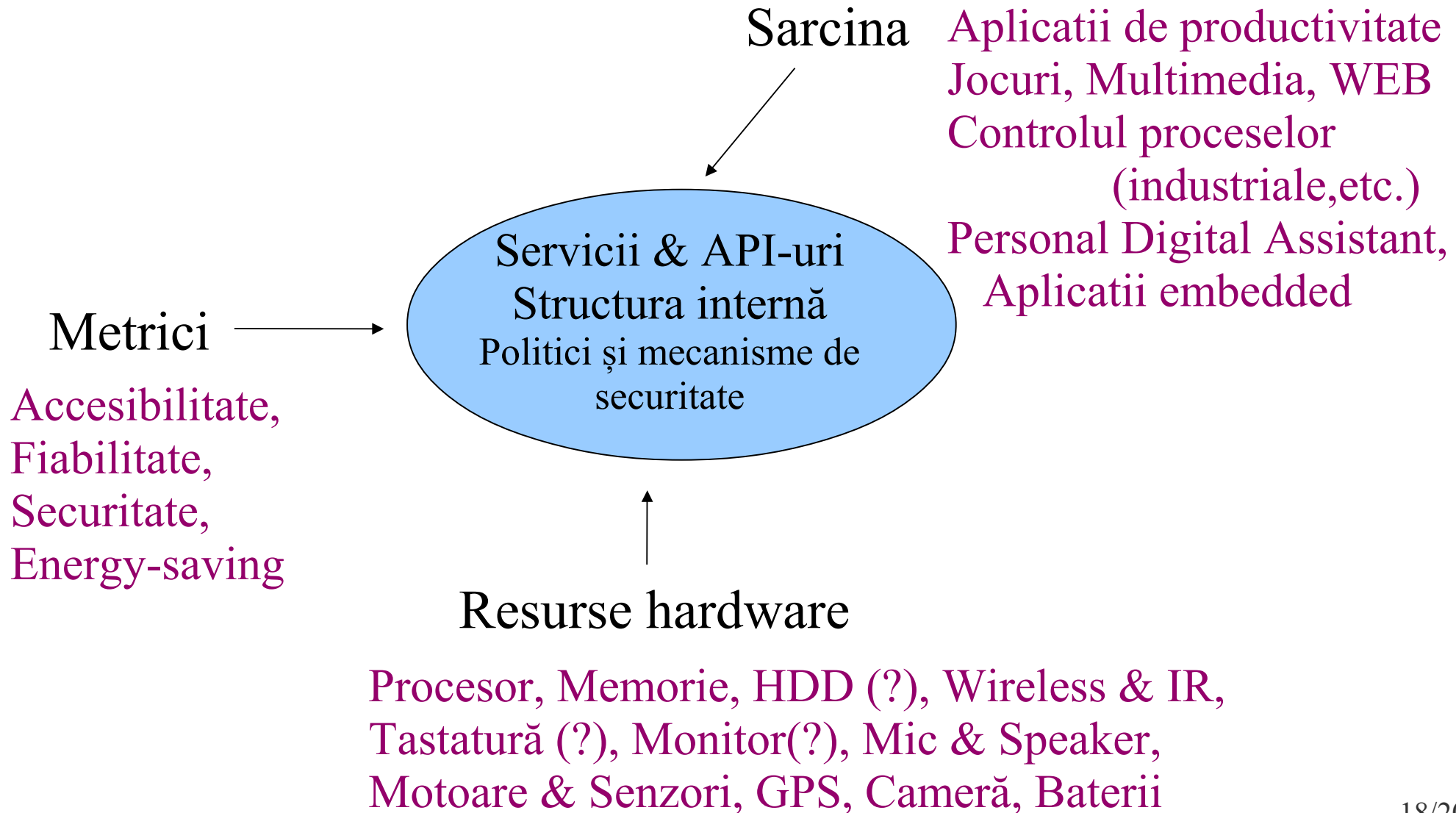
Proiectarea S.O. : influente /2



Proiectarea S.O. : influente /3



Proiectarea S.O. : influente /4



- **Bibliografie obligatorie**
capitolul introductiv din

- Silberschatz : “*Operating System Concepts*”
(cap.1 din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”
(prima parte a cap.1 din [MOS3])

Sumar

- Ce este un sistem de operare ?
 - definiții tradiționale
- Arhitectura și Proiectarea S.O.
 - evoluția S.O.
 - tipuri
 - exemple
 - influențe în proiectarea S.O.

Întrebări ?

Sisteme de Operare

Structură, componente, servicii

Cristian Vidraşcu

<http://www.info.uaic.ro/~vidrascu>

Cuprins

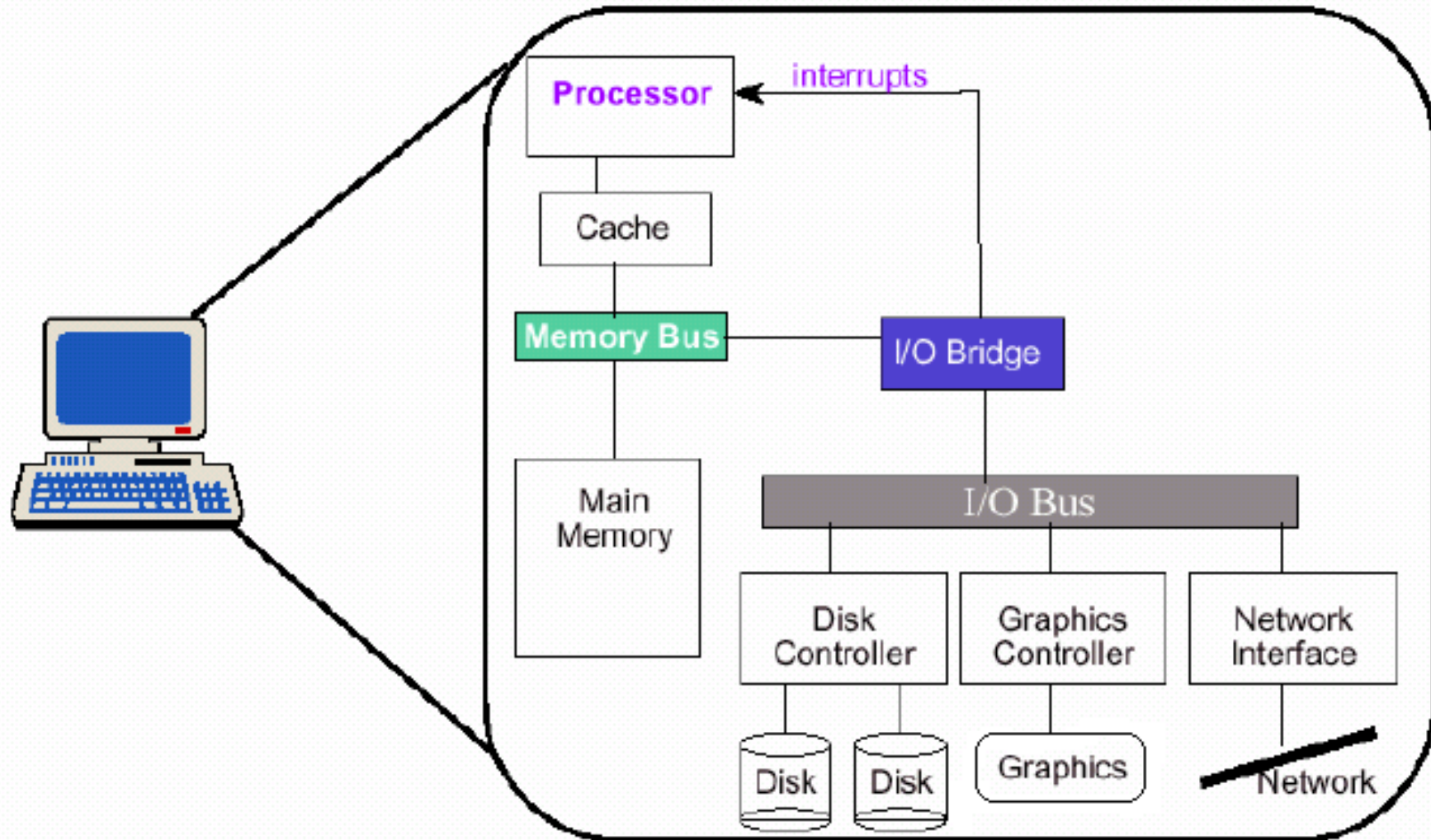
- Organizarea unui sistem de calcul
- Structura unui S.O.
- Servicii oferite de S.O.
- Abstractizări și API-uri S.O.
- Programe de sistem
- Nucleul S.O.

Preliminarii

Pentru ca operațiile CPU și I/E să se suprapună în timp (i.e., să se execute în paralel), trebuie să existe un mecanism, oferit de hardware-ul sistemului de calcul, care să permită sincronizarea operațiilor:

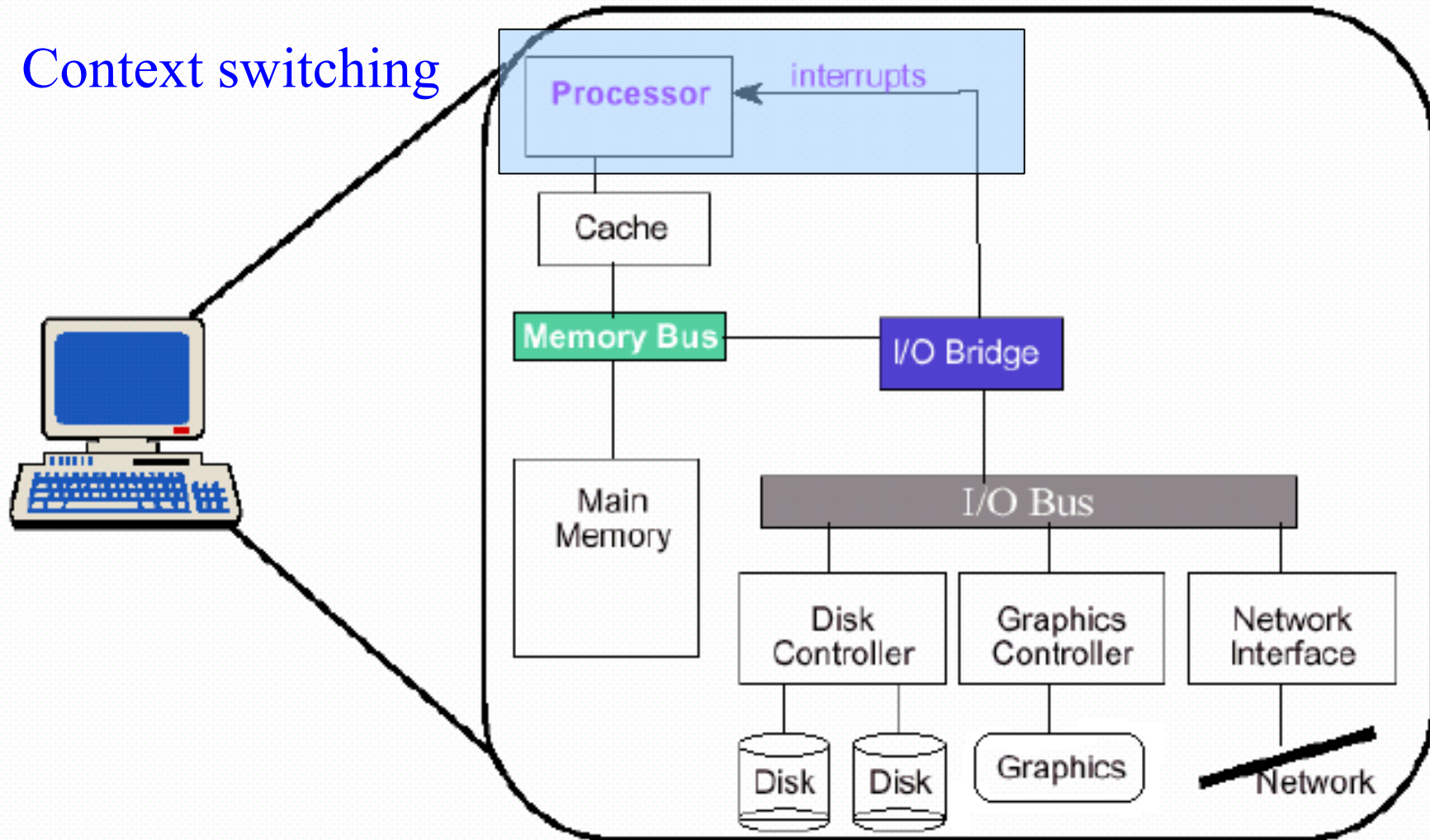
- transfer de date controlat prin întreruperi
- transfer de date prin DMA (direct memory access)

Organizarea sistemului /1a

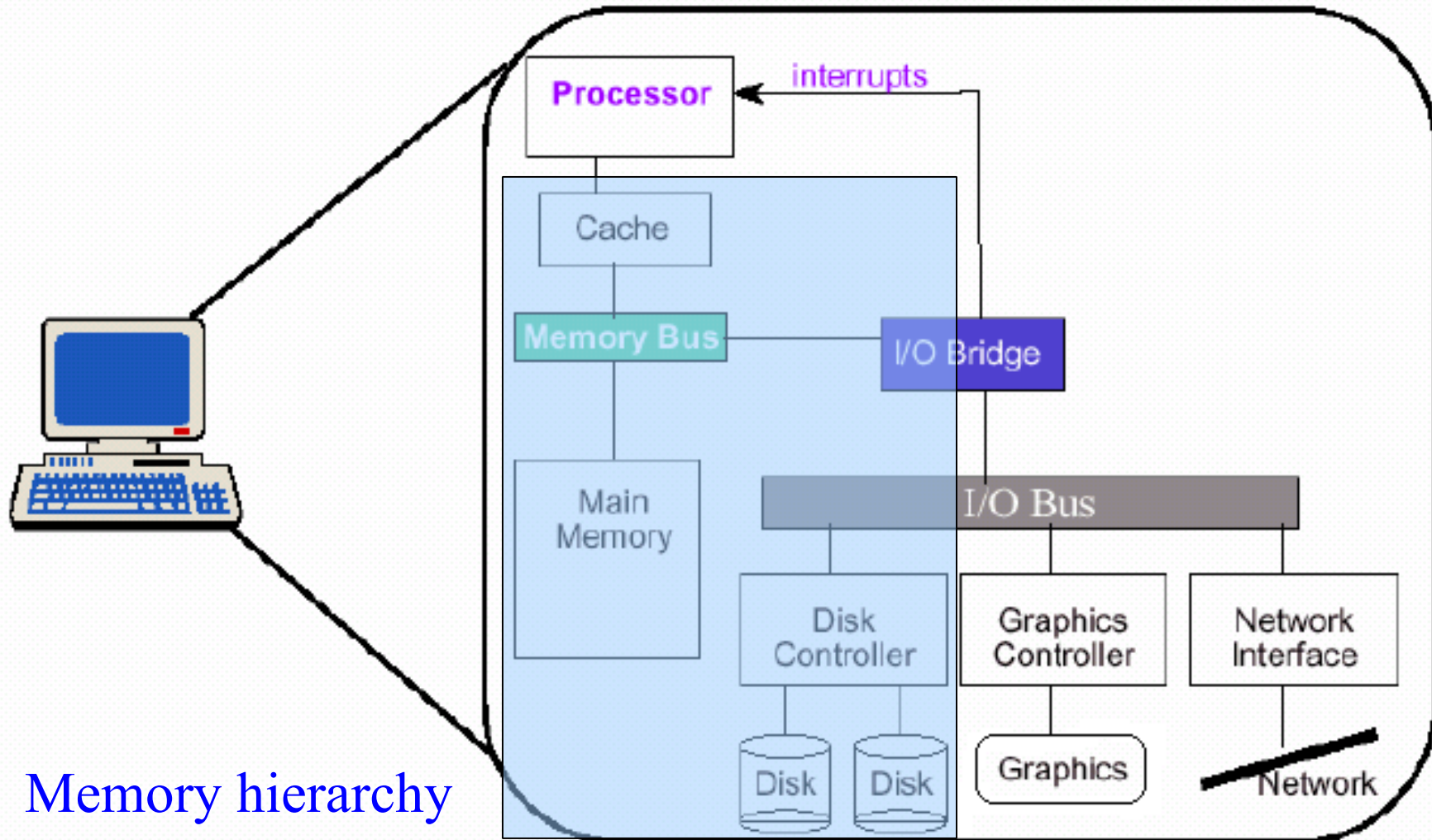


Organizarea sistemului /1b

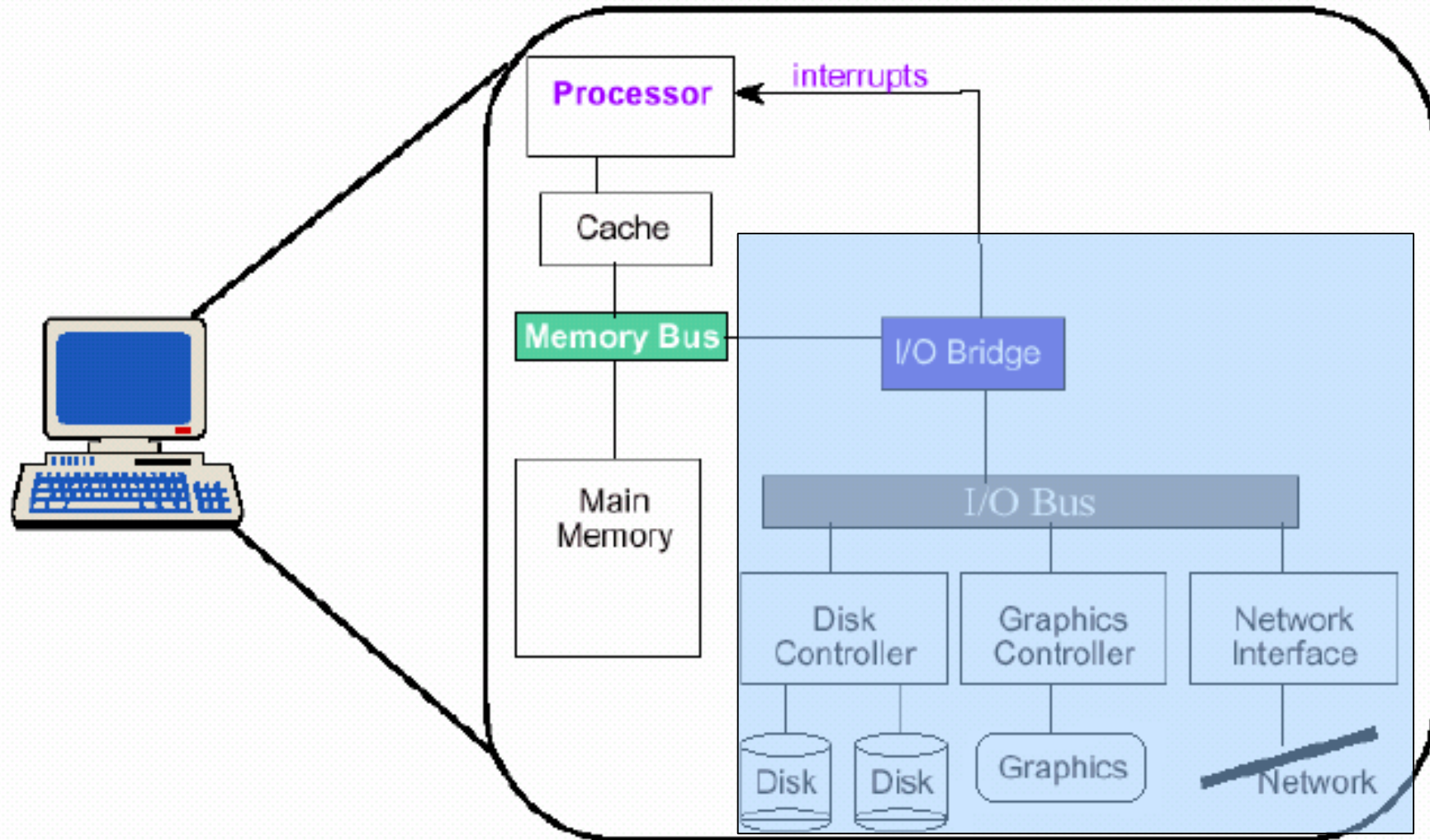
Context switching



Organizarea sistemului /1c



Organizarea sistemului /1d



DMA
I/O

Organizarea sistemului /2

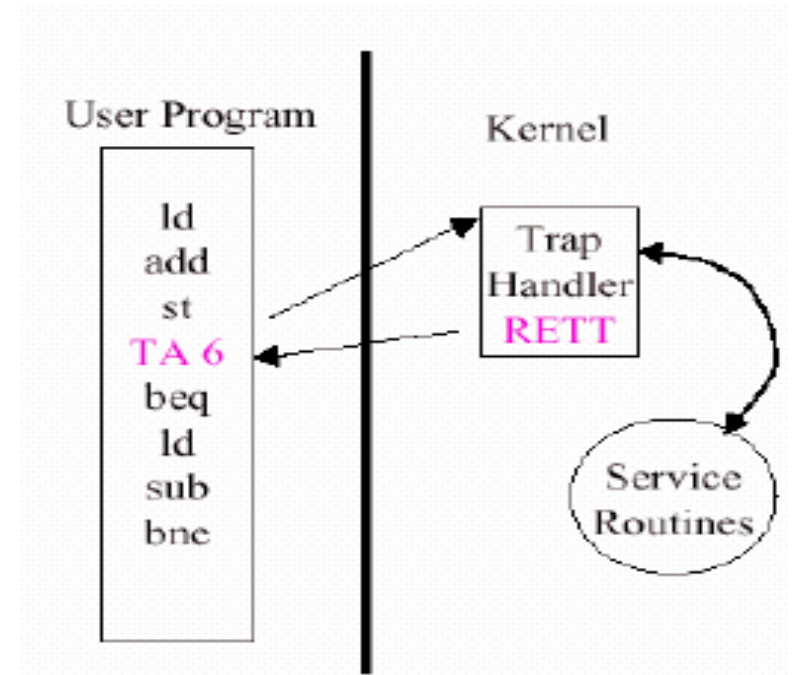
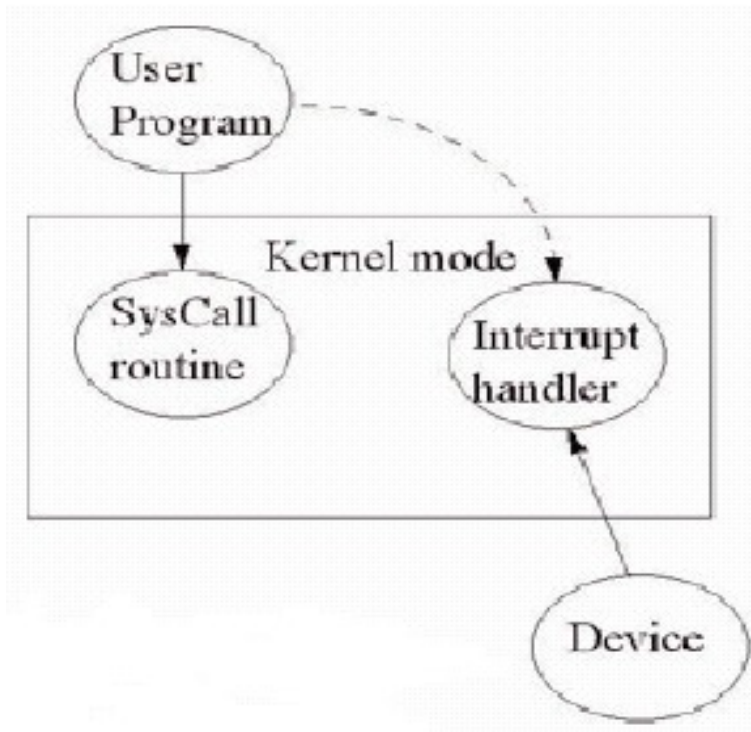
- Termeni I/E
 - **Trap** (excepție) – o întrerupere generată software, cauzată de o eroare sau de o cerere explicită dintr-un program utilizator
 - **Polling** – activitatea de determinare a tipului de întrerupere apărută

- Operarea în mod dual a CPU-ului
 - **User-mode** (modul utilizator):
modul de lucru neprivilegiat
 - **Kernel-mode** (modul sistem, sau supervizor):
modul de lucru privilegiat

Organizarea sistemului /3

Pentru ca un utilizator să poată face ceva “privilegiat”, el trebuie să apeleze acea procedură a S.O. ce furnizează acel serviciu.

Un *apel de sistem* (system call).



- O instrucțiune **trap** specială, care salvează contextul, schimbă modul de lucru (neprivilegiat – privilegiat) și transferă controlul la o rutină handler

Structura unui S.O.

Componentele sistemului de operare:

- Gestiunea proceselor
- Administrarea memoriei principale
- Administrarea memoriei secundare
- Gestiunea sistemului I/E
- Gestiunea fișierelor
- Sistemul de protecție a accesului la resurse
- Networking (gestiunea legăturii la rețea)
- Sistemul de interpretare a comenzilor

Gestiunea proceselor

➤ Proces

- Un program în curs de execuție
- O entitate activă (dinamică) a S.O.-ului
- Unitatea de lucru într-un sistem de calcul

➤ Activități S.O.

- Crearea și distrugerea proceselor user/sistem
- Suspendarea și reluarea execuției proceselor
- Mecanisme pentru sincronizarea proceselor
- Mecanisme pentru comunicații între procese
- Mecanisme pentru tratarea inter-blocajului

Gestiunea memoriei principale

➤ Memorie

- Un vector f. mare de octeți (cuvinte de memorie), având fiecare propria sa adresă
- Pentru a putea fi executat, un program trebuie să se afle în memoria principală

➤ Activități S.O.

- Alocarea și dealocarea spațiilor de memorie la cerere
- Gestiunea spațiilor de memorie libere și a celor ocupate, precum și a proceselor ce le ocupă
- Decide ce procese vor fi încărcate în memorie atunci când un spațiu de memorie devine disponibil

Gestiunea memoriei secundare

- Memoria secundară (dispozitive de stocare)
 - O extensie a memoriei principale
 - Pentru stocarea programelor și datelor
 - Dispozitive de stocare: hard-discuri, benzi magnetice, CD-ROM-uri, memorii flash, ...
- Activități S.O.
 - Gestiunea spațiului liber
 - Alocarea spațiului pentru stocare
 - Planificarea acceselor la discuri

Gestiunea sistemului I/E

➤ Sistemul I/E

- Lucrează pe bază de zone tampon (*caching*)
- Sistem de drivere pentru periferice generale
- Drivere pentru diverse periferice hardware specifice

➤ Activități S.O.

- Ascunderea detaliilor specifice diverselor periferice hardware, față de utilizatori/programe de aplicații
- Rutine de tratare (*handlers*) a întreruperilor

Gestiunea fișierelor

➤ Fișier

- O colecție de informații definită de creatorul ei
- Unitatea logică de stocare a informațiilor
- Fișierele sunt mapate pe perifericele fizice

➤ Activități S.O.

- Crearea și ștergerea fișierelor/directoarelor
- Suport pentru manipularea fișierelor/directoarelor
- Maparea fișierelor pe dispozitivele de stocare
- Backup-ul fișierelor pe medii de stocare nevolatile

Sistemul de protecție a accesului

- Protecție
 - Un mecanism pentru a controla accesul programelor / utilizatorilor la resursele sistemului de calcul
- Activități S.O.
 - Controlează utilizarea (autorizată vs. neautorizată) a resurselor sistemului de calcul

Networking

- Sistem de calcul distribuit
 - O colecție de procesoare care nu partajează o memorie comună sau un ceas intern comun
 - Fiecare procesor are propria sa memorie locală
 - Procesoarele (PCs, workstations, minicomputers, ...) sunt conectate printr-o **rețea de comunicație**
- Activități S.O.
 - Oferă acces la resursele partajate
 - “Ascunde” detaliile legate de topologia fizică a rețelei de comunicație

Sistemul de interpretare a comenzilor

- Interpretorul de comenzi
 - Un program folosit pe post de interfață cu utilizatorul
 - Inclus în nucleu sau privit ca un program special
 - Numit **interpretor linie de comandă** sau **shell**
- Instrucțiuni de comandă
 - Interfața utilizator text (e.g. DOS, UNIX)
 - Interfața utilizator grafică (e.g. Macintosh, Windows)

Servicii oferite de S.O.

- Execuția programelor
- Operații I/E
- Manipularea sistemului de fișiere
- Comunicații
- Detecția erorilor
- Alocarea resurselor
- *Accounting* (raportări)
- Protecția accesului

Abstractizări și API-uri S.O.

- Mediu de tip **Mașină Abstractă**
 - S.O.-ul definește un set de resurse logice (abstracte) și de operații asupra acelor resurse (i.e., o interfață de utilizare a acelor resurse)
- “Ascunde” partea de hardware
- Apel sistem (= o primitivă a S.O.-ului)
 - Interfața între un program în curs de execuție și S.O.
 - Apelează o rutină a nucleului ($\neq$ funcție de bibliotecă)

Abstractizări și API-uri S.O./1

Categorii de apeluri sistem

- Controlul proceselor

end, abort, load, execute, wait, wait event, signal event, allocate, free, get/set attributes

- Manipularea fișierelor

create, delete, open, close, read, write, reposition, get/set attributes

- Manipularea dispozitivelor periferice

request, release, read, write, reposition, attach, detach, get/set attributes

- Administrarea informațiilor

get/set date, get/set system data, ...

- Comunicații

create, delete, send, receive, write, reposition, get/set attributes

Abstractizări și API-uri S.O./2

Abstractizări UNIX (tradiționale)

- Procese

fir de execuție (thread) cu context

- Fișiere

un flux liniar, cu nume, de octeți de date

- Socket-uri

capete ale canalului de comunicație dintre două procese neînrudite

Abstractizări și API-uri S.O./3

Modelul de proces în UNIX și API-ul POSIX

- Primitive simple, dar puternice, pentru crearea și inițializarea procesului
 - `fork()` creează un proces fiu (inițial) ca o clonă a procesului părinte
 - programul părintelui se execută în procesul fiu pentru a-l pregăti pentru `exec()`
 - fiul poate `exit()`, părintele poate `wait()` fiul înainte de a face ceva

Exemple din Windows API: `CreateProcess()`, `WaitForSingleObject()`, `ExitProcess()`

Abstractizări și API-uri S.O./4

Modelul de proces în UNIX (cont.)

- Facilități puternice pentru controlul proceselor prin **semnale** asincrone
 - notificări ale apariției unor evenimente interne și/sau externe pentru procese sau grupuri de procese
 - sunt asemenea și au puterea întreruperilor și excepțiilor
 - acțiuni de tratare implicite: stoparea procesului, omorârea procesului, *dump core*, nici un efect (ignorare)
 - rutine de tratare (*handlers*) definite de utilizator

Abstractizări și API-uri S.O./5

Fișiere UNIX și API-ul POSIX

- **Sesiune de lucru** - *descriptorii de fișiere* sunt numere întregi pozitive folosite ca “handles” (referințe) pentru a manipula toate obiectele din sistem care se aseamănă cu fișierele
- `open()` cu un nume de fișier returnează un descriptor
- `read()` și `write()` operează la poziția (*offset*) curentă din fișier
- `lseek()` repoziționează fișierul, `close()` termină sesiunea de lucru
- Pipe-urile sunt fluxuri I/E unidirecționale fără nume create de `pipe()`; pentru pipe-uri cu nume se folosește `mkfifo()`
- Dispozitivele periferice sunt fișiere speciale create de `mknod()`, cu parametrii specifici dispozitivului setați cu `ioctl()`
- Socket-urile oferă câte 3 forme de apel `sendmsg()` și `recvmsg()`

Abstractizari si API-uri S.O./6

Abstractizări PalmOS

- Aplicații : programe single threaded, event-driven
- Elemente administrate de S.O.:
 - Obiecte din interfața utilizator: Windows, Forms, Menus
 - Obiecte din memorie: resurse, zone de memorie, databases
 - Facilități de comunicație: linie serială, IR (infra-roșu)
 - Facilități de sistem: alarme (*timers*), generator de sunete, funcții de administrare a puterii electrice consumate, acces la evenimentele de nivel scăzut generate de tastatură și *pen*

Programe de sistem

- Interpretoare de comenzi (shell-uri)
- Manipularea fișierelor
- Modificarea fișierelor
- Informații de stare
- Suport pentru limbajele de programare
- Incărcarea și execuția programelor
- Comunicații
- Programe de aplicații

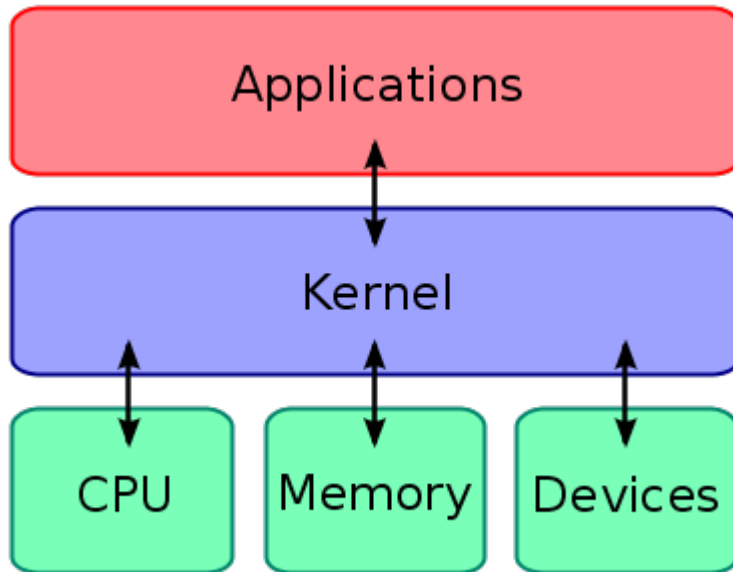
Nucleul /1

- Este cea mai importantă componentă a S.O.-ului
- Oferă servicii pentru procesele utilizator prin intermediul **primitivelor (apelurilor de sistem)**
- Activități importante desfășurate de către nucleu:
 - Gestiunea proceselor
 - Gestiunea memoriei
 - Gestiunea sistemelor de fișiere
 - Gestiunea perifericelor I/E

Plasarea serviciilor oferite de S.O.:

- Incluse în spațiul proceselor utilizator:
funcțiile de bibliotecă
(cu legare statică vs. legare dinamică)
- În nucleu: apelurile de sistem
- În *userspace* ca procese separate: procese server

Nucleul /3



- Tipuri de nucleu:
 - Nucleu monolithic
 - Micro-nucleu
 - Nucleu hibrid
 - Exo-nucleu

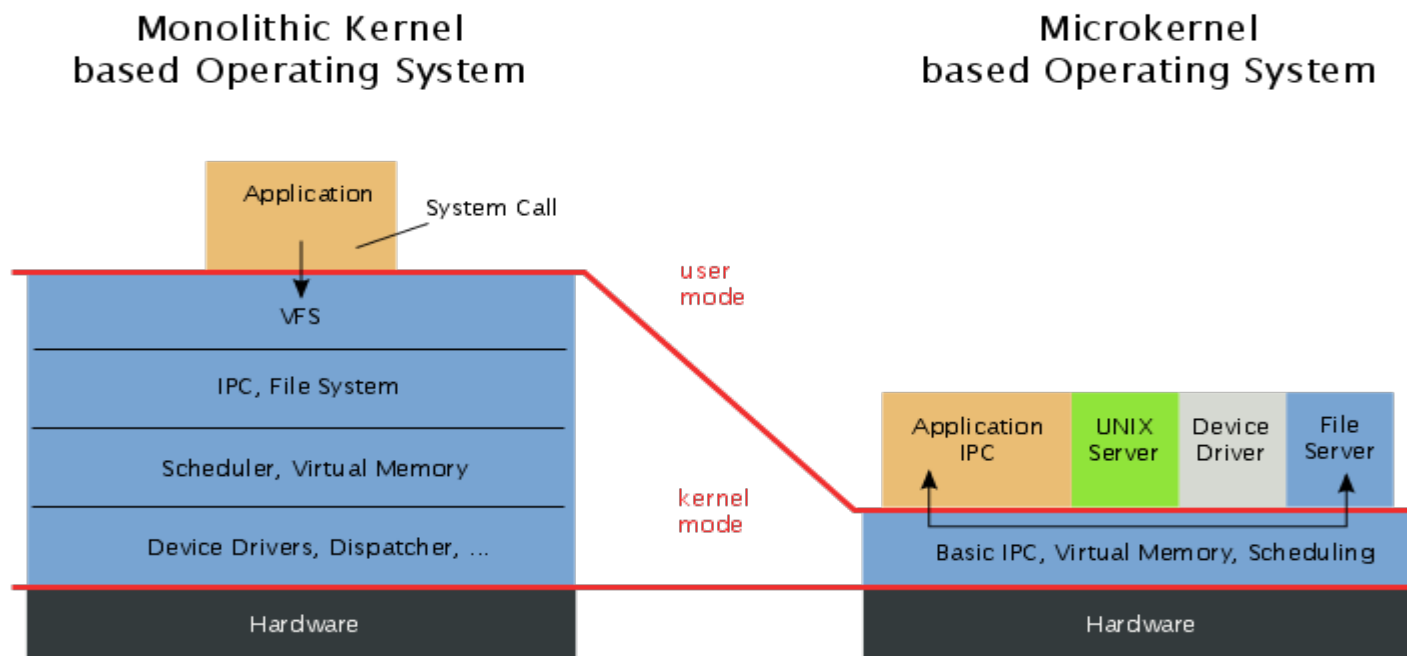
Nucleu monolitic:

- toate componentele S.O.-ului rulează în *kernelspace* (i.e. în mod privilegiat) și oferă servicii aplicațiilor prin intermediul apelurilor de sistem
- exemple: UNIX, VMS, DOS, Windows 3.x/9x
- dezavantaje: greu de scris, inflexibilitate
- Nuclee monolitice modulare: au module executabile ce se pot încărca/descărca la cerere în/din memorie la *runtime*
Exemple: Linux, Solaris, FreeBSD și alte variante

Micro-nucleu:

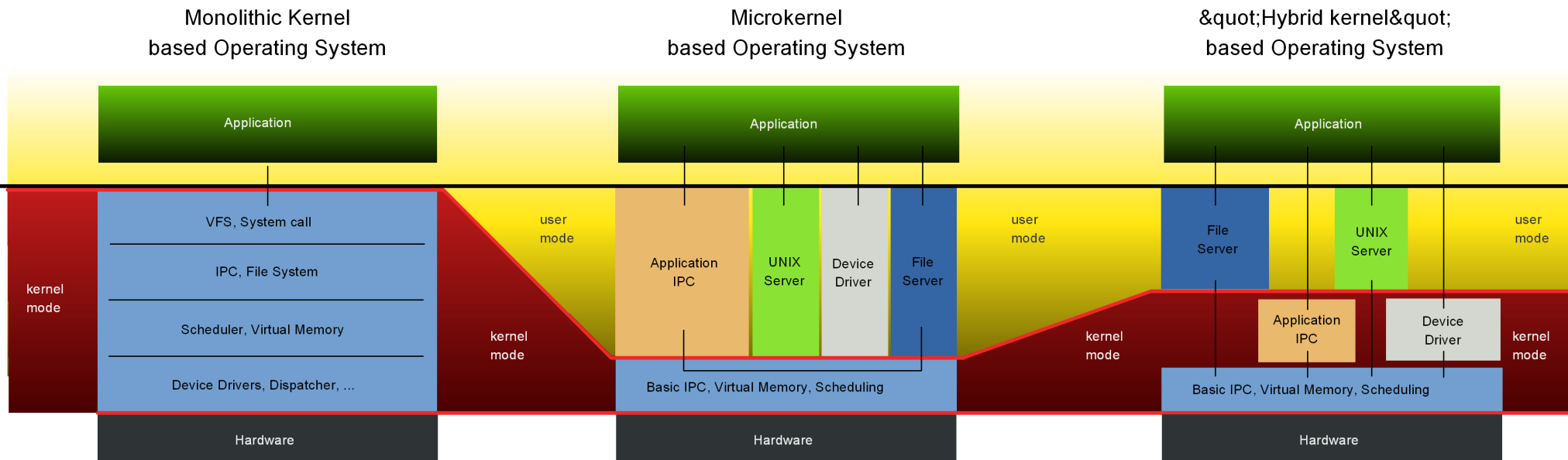
Nucleul /5

- principiul minimalității: doar componentele esențiale rulează în *kernelspace*, majoritatea serviciilor rulează în *userspace* (i.e. în mod neprivilégiat) sub formă de procese server
- modularitate, scalabilitate, adaptabilitate, dimensiune mică
- exemple: Mach (@CMU), L4 microkernel, MINIX, QNX



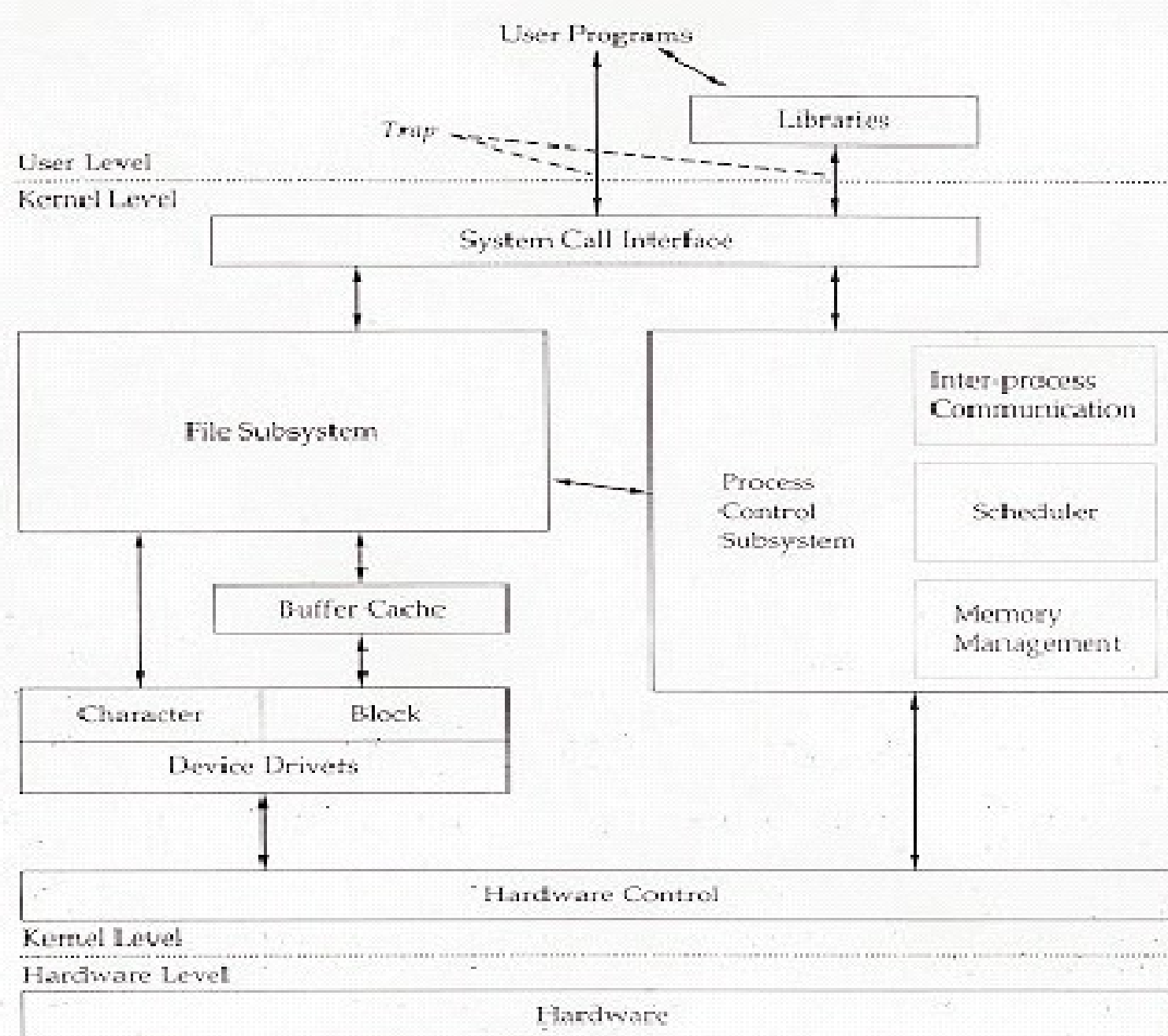
Nucleu hibrid:

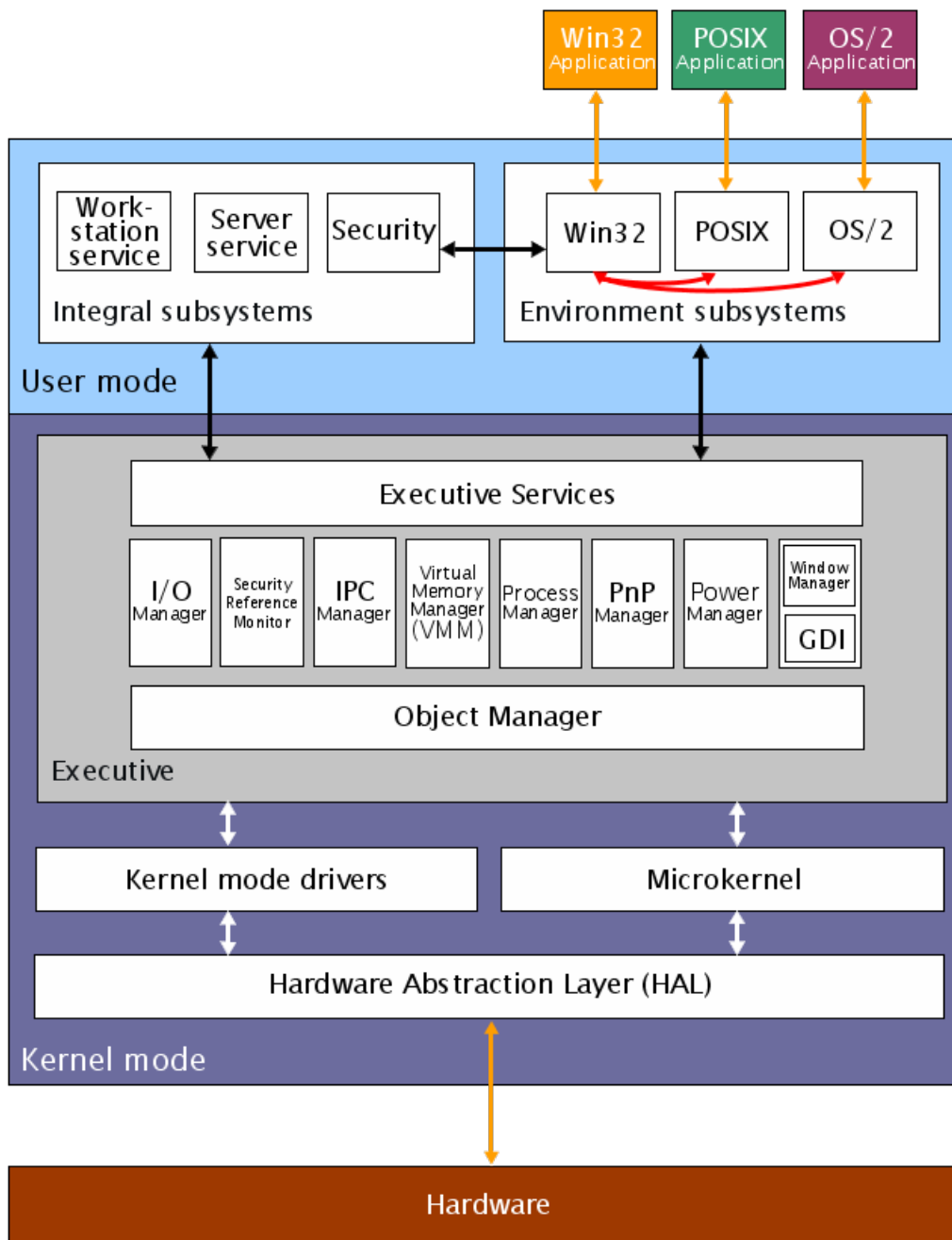
- o combinație a arhitecturilor monolitică și micro
- exemple: NT kernel (familia Windows NT4/2000/XP/2003/Vista/2008/7), Plan 9 (@Bell Labs), ReactOS



Nucleul /7

Nucleul UNIX





Nucleul /8

Nucleul
WindowsNT

Bibliografie

- **Bibliografie obligatorie**
capitolele introductive din
 - Silberschatz : “*Operating System Concepts*”
(cap.2 din [OSCE8])
 - sau
 - Tanenbaum : “*Modern Operating Systems*”
(a doua parte a cap.1 din [MOS3])

Sumar

- Organizarea unui sistem de calcul
- Structura unui S.O.
- Servicii oferite de S.O.
- Abstractizări și API-uri S.O.
- Programe de sistem
- Nucleul S.O.

Întrebări ?

Sisteme de Operare

Gestiunea proceselor – partea I

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

Cuprins

- Conceptul de proces
- Stările procesului
- Relații între procese
- Procese concurente
- Planificarea proceselor

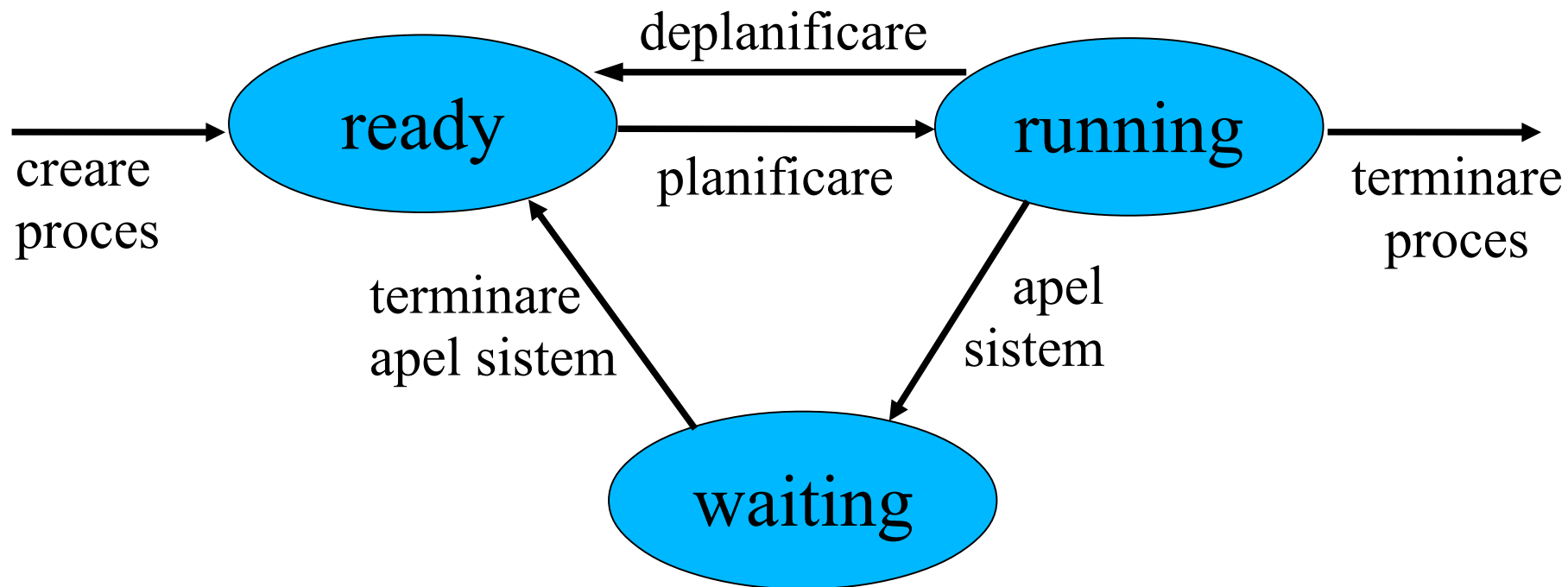
Conceptul de proces

- Proces (task, job)
 - **Job** : un program în curs de execuție, fiind o secvență formată din unul sau mai multe procese
 - **Proces** : o entitate activă (dinamică) a S.O.-ului, fiind unitatea de lucru într-un sistem de calcul
- Un proces include, printre alte resurse:
 - **zona de cod** (ce conține codul programului)
 - **zona de date** (ce conține variabilele globale)
 - **stiva de lucru** (ce conține informații temporare, e.g. parametrii subrutinelor, adresele de return, variabilele temporare)

Stările procesului /1

- Pe parcursul execuției sale, un proces își schimbă **starea**
- Fiecare proces (mai exact, fir de execuție) poate fi într-una din următoarele stări:
 - **running** (în execuție)
 - **waiting** (în așteptare)
 - **ready** (gata de execuție)
- În orice moment, un singur proces poate fi în starea running (în cazul sistemelor uniprocessor)

Stările procesului /2



Blocul de control al procesului

- **PCB** (Process Control Block) este o structură de date reprezentând un proces în cadrul S.O.-ului, ce păstrează următoarele informații:
 - ID-ul procesului (i.e. PID-ul)
 - PID-ul procesului părinte (cel care a creat respectivul proces)
 - starea procesului
 - contorul de program (*program counter*) și ceilalți regiștri CPU
 - directorul curent de lucru; linia de comandă; variabilele de mediu
 - drepturile de acces la resursele sistemului
 - fișierele deschise de respectivul proces
 - informații de planificare a CPU
 - informații de gestiune a memoriei
 - informații pentru raportări (*accounting*)
 - informații despre starea I/E

ș.a.

Relații între procese /1

- Un proces este **independent** dacă nu poate afecta și nici nu poate fi afectat de celelalte procese ce se execută în sistem
 - “starea” procesului **nu este partajată** de alte procese
 - execuția procesului este **deterministă** (depinde în întregime numai de datele de intrare)
 - execuția procesului este **reproductibilă** (rezultatul execuției va fi mereu același pentru aceleași date de intrare)
 - execuția procesului **poate fi suspendată** și apoi **poate fi reluată** fără a cauza efecte nedorite

Relații între procese /2

- Un proces este **cooperant** dacă poate afecta sau poate fi afectat de celelalte procese ce se execută în sistem
 - “starea” procesului **este partajată** de alte procese
 - rezultatul execuției procesului **nu poate fi prevăzut** în avans
 - rezultatul execuției procesului este **nedeterminist** (nu depinde numai de datele de intrare)

Relații între procese /3

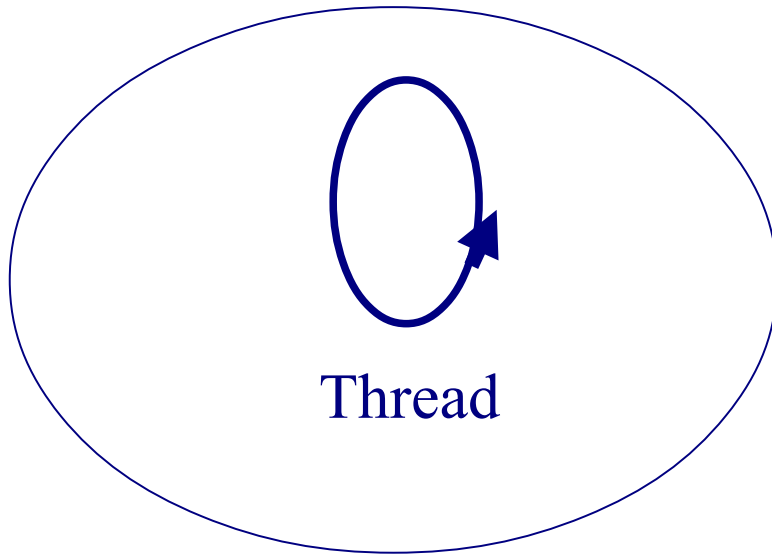
- De ce se utilizează mai multe procese / thread-uri ?
 - Pentru a captura activități natural concurente în cadrul sistemului programat
 - Tratarea evenimentelor asincrone
 - Pentru a câștiga viteză de execuție (*speedup*) prin suprapunerea activităților de calcul cu cele de I/E sau prin exploatarea hardware-ului paralel

Relații între procese /4

- Abstractizarea **thread** – definește un singur flux secvențial de instrucțiuni (contor program, stivă, valori regiștri)
 - Un thread este unitatea de bază de utilizare a CPU
 - Poate fi suportat de nucleul S.O.-ului (e.g. OS/2, Windows NT/2000/XP/2003/Vista/2008/7/8..., UNIX)
- **Proces** – resursa context, cu rol de “container” pentru unul sau mai multe thread-uri (spațiu de adrese partajat de către acestea)

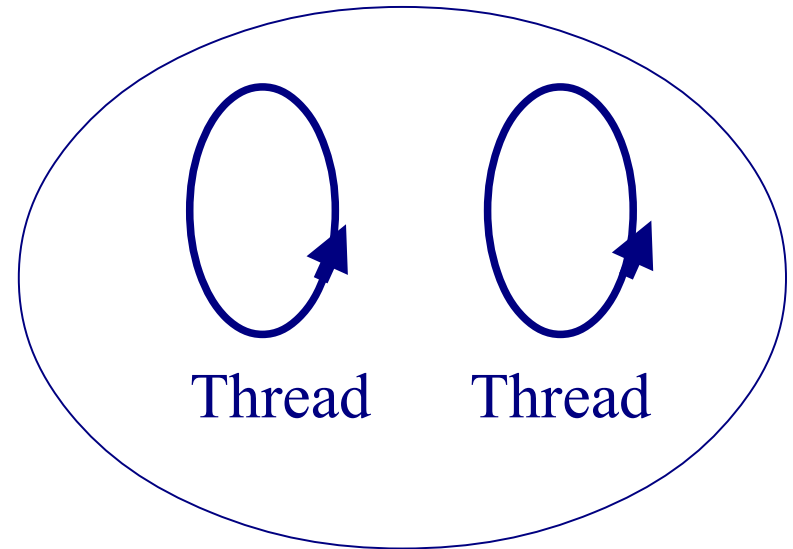
Relații între procese /5

Thread



Address Space

Process



Address Space

Procese concurente /1

- Procese **multiple** pot fi multiprogramate pe un **singur** CPU
- Motive:
 - partajarea resurselor fizice
 - partajarea resurselor logice
 - creșterea vitezei de calcul (*speedup* computațional)
 - modularitate
 - comoditate de utilizare a sistemului

Procese concurente /2

- Crearea și terminarea proceselor (modelul Unix)

```
int pid;  
int status = 0;  
  
if (pid = fork())  
{ /* parent */  
    .....  
    pid = wait (&status);  
}  
else  
{ /* child */  
    .....  
    exit(status);  
}
```

*Primitiva **fork** returnează zero fiului și PID-ul fiului părintelui*

***Fork** creează o copie exactă a procesului părinte*

*Părintele folosește **wait** pentru a dormi până când fiul se termină; apelul wait returnează PID-ul fiului și codul de terminare. Variantele de **wait** permit așteptarea unui anumit fiu, sau notificarea stopării fiului sau a altor semnale.*

*Fiul întoarce părintelui codul de terminare la **exit**, pentru a raporta succesul/eșecul.*

- Procese **părinte** și **fiu**

Procese concurente /3

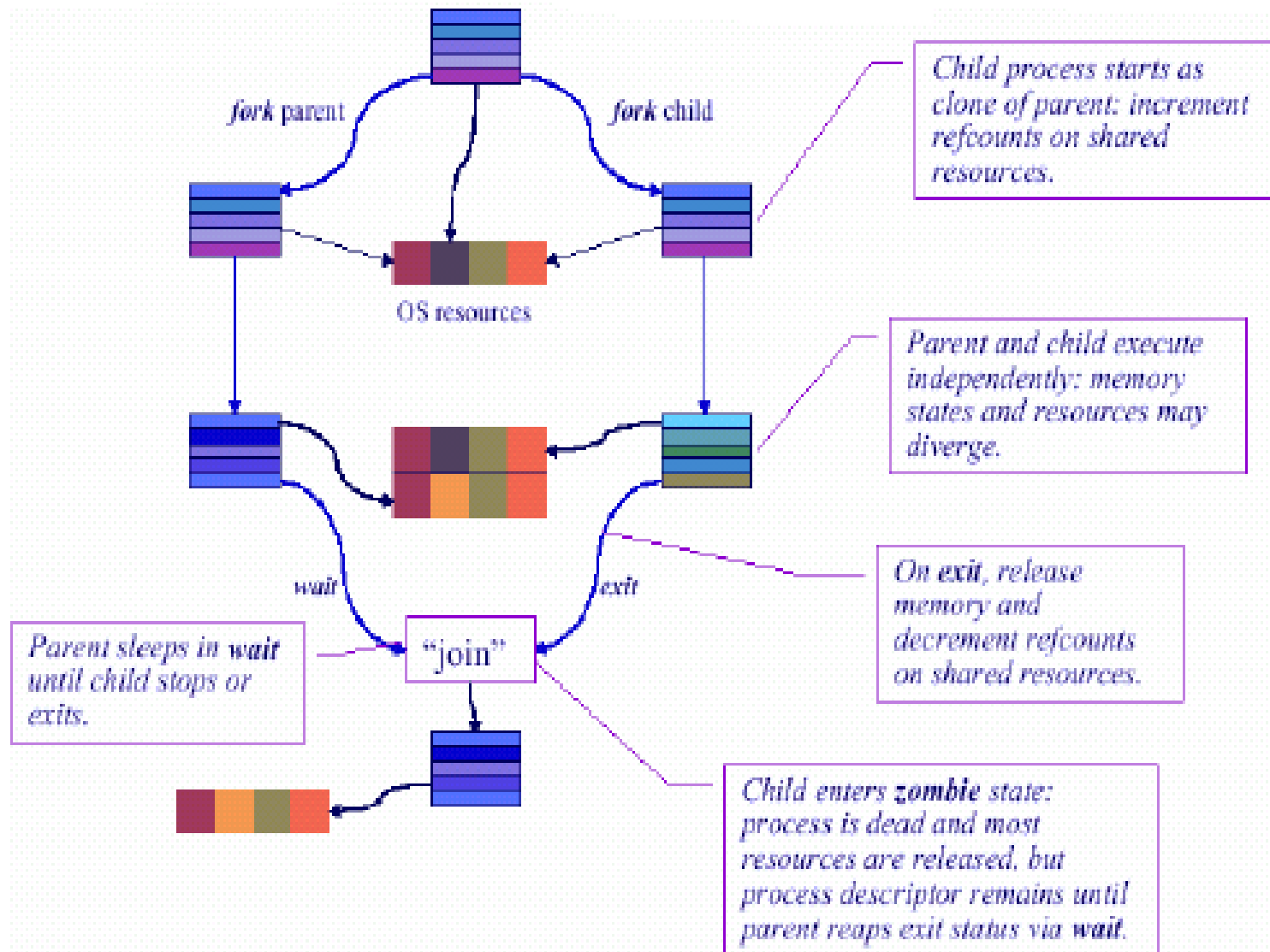
- Disciplina fiului
 - După un apel `fork()`, programul părinte are controlul total asupra comportamentului fiului său
 - Fiul își moștenește mediul de execuție de la părinte (dar programul părinte îl poate schimba)
 - asignările descriptorilor de fișiere sunt setate cu `open()`, `close()`, `dup()`
 - `pipe()` inițializează canalele de comunicație între procese
 - Programul părinte poate pune fiul să execute un program diferit, apelând `exec()` în contextul fiului

Procese concurente /4

- Procesul fiu trebuie să poată fi diferit de părinte
 - Primitivele `exec()` “bootează” fiul cu o imagine executabilă diferită de cea a părintelui
 - programul părinte apelează primitiva `exec()` (în contextul fiului creat) pentru a executa în acesta un nou program
 - `exec()` reacoperă procesul fiu cu o nouă imagine executabilă
 - restartează fiul în mod utilizator la un punct de intrare predeterminat
 - nu returnează nici o valoare programului apelant (acesta nu mai există)
 - argumentele liniei de comandă și variabilele de mediu sunt transferate în memorie
 - descriptorii de fișiere, PID-ul, ș.a. rămân neschimbate

Procese concurente /5

Exemplu



Procese concurente /6

- Mai multe cazuri trebuie considerate pentru “join”
e.g. `exit()` și `wait()`
 - Ce se întâmplă dacă fiul face `exit` (se termină) înainte ca tatăl să facă `join`?
 - Un obiect proces “zombie” păstrează codul de terminare și informațiile de stare ale fiului
 - Ce se întâmplă dacă tatăl se termină înaintea fiului?
 - Orfanii devin copii ai procesului **init** (cu PID-ul 1)
 - Ce se întâmplă dacă tatăl nu-și poate permite să aștepte la un punct de `join`?
 - Facilități pentru notificări asincrone (prin semnale Unix)

Planificarea proceselor

- Planificarea proceselor (va fi continuată)
 - Obiective
 - Cozi de planificare
 - Planificatoare
 - Structura planificării
 - Schimbarea contextului
 - Priorități
 - Algoritmi de planificare

Planificarea proceselor /1

- Obiectivele gestiunii procesorului
 - de a aloca timp CPU la joburile/procese de executat, într-o asemenea manieră încât să optimizeze un anumit aspect (sau mai multe aspecte) ale performanței utilizării sistemului de calcul

Planificarea proceselor /2

Obiective urmărite:

- **Echitate**

Asigurarea faptului că fiecare proces are șanse echitabile la CPU

- **Timp de răspuns**

Minimizarea timpului de răspuns pentru utilizatorii interactivi

- **Predictibilitate**

Asigurarea faptului că un același job va avea o aceeași durată de execuție indiferent de variabilele sistemului

- **Eficiența**

Furnizarea unui grad ridicat de utilizare a CPU

- **Utilizarea resurselor**

Asigurarea faptului că toate resursele sunt folosite la maxim

- **Throughput** (rata de servire)

Maximizarea numărului de joburi executate pe oră

- **Evitarea amânării la infinit**

Asigurarea faptului că toate joburile se termină de executat

- **Deadlines** (termene limită)

Asigurarea îndeplinirii termenelor limită specificate de utilizatori

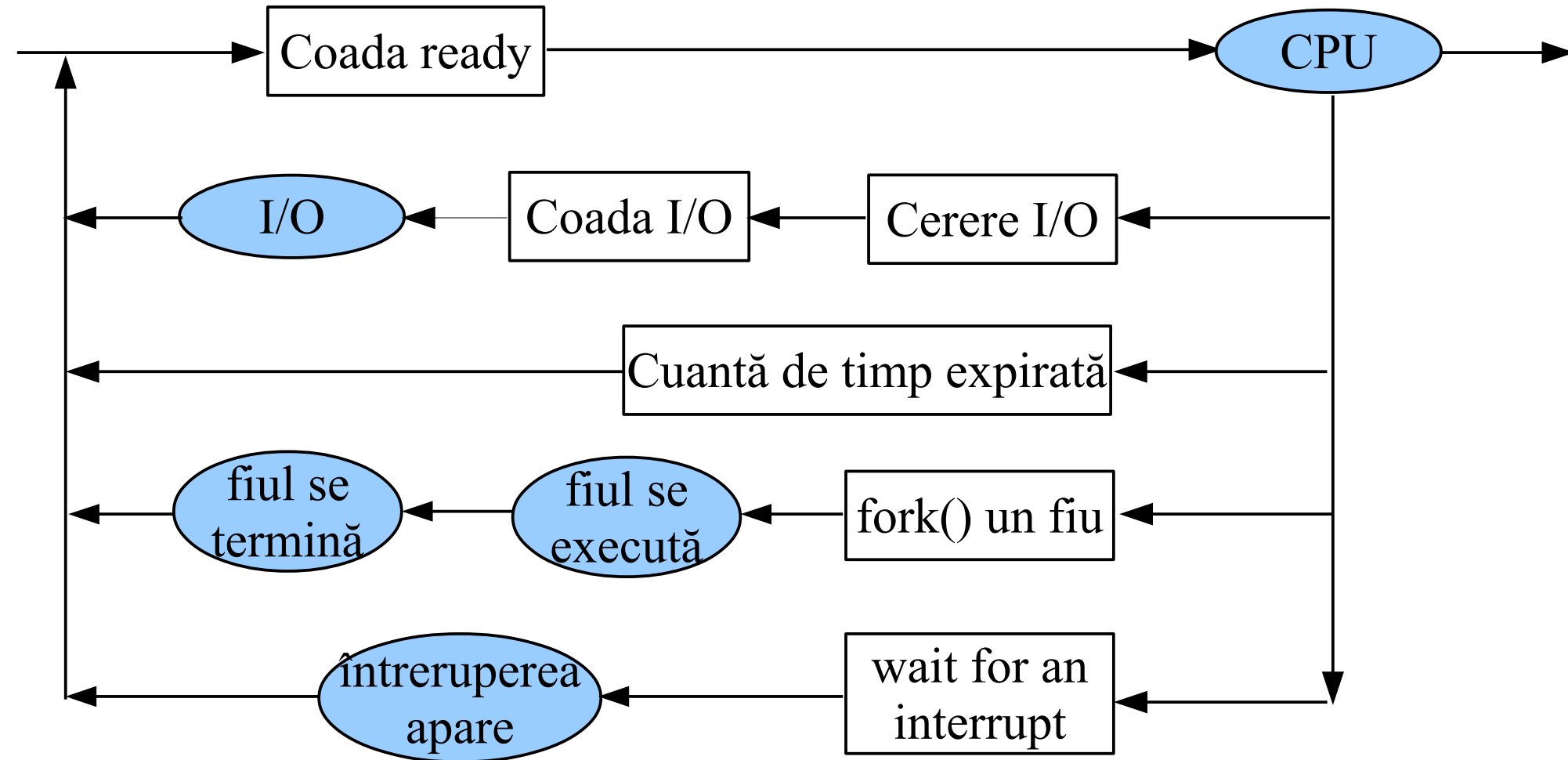
Cozi de planificare /1

- Cozi de planificare
 - pe măsură ce procesele intră în sistem, sunt depuse într-o **coadă de joburi** (cu toate procesele ce așteaptă să li se aloce memoria principală)
 - procesele ce sunt rezidente în memoria principală și care sunt gata de execuție și așteaptă să fie executate, sunt păstrate în **coada ready** (gata de execuție)
 - procesele ce așteaptă un dispozitiv periferic I/O sunt păstrate într-o **coadă I/O** (coada periferic)

Cozi de planificare /2

- Un nou proces este pus inițial în coada ready
- El așteaptă în coada ready până când este selectat pentru execuție și i se dă CPU-ul
- După ce CPU-ul îi este alocat procesului și începe să-l execute, pot apare mai multe evenimente:
 - procesul poate lansa o cerere I/O și apoi este plasat într-o coadă I/O
 - procesul poate `fork()` un nou proces și `wait()` terminarea fiului
 - procesul poate fi înlăturat forțat de pe CPU (ca urmare a unei întreruperi) și plasat înapoi în coada ready

Cozi de planificare /3



Planificatoare /1

Activitatea S.O.-ului de planificare poate fi considerată că se desfășoară la trei nivele:

- 1. La nivelul înalt (planificarea joburilor)** se decide care joburi pot intra în sistem pentru a concura pentru resursele acestuia
- 2. La nivelul de mijloc (planificarea proceselor)** se ajustează prioritățile proceselor și se pot suspenda procese, determinând astfel care procese vor concura pentru CPU
- 3. La nivelul scăzut (dispecerat)** se decide cărui thread i se va da efectiv CPU-ul

Planificatoare:

1. Planificator pe termen lung (planificator de joburi)

- selectează procesele și le încarcă în memorie pentru execuție
- controlează *gradul de multi-programare* (i.e., numărul de procese din memorie)

2. Planificator pe termen scurt (planificator CPU)

- selectează dintre procesele care sunt în starea ready (gata de execuție) unul căruia îi alocă CPU-ul pentru următoarea cantă de timp procesor
- trebuie să fie foarte rapid, deoarece va fi executat cel puțin o dată la ~ 10 ms

Tipuri de procese:

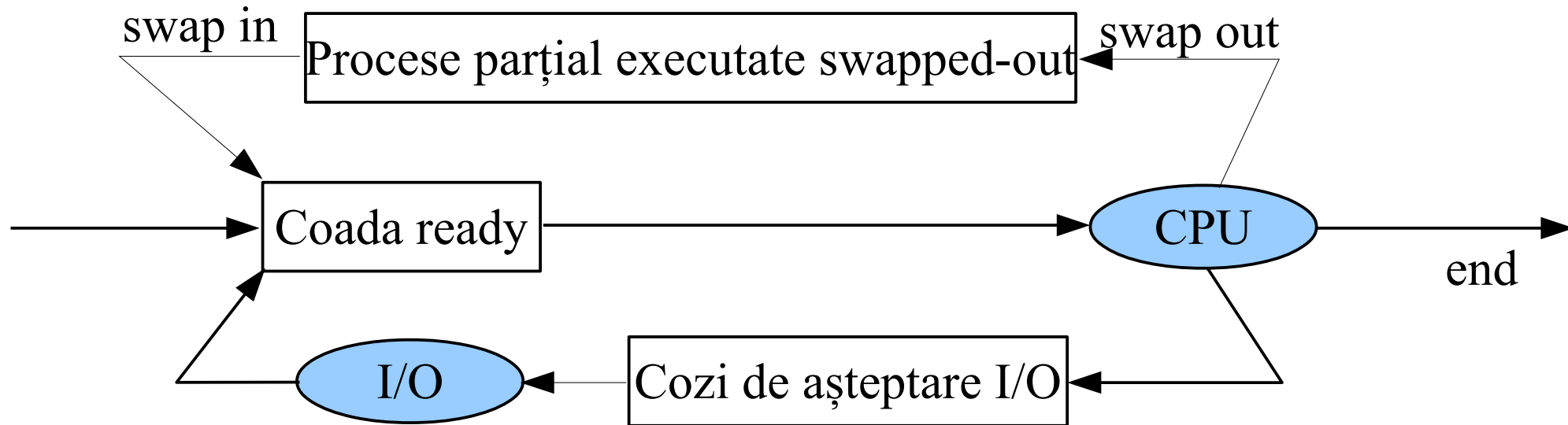
- Procese I/O-intensive
 - un proces care generează des cereri I/O, i.e. care-și petrece mai mult timp făcând operații I/O decât efectuând calcule
- Procese CPU-intensive
 - un proces care generează rar cereri I/O, petrecându-și timpul mai mult făcând calcule decât operații I/O

Planificatoare /4

- Orice algoritm de planificare trebuie să ia în calcul următorii factori:
 - i) dacă un task este I/O-intensiv sau CPU-intensiv,
 - ii) dacă un task este de tip batch sau interactiv, și
 - iii) cât de urgent se cere a fi răspunsul.
- Sistemul cu cea mai bună performanță va avea o combinație de procese CPU-intensive și I/O-intensive.
- Pentru sistemele moderne, planificatorul pe termen lung poate fi minimal sau chiar absent.

Planificatoare /5

- Unele S.O.-uri (e.g. sisteme cu time-sharing) pot introduce un nivel intermediar de planificare:
planificatorul pe termen mediu



Schema de swapping

- **Bibliografie obligatorie**
capitolele despre *gestiunea proceselor* din
 - Silberschatz : “*Operating System Concepts*”
(cap.3,5 din [OSCE8])
 - sau
 - Tanenbaum : “*Modern Operating Systems*”
(prima parte a cap.2 din [MOS3])

Sumar

- Conceptul de proces
- Stările procesului
- Relații între procese
- Procese concurente
- Planificarea proceselor
 - Obiective
 - Cozi de planificare
 - Planificatoare

va fi continuată

Întrebări ?

Sisteme de Operare

Gestiunea proceselor (partea a II-a)

Cristian Vidrașcu

<https://profs.info.uaic.ro/~vidrascu>

Cursul precedent

- Conceptul de proces
- Stările procesului
- Relații între procese
- Procese concurente
- Planificarea proceselor
 - Obiective
 - Cozi de planificare
 - Planificatoare

- Planificarea proceselor (continuare)
 - Structura planificării
 - Schimbarea contextului
 - Priorități
 - Algoritmi de planificare:
FCFS, SJF, Priorități, RR, ș.a.

Structura planificării /1

- Structura planificării

- Deciziile de planificare a CPU se iau în următoarele situații:

1. când un proces trece din starea running în starea waiting (e.g. cerere I/O, sau apel wait)
2. când un proces trece din starea running în starea ready (e.g. când apare o întrerupere hardware de ceas ce marchează sfârșitul unei cuante de timp procesor)
3. când un proces trece din starea waiting în starea ready (e.g. terminarea unei operații I/O)
4. când un proces se termină

Structura planificării /2

- Structura planificării

- Pentru situațiile 1. și 4., un nou proces (dacă există vreunul în starea ready) trebuie să fie selectat pentru execuție.
- Când planificarea se face numai datorită situațiilor 1. și 4., schema de planificare este numită **ne-preemptivă**; altfel, ea este **preemptivă**.
- O politică de planificare este numită **preemptivă** dacă, o dată ce unui proces i s-a dat CPU-ul, acesta poate mai târziu să-i fie luat.

Și este **ne-preemptivă** dacă CPU-ul nu mai poate fi luat procesului ce-l deține, i.e. fiecare proces rulează până la terminare sau la efectuarea unei cereri I/O, sau apel wait.

Structura planificării /3

- Structura planificării
 - O politică ne-preemptivă implică mai puțină încărcătură (*overhead*) a sistemului și face ca timpul total de la startul execuției unui program și până la terminarea lui să fie mai ușor de anticipat.
 - Schemele preemptive sunt importante în sistemele în timp real și în cele cu timp partajat, dar acestea implică schimbarea frecventă a proceselor pe CPU (*process switching*), ceea ce poate determina o încărcare suplimentară semnificativă a sistemului.

Structura planificării /4

- **Preempție**
 - Politicile de planificare pot fi *preemptive* sau *ne-preemptive*.
Preempție: planificatorul poate forța un proces să renunțe la procesor înainte ca procesul să se blocheze (i.e. să inițieze o operație I/O), să renunțe singur la CPU, sau să se termine.
- **Cuantificarea timpului CPU (*timeslicing*)** previne monopolizarea CPU-ului de către vreun proces
 - Planificatorul alege un proces ready și-l execută o *cuantă* de timp.
 - Un proces ce se execută mai mult decât cuanta sa de timp, este forțat să renunțe la CPU de către codul planificatorului rulat prin *handler*-ul întreruperii hardware de ceas.
- In politicile de planificare pe bază de **priorități** se folosește preempția pentru a onora prioritățile
 - Procesul curent running este preemptat dacă un proces cu o prioritate mai mare intră în starea ready.

Schimbarea contextului

- **Schimbarea contextului** (*context switch*)
 - Comutarea CPU-ului către alt proces necesită salvarea stării vechiului proces și încărcarea stării salvate a noului proces ce urmează să se execute
 - Timpul necesar pentru schimbarea contextului constituie o încărcare suplimentară a sistemului (de ordinul 1-100 μs (microsecunde)), dar depinde foarte mult de suportul oferit de hardware

- **Prioritate**

- Anumite obiective pot fi îndeplinite prin încorporarea în disciplina de planificare de bază (gen round-robin) a unei noțiuni de *prioritate* a proceselor.
- Fiecare proces din coada ready are asociată o anumită valoare a priorității; planificatorul favorizează procesele cu valori mai ridicate ale priorității.

Priorități /2

- Valori *externe* ale priorității
 - sunt impuse sistemului din afara sa
 - reflectă preferințe externe pentru anumiți utilizatori sau joburi
 (“Toate joburile sunt egale, dar unele sunt mai egale decât altele...”)
 - exemplu: primitiva Unix `nice()` micșorează prioritatea unui job
 - exemplu: joburile urgente într-un sistem de control în timp real
- Manipularea priorităților
 - **extern** – valori statice, în funcție de rangul utilizatorului, ș.a.
 - **intern** – planificatorul calculează dinamic prioritățile și le utilizează pentru gestiunea cozilor de planificare
 (i.e., sistemul ajustează intern valorile priorităților, pe parcursul execuției joburilor, printr-o tehnică implementată în planificator.)

Priorități /3

Prioritățile trebuie manevrate cu grijă atunci când există dependențe între procese cu priorități diferite.

- Un proces cu prioritatea P ar trebui să nu împiedice niciodată progresul unui proces cu prioritatea $Q > P$.

O astfel de situație se numește *inversiunea priorității* și trebuie să se evite apariția sa.

- Soluția cea mai simplă constă într-o *moștenire a priorității*:

Când un proces cu prioritatea Q așteaptă o anumită resursă, deținătorul ei (cu prioritatea P) moștenește temporar prioritatea Q dacă $Q > P$.

Moștenirea s-ar putea să fie necesară și atunci când procesele se coordonează prin IPC (Inter-Process Communication).

- Moștenirea este utilă și în alte situații, spre exemplu, pentru a îndeplini anumite termene limită.

Planificarea proceselor

➤ Algoritmi de planificare

– Criterii:

- Gradul de utilizare a CPU-ului (% timp non-idle; 40%-90%)
- Rata de servire (numărul de procese/unitatea de timp)
- Timpul *turnaround* (intervalul scurs între momentul submiterii și cel al terminării unui proces; timpul de viață)
- Timpul de așteptare (timpul petrecut în coada ready)
- Timpul de răspuns (timpul scurs între emiterea unei comenzi de către utilizator și producerea primului răspuns la acea comandă)

– Scopuri:

- Maximizarea utilizării CPU și a ratei de servire
- Minimizarea timpilor turnaround, de așteptare și de răspuns

Algoritmi de planificare /1

➤ **Algoritmi de planificare**

- First-Come, First-Served (FCFS)
- Shortest-Job-First (SJF)
- Planificarea cu priorități
- Round-Robin (RR)
- Planificarea cu cozi pe nivele multiple
- Planificarea în timp real
- Planificarea cu procesoare multiple

Algoritmi de planificare /2

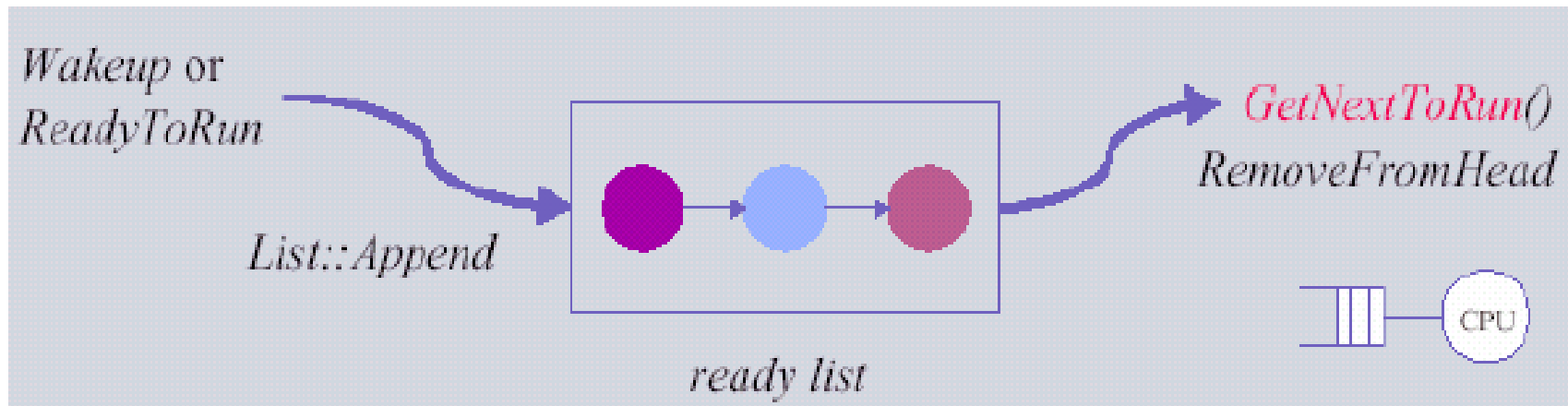
- **First-Come, First-Served (FCFS)**

- procesul care solicită primul să i se acorde timp CPU, este primul căruia i se va aloca CPU-ul
- implementare: o simplă structură FIFO
- algoritmul este simplu de scris și de înțeles
- alg. de planificare FCFS este ne-preemptiv (deci nu poate fi utilizat pentru medii interactive)
- procesele lungi sunt favorizate de politica de planificare FCFS, iar cele scurte sunt defavorizate

Algoritmi de planificare /3

- **First-Come, First-Served (FCFS)**

- **rata de servire** – alg. FCFS este la fel de bun ca orice altă politică de planificare ne-preemptivă ...
... dacă CPU-ul ar fi singura resursă planificabilă din sistem
- **echitate** – alg. FCFS este intuitiv echitabil
- **timpul de răspuns** – procesele lungi le țin pe toate celelalte în așteptare



Algoritmi de planificare /4

• First-Come, First-Served (FCFS)

Scenariu:
(exemplu)

Ipoteză de lucru:
doar procese
CPU-intensive
(fără nici o op. I/O)

Procese	Momentul sosirii	Timpul de serviciu solicitat
A	0	3
B	1	5
C	3	2
D	9	5
E	12	5

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	A	A	B	B	B	B	B	C	C	D	D	D	D	D	E	E	E	E	E	

Algoritmi de planificare /5

• First-Come, First-Served (FCFS)

Rezultatele
planificării
FCFS

Proces	Sosire	Serviciu	Start	Finish	T	W	P
A	0	3	0	3	3	0	1.0
B	1	5	3	8	7	2	1.4
C	3	2	8	10	7	5	3.5
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6

Notatii:

Start = momentul când devine *running*; Finish = momentul când se termină de executat

t = timpul de execuție propriu-zisă (i.e. timpul de serviciu)

T = timpul de viață (= moment finish – moment sosire)

W = timpul de așteptare (*în coada ready*) (= $T - t$)

P = rata de penalitate = T/t ; **R** = rata de răspuns = t/T

Algoritmi de planificare /6

• First-Come, First-Served (FCFS)

Al doilea scenariu (exemplu): procesele pot face și operații I/O (pentru simplitate, în cazul lor în coloana Serviciu se vor specifica atât duratele rafalelor CPU, cu negru, cât și duratele operațiilor IO, cu roșu)

Rezultatele
planificării
FCFS

Proces	Sosire	Serviciu	Start	Finish	T	W	P
A	0	2; 2 ;1	0	8	8	3	1.6
B	1	5	2	7	6	1	1.2
C	3	2	8	10	7	5	3.5
D	9	3; 2 ;2	10	20	11	4	1.57
E	12	5	13	18	6	1	1.2

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	A	B	B	B	B	B	A	C	C	D	D	D	E	E	E	E	E	D	D	

Algoritmi de planificare /7

- **Shortest-Job-First (SJF)**

- **Ideea:** scoaterea rapidă din sistem a proceselor scurte pentru a minimiza numărul de procese aflate în așteptare cât timp rulează un proces lung
- Intuitiv: procesele cele mai lungi dăunează cel mai mult pentru timpii de așteptare ai competitorilor lor
- SJF este **optimal** (lucru demonstrabil matematic), în sensul că produce cel mai mic timp mediu de așteptare pentru o mulțime dată de procese
- Este nevoie de **anticiparea** timpilor de serviciu CPU

Algoritmi de planificare /8

- **Shortest-Job-First (SJF)**

- Planificarea SJF poate fi ne-preemptivă sau preemptivă
- Planificarea SJF preemptivă este numită planificare **shortest-remaining-time-first (SRTF)**
- SJF favorizează procesele interactive, ce necesită răspuns rapid și care interacționează cu utilizatorul în mod repetat
- SJF favorizează procesele ce produc rafale (*bursts*) I/O – care se blochează curând, țin perifericele ocupate, eliberând astfel CPU-ul
- Atenția este îndreptată spre o măsură *medie* a performanței, unele procese lungi pot fi înfometate în cazul unei încărcări masive a sistemului sau a unui flux constant de noi procese scurte ce intră în sistem

Algoritmi de planificare /9

- **Shortest-Job-First (SJF)**

- Sacrifică echitatea pentru a micșora timpul mediu de răspuns
- Planificarea SJF pură este impracticabilă: planificatorul nu poate anticipa durata unui proces
- Totuși, SJF are valoare în sistemele reale:
 - Multe aplicații execută o secvență de rafale CPU scurte cu operații I/O între acestea
 - E.g., joburile *interactive* se blochează în mod repetat pentru a accepta input din partea utilizatorului
Scop: furnizarea celui mai bun timp de răspuns pentru utilizator
 - E.g., joburile pot trece prin perioade de activitate I/O intensivă
Scop: cererea următoarei operații I/O cât mai repede posibil pentru a ține perifericele ocupate și a furniza cea mai bună rată de servire pe ansamblu
 - Folosirea *priorității interne adaptive* pentru a încorpora SJF în RR
Strategia meteorologilor: previziunea viitorului apropiat pe baza trecutului recent

Algoritmi de planificare /10

- **Shortest-Job-First (SJF)**

Rezultatele
planificării
SJF (nepreemptiv)

Proces	Sosire	Serviciu	Start	Finish	T	W	P
A	0	3	0	3	3	0	1.0
B	1	5	5	10	9	4	1.8
C	3	2	3	5	2	0	1.0
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	A	A	C	C	B	B	B	B	B	D	D	D	D	D	E	E	E	E	E	

Algoritmi de planificare /11

- **Planificarea cu priorități**

- Fiecare proces are asociată o prioritate, iar CPU-ul este alocat procesului ready cu prioritatea cea mai mare
- Procesele cu priorități egale sunt planificate în ordinea FCFS
- Valorile priorităților depind de implementarea S.O. (numerele mici pot semnifica prioritati mari, sau invers)
- Observație: SJF poate fi privit ca un algoritm cu priorități, unde prioritatea (p) este inversul duratei următoarei rafale CPU anticipate (c) : $p = 1/c$

- **Planificarea cu priorități**

- Poate fi preemptivă sau ne-preemptivă

- Problemă:

- Blocarea nelimitată sau înfometarea (starvation)*
proceselor cu priorități mici

- Soluție:

- Imbătrânirea (aging) : creșterea graduală a priorității*
proceselor care așteaptă în sistem pentru o perioadă
mare de timp

Algoritmi de planificare /13

• Planificarea cu priorități

Rezultatele
planificării
cu priorități
stative,
preemptivă

Proces	Sosire	Serviciu	Prioritate	Start	Finish	T	W	P
A	0	3	1	0	20	20	17	6.6
B	1	5	2	1	8	7	2	1.4
C	3	2	4	3	5	2	0	1.0
D	9	5	3	9	19	10	5	2.0
E	12	5	5	12	17	5	0	1.0

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	B	B	C	C	B	B	B	A	D	D	D	E	E	E	E	E	D	D	A	

Algoritmi de planificare /14

• Planificarea Round-Robin (RR)

- Fiecărui proces i se oferă serviciul CPU pentru o perioadă scurtă de timp (numită *cuantă* sau *felie de timp*), după care, doar dacă nu a fost terminat, revine la sfârșitul cozii ready și așteaptă să-i vină din nou rândul la CPU
- Această perioadă este adesea de ordinul $10 \div 100$ ms
- A fost proiectată special pentru sistemele cu timp partajat
- Coadă ready este tratată ca o coadă circulară
- Comutarea CPU-ului de la un proces la altul (schimbarea contextului) implică o încărcare suplimentară considerabilă a sistemului

Algoritmi de planificare /15

• Planificarea Round-Robin (RR)

– *Timpul de viață*

RR reduce timpul de viață pentru procesele scurte

- Pentru o încărcare dată a sistemului, timpul de viață al unui proces este direct proporțional cu durata sa de serviciu

– *Echitate*

RR reduce variația în timpii de așteptare

- Dar: RR forțează procesele să aștepte pentru alte procese intrate mai târziu în sistem

– *Throughput (rata de servire)*

RR impune o încărcare suplimentară a sistemului pentru schimbarea contextului, ceea ce dăunează ratei.

Pe un sistem *multiprocesor*, RR poate îmbunătăți rata de servire sub o încărcare ușoară (i.e. număr mic de joburi).

Algoritmi de planificare /16

- **Planificarea Round-Robin (RR)**

Primul scenariu (o cuantă = 1 unitate de timp)

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	B	A	B	C	A	B	C	B	D	B	D	E	D	E	D	E	D	E	E	E

Rezultatele
planificării
RR, cuanta=1

Proces	Sosire	Serviciu	Start	Finish	T	W	P
A	0	3	0	6	6	3	2.0
B	1	5	1	11	10	5	2.0
C	3	2	4	8	5	3	2.5
D	9	5	9	18	9	4	1.8
E	12	5	12	20	8	3	1.6

Algoritmi de planificare /17

• Planificarea Round-Robin (RR)

Al doilea scenariu (o cuantă = 4 unități de timp)

Sumarul planificării:

00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20
A	A	A	B	B	B	B	C	C	B	D	D	D	D	E	E	E	E	D	E	

Rezultatele
planificării
RR, cuanta=4

Proces	Sosire	Serviciu	Start	Finish	T	W	P
A	0	3	0	3	3	0	1.0
B	1	5	3	10	9	4	1.8
C	3	2	7	9	6	4	3.0
D	9	5	10	19	10	5	2.0
E	12	5	14	20	8	3	1.6

Algoritmi de planificare /18

- **Planificarea Round-Robin (RR)**

- Problemă: Cât ar trebui să fie cuanta de timp ?
- Dacă e prea mare, pur și simplu RR devine FIFO
- Dacă e prea mică, CPU-ul este folosit (aproape) în întregime doar de către rutina dispecer pentru selecție și comutarea contextului, deci joburile utilizatorilor nu mai apucă să fie executate
- Durata ideală a cuantei de timp depinde de mulți factori
- O bună regulă generală este: acea perioadă de timp în care marea majoritate a utilizatorilor interactivi pot fi satisfăcuți ($10 \div 100$ ms)
- Durata cuantei este adesea schimbată de două ori pe zi (este mai lungă noaptea, când rulează joburi de tip batch)

Algoritmi de planificare /19

- **Planificarea cu cozi pe nivele multiple**

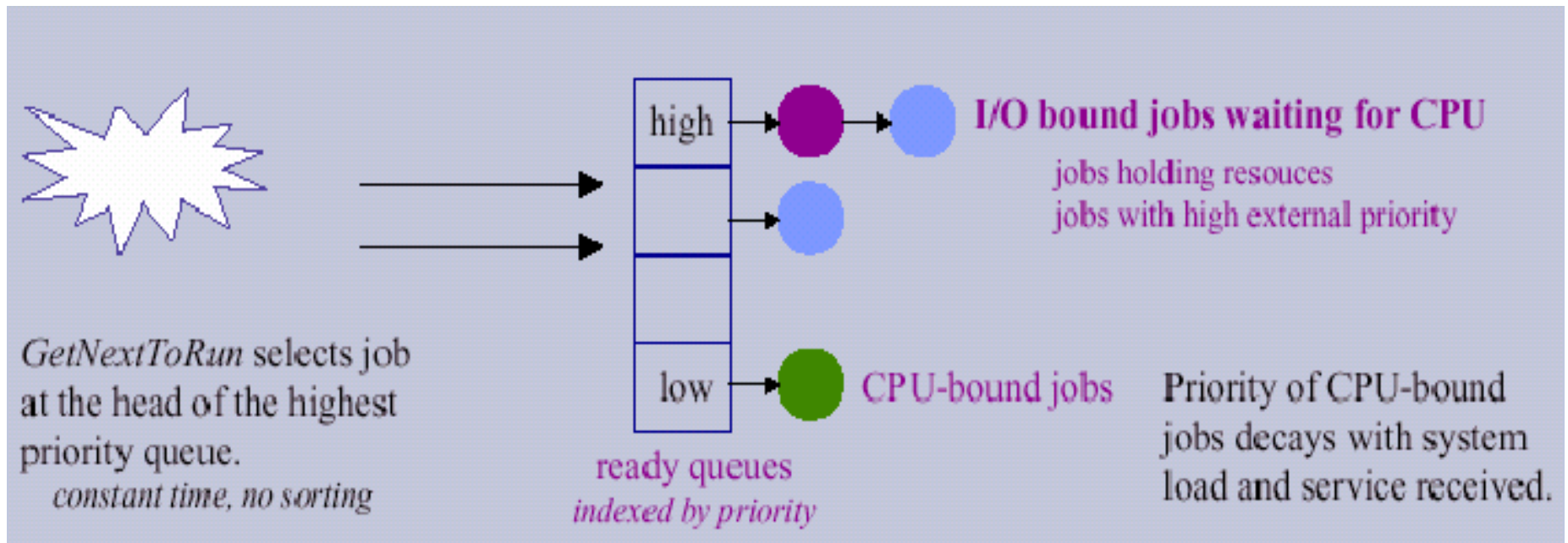
- Divizarea cozii ready în mai multe (sub-)cozi ready separate

E.g., coada proceselor *foreground* (joburi interactive) și coada proceselor *background* (batch-joburi)

- Procesele sunt asignate permanent uneia dintre cozi, în general pe baza unei proprietăți a procesului (dimensiunea memoriei, tipul procesului, etc.)
- Fiecare coadă are propriul algoritm de planificare
- Trebuie să existe o planificare între cozi (e.g., poate fi utilizată o planificare preemptivă cu priorități fixe)

Algoritmi de planificare /20

- Planificarea cu cozi pe nivele multiple

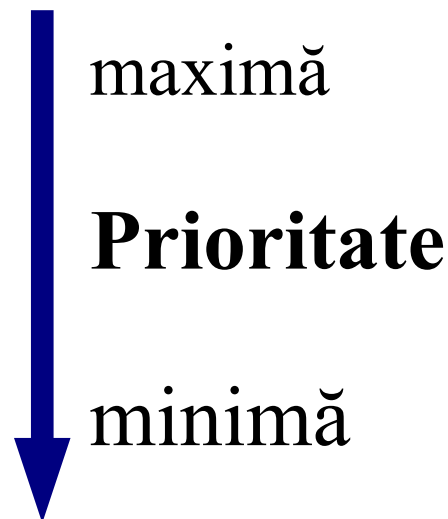


Algoritmi de planificare /21

- **Planificarea cu cozi pe nivele multiple**

- Exemplu:

- Procese de sistem
- Procese interactive
- Procese de editare de texte
- Procese batch



- Fiecare coadă are întâietate absolută asupra cozilor cu priorități mai joase
- Altă soluție: de împărțit timpul în cuante (*time slice*) între cozi

Algoritmi de planificare /22

- **Planificarea cu cozi pe nivele multiple, cu feedback**
 - Se permite unui proces să migreze între cozi
 - Parametri:
 - Numărul de cozi
 - Algoritmul de planificare pentru fiecare coadă
 - Metoda utilizată pentru a decide când un proces să fie avansat într-o coadă cu prioritate mai mare
 - Metoda utilizată pentru a decide când un proces să fie degradat într-o coadă cu prioritate mai mică
 - Metoda utilizată pentru a decide în ce coadă va intra inițial un proces (i.e., atunci când acesta este lansat în execuție)

Algoritmi de planificare /23

- **Planificarea în timp real**

- Planificatoarele în timp real trebuie să suporte execuții regulate, periodice, ale taskurilor (e.g., flux media continuu)

- *Rezervări timp CPU*

- “Am nevoie să execut X din fiecare Y unități de timp.”

- Planificatorul exercită *controlul admiterii* la momentul rezervării: aplicația trebuie să gestioneze eșecul unei cereri de rezervare

- *Constrângeri temporale*

- “Rulează această aplicație înainte de termenul meu limită: momentul T.”

- **Planificarea cu procesoare multiple**

(multiprocesare)

- Probleme:

- Tipul procesoarelor:

- sisteme omogene vs. sisteme heterogene

- Tipul cozilor de planificare:

- coada ready comună vs. câte o coadă ready separată pentru fiecare procesor

- Sisteme SMP: toate procesele intră într-o coadă și sunt planificate pe oricare dintre procesoarele disponibile

- Planificare master-slave (multiprocesare asimetrică)

Algoritmi de planificare /25

• Planificarea multiprocesor

Scenariu: planificare SMP (o singură coadă ready), cu două procesoare

Rezultatele
planificării
cu priorități
statice,
preemptivă

Proces	Sosire	Serviciu	Prioritate	Start	Finish	T	W	P
A	0	3	1	0	5	5	2	1.66
B	1	7	2	1	12	11	4	1.57
C	2	2	4	2	4	2	0	1.0
D	6	5	3	6	11	5	0	1.0
E	7	5	5	7	12	5	0	1.0

Sumarul
planificării:

Procesor	00	01	02	03	04	05	06	07	08	09	10	11	12	13
CPU1	A	A	C	C	A		D	D	D	D	D	B		
CPU2		B	B	B	B	B	B	E	E	E	E	E		

- **Bibliografie obligatorie**
capitolele despre *gestiunea proceselor* din
 - Silberschatz : “*Operating System Concepts*”
(cap.5 din [OSCE8])
 - sau
 - Tanenbaum : “*Modern Operating Systems*”
(a patra parte a cap.2 din [MOS3])

- Planificarea proceselor (continuare)
 - Structura planificării
 - Schimbarea contextului
 - Priorități
 - Algoritmi de planificare:
FCFS, SJF, Priorități, RR, ș.a.

Întrebări ?

Sisteme de Operare

Sincronizarea proceselor – partea I

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

- Introducere
- Problema secțiunii critice
 - Enunțul problemei și cerințele de rezolvare
 - Soluții pentru cazul a 2 procese
 - Soluții pentru cazul a $n > 2$ procese
 - Soluții hardware
 - Semafoare
- Interblocajul și înfometarea
- Probleme clasice de sincronizare
 - Problema Producător-Consumator
 - Problema Cititori și Scriitori
 - Problema Cina Filozofilor
 - Problema Bărbierului Adormit
- Monitoare

- Procesarea concurentă este fundamentul sistemelor de operare multiprogramate
- Pentru a asigura execuția ordonată, sistemul de operare trebuie să ofere mecanisme pentru sincronizarea și comunicația proceselor

➤ “race conditions”:

Uneori procesele cooperante pot partaja un spațiu de stocare pe care fiecare proces îl poate citi și scrie (acest spațiu comun poate fi e.g. o zonă în memoria principală, ori un fișier pe disc).

În acest context, rezultatul final al execuției acelor procese poate depinde nu numai de datele de intrare, ci și de “cine execută exact ce anume și când anume” (i.e. de ordinea exactă de execuție a instrucțiunilor atomice ale acelor procese).

Asemenea situații sunt numite “race conditions”.

Introducere

➤ Exemplu de “race condition”:

Două procese A și B care partajează o variabilă v , inițializată cu valoarea 0. Procesul A execută operația $v:=v+1$, iar B execută operația $v:=v-1$. Rezultatul firesc al execuției celor două procese ar fi ca variabila v să aibă în final tot valoarea 0.

Cele două operații constau de fapt din următoarele secvențe de instrucțiuni atomice (în limbaj mașină):

A: LD reg, adr_v
INC reg
ST reg, adr_v

B: LD reg, adr_v
DEC reg
ST reg, adr_v

Dacă ambele instrucțiuni Load sunt executate înaintea ambelor instrucțiuni Store, atunci rezultatul final este eronat.

Introducere

➤ Cum evităm “race conditions” ?

Ideea de evitare a acestor situații, ce sunt nedorite în orice context ce implică memorie partajată, fișiere partajate, sau orice alt fel de resursă partajată, constă în a găsi o cale de a împiedica să avem mai mult de un singur proces, în același timp, care să citească sau să scrie resursa partajată.

Introducere

- Cum evităm “race conditions” ? (cont.)

Cu alte cuvinte, avem nevoie de **excludere mutuală**, adică de o tehnică care să ne garanteze că dacă un proces utilizează o resursă partajată, atunci toate celelalte procese vor fi împiedicate să o utilizeze.

Notă: alegerea operațiilor primitive adecvate pentru realizarea excluderii mutuale este o problemă de proiectare majoră pentru fiecare sistem de operare, pe care o vom studia în continuare.

Problema Secțiunii Critice

- Enunțul problemei secțiunii critice
- Cerințele de rezolvare
- Soluții pentru cazul a 2 procese
- Soluții pentru cazul a N procese
- Soluții concrete (cu instrucțiuni hardware specializate)
- Semafoare

Problema Secțiunii Critice

Enunțul problemei secțiunii critice:

- Avem n procese P_i , $i=0, \dots, n-1$, cu viteze de execuție necunoscute
- Fiecare proces are o zonă de cod, numită **secțiune critică (SC)**, în care efectuează diverse operații asupra unei resurse partajate
- Execuția secțiunilor critice de către procese trebuie să se producă **mutual exclusiv** în timp: la orice moment de timp cel mult un proces să se afle în SC proprie
- Oprirea oricărui proces are loc numai în afara SC
- Fiecare proces trebuie să ceară permisiunea să intre în secțiunea lui critică. Secvența de cod ce implementează această cerere este numită **secțiunea de intrare**
- Secțiunea critică poate fi urmată de o **secțiune de ieșire**
- Restul codului din fiecare proces este **secțiunea rest**

Problema Secțiunii Critice

Soluția problemei secțiunii critice trebuie să satisfacă următoarele 3 cerințe:

- **Excluderea mutuală** – dacă procesul P_i execută instrucțiuni în SC proprie, atunci nici un alt proces nu poate executa în propria SC.
- **Progresul** – dacă nici un proces nu execută în SC proprie, și există unele procese care doresc să intre în propriile lor SC, atunci numai acele procese care nu execută în secțiunile lor rest pot participa la luarea deciziei care va fi următorul proces ce va intra în SC proprie, iar aceasta selecție nu poate fi amânată la infinit.
- **Așteptarea limitată** – trebuie să existe o limită a numărului de permisiuni acordate altor procese de a intra în SC proprii, între momentul când un proces a cerut accesul în propria SC și momentul când a primit permisiunea de intrare.

Problema Secțiunii Critice

Opțiuni de implementare a excluderii mutuale:

- **Dezactivarea întreruperilor**

(posibilă doar pentru sistemele uniprocessor și eficientă doar pentru secvențe critice scurte)

- **Soluții de așteptare ocupată**

- execută o buclă while de așteptare dacă SC este ocupată
- folosirea unor instrucțiuni atomice specializate

- **Sincronizare blocantă**

- sleep (inserarea în coada de așteptare) cât timp SC este ocupată

Primitivele de sincronizare (diverse abstracții, precum lacătele pe fișiere), ce sunt puse la dispoziție de un sistem, pot fi implementate prin aceste tehnici sau prin combinații ale unora dintre aceste tehnici.

Problema Secțiunii Critice

Un șablon tipic de proces:

repeat

secțiunea de intrare

secțiunea critică

secțiunea de ieșire

secțiunea rest

forever

Problema Secțiunii Critice

Soluții pentru cazul a $n=2$ procese:

- Două procese P_0 și P_1 ce execută fiecare într-o buclă infinită câte un program ce constă din două secțiuni: secțiunea critică c_0 , respectiv c_1 , și restul programului – secțiunea necritică r_0 , respectiv r_1 . Execuția secțiunilor c_0 și c_1 nu trebuie să se suprapună în timp.
- Când se va prezenta procesul P_i , se va utiliza P_j pentru a ne referi la celălalt proces ($j=1-i$).

Problema Secțiunii Critice

Soluția 1 (o primă idee de rezolvare)

- Cele două procese vor partaja o variabilă întreagă comună **turn** inițializată cu 0 (sau cu 1).
- Dacă **turn = i**, atunci procesul P_i este cel căruia i se permite să-și execute SC.

Problema Secțiunii Critice

Soluția 1

repeat

while turn $\neq$ i **do** *nothing*;

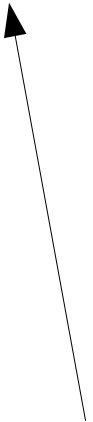
secțiunea critică

turn := j;

secțiunea rest

forever

Așteptare ocupată



Problema Secțiunii Critice

Soluția 1 **este incompletă !**

- Motivul: această soluție satisface condițiile de excludere mutuală și de așteptare limitată, în schimb cerința de progres nu este îndeplinită (e.g., dacă **turn=0** și procesul P_1 este gata să intre în SC proprie, nu poate face aceasta, chiar dacă procesul P_0 este în secțiunea sa rest).

```
P0:  
repeat  
  while turn ≠ 0 do nothing;  
  secțiunea critică  
  turn := 1;  
  secțiunea rest  
forever
```

```
P1:  
repeat  
  while turn ≠ 1 do nothing;  
  secțiunea critică  
  turn := 0;  
  secțiunea rest  
forever
```

Problema Secțiunii Critice

Soluția 2 (o a doua idee de rezolvare)

- Variabila comună **turn** este înlocuită cu un tablou comun **flag[]**, inițializat cu valoarea false:
flag[0] = false , **flag[1] = false** .
- Prin **flag[i] = true** se indică faptul că procesul P_i dorește să-și execute SC.

Problema Secțiunii Critice

Soluția 2

repeat

flag[i] := true;

while flag[j] **do** *nothing*;

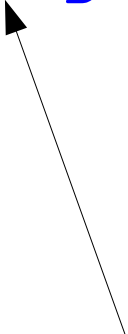
secțiunea critică

flag[i] := false;

secțiunea rest

forever

Așteptare ocupată



Problema Secțiunii Critice

Solutia 2 **este incompletă !**

- Motivul: condițiile de excludere mutuală și de așteptare limitată sunt satisfacute, în schimb cerința de progres nu este îndeplinită
[soluția e dependentă de *timing*-ul (i.e. sincronizarea) proceselor]

```
P0:  
repeat  
  flag[0] := true;  
  while flag[1] do nothing;  
  secțiunea critică  
  flag[0] := false;  
  secțiunea rest  
forever
```

```
P1:  
repeat  
  flag[1] := true;  
  while flag[0] do nothing;  
  secțiunea critică  
  flag[1] := false;  
  secțiunea rest  
forever
```



Problema Secțiunii Critice

Soluția 3 (completă !) (Peterson '81)

- Este o combinație a soluțiilor 1 și 2.
- Procesele partajează variabila `turn` și tabloul `flag[]`.
- Inițializări: `flag[0] = flag[1] = false`, `turn = 0` (sau `1`).
- Pentru a intra în SC, procesul P_i setează `flag[i] = true` și apoi îi dă voie celui alt proces P_j să intre în propria sa SC, dacă dorește acest lucru (`turn = j`).
- Dacă ambele procese doresc să intre în același timp, valoarea lui `turn` va decide căruia dintre cele două procese îi este permis să intre primul în SC proprie.
- Istoric, prima soluție completă e cea datorată lui Dekker ('65)

Problema Secțiunii Critice

Solutia 3

repeat

flag[i] := true;

turn := j;

while (flag[j] **and** turn=j)

do *nothing*;

secțiunea critică

flag[i] := false;

secțiunea rest

forever

Așteptare ocupată



Problema Secțiunii Critice

Soluția 3 este corectă și completă ! Justificare:

- Motivul #1: condiția de excludere mutuală este satisfăcută. Într-adevăr:

Fiecare proces P_i intră în SC proprie doar dacă fie $flag[j]=false$, fie $turn=i$. Dacă ambele procese ar putea să execute în SC proprie în același timp, atunci am avea $flag[0]=flag[1]=true$. Aceasta înseamnă că P_0 și P_1 nu ar fi putut să-și execute cu succes instrucțiunile *while* proprii în același timp.

Problema Secțiunii Critice

Soluția 3 **este corectă și completă !** Justificare:

- Motivul #2: condițiile de progres și de așteptare limitată sunt satisfăcute. Într-adevăr:

```
P0:  
repeat  
  flag[0] := true;  
  turn := 1;  
  while (flag[1] and turn=1)  
    do nothing;  
  secțiunea critică  
  flag[0] := false;  
  secțiunea rest  
forever
```



```
P1:  
repeat  
  flag[1] := true;  
  turn := 0;  
  while (flag[0] and turn=0)  
    do nothing;  
  secțiunea critică  
  flag[1] := false;  
  secțiunea rest  
forever
```

Problema Secțiunii Critice

Soluția 3 **este corectă și completă !** Justificare:

– Motivul #2 (cont.):

Un proces P_i poate fi împiedicat să intre în SC (i.e., să **progreseze**) doar dacă este blocat în bucla *while*. În acest caz, dacă P_j nu-i gata să intre în SC, atunci **flag[j]=false** și P_i poate intra în SC proprie.

Dacă însă P_j a setat deja **flag[j]=true** și-și execută și el bucla *while*, atunci avem **turn=i** sau **turn=j**. Dacă **turn=i**, atunci P_i va intra în SC.

Dacă însă **turn=j**, atunci P_j va intra în SC, iar după ce P_j va ieși din SC, își va reseta **flag[j]** la **false**, permițându-i lui P_i să intre în SC.

Aceasta deoarece, chiar dacă apoi P_j setează **flag[j]** la **true** pentru a intra din nou în SC, el va seta de asemenea și **turn=i**. Astfel, P_i va termina bucla *while* și va intra în SC (**progres**) după cel mult o intrare a lui P_j în SC (**așteptare limitată**).

Problema Secțiunii Critice

Prima soluție corectă și completă: Dekker '65

- Inițial propusă de Dekker într-un context diferit, a fost aplicată de Dijkstra pentru rezolvarea problemei SC.
- Dekker a adus ideea unui proces favorit și a permiterii accesului în SC a oricărui dintre procese în cazul când o cerere de acces este necontestată de celălalt
- În schimb, dacă este un conflict (i.e. ambele procese vor să intre simultan în SC-urile proprii), unul dintre procese este favorizat, iar prioritatea se inversează după execuția cu succes a SC.
- Procesele partajează variabila `turn` și tabloul `flag[]`.
- Inițializări: `flag[0] = flag[1] = false`, `turn = 0` (sau `1`).

Problema Secțiunii Critice

repeat

flag[i] := true;

while (flag[j]) **do**

if (turn=j) **then begin**

 flag[i] := false;

while (turn=j) **do nothing;**

 flag[i] := true;

end

secțiunea critică

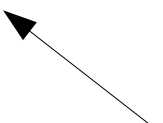
turn := j;

flag[i] := false;

secțiunea rest

forever

Așteptare ocupată



Temă:
demonstrați
corectitudinea
acestui algoritm.

Problema Secțiunii Critice

Soluții pentru procese multiple:

- Trebuie dezvoltați algoritmi diferiți de cei anteriori pentru a rezolva problema secțiunii critice pentru cazul a n procese ($n > 2$)

Problema Secțiunii Critice

Soluția 1 (Eisenberg & McGuire '72)

(Prima soluție corectă, fiind o generalizare a soluției lui Peterson)

- Structurile de date comune (partajate de cele n procese) sunt variabila **turn** și tabloul **flag[]**, cu $\text{turn} \in \{0, 1, \dots, n-1\}$,
 $\text{flag}[i] \in \{\text{idle}, \text{want-in}, \text{in-cs}\}$, $0 \leq i \leq n-1$.
- Inițializări:
 $\text{flag}[i] = \text{idle}$, $0 \leq i \leq n-1$, și
 $\text{turn} = 0$ (sau orice valoare între 0 și $n-1$).

Problema Secțiunii Critice

```
var k:0..n;
```

```
repeat
```

```
  repeat
```

```
    flag[i] := want-in;
```

```
    k := turn;
```

```
  while k ≠ i do
```

```
    if flag[k]=idle then k := (k + 1) mod n;
```

```
    else k := turn;
```

```
    flag[i] := in-cs;
```

```
    k := 0;
```

```
  while (k<n) and (k=i or flag[k]≠in-cs) do k := k + 1;
```

```
until (k≥n) and (turn=i or flag[turn]=idle);
```

```
turn := i;
```

```
secțiunea critică
```

```
k := (turn + 1) mod n;
```

```
while (flag[k]=idle) do k := (k + 1) mod n;
```

```
turn := k;
```

```
flag[i] := idle;
```

```
secțiunea rest
```

```
forever
```

Soluția 1

Temă:
demonstrați
corectitudinea
acestui algoritm.

Problema Secțiunii Critice

Soluția 2 – Algoritmul brutarului (Lamport '74)

- La intrarea în magazin, fiecare client primește un număr de ordine.
- Clientul cu cel mai mic număr este servit primul.
- Dacă P_i și P_j primesc același număr și dacă $i < j$, atunci P_i este servit primul.
- Algoritmul este deterministic
(numele proceselor sunt unice și total ordonate).

Problema Secțiunii Critice

Soluția 2 – Algoritmul brutarului (bakery alg.)

- Structurile de date comune (partajate de cele n procese) sunt tablourile `choosing[]` și `number[]`.
- Inițializări: `choosing[i] = false` , `number[i] = 0`.
- Notății:
 - $(a,b) < (c,d)$ dacă $a < c$ sau ($a = c$ și $b < d$) ;
 - $\max(a_0, \dots, a_{n-1})$ = un număr k astfel încât
 $k \geq a_i$, pentru $i=0, \dots, n-1$.

Problema Secțiunii Critice

Soluția 2 – Algoritmul brutarului (bakery alg.)

repeat

 choosing[i] := true;

 number[i] := max(number[0], ..., number[n-1]) + 1;

 choosing[i] := false;

for j := 0 **to** n-1 **do begin**

while choosing[j] **do** *nothing*;

while number[j] ≠ 0 **and**

 (number[j], j) < (number[i], i) **do** *nothing*;

end

secțiunea critică

 number[i] := 0;

secțiunea rest

forever

Problema Secțiunii Critice

Soluții hardware (1)

- Instrucțiunea atomică specializată **Test-and-Set**

Semantica ei (în pseudo-cod):

```
function Test-and-Set (var target: boolean): boolean;  
begin  
    Test-and-Set := target;  
    target := true;  
end;
```

Exemplu: majoritatea arhitecturilor de calcul multiprocesor posedă o instrucțiune de tipul

`TSL reg, adr`

e.g. pentru μ P Intel x86 avem instrucțiunea

`lock bts op1, op2`

Problema Secțiunii Critice

Soluții hardware (1)

- n procese; variabila comună **lock**, inițializată cu false.

repeat

while Test-and-Set (*lock*)

do *nothing*;

secțiunea critică

lock := false;

secțiunea rest

forever

Problema Secțiunii Critice

Soluții hardware (2)

- Instrucțiunea atomică specializată **Swap**

Semantica ei (în pseudo-cod):

```
procedure Swap (var a, b: boolean);  
var temp: boolean;  
begin  
    temp := a;  
    a := b;  
    b := temp;  
end;
```

Exemplu: pentru μ P Intel x86 avem instrucțiunea

xchg *op1, op2*

unde operanzii sunt doi regiștri, sau un registru și o adresă de memorie.

Problema Secțiunii Critice

Soluții hardware (2)

- n procese; variabila comună **lock**, inițializată cu false.

```
var key:boolean;    // variabila locală key
```

```
repeat
```

```
    key := true;
```

```
    repeat
```

```
        Swap(lock, key);
```

```
    until key = false;
```

```
    secțiunea critică
```

```
    lock := false;
```

```
    secțiunea rest
```

```
forever
```

Problema Secțiunii Critice

Soluții hardware (3)

- *Notă*: soluțiile anterioare satisfac condițiile de excludere mutuală și de progres, dar nu îndeplinesc și cerința de așteptare limitată.
- O soluție **completă** folosind **Test-and-Set** (sau **Swap**) :
 - n procese;
 - variabila comună **lock**, inițializată cu false, și vectorul comun **waiting**[0.. $n-1$] , inițializat cu false
 - limita de așteptare: $n-1$

Problema Secțiunii Critice

Soluții hardware (3)

```
var j:0..n-1; key:boolean;
```

```
repeat
```

```
  waiting[i]:=true;
```

```
  key:=true;
```

```
  while waiting[i] and key do key:=Test-and-Set(lock);
```

```
  waiting[i]:=false;
```

```
  secțiunea critică
```

```
  j:=i+1 mod n;
```

```
  while j≠i and not waiting[j] do j:=j+1 mod n;
```

```
  if j=i then lock:=false;
```

```
    else waiting[j]:=false;
```

```
  secțiunea rest
```

```
forever
```

Sau : ... **do** Swap(*key*,*lock*) ;

Temă:
demonstrați
corectitudinea
acestui algoritm.

Problema Secțiunii Critice

Soluții concrete: Semafoarele

- Concept introdus de E.W. Dijkstra
- Un **semafor** S este o variabilă întreagă care este accesată (exceptând operația de inițializare) numai prin intermediul a două operații standard, atomice:
 - operația P sau **wait()** (*proberen* = a testa), și
 - operația V sau **signal()** (*verhogen* = a incrementa).
- Semantica operației **wait(S)** :
$$\text{while } S \leq 0 \text{ do nothing; } S := S - 1;$$
- Semantica operației **signal(S)** : $S := S + 1;$

Problema Secțiunii Critice

Excluderea mutuală implementată cu semafoare

- Problema SC cu n procese; variabila comună **mutex** este un semafor binar (i.e, semafor inițializat cu 1).

repeat

`wait(mutex) ;`

secțiunea critică

`signal(mutex) ;`

secțiunea rest

forever

Problema Secțiunii Critice

Semafoare – implementare la nivelul SO:

- Semafoarele pot suspenda și reporni procese/thread-uri, pentru a evita *așteptarea ocupată* (i.e. risipa de cicli CPU)
- Semaforul se definește ca un articol:

```
typedef struct {  
    int value;  
    struct thread *ListHead;  
} semaphore;
```

- Se consideră următoarele 2 operații:

`suspend()` : suspendă execuția thread-ului care-l apelează

`resume(T)` : reia execuția unui thread T blocat anterior
(printr-un apel `suspend()`)

Problema Secțiunii Critice

Semafoare – implementare la nivelul SO:

– Cele 2 operații atomice cu semafoare se definesc atunci astfel:

1) operația ***wait(S)*** :

```
S.value--;  
if (S.value < 0) {  
    add this thread to S.ListHead;  
    suspend();  
}
```

2) operația ***signal(S)*** :

```
S.value++;  
if (S.value <= 0) {  
    remove a thread T from S.ListHead;  
    resume(T);  
}
```


Problema Secțiunii Critice

Semafoare – implementare la nivelul SO:

- mai multe detalii de implementare în S.O.-urile moderne: a se citi, de exemplu, §6.5.2, pag. 227-230, din Silberschatz: “*Operating System Concepts*”, ediția 8 [OSCE8]
- La nivelul aplicațiilor, semafoarele pot fi simulate prin diverse entități logice (e.g. fișiere, canale fifo, semnale, ș.a.)
- Biblioteca IPC (introdusă în UNIX System V) permite lucrul cu semafoare în aplicații (inclusiv în Linux)

Problema Secțiunii Critice

Semafoare – utilizare

- Pot fi folosite pentru a rezolva diverse probleme de sincronizare între procese
- Ex: execută B în P_2 numai după ce s-a executat A în P_1
- Soluție: folosim un semafor *flag* inițializat cu 0

P_1 :

.....

A

signal(*flag*);

.....

P_2 :

.....

wait(*flag*);

B

.....

Problema Secțiunii Critice

Semafoare – utilizare (cont.)

– Putem clasifica semafoarele în 2 tipuri:

- **Semafoarele binare** (i.e., semafoare inițializate cu valoarea 1) **pot asigura excluderea mutuală** (e.g., pot soluționa problema secțiunii critice)
- **Semafoarele generale** (i.e., semafoare inițializate cu valoarea $n > 1$) **pot reprezenta o resursă cu instanțe multiple** (i.e., cu n instanțe) (e.g., pot soluționa problema producător-consumator)

Interblocajul și înfometarea

- **Interblocajul** (*deadlock*)

- O situație în care două sau mai multe procese așteaptă pe termen nelimitat producerea câte unui eveniment (e.g., execuția unei operații signal pe un semafor), eveniment ce ar putea fi cauzat doar de către unul dintre celelalte procese ce așteaptă.
- Aceste procese se spune că sunt **interblocate**.

- **Blocajul nelimitat** sau **înfometarea** (*starvation*)

- O situație în care un(ele) proces(e) așteaptă nelimitat (e.g., la un semafor: procesul ar putea sta suspendat în coada de așteptare asociată acelui semafor pe termen nelimitat).

Interblocajul si infometarea

- **Interblocajul** (*deadlock*)

– Exemplu: 2 procese folosesc 2 semafoare binare S și Q (i.e. inițializate cu 1) în ordinea de mai jos:

P_1 :	P_2 :
<code>wait(S);</code>	<code>wait(Q);</code>
<code>wait(Q);</code>	<code>wait(S);</code>
.....	
<code>signal(S);</code>	<code>signal(Q);</code>
<code>signal(Q);</code>	<code>signal(S);</code>

Se observă că este posibil să apară interblocaj. În ce situație apare? (specificați un scenariu de execuție care duce la interblocaj)

Soluție: ambele programe ar trebui să execute operațiile wait() pe cele două semafoare în aceeași ordine !

Probleme clasice de sincronizare

- Problema Producător-Consumator
(Producer-Consumer or Sender-Receiver problem)
- Problema Cititori și Scriitori
(Readers and Writers problem)
- Problema Cina Filozofilor
(Dining-Philosophers problem)
- Problema Bărbierului Adormit
(Sleeping Barber problem)

(va urma)

- **Bibliografie obligatorie**

capitolele despre *sincronizarea proceselor* din

- Silberschatz : “*Operating System Concepts*”

(cap.6 din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(a treia parte a cap.2 din [MOS3])

- Introducere
 - Problema secțiunii critice
 - Enunțul problemei și cerințele de rezolvare
 - Soluții pentru cazul a 2 procese
 - Soluții pentru cazul a $n > 2$ procese
 - Soluții hardware
 - Semafoare
 - Interblocajul și înfometarea
- (va urma)
- Probleme clasice de sincronizare
 - Monitoare

Întrebări ?

Sisteme de Operare

Sincronizarea proceselor – partea II

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

Am discutat despre:

- Introducere
- Problema secțiunii critice
- Interblocajul și înfometarea

Continuăm cu:

- Probleme clasice de sincronizare
 - Problema Producător-Consumator
 - Problema Cititori și Scriitori
 - Problema Cina Filozofilor
 - Problema Bărbierului Adormit
- Monitoare

Probleme clasice de sincronizare

- Problema Producător-Consumator
(Producer-Consumer or Sender-Receiver problem)
- Problema Cititori și Scriitori
(Readers and Writers problem)
- Problema Cina Filozofilor
(Dining-Philosophers problem)
- Problema Bărbierului Adormit
(Sleeping Barber problem)

Probleme clasice de sincronizare (1)

- Problema Producător-Consumator
 - Enunțată de Dijkstra în '65
 - Este o problemă reprezentativă pentru ilustrarea conceptului de *procese cooperante*:

Un proces, cu rol de *producător*, produce informații ce sunt consumate de un alt proces, cu rol de *consumator*.

Producător-Consumator

- Enunțul problemei:
 - Un proces produce niște date (e.g. informații, mesaje, ...) și le depune într-o zonă tampon (*buffer*), de unde le ia un al doilea proces și le consumă.
 - Accesul la zona tampon se face în mod exclusiv.
 - Zona tampon are capacitate nelimitată.
 - Consumatorul trebuie să aștepte când bufferul este gol.
 - Altă variantă a acestei probleme este cu zona tampon de capacitate finită. În acest caz și producătorul trebuie să aștepte și, anume, atunci când zona tampon este plină.
 - *Notă*: problema aceasta se poate formula și cu mai mulți producători și consumatori (ce utilizează un singur buffer).

Producător-Consumator

- Soluția problemei (varianta cu buffer nelimitat):
 - un semafor binar **mutex** – care va controla secțiunea critică (i.e., accesul exclusiv la zona tampon)
 - un semafor (binar) **delay** – care va bloca consumatorul dacă zona tampon este goală
 - o variabilă întreagă **n** – care va număra elementele din zona tampon
 - Inițializări: **$n = 0$, $mutex = 1$, $delay = 0$**

Producător-Consumator

Procesul Producător:

repeat

producere_element() ;

wait(mutex) ;

adaugare_element_in_buffer() ;

n:=n+1;

if *n=1* **then** *signal(delay)* ;

signal(mutex) ;

forever

Producător-Consumator

Procesul Consumator:

```
wait(delay);  
repeat  
    wait(mutex);  
    extragere_element_din_buffer();  
    n:=n-1; nlocal:=n;  
    signal(mutex);  
    consumare_element();  
    if nlocal=0 then wait(delay);  
forever
```


Producător-Consumator

Producător:

repeat

```
    producere_element() ;  
    wait(mutex) ;  
    adaugare_element_in_buffer() ;  
    n:=n+1 ;  
    if n=1 then signal(delay) ;  
    signal(mutex) ;
```

forever

Consumator:

```
wait(delay) ;
```

repeat

```
    wait(mutex) ;  
    extragere_element_din_buffer() ;  
    n:=n-1 ; nlocal:=n ;  
    signal(mutex) ;  
    consumare_element() ;  
    if nlocal=0 then wait(delay) ;
```

forever

- Instrucțiunea if din producător poate fi scoasă în afara SC (i.e. asemănător ca în consumator, folosind o variabilă nlocal), în schimb if-ul din consumator nu poate fi pus în SC (i.e. asemănător ca în producător) deoarece ar putea apare interblocaj.

Producător-Consumator

- Soluția problemei (varianta cu buffer limitat):
 - Se poate adapta soluția de la versiunea cu buffer nelimitat, adăugând un semafor binar **delayPro** – care va bloca producătorul dacă zona tampon este plină
- (Observație: if-ul din producător trebuie scos în afara SC, asemănător ca în consumator, folosind o variabilă nlocal, pentru ca altfel poate să apară interblocaj)

Temă: încercați să scrieți soluția pe baza acestei idei.

Producător-Consumator

- Altă soluție pentru varianta cu buffer limitat:
 - un semafor binar **mutex** – care va controla secțiunea critică (i.e., accesul exclusiv la zona tampon)
 - un semafor general **empty** – care va număra locațiile goale din zona tampon
 - un semafor general **full** – care va număra locațiile pline din zona tampon
 - Inițializări: **mutex** = 1, **full** = 0, **empty** = n

Producător-Consumator

Procesul Producător:

repeat

producere_element();

wait(empty);

wait(mutex);

adaugare_element_in_buffer();

signal(mutex);

signal(full);

forever

Producător-Consumator

Procesul Consumator:

repeat

wait(full);

wait(mutex);

extragere_element_din_buffer();

signal(mutex);

signal(empty);

consumare_element();

forever

Producător-Consumator

Producător:

repeat

```
    producere_element();  
    wait(empty);  
    wait(mutex);  
    adaugare_element_in_buffer();  
    signal(mutex);  
    signal(full);
```

forever

Consumator:

repeat

```
    wait(full);  
    wait(mutex);  
    extragere_element_din_buffer();  
    signal(mutex);  
    signal(empty);  
    consumare_element();
```

forever

- Ordinea celor două apeluri wait(), atât în producător, cât și în consumator, este esențială; prin schimbarea ordinii ar putea apare interblocaj.

Probleme clasice de sincronizare (2)

- Problema Cititori și Scriitori (CREW)
 - Enunțată de Courtois, Heymans și Parnas în '71
 - Este o problemă reprezentativă pentru accesul la o bază de date
(e.g. un sistem de rezervare a biletelor de avion, cu multe procese competitive dorind să citească și să scrie în baza de date)
 - CREW: este acceptabil să avem mai multe procese care să citească baza de date în același timp, dar, dacă un proces actualizează (i.e. scrie) baza de date, nici un alt proces nu trebuie să aibă acces la ea, nici măcar în citire.

Cititori și Scriitori

- Enunțul problemei:
 - Un obiect (i.e. o resursă, e.g. un fișier, o zonă de memorie, etc.) trebuie să fie partajat de mai multe procese concurente
 - Unele dintre aceste procese ar putea dori doar să citească conținutul obiectului partajat : **Cititorii**
 - Alte procese ar putea dori însă să actualizeze (citire și scriere) conținutul obiectului partajat : **Scriitorii**
 - Se cere ca scriitorii să aibă acces exclusiv la obiectul partajat, în schimb cititorii să îl poată accesa în mod concurent (non-exclusiv)

Cititori și Scriitori

- Versiunea #1: nici un cititor nu va fi ținut în așteptare decât dacă un scriitor a obținut deja permisiunea de acces la obiectul partajat
 - Cu alte cuvinte, nici un cititor nu trebuie să aștepte alți cititori să termine de citit, doar din cauză că un scriitor așteaptă deja permisiunea de acces
 - La acces simultan, cititorii sunt mai prioritari decât scriitorii
 - *Notă*: unele procese (scriitorii în acest caz) pot deveni înfometate

Cititori și Scriitori

- Versiunea #2: o dată ce un scriitor este gata de scriere, el va executa acea scriere cât mai curând posibil
 - Cu alte cuvinte, dacă un scriitor așteaptă deja permisiunea de acces, nici un nou cititor (i.e. care cere permisiunea de acces după scriitor) nu trebuie să primească permisiunea de acces
 - La acces simultan, scriitorii sunt mai prioritari decât cititorii
 - *Notă*: unele procese (cititorii în acest caz) pot deveni înfometate. De asemenea, această soluție permite un grad de concurență mai scăzut și, deci, o performanță mai slabă decât prima soluție.

Cititori și Scriitori

– Soluția pentru versiunea #1:

- Procesele cititori partajează două semafoare binare **mutex** și **wrt**, precum și o variabilă întreagă **readcount** ; semaforul **wrt** este partajat și de către procesele scriitori.
- Inițializări: **mutex = wrt = 1** , **readcount = 0** .
- **readcount** ține evidența numărului de procese cititori ce sunt în cursul citirii obiectului partajat
- **mutex** este folosit pentru a asigura excluderea mutuală când este actualizată variabila **readcount**
- **wrt** este utilizat pentru a asigura excluderea mutuală pentru scriitori la accesul obiectului partajat

Cititori și Scriitori

- Structura procesului scriitor:

repeat

...

`wait(wrt);`

`scriere_obiect();`

`signal(wrt);`

...

forever

- Dacă un scriitor este în SC și n cititori așteaptă, atunci un cititor este în așteptare la **wrt**, iar ceilalți $n-1$ cititori așteaptă la **mutex**.

Cititori și Scriitori

– Structura procesului cititor:

repeat

...

wait(mutex);

readcount:=readcount+1;

if readcount=1 **then** wait(wrt);

signal(mutex);

citire_obiect();

wait(mutex);

readcount:=readcount-1;

if readcount=0 **then** signal(wrt);

signal(mutex);

...

forever

Cititori și Scriitori

- Soluția versiunii #1

Proces scriitor:

repeat

...

wait(wrt);

scriere_obiect();

signal(wrt);

...

forever

Proces cititor:

repeat

...

wait(mutex);

readcount:=readcount+1;

if readcount=1 **then** wait(wrt);

signal(mutex);

citire_obiect();

wait(mutex);

readcount:=readcount-1;

if readcount=0 **then** signal(wrt);

signal(mutex);

...

forever

Temă: proiectați o soluție pentru vers. #2 (Atenție: nu este simetrică!)

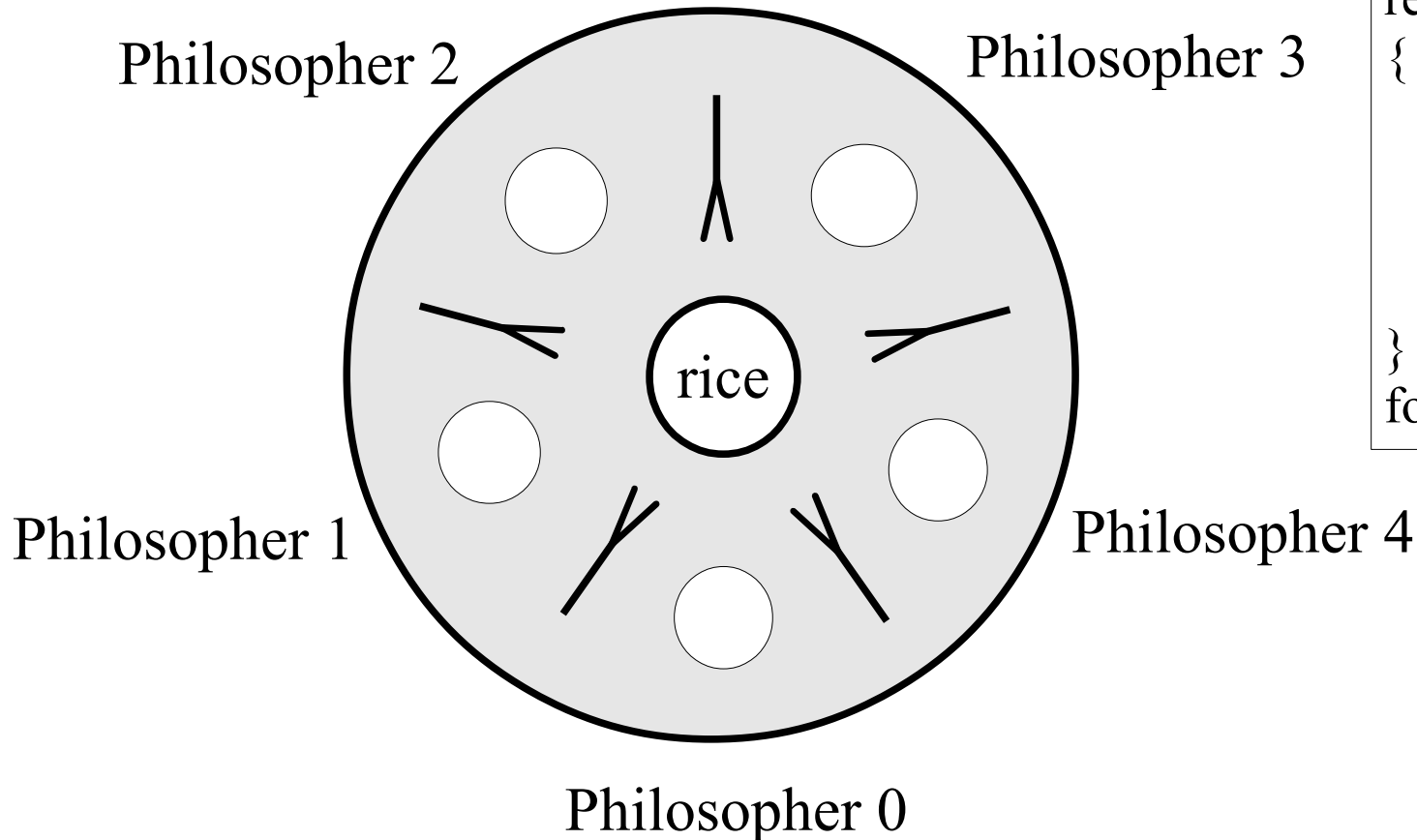
Probleme clasice de sincronizare (3)

- Problema Cina Filozofilor Chinezi
 - Enunțată de Dijkstra în '65
 - Este o problemă reprezentativă pentru nevoia de a aloca un număr limitat de resurse (nepartajabile) la mai multe procese, ce concurează pentru acces exclusiv la aceste resurse, alocare care să se facă fără să apară fenomenul de interblocaj sau cel de înfometare.

Cina Filozofilor

- Enunțul problemei:
 - 5 filozofi chinezi își petrec viețile gândind și mâncând
 - Ei partajează o masă circulară comună, înconjurată de 5 scaune, fiecare aparținând unuia dintre ei
 - În centrul mesei este un platou cu (foarte mult!) orez și mai există pe masă 5 farfurii și (doar!) 5 bețișoare pentru mâncat
 - Când un filozof gândește, el nu interacționează cu colegii lui
 - Din când în când, unui filozof i se face foame și vrea să mănânce, încercând să ridice cele 2 bețișoare din dreptul lui
 - Fiecare filozof poate ridica un singur bețișor o dată
 - După ce termină de mâncat, pune bețișoarele înapoi pe masă și începe să filozofeze din nou

Cina Filozofilor



Philosopher life:

```
repeat
{
  pick up the forks;
  eat awhile;
  put down the forks;
  think awhile;
}
forever
```

Cina Filozofilor

- Soluția problemei:
 - Fiecare bețișor (i.e. resursă nepartajabilă) este reprezentat printr-un semafor binar
 - Un filozof încearcă să ridice bețișorul executând operația `wait()` pe acel semafor, respectiv lasă bețișorul jos pe masă executând operația `signal()`
 - Datele partajate de procesele filozofi sunt:
`chopstick[0..4]` of semaphore;
 - Inițializări: `chopstick[i] = 1`, $i=0, \dots, 4$

Cina Filozofilor

- Structura procesului filozof al i -lea ($i=0,\dots,4$):

repeat

wait(chopstick[i]);

wait(chopstick[(i+1) mod 5]);

mananca() ;

signal(chopstick[i]);

signal(chopstick[(i+1) mod 5]);

gandeste() ;

forever

Cina Filozofilor

- Comentarii
 - Soluția este **incompletă !** Motivul:
 - Este posibil să se creeze un interblocaj
 - Unii filozofi pot deveni înfometați
- Temă:
 - Proiectați o altă soluție pentru problema cinei filozofilor, care să nu permită apariția interblocajului sau a înfometării

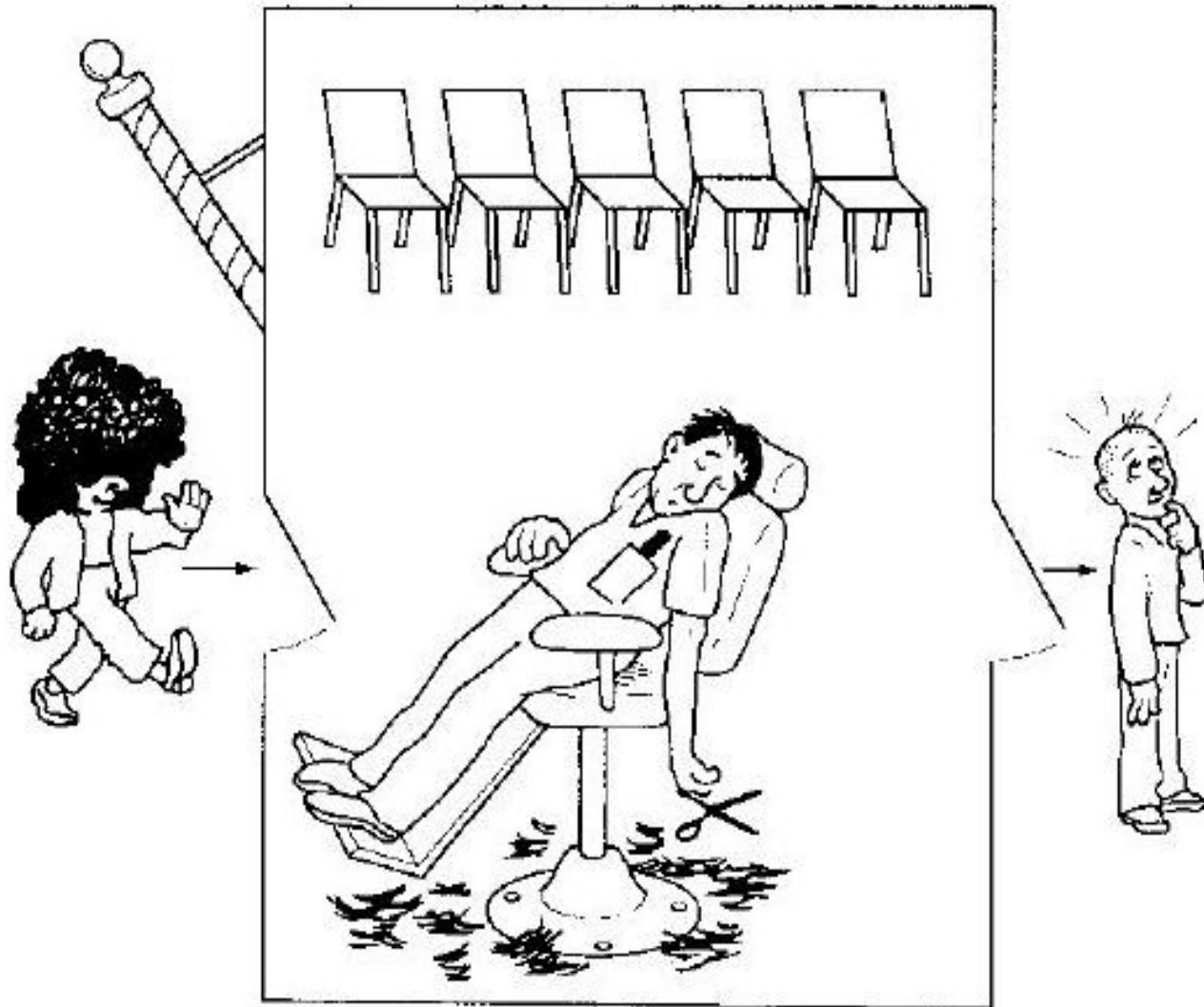
Probleme clasice de sincronizare (4)

- Problema Bărbierului Adormit
 - Enunțată de Dijkstra în '65
 - Este o problemă reprezentativă pentru situații diverse de așteptare la coadă
(e.g. un secretariat cu un sistem computerizat de preluare a apelurilor telefonice și punere în așteptare, cu o capacitate limitată de păstrare a apelurilor în așteptare)

Bărbierul Adormit

- Enunțul problemei:
 - O frizerie cu un bărbier, un scaun de lucru și n scaune pentru clienții ce așteaptă să le vină rândul la tuns
 - Dacă nu are clienți, bărbierul stă în scaunul de lucru și doarme (i.e., se odihnește)
 - Când vine un prim client, el trebuie să-l trezească pe bărbier
 - Dacă mai vin și alți clienți cât timp bărbierul tunde un client, aceștia fie iau loc pe scaunele de așteptare (dacă mai sunt locuri libere), fie pleacă netunși (în caz contrar)
 - Problema constă în a programa bărbierul și clienții de așa manieră încât să nu apară blocaje datorită fenomenelor de tip “*race conditions*”
 - *Notă*: problema are și o variantă cu mai mulți bărbieri

Bărbierul Adormit



Bărbierul Adormit

- Soluția problemei:
 - Procesele bărbier și clienți partajează trei semafoare: **mutex**, **customers** și **barber**, și o variabilă întreagă: **freeseatscount**
 - **mutex** este folosit pentru a asigura excluderea mutuală când este accesată variabila **freeseatscount**
 - **freeseatscount** ține evidența numărului de scaune libere (i.e. neocupate de clienți ce stau în așteptare)
 - semaforul general **customers** ține evidența numărului de clienți în așteptare pe scaune (exclusiv cel ce este tuns)
 - semaforul binar **barber** este utilizat pentru a indica dacă bărbierul este ocupat (1) sau liber (0)
 - Inițializări: **customers = barber = 0** , **freeseatscount = n** , **mutex = 1**

Bărbierul Adormit

– Structura procesului bărbier:

repeat

```
wait(customers);  
wait(mutex);  
freeseatscount++;  
signal(barber);  
signal(mutex);  
tunde_client();
```

forever

Bărbierul Adormit

– Structura proceselor clienți:

```
wait(mutex);  
if(freeseatscount > 0)  
{  
    freeseatscount--;  
    signal(customers);  
    signal(mutex);  
    wait(barber);  
    este_tuns_de_barbier();  
}  
else  
    signal(mutex);
```

Bărbierul Adormit

Procesul bărbier:

```
repeat
    wait(customers);
    wait(mutex);
    freeseatscount++;
    signal(barber);
    signal(mutex);
    tunde_client();
forever
```

Procesele clienți:

```
wait(mutex);
if(freeseatscount > 0)
{
    freeseatscount--;
    signal(customers);
    signal(mutex);
    wait(barber);
    este_tuns_de_barbier();
}
else
    signal(mutex);
```

Monitoare

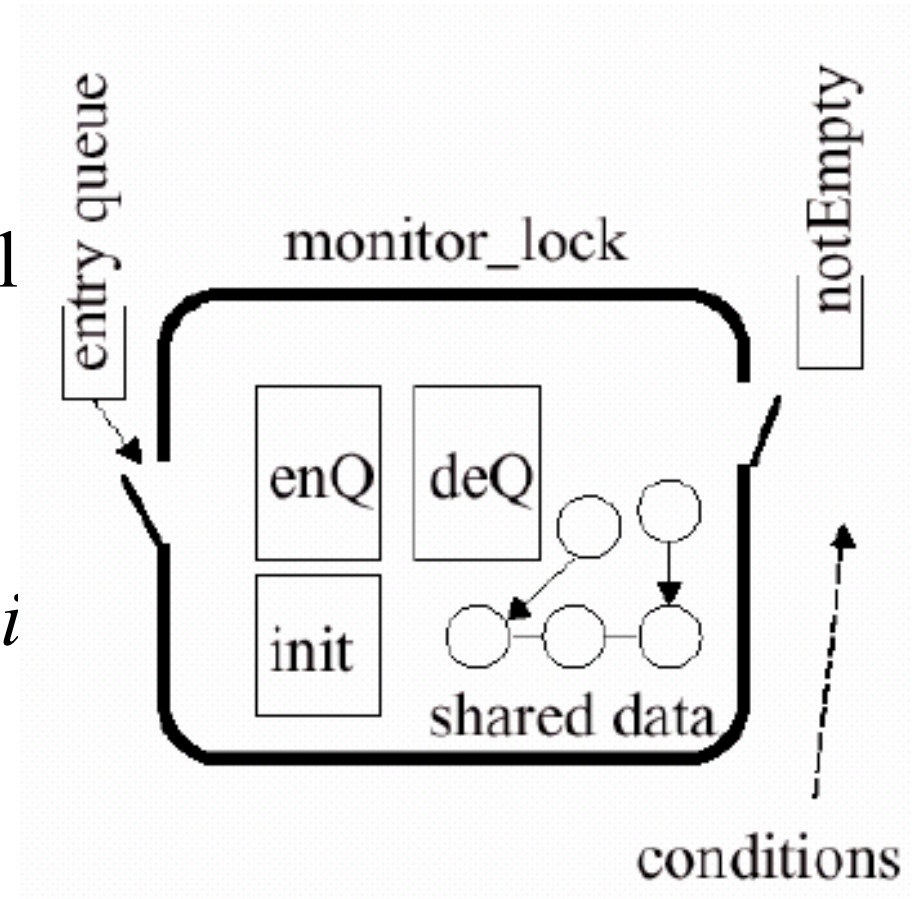
- Construcții de sincronizare în limbaje de nivel înalt

La folosirea semafoarelor pot apare erori de sincronizare datorită unei ordini incorecte a apelurilor wait și signal; este suficient ca un singur proces să nu coopereze corect (fie datorită unei erori de programare, fie în mod intenționat), pentru a “strica” sincronizarea tuturor proceselor cooperante.

De aceea, s-au introdus o serie de construcții de sincronizare în unele limbaje de programare de nivel înalt, care să “ascundă” programatorului detaliile legate de apelurile wait și signal (tratarea corectă a acestora cade în sarcina compilatorului).

Monitoare

- Monitor = este un tip de dată abstractă care:
 - Încapsulează date partajate împreună cu operații asupra lor ce se efectuează cu accesul mutual exclusiv la obiect (practic, un monitor = o “clasă” cu un *zăvor* asociat)
 - Se pot utiliza *variabile condiții* ce au asociate doar operațiile `wait()` și `signal()`



Monitoare

- Monitor
 - Concept dezvoltat de B.Hansen '73 & C.A.R.Hoare '74
 - Este o construcție de sincronizare de nivel înalt (i.e. implementată în unele limbaje de programare, cum ar fi limbajul *Concurrent Pascal*), introdusă pentru a ușura sarcina programatorilor: se elimină erorile de programare ce pot apărea la folosirea semafoarelor datorită unei ordini incorecte a apelurilor wait și signal
- Altă construcție de acest gen – Regiunile critice condiționale
 - `var var-shared : shared type ;`
 - `region var-shared when condiție do cod ;`
- În limbajul Java: metode `synchronized` într-o clasă

- **Bibliografie obligatorie**

capitolele despre *sincronizarea proceselor* din

- Silberschatz : “*Operating System Concepts*”

(cap.6 din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(a treia parte a cap.2 din [MOS3])

- Probleme clasice de sincronizare
 - Problema Producător-Consumator
 - Problema Cititori și Scriitori
 - Problema Cina Filozofilor
 - Problema Bărbierului Adormit
- Monitoare

Sisteme de Operare

Comunicații inter-procese și interblocajul proceselor

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

- **Comunicații inter-procese (IPC)**
 - Problema comunicației
 - Tipuri de comunicație
 - Comunicația directă
 - Comunicația indirectă
 - Excepții
 - IPC sub UNIX
- **Interblocajul proceselor**
 - Definiție
 - Context
 - Model
 - Cerințele interblocajului
 - Graful de alocare a resurselor
 - Strategii: prevenire, evitare, detecție și restabilire
- **Înfometarea proceselor**

Comunicații inter-procese

- Comunicații inter-procese
(IPC = Inter-Process Communication)
 - Problema comunicației
 - Proiectarea unor mecanisme care să permită **comunicația** între procesele ce doresc să coopereze; există 2 modele:
 - **Sisteme cu memorie partajată (modelul *shared memory*)** – necesită ca procesele comunicante să partajeze niște variabile (zone) de memorie
 - **Sisteme cu mesagerie (modelul *message-passing*)** – permit proceselor să schimbe mesaje între ele (chiar fără a avea memorie partajată)
 - Procesele ce comunică pot fi executate local sau la distanță

Comunicații inter-procese

- Primitivele sistemelor cu mesaje:
 - send (*mesaj*)
 - receive (*mesaj*)
- Mesajele pot fi de lungime fixă sau de lungime variabilă
- Între procesele comunicante trebuie să existe o **legătură de comunicație**

Comunicații inter-procese

- Întrebări legate de implementare:
 - Cum se stabilesc legăturile de comunicație între procese?
 - O legătură poate să fie asociată cu mai mult de două procese?
 - Câte legături pot exista între o pereche de procese?
 - Care este capacitatea unei legături?
 - Care este dimensiunea mesajelor?
 - O legătură este unidirecțională sau bidirecțională?

Comunicații inter-procese

- Modalități folosite pentru implementarea logică a legăturii de comunicație și a operațiilor `send()` și `receive()`:
 - comunicație directă sau indirectă
 - comunicație simetrică sau asimetrică
 - *buffering* (=stocarea mesajului într-o zonă tampon pentru preluarea ulterioară de către destinatar) implicit sau explicit
 - trimiterea mesajului prin copie sau prin referință
 - mesaje de lungime fixă sau de lungime variabilă

Comunicații inter-procese

- Comunicație directă:
 - Primitivele `send()` și `receive()` specifică explicit numele procesului destinatar, respectiv expeditor (*comunicare simetrică*)
 - Legătura este stabilită automat între fiecare pereche de procese ce doresc să comunice; ele trebuie să cunoască doar identitatea celuilalt
 - O legătură este asociată cu exact două procese
 - Între fiecare pereche de procese comunicante există exact o legătură de comunicație
 - Legătura poate fi și unidirecțională, însă de obicei este bidirecțională

Comunicații inter-procese

- Problema Producător-Consumator (reluare)
 - O soluție bazată pe comunicație directă
 - Procesul producător:

repeat

```
produ_element_in(nextpro) ;  
send(consumer, nextpro) ;
```

forever

Comunicații inter-procese

- Problema Producător-Consumator (reluare)

- Procesul consumator:

- repeat**

- `receive(producer, nextcon) ;`

- `consuma_element_din(nextcon) ;`

- forever**

- Varianta: *comunicație asimetrică* – destinatarul poate primi de la oricine: `receive(id, mesaj)`
 - O altă soluție posibilă: bazată pe comunicație indirectă

Comunicații inter-procese

- Comunicația indirectă
 - Mesajele sunt trimise/primate prin intermediul unor cutii poștale (numite și **porturi**)
 - Două procese pot comunica numai dacă au o cutie poștală în comun (i.e. partajată)
 - Primitivele de comunicație au forma:
 - **send(PORT,mesaj)**
 - **receive(PORT,mesaj)**

Comunicații inter-procese

- Comunicația indirectă
 - Permite folosirea unei legături de comunicație de către mai multe procese
 - Permite la un moment dat cel mult unui proces să execute o operație `receive()` pe un anumit port
 - Permite sistemului să selecteze arbitrar care proces va primi mesajul (în caz de mai multe operații `receive()` efectuate “simultan” de procese distincte, acestea nu au nici un control asupra ordinii de satisfacere a cererilor lor)

Comunicații inter-procese

- Comunicația indirectă
 - Operații asupra cutiilor poștale:
 - crearea unei cutii poștale
 - trimiterea/primirea de mesaje prin intermediul unei cutii poștale
 - distrugerea unei cutii poștale
 - Temă: reformulați soluția problemei producător-consumator folosind comunicația indirectă

Comunicații inter-procese

- Excepții
 - Terminarea unui proces înainte de primirea mesajului
 - Pierderea mesajelor
 - “Amestecarea” (*scrambling*-ul) mesajelor
- Soluții
 - S.O.-ul trebuie să detecteze excepțiile, folosind diverse mecanisme:
 - notificări de terminare trimise celuilalt proces
 - *timeout*-uri și protocoale de confirmare
 - sume de control, de paritate, CRC, etc.

Comunicații inter-procese

- IPC sub UNIX

- *pipe*-uri (canale anonime)

- `pipe()`

- pot fi utilizate doar de către procese înrudite prin `fork()`

- *named pipe*-uri (canale cu nume), i.e. fișiere `fifo`

- `mkfifo()`

- pot implementa schimbul de mesaje între procese oarecare, dar locale (i.e. executate pe același calculator)

- *socket*-uri (Notă : se vor studia la rețele de calculatoare, în anul 2)

- pot implementa schimbul de mesaje între procese oarecare, la distanță (i.e. executate pe calculatoare diferite)

- *semnale* UNIX

Comunicații inter-procese

- IPC sub UNIX (cont.)
 - semafoare (UNIX System V)
 - implementate cu mesaje
 - `semget()`, `semop()`, `semctl()` - UNIX System V
 - `creatsem()`, `opensem()`, `waitsem()`, `sigsem()` - Xenix
 - zone de memorie partajată (UNIX System V)
 - `shmget()`, `shmat()`, `shmdt()`, `shmctl()`
 - cozi de mesaje (UNIX System V)
 - `msgget()`, `msgsnd()`, `msgrcv()`, `msgctl()`

- Interblocajul (*deadlock*-ul) proceselor
 - Definiție
 - Context
 - Model
 - Cerințele interblocajului
 - Graful de alocare a resurselor
 - Strategii de rezolvare: prevenire, evitare, detecție și restabilire

➤ Definiție

- **Interblocaj:** două sau mai multe procese așteaptă la infinit producerea unor evenimente ce pot fi cauzate doar de unul sau mai multe dintre procesele ce așteaptă, prin urmare nici un eveniment nu se va produce niciodată.
- Există o “așteptare circulară” (necesară dar nu și suficientă pentru a avea interblocaj, după cum vom vedea puțin mai încolo).

➤ Context

- Problema interblocajului proceselor va fi studiată în contextul unui sistem de calcul abstract, în care procesele sunt în competiție pentru resurse (CPU, memorie, periferice I/O, ...)

➤ Modelul matematic:

- Tipurile de resurse: $R_1, R_2, R_3, \dots$
e.g. R_1 =procesor, R_2 =imprimantă
- Fiecare resursă R_i are W_i instanțe (i.e. numărul de “unități” ale acelei resurse)
e.g. R_2 are $W_2=2$ instanțe (adică sistemul respectiv are 2 imprimante)
- Procesele utilizează resursele astfel:
 - cerere de alocare a resursei necesare
 - utilizarea resursei pentru o perioadă finită de timp
 - eliberarea resursei

Interblocaj – Cerințe

- Sunt *necesare* 4 condiții pentru a fi posibilă apariția interblocajului (Coffman, Elphic, Shoshani – 1971):
- 1. excluderea mutuală:** procesele solicită controlul în mod exclusiv asupra resurselor pe care le cer S.O.-ului
 - 2. hold & wait** (păstrare & așteptare): procesele păstrează resursele deja deținute în timp ce așteaptă să obțină alte resurse
 - 3. no preemption** (ne-preemptie): resursele deținute de un proces nu-i pot fi luate de către S.O. fără voia sa
 - 4. așteptare circulară:** se poate stabili o ordonare $P_1, P_2, \dots, P_n$ a proceselor astfel încât P_1 așteaptă o resursă deținută de P_2 , P_2 așteaptă o resursă deținută de P_3 , ... ș.a.m.d. ... , P_n așteaptă o resursă deținută de P_1

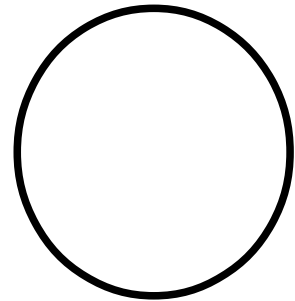
Interblocaj – RAG /1

- Pentru modelarea sistemelor se folosește **graful de alocare a resurselor** (RAG = resource allocation graph), care ne arată:
 - Ce procese cer resurse și ce resurse cer ele
 - Ce resurse au fost acordate și căror procese au fost ele acordate
 - Câte unități din fiecare tip de resursă sunt disponibile

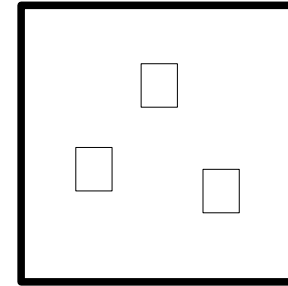
Interblocaj – RAG /2

- Un RAG G este un graf bipartit orientat (V, E) , cu V mulțimea vârfurilor și E mulțimea arcelor
- Vârfurile din V sunt elemente de două tipuri:
 - $P = \{P_1, P_2, \dots, P_n\}$: toate procesele din sistem
 - $R = \{R_1, R_2, \dots, R_m\}$: toate resursele din sistem
- Arcele din E sunt de două tipuri:
 - arce de cerere: arc orientat $P_i \rightarrow R_j$
 - arce de alocare: arc orientat $R_j \rightarrow P_i$

Interblocaj – RAG /3



Proces

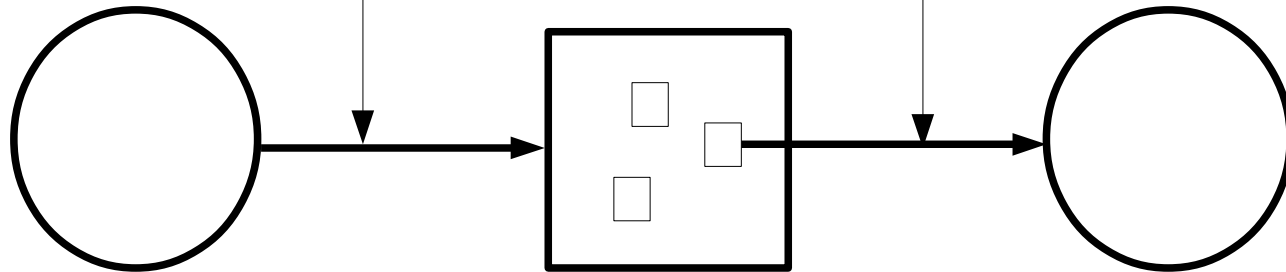


Resursă
(3 unități)

Cerere
și
Alocare

Arc de cerere

Arc de alocare

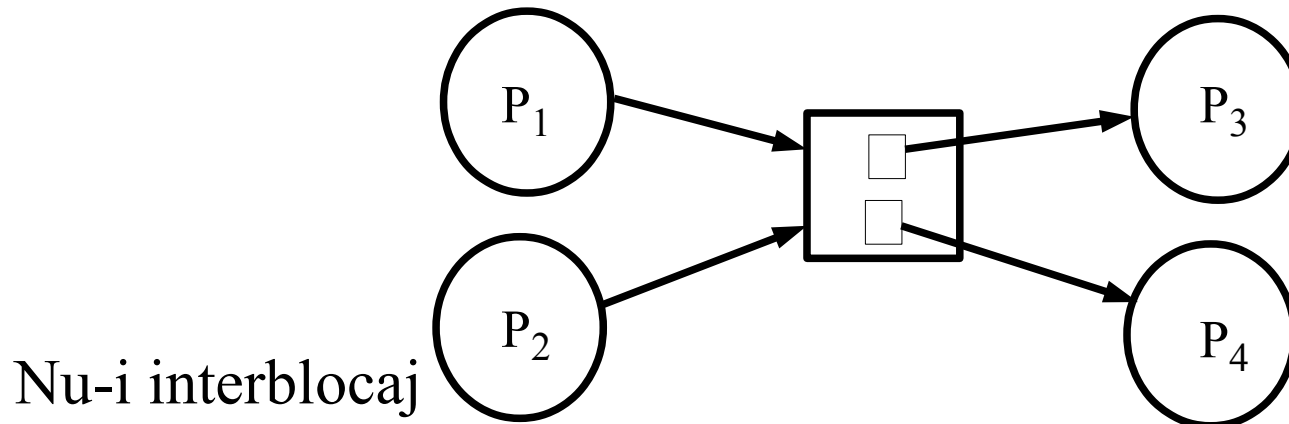
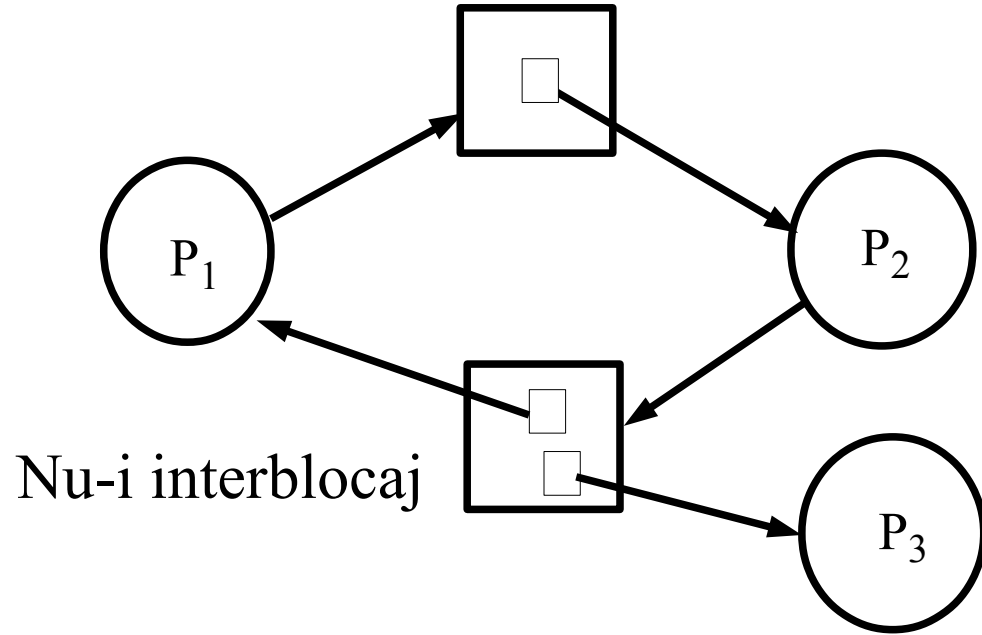
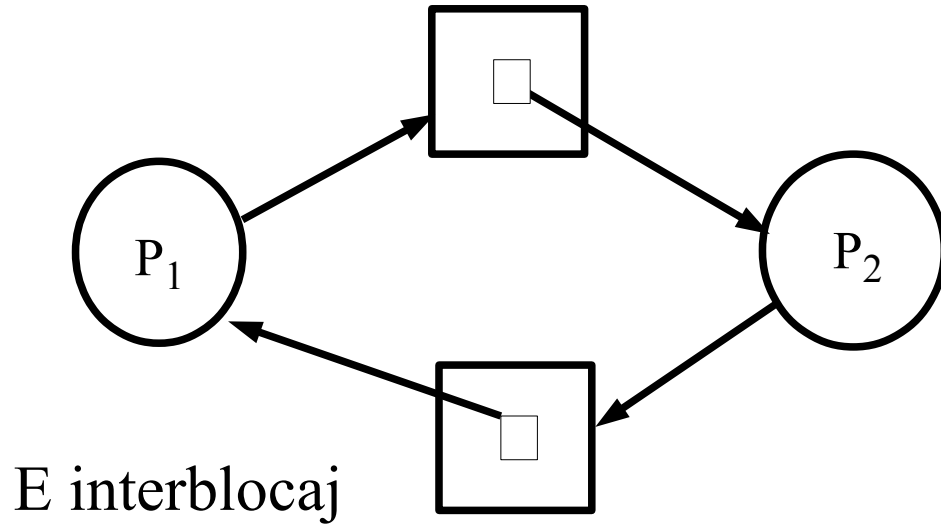


- Observații

- Dacă este interblocaj $\rightarrow$ există circuit în RAG
- Dacă nu există circuite (orientate!) în graful de alocare a resurselor, atunci nu există interblocaj
- Dacă, în schimb, există un circuit în RAG, aceasta în general este o condiție necesară pentru interblocaj, dar *nu este și suficientă*
(Observație: este și suficientă numai în cazul resurselor cu instanțe unice ; mai precis, doar resursele care intervin în circuit trebuie să fie cu instanțe unice)

Interblocaj – RAG /5

Exemple



Interblocaj – Strategii

➤ Strategii de rezolvare a interblocajului:

- **Ignorare**

- majoritatea S.O.-urilor ignoră interblocajele!

- **Prevenire**

- **Evitare**

➔ Ambele presupun utilizarea unui protocol care asigură faptul că sistemul nu va intra niciodată într-o stare de interblocaj

- **Detecție și restabilire**

➔ Se permite intrarea sistemului într-o stare de interblocaj, situație în care se face *recovery* (se iese din interblocaj)

Interblocaj – Ignorare

➤ Ignorarea interblocajului

- Cea mai simplă metodă de rezolvare a interblocajului: nu-l rezolva!
- Te prefaci că nu va apare niciodată un interblocaj, sau presupui că dacă va apare, atunci sistemul se va comporta “ciudat”, moment la care va putea fi rebootat
- Avantaj: simplitate
- Dezavantaje:
 - Necesită intervenție inteligentă umană
 - Nesiguranța faptului dacă există într-adevăr interblocaj
 - S-ar putea să nu fie aplicabilă în sisteme reale critice

Interblocaj – Prevenire /1

➤ **Prevenirea interblocajului**

- Presupune eliminarea a cel puțin uneia din cele 4 condiții necesare pentru interblocaj, făcând astfel interblocajul imposibil
- **Excluderea mutuală**
 - Această condiție poate fi relaxată numai pentru resurse partajabile, nu și pentru resurse exclusive, e.g. imprimanta
- **Hold & wait**
 - Nu lăsa procesul să păstreze resursele în timpul cât așteaptă să obțină alte resurse
 - Procesul trebuie să obțină deodată toate resursele necesare (**preallocarea** = alocarea la începutul execuției a tuturor resurselor)
 - Dezavantaje: proasta utilizare a resurselor și posibilitatea de înfometare a proceselor

Interblocaj – Prevenire /2

➤ **Prevenirea interblocajului (cont.)**

– **Așteptarea circulară**

- Se impune o ordonare a resurselor
 - discuri hard → 1
 - unitati CD-ROM → 2
 - imprimante → 3
- Procesele trebuie să ceară resursele în ordine crescătoare (sau înainte de a cere o resursă, trebuie să elibereze toate resursele deținute deja ce au numere de ordine mai mari), ceea ce elimină posibilitatea de așteptare circulară:

P_1 : request(disc);

request(imprimantă);

P_2 : request(imprimantă);

request(disc);

Interblocaj – Prevenire /3

➤ **Prevenirea interblocajului (cont.)**

– **Ne-preemptie**

- Se permite S.O.-ului să ia înapoi resursele de la acele procese ce dețin resurse, dar așteaptă pentru alte resurse
- Resursele luate sunt adăugate la lista resurselor de care procesul “victimă” are nevoie pentru a-și continua execuția (ca și cum acel proces nu le-ar fi primit niciodată)
- Dezavantaj: pot apare complicații la luarea înapoi a resurselor. Soluția este aplicabilă doar pentru resurse pentru care contextul poate fi salvat și restaurat cu ușurință (e.g. CPU și memoria internă)

Interblocaj – Evitare /1

➤ **Evitarea interblocajului**

- Strategia de evitare nu elimină vreuna din cele 4 condiții necesare pentru apariția interblocajului (ca la strategia de prevenire), ci în schimb folosește **alocarea controlată**: se examinează toate cererile de alocare a resurselor și se iau decizii astfel încât să se împiedice apariția interblocajului
- **Starea curentă** (i.e., la un moment dat) a sistemului: numărul și lista resurselor disponibile și respectiv alocate, precum și cerințele maxime de resurse ale proceselor
- **Stare sigură**: o stare în care sistemul poate aloca, într-o ordine oarecare, fiecărui proces alte resurse solicitate de către acesta, în limita maximului declarat, fără să apară un interblocaj

Interblocaj – Evitare /2

➤ Evitarea interblocajului (cont.)

– Stare “nesigură” = *posibilitatea* de apariție a unui interblocaj, nu și *necesitatea* apariției lui

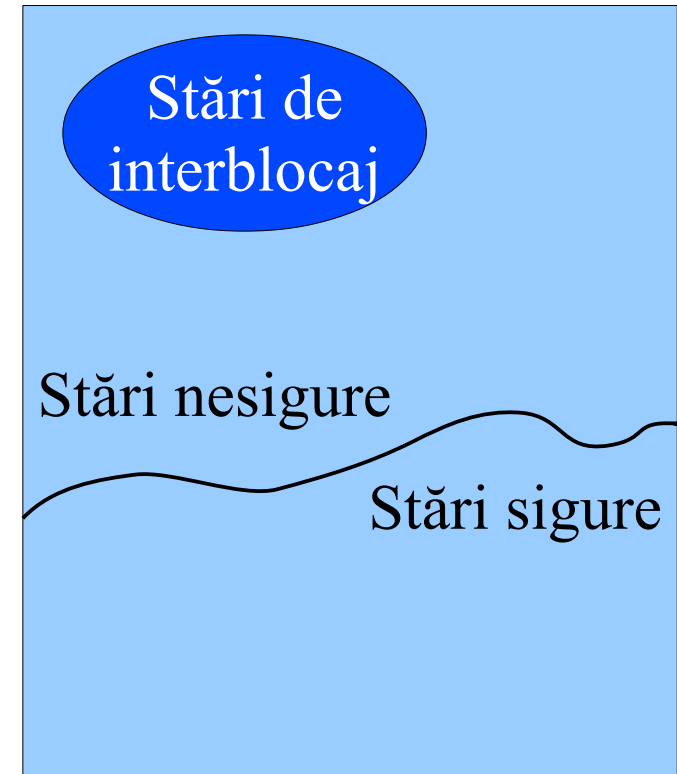
– Soluție:

- Un algoritm pentru păstrarea sistemului într-o stare sigură:

algoritmul bancherului

(the banker's algorithm)

[Dijkstra '65-'68, Habermann '69]



Spațiul stărilor

Interblocaj – Evitare /3

- Algoritmul bancherului
 - Procesele trebuie să-și declare necesitățile maximale de resurse
 - Când un proces face o cerere de alocare de resurse, algoritmul bancherului determină dacă satisfacerea acestei cereri ar cauza intrarea sistemului într-o stare nesigură
 - Dacă da, atunci cererea este respinsă
 - Dacă nu, atunci cererea este aprobată

Interblocaj – Evitare /4

- Algoritmul bancherului – structuri de date
 - n = nr. proceselor , m = nr. tipurilor de resurse
 - $\mathbf{Max}[1..n, 1..m]$ = cerința maximală din fiecare resursă a fiecărui proces

Dacă $\mathbf{Max}[i,j] = k$, atunci procesul P_i va solicita cel mult k instanțe din resursa R_j pe parcursul execuției sale
 - $\mathbf{Disponibil}[1..m] (t)$ = nr. de instanțe disponibile curent (la momentul t) în sistem, din fiecare resursă

Dacă $\mathbf{Disponibil}[j] = k$, atunci există k instanțe disponibile din resursa R_j la momentul respectiv

Inițial (la $t=0$) : $\mathbf{Disponibil}[j] = W_j$ (nr. de instanțe din R_j)

Interblocaj – Evitare /5

- Algoritmul bancherului – structuri de date (cont.)
 - **Alocare**[1..n,1..m] (t) = alocarea curentă (i.e., la momentul t) a fiecărei resurse pentru fiecare proces
Dacă **Alocare**[i,j] = k, atunci procesul P_i are alocate k instanțe din resursa R_j la momentul respectiv
 - **Necesar**[1..n,1..m] (t) = necesarul curent (i.e., la momentul t) din fiecare resursă a fiecărui proces
Dacă **Necesar**[i,j] = k, atunci procesul P_i mai are nevoie de încă cel mult k instanțe din resursa R_j la momentul respectiv
 - Observații: **Necesar** (t) = **Max** – **Alocare** (t)
și **Disponibil**[j] (t) = $W_j - (\sum_{1 \leq i \leq n} \text{Alocare}[i,j] (t))$

Interblocaj – Evitare /6

- Algoritmul bancherului – structuri de date (cont.)
 - **Cerere**[1..n,1..m] (t) = cererea curentă (i.e., la momentul t) din fiecare resursă a fiecărui proces
Dacă **Cerere**[i,j] = k, atunci procesul P_i cere k instanțe din resursa R_j la momentul respectiv
 - Notatii:
 - **Cerere_i** = vectorul cererii curente a procesului P_i
(i.e. **Cerere_i** [j] (t) = **Cerere**[i,j] (t))
 - **Necesar_i** = vectorul necesarului curent al procesului P_i
(i.e. **Necesar_i** [j] (t) = **Necesar**[i,j] (t))
 - **Alocare_i** = vectorul alocării curente a procesului P_i
(i.e. **Alocare_i** [j] (t) = **Alocare**[i,j] (t))

Interblocaj – Evitare /7

- Algoritmul bancherului
 - Ideea: cunoașterea numărului maxim de instanțe din fiecare resursă pe care le poate cere un proces
 - Cunoașterea alocării de resurse curente a fiecărui proces și a necesarului curent (**max – alocarea curentă**)
 - Când apare o cerere, sistemul “pretinde” că o onorează ...
 - Și apoi, consideră cazul cel mai rău posibil: încearcă să satisfacă necesarul curent al tuturor proceselor într-o ordine oarecare ...
 - Dacă îi este imposibil, atunci refuză cererea

Interblocaj – Evitare /8

- Algoritmul de soluționare a cererii – executat când este făcută o cerere de resurse de către un proces P_i
 1. If $Cerere_i > Necesari_i$ then { eroare; exit }
// procesul și-a depășit cerința maximală pretinsă la început
 2. If $Cerere_i > Disponibil$ then { wait(P_i); exit }
// P_i trebuie să aștepte deoarece nu există resurse disponibile
 3. $Disponibil := Disponibil - Cerere_i$;
 $Alocare_i := Alocare_i + Cerere_i$;
 $Necesari_i := Necesari_i - Cerere_i$;
// sistemul “pretinde” că a alocat resursele solicitate de P_i
 4. If $EStareSigură()$ then {alocă resursele procesului P_i }
else { wait(P_i); **rollback** pasul 3. } // refă vechea stare

Interblocaj – Evitare /9

- Algoritmul de siguranță **EStareSigură()** – testează dacă starea curentă a sistemului este sigură sau nu
 1. Date: doi vectori **Work[1..m]**, **Finish[1..n]**
Inițializări: **Work := Disponibil**, **Finish[i] := false**, $i=1..n$
 2. Caută un i ce satisface condițiile
Finish[i] = false and **Necesar_i ≤ Work**
// caut procesul P_i care ar putea fi servit din resursele disponibile
 3. If (există un astfel de i) then
{ **Work := Work + Alocare_i**; **Finish[i] := true**; goto 2.; }
// sper că P_i se va termina și voi putea folosi resursele lui
else { goto 4.; }
 4. If (**Finish[i]=true for all i**) then { starea este sigură }
else { starea este nesigură }

Interblocaj – Evitare /10

➤ Algoritmul de siguranță **EStareSigură()** – exemplu

Considerăm un sistem cu un singur tip de resursă: unități de bandă magnetică, în număr de 12 unități, și cu 3 procese P_1, P_2 și P_3 , care la un moment dat t au alocate respectiv câte 5, 2 și 2 unități de bandă, și care și-au declarat la început ca cerințe maximele respectiv câte 10, 4 și 6 unități de bandă.

Deci: $\text{Max} = (10, 4, 6)$, $\text{Disponibil}(t) = 3$,

$\text{Alocare}(t) = (5, 2, 2)$ și $\text{Necesar}(t) = (5, 2, 4)$

Este starea sistemului sigură? Da, este sigură, deoarece aplicând alg. de siguranță obținem o secvență sigură: $\langle P_2, P_1, P_3 \rangle$ (de asemenea, și secvența $\langle P_2, P_3, P_1 \rangle$ este sigură).

➤ Obs.: cu o mică modificare, alg. poate furniza și secvența sigură

Interblocaj – Detecție /1

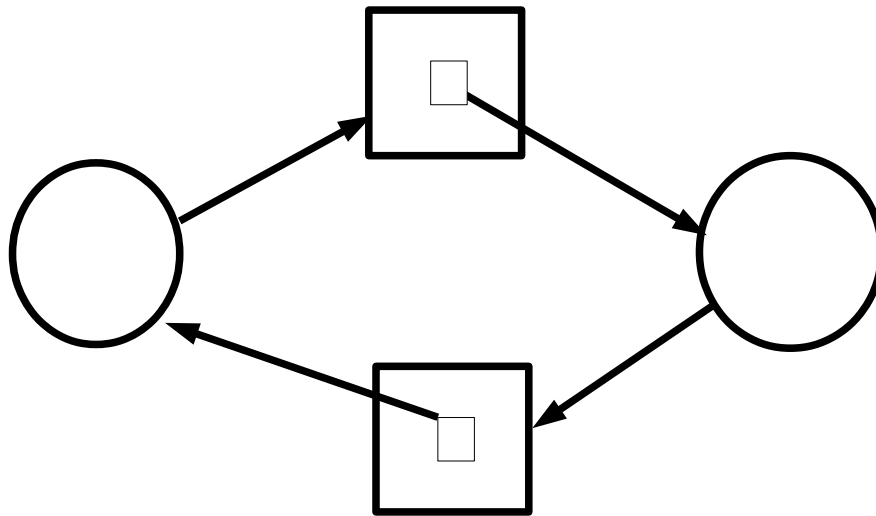
➤ Detecția interblocajului

- Pentru a detecta interblocajul, este nevoie de:
 - Un algoritm de detecție a interblocajului
 - Un mecanism de rezolvare (i.e., de a ieși din interblocaj)
 - Este mai ușor în cazul când toate resursele au instanțe unice, deoarece în acest caz un circuit în RAG este condiție necesară și suficientă pentru interblocaj
- Pentru a detecta interblocajul în cazul resurselor cu *instanțe unice*, graful de alocare a resurselor este “strâns” într-un graf de așteptare (**wait-for graph**) și se caută circuite în acest graf. Algoritmul de detecție a circuitelor are în acest caz complexitatea $O(n^2)$.

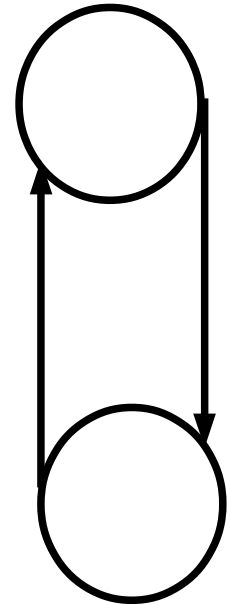
Interblocaj – Detecție /2

➤ **Detecția interblocajului** (cont.)

– “Strângerea” unui RAG într-un graf wait-for:



devine:



– Observație: această tehnică nu funcționează în cazul resurselor cu instanțe multiple

Interblocaj – Detecție /3

➤ **Detecția interblocajului** (cont.)

- Pentru cazul resurselor cu *instanțe multiple*, algoritmul de detecție a interblocajului operează în mod brut, încercând să reducă graful RAG prin verificarea tuturor posibilităților de alocare a resurselor
- Dacă nu există nici o posibilitate de alocare a instanțelor resurselor care să permită fiecărui proces să-și termine execuția, atunci este interblocaj
- Complexitatea algoritmului este în acest caz $O(n^2 \cdot m)$, cu n = numărul de procese și m = numărul de resurse
Algoritmul este prezentat în continuare ...

Interblocaj – Detecție /4

- Algoritmul de detecție – structuri de date
 - n = nr. proceselor , m = nr. tipurilor de resurse
 - **Disponibil**[1.. m] (t) = nr. de instanțe disponibile curent (la momentul t) în sistem, din fiecare resursă
Dacă **Disponibil**[j] = k , atunci există k instanțe disponibile din resursa R_j la momentul respectiv
Inițial (la $t=0$) : **Disponibil**[j] = W_j (nr. de instanțe din R_j)
 - **Alocare**[1.. n ,1.. m] (t) = alocarea curentă (i.e., la momentul t) a fiecărei resurse pentru fiecare proces
Dacă **Alocare**[i,j] = k , atunci procesul P_i are alocate k instanțe din resursa R_j la momentul respectiv

Interblocaj – Detecție /5

- Algoritmul de detecție – structuri de date (cont.)
 - **Cerere**[1..n,1..m] (t) = cererea curentă (la momentul t) din fiecare resursă a fiecărui proces
Dacă **Cerere**[i,j] = k, atunci procesul P_i cere k instanțe din resursa R_j la momentul respectiv
 - Notatii:
 - **Cerere_i** = vectorul cererii curente a procesului P_i
(i.e. **Cerere_i** [j] (t) = **Cerere**[i,j] (t))
 - **Alocare_i** = vectorul alocării curente a procesului P_i
(i.e. **Alocare_i** [j] (t) = **Alocare**[i,j] (t))
 - Obs.: **Disponibil**[j] (t) = $W_j - (\sum_{1 \leq i \leq n} \text{Alocare}[i,j] (t))$

Interblocaj – Detecție /6

➤ Algoritmul de detecție – testează dacă starea curentă a sistemului este interblocaj sau nu

1. Date: doi vectori $Work[1..m]$, $Finish[1..n]$

Inițializări: $Work := Disponibil$,

$Finish[i] := (Alocare[i,j]=0 \text{ for all } j ? \text{true} : \text{false})$, $i=1..n$

2. Caută un i a.î. $Finish[i] = \text{false}$ and $Cerere_i \leq Work$

// caut, printre procesele care dețin deja resurse, un proces P_i a cărui

// cerere ar putea fi servită din resursele disponibile

3. If (există un astfel de i) then

{ $Work := Work + Alocare_i$; $Finish[i] := \text{true}$; goto 2.; }

// sper că P_i se va termina și voi putea folosi resursele eliberate de el

else { goto 4.; }

4. If ($Finish[i]=\text{true}$ for all i) then { starea nu este interblocaj }
else {este un interblocaj al proceselor i cu $Finish[i]=\text{false}$ }

Interblocaj – Detecție /7

➤ Algoritmul de detecție – exemplu

Considerăm un sistem cu un singur tip de resursă: unități de bandă magnetică, în număr de 12 unități, și cu 3 procese P_1, P_2 și P_3 , care la un moment dat t au alocate respectiv câte 5, 2 și 3 unități de bandă, și care mai solicită încă respectiv câte 5, 2 și 4 unități.

➔ Disponibil (t) = 2, Alocare (t) = (5,2,3) și Cerere (t) = (5,2,4)

Este starea curentă a sistemului un interblocaj?

Nu, nu este, deoarece aplicând alg. de detecție obținem o secvență de servire a tuturor cererilor: $\langle P_2, P_3, P_1 \rangle$.

➤ Observație: cu o mică modificare, algoritmul poate furniza și secvența de servire.

Interblocaj – Detecție /8

➤ **Detecția interblocajului** (cont.)

– Problemă:

- Cât de des ar trebui executat algoritmul de detecție?

– Răspunsul depinde de mai mulți factori:

- Cât de des este posibil să apară un interblocaj?
- Câte procese pot fi afectate de interblocaj?
- La apariția unui interblocaj, se poate întâmpla ceva foarte grav mai înainte ca un operator uman să observe ceva suspect în sistem?
- Cât de mult *overhead* al sistemului ne putem permite?

Interblocaj – Restabilire /1

➤ **Ieșirea din interblocaj**

- Dacă s-a detectat un interblocaj, cum se poate restabili sistemul (i.e., ieși din blocaj) ?
- Răspunsuri posibile:
 - **Violarea excluderii mutuale** (numai pentru resurse partajabile, e.g. memorie, discuri hard, ...)
 - ***Abort*-ul** (i.e., terminarea anormală a) **tuturor proceselor**
 - ***Abort*-ul unui(or) proces(e) a.î. să dispară interblocajul** (Ce efect are *abort*-ul asupra integrității aplicațiilor?)
 - **Preemția unor resurse: selectarea unui(or) proces(e) victimă și luarea înapoi a resurselor deținute de el(e)** (Dezavantaje: necesitatea de a face *rollback* pentru victimă, posibilitatea de înfometare a victimei)

Interblocaj – Restabilire /2

➤ **Ieșirea din interblocaj** (cont.)

- Ce proces să fie ales drept victimă (pentru abort sau pentru preempție & rollback)?
- Răspunsul depinde de mai mulți factori:
 - Prioritatea proceselor
 - Cât timp a rulat procesul până la interblocaj și cât mai are de rulat până la terminarea sa
 - Câte resurse a utilizat procesul și ce fel de tip de resurse (ușor/difícil de preemptat)
 - De câte resurse mai are nevoie procesul pentru a se termina
 - Tipul procesului (interactiv sau serial)
 - Câte procese va fi nevoie să fie abortate sau preemptate

- Înfometare (*starvation*) a proceselor
 - Definiție
 - Strategii de rezolvare

➤ Definiție

- **Înfometare:** situația care apare atunci când un proces, ce a cerut permisiunea de acces la o resursă (e.g. la CPU, în cazul alg. de planificare a proceselor, sau la un semafor, în cazul alg. de sincronizare, etc.), așteaptă la infinit primirea acelei resurse, deoarece există un flux constant de alte procese care solicită acea resursă și o și primesc, în defavoarea procesului înfometat (datorită politicii de servire a acelei resurse implementate de acel S.O.).

➤ Strategii de rezolvare

- **Ignorare:** o lăsăm în sarcina operatorului uman
- **FIFO:** implementarea unei politici de servire care să respecte ordinea de primire a cererilor (e.g. la alocarea CPU-ului: alg. de planificare FCFS sau RR, sau pentru semafoare: implementarea cozii de așteptare sub formă de coadă FIFO, ș.a.).
- ***Aging-ul*:** implementarea unei politici de servire care să favorizeze procesele ce așteaptă de mult timp primirea resursei solicitate (e.g. la alocarea CPU-ului: alg. de planificare cu priorități dinamice, în care prioritatea unui proces este mărită treptat în timp ce este *ready*, cu revenirea la valoarea inițială a priorității după ce rulează o cuantă).

- **Bibliografie obligatorie**

capitolele despre *IPC* și *deadlock* din

- Silberschatz : “*Operating System Concepts*”

(cap.3,§3.4–6 despre IPC + cap.6,§6.9 despre deadlock, din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.2,§2.3.8–9 despre IPC + cap.6 despre deadlock, din [MOS3])

- **Comunicații inter-procese (IPC)**
 - Problema comunicației
 - Tipuri de comunicație
 - Comunicația directă
 - Comunicația indirectă
 - Excepții
 - IPC sub UNIX
- **Interblocajul proceselor**
 - Definiție
 - Context
 - Model
 - Cerințele interblocajului
 - Graful de alocare a resurselor
 - Strategii: prevenire, evitare, detecție și restabilire
- **Înfometarea proceselor**

Gestiunea proceselor

➤ **Recapitulare**

- Definiția procesului
- Stările procesului
- Concurență
- Planificare
- Problema secțiunii critice
- Probleme clasice de sincronizare
- Comunicații inter-procese
- Interblocajul și înfometarea proceselor

➤ Urmează: **Administrarea memoriei**

~~Urmează: Primul parțial scris~~

Sisteme de Operare

Administrarea memoriei – partea I

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

- Introducere
 - Probleme
 - Ierarhii de memorie
 - Alocarea memoriei
 - Adrese de memorie logice vs. fizice
 - Scheme de alocare contigue
 - Memorie virtuală
- (va urma)
- Scheme de alocare necontigue

Introducere

- Codul executabil și datele programului trebuie să fie rezidente în memorie pentru a putea fi accesate de către procesor/procesoare
- Este nevoie de *partiționarea* memoriei (pentru a permite utilizarea ei simultan de către mai multe procese) și de *protecția* memoriei (pentru a împiedica procesele să se “deranjeze” unele pe altele)
- Utilizatorii pot ignora modul în care adresele de memorie sunt generate de programe

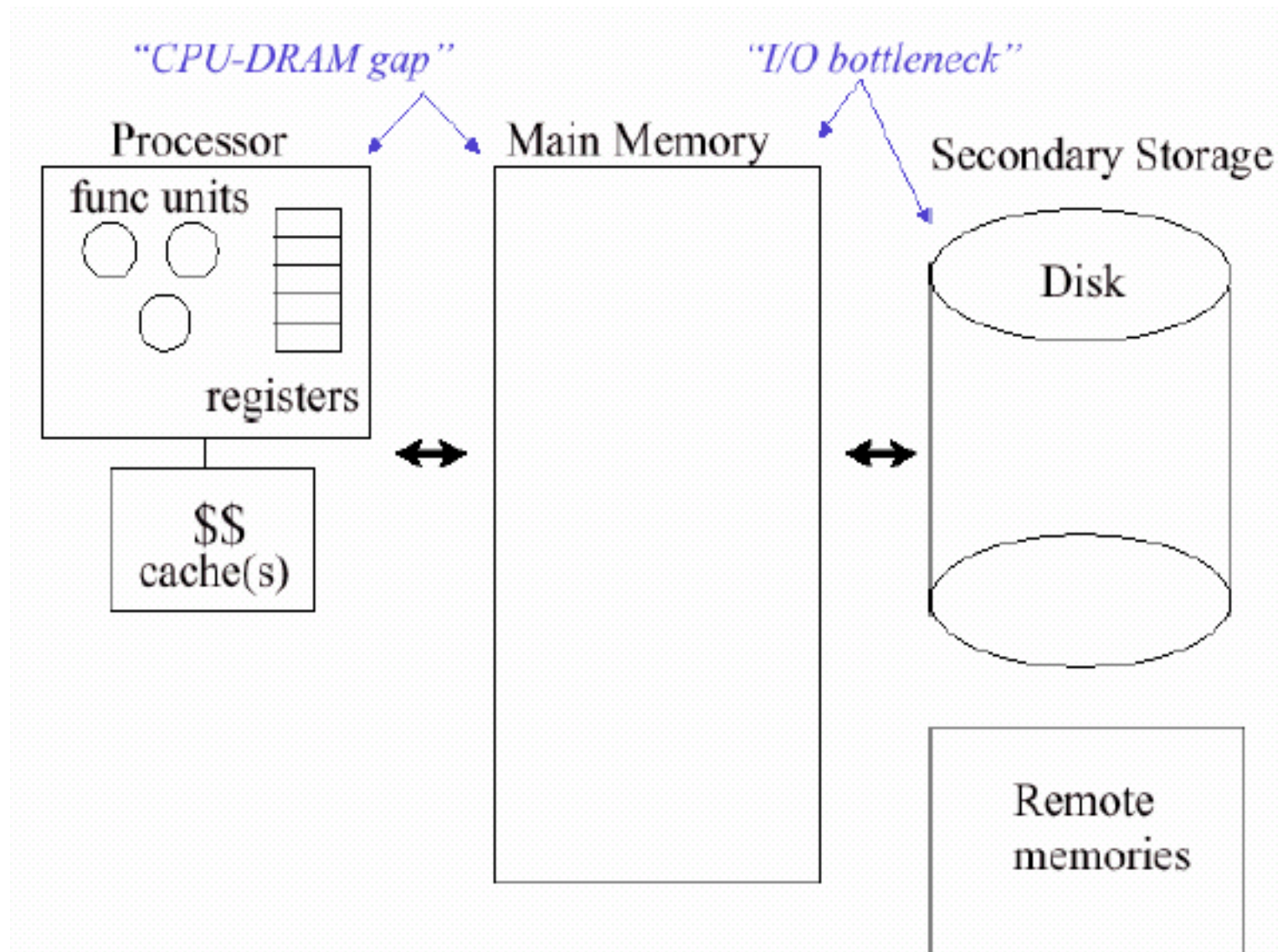
Probleme

- “Legarea” adreselor (*address binding*)
- Optimizări
 - încărcare dinamică
 - legare dinamică
 - *overlay*-uri
- Alocarea memoriei și fragmentarea ei

Întrebări

- Ce este *ierarhia de memorii* ?
- Ce este un *spațiu de adresare* ?
- Cum este *partajată* resursa memorie între toate procesele ce rulează în sistem?
- Cum poate un spațiu de adresare să fie *protejat* de operațiile executate de alte procese?
- Care este *overhead*-ul (i.e. încărcarea suplimentară) generat de operațiile cu memoria?

Ierarhii de memorii /1



Ierarhii de memorii /2

- Ierarhii de memorii:
memorii cu acces mai rapid, dar realizate în tehnologie mai costisitoare și deci de dimensiune mai mică
vs.
memorii cu acces mai lent, dar realizate în tehnologie mai ieftină și deci de dimensiune mai mare
- Memorii: – internă: principală (RAM) ; cache
– externă: secundară (discuri hard) ; de arhivare
- Astfel se realizează un mecanism, transparent pentru utilizator, prin care SC-ul lucrează cu o cantitate mică de memorie rapidă și una mare de memorie lentă, ca și cum ar avea numai memorie rapidă și în cantitate mare

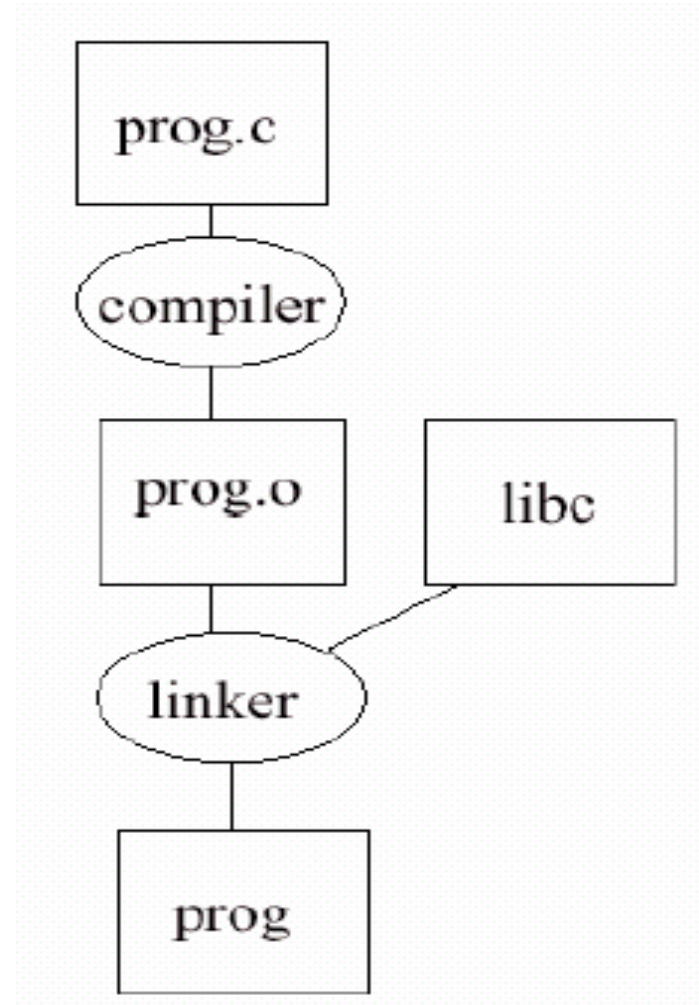
Legarea adreselor

- “Legarea” adreselor (*address binding*) : necesitatea de a decide unde să fie plasate codul executabil și datele
- Se poate face la:
 1. **momentul compilării** – se generează cod executabil cu adrese fixe. Programul trebuie recompilat dacă codul trebuie încărcat la o altă adresă de memorie. (e.g. DOS fișiere .com)
 2. **momentul încărcării** – se generează adresele atunci când programul este încărcat în memorie. (e.g. DOS fișiere .exe)
 3. **momentul execuției** (dinamic) – adresele sunt relative la o valoare care se poate modifica pe parcursul execuției (codul este **relocabil**).

De la Program la Executabil

Fișierul executabil, rezultat prin compilarea codului sursă și link-editarea cu alte module compilate, conține:

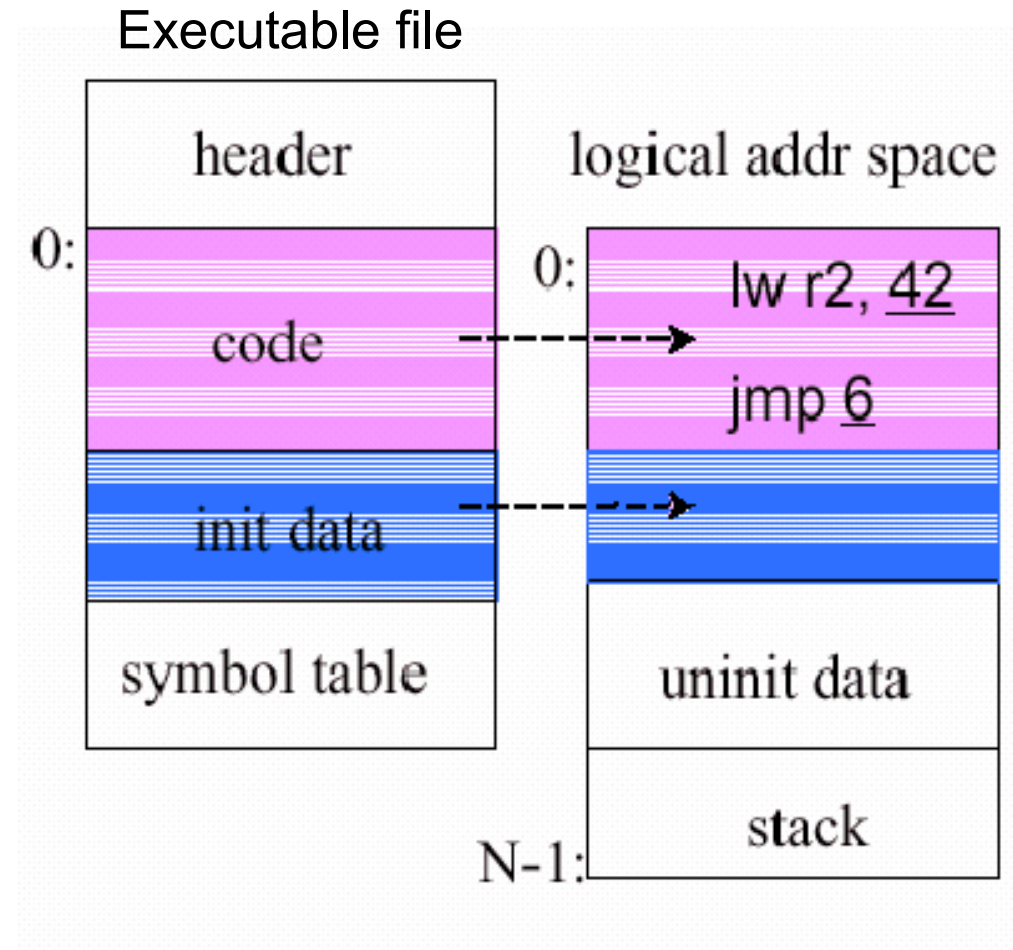
- instrucțiuni în limbaj mașina (ca și cum adresele încep de la zero)
- datele inițializate
- cât spațiu este necesar pentru datele neinițializate



De la Executabil la Spațiul de adresare

Atunci când este creat procesul, pe lângă codul și datele inițializate ce sunt copiate din fișierul executabil, mai trebuie rezervate adrese pentru zonele de date neinițializate și stivă.

Când și cum sunt asignate *adresele fizice* reale?



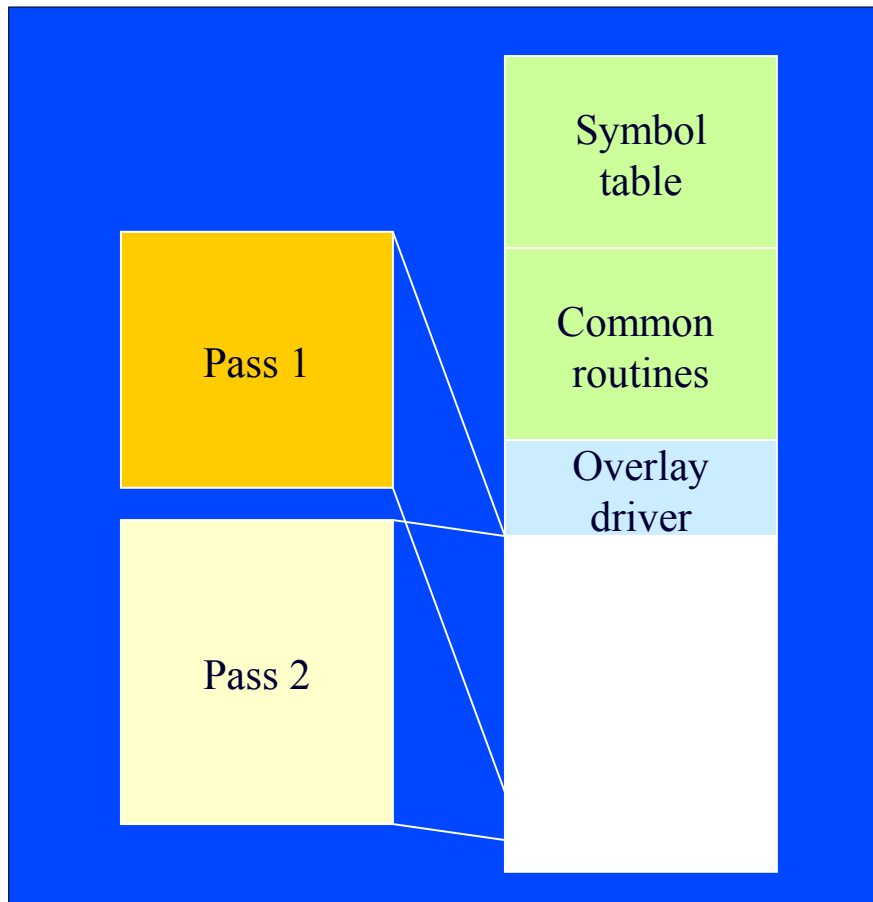
- **Încărcare dinamică**

- Încărcarea rutinelor numai atunci când sunt necesare
- O mai bună utilizare a memoriei deoarece codul utilizat rar nu este necesar să fie prezent în memorie pe toată durata execuției programului
- Este implementată de programator prin intermediul unor apeluri de sistem ce încarcă noi “bucăți” de cod; complexitatea facilităților puse la dispoziție variază de la un SC la altul

- ***Overlay-uri***

- *Overlay*-urile furnizează un mod de scriere a programelor ce necesită mai multă memorie decât este fizic disponibilă (pe sisteme fără memorie virtuală)
- Programatorul partiționează programul în “bucăți” de cod ce se pot suprapune; o “bucată” diferită de cea curentă este încărcată atunci când este nevoie de ea
- Programarea este complexă și dificilă; programatorul este responsabil cu proiectarea “bucăților” de cod
- ex.: DOS; suport din partea compilatorului Turbo Pascal

- ***Overlay-uri*** (cont.)
 - Exemplu: compiler multi-pass



- **Legarea dinamică (.DLL) – Windows, OS/2**
 - Legarea (*linking*-ul) subrutinelor este amânată până la momentul execuției programului
 - Rutinele din biblioteci (e.g. funcțiile C de bibliotecă) nu sunt incluse în codul obiect generat de compilator; în locul lor, este inclus un *stub* (ce conține informații despre cum poate fi localizată în memorie rutina respectivă ...)
 - Când este apelat, *stub*-ul se înlocuiește cu adresa rutinei respective și se execută aceasta
 - S.O.-ul trebuie să cunoască dacă rutina este prezentă în memorie, iar dacă nu este, trebuie să o încarce

- **Legarea dinamică (.DLL) (cont.)**

- Avantaj: se permite *partajarea* codului
- Probleme:
 - Programul depinde de versiunea .DLL-ului
→ **.DLL hell**
(soluția: *assemblies* utilizate în .NET)
 - Programul nu va funcționa dacă .DLL-urile necesare nu sunt prezente în sistem
- Echivalentul .DLL-urilor în UNIX/Linux:
biblioteci partajate – bibliotecile **.so** (vezi **/lib**)

Alocarea memoriei /1

- Adrese de memorie logice vs. fizice
 - **adrese logice (sau *virtuale*)**: adresele de accesare a memoriei generate de CPU (adresele din codul programului incarcata)
 - **adrese fizice**: adresele de accesare a hardware-ului memoriei (memoria fizică din SC)
 - Pentru SC cu “legarea” adreselor la momentul compilării sau al încărcării, adresele logice coincid cu cele fizice
 - Pentru SC cu “legarea” adreselor la momentul execuției, adresele logice nu mai coincid neapărat cu cele fizice

Translatarea adreselor logice în adrese fizice este executată de către hardware-ul de mapare a memoriei (i.e. MMU); ea depinde de tipul acelui SC, în particular de mecanismul de administrare a memoriei utilizat de acel SC.

Alocarea memoriei /2

- Alocarea memoriei contiguă vs. necontiguă
 - **alocare contiguă**: pentru un proces (cod+date) se alocă o singură porțiune, continuă, din memoria fizică
 - **alocare necontiguă**: pentru un proces (cod+date) se alocă mai multe porțiuni separate din memoria fizică
- Memorie reală vs. virtuală
 - **memoria reală**: spațiul de adresare al proceselor este limitat de capacitatea memoriei interne
 - **memoria virtuală**: spațiul de adresare nu este limitat de capacitatea memoriei interne (aceasta este suplimentată de cea externă, prin tehnica de *swapping*)

Alocarea memoriei /3

- Scheme de alocare a memoriei:
 - alocarea contiguă:
 - alocarea unică, la S.O. mono-utilizator
 - alocarea cu partiții (la S.O. multi-utilizator)
 - ➔ cu partiții fixe (alocare statică)
 - ➔ cu partiții variabile (alocare dinamică)
 - alocarea cu *swapping*
 - alocarea necontiguă:
 - paginată (simplă și la cerere)
 - segmentată (simplă și la cerere)
 - segmentată-paginată (simplă și la cerere)

Scheme de alocare contiguă /1

➤ **Alocarea unică (cu o singură partiție)**

– **Nici o schemă de administrare a memoriei**

- Prima formă de alocare d.p.d.v. istoric, folosită de primele SC, monoprogramate, fără S.O.
- Furnizează utilizatorului o flexibilitate maximă
- Fără servicii oferite de S.O. - fără control al întreruperilor, fără mecanisme de procesare a apelurilor de sistem și a erorilor, fără posibilități de multiprogramare

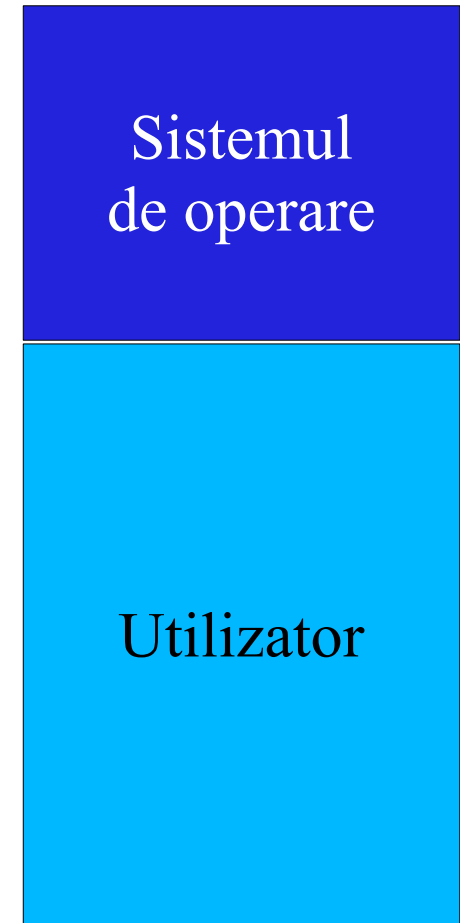
Scheme de alocare contiguă /2

➤ **Alocarea unică (cu două partiții)**

- o partiție de memorie pentru utilizator
- o partiție de memorie pentru S.O.
 - S.O.-ul este plasat fie în zona inferioară a memoriei (e.g. CP/M), fie în zona superioară a memoriei (e.g. DOS)

➤ **Dezavantaje:**

- neutilizarea optimă a memoriei și a CPU;
- imposibilitatea multiprogramării
- programele s-ar putea să nu încapă în memoria disponibilă (excepție – programele cu *overlay*)



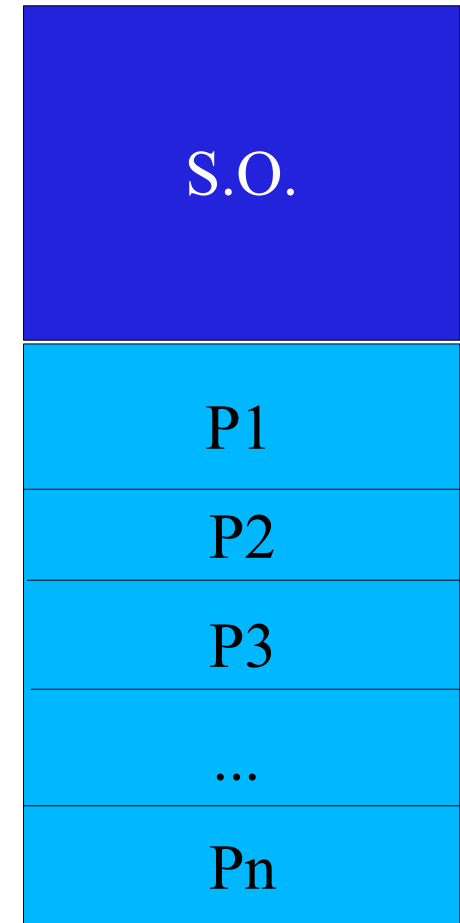
Scheme de alocare contiguă /3

➤ **Alocarea memoriei în partiții fixe**

- memoria este împărțită static în mai multe partiții (de dimensiuni nu neapărat egale)
- fiecare partiție poate conține exact un proces (gradul de multiprogramare este limitat deci de numărul de partiții)
- *alocarea absolută* – pentru SC cu legarea adreselor la compilare sau încărcare

vs.

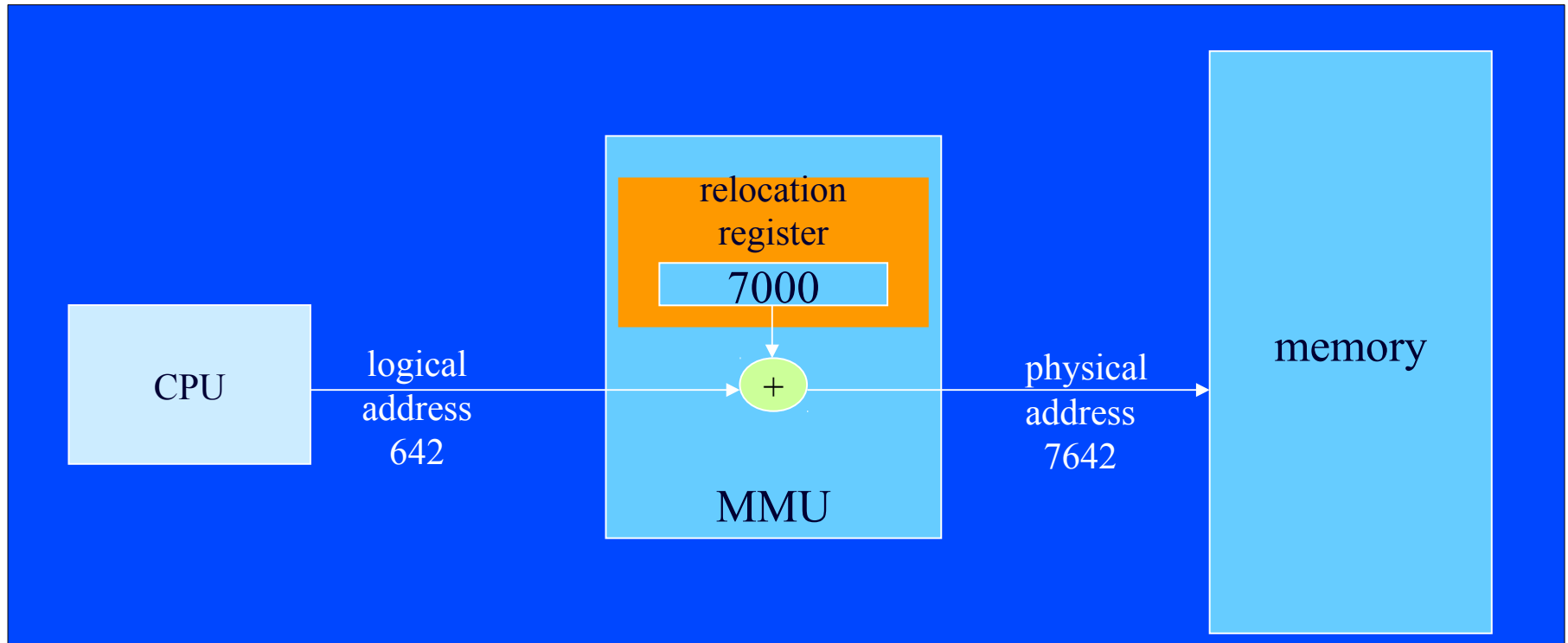
- *alocarea relocabilă* – pentru SC cu legarea adreselor la execuție
- utilizată la sistemele seriale (e.g. IBM OS/360)



Scheme de alocare contiguă /4

➤ Alocarea memoriei în partiții fixe (cont.)

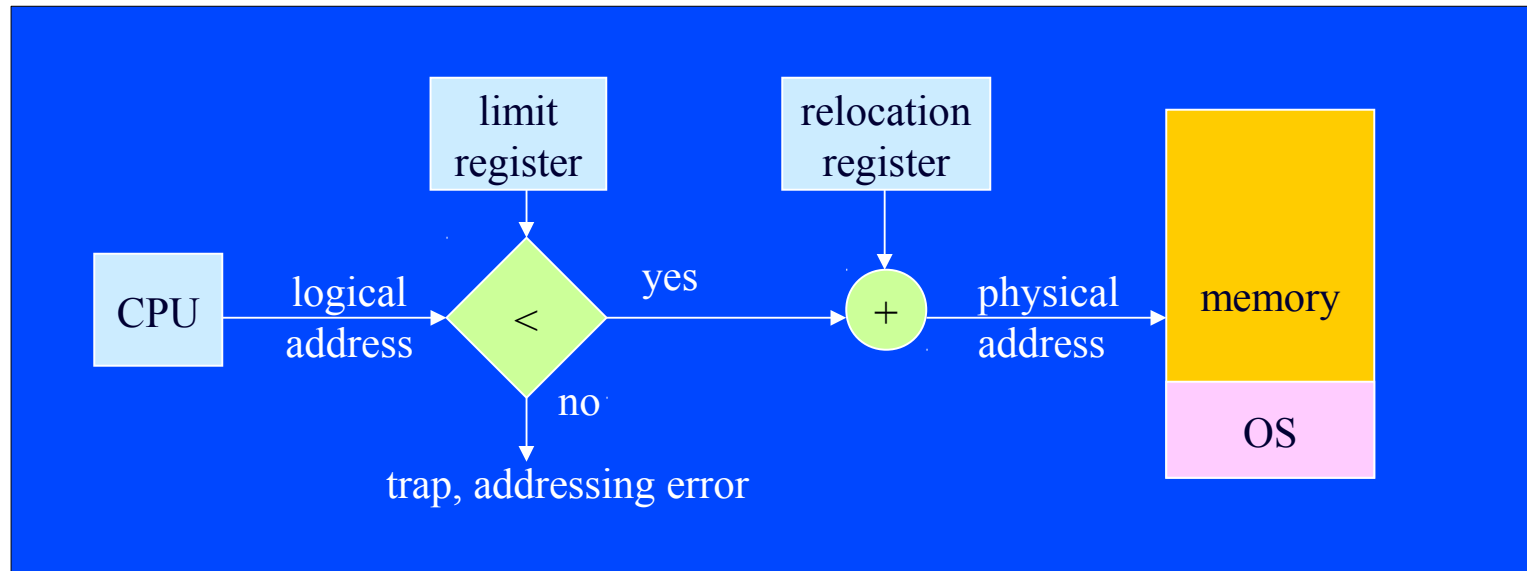
- la alocarea relocabilă se folosește un **registru de relocare**
- **adresa fizică** = **adresa de bază** (valoarea acestui registru) + **deplasamentul** (adresa logică)



Scheme de alocare contiguă /5

➤ Alocarea memoriei în partiții fixe (cont.)

- S.O.-ul trebuie să asigure protecția partițiilor (proceselor)
- un alt registru, **registru de limită**, furnizează deplasamentul maxim ce se permite a se aduna la adresa de bază



- Alt mecanism de protecție folosit de unele S.O.-uri: împărțirea partițiilor în pagini de memorie de lungime fixă (e.g. 2Ko) și folosirea de chei de acces la ele

Scheme de alocare contiguă /6

➤ **Alocarea memoriei în partiții fixe (cont.)**

- când un proces se termină, se încarcă în partiția respectivă un alt proces dintre cele ce așteptau să le vină rândul la execuție
- la primele SC seriale se folosea câte o coadă de așteptare pentru fiecare partiție, iar plasarea job-urilor pe partiții o realiza la început operatorul uman
- apoi s-a folosit o singură coadă de așteptare pentru toate partițiile, ceea ce permitea optimizarea plasării job-urilor
- Dezavantaje:
 - programele s-ar putea să nu încapă în partițiile disponibile
 - neutilizarea optimă a memoriei: poate rămâne spațiu nefolosit în fiecare partiție
 - fenomenul de fragmentare (internă) a memoriei

Scheme de alocare contiguă /7

➤ **Alocarea dinamică (în partiții variabile)**

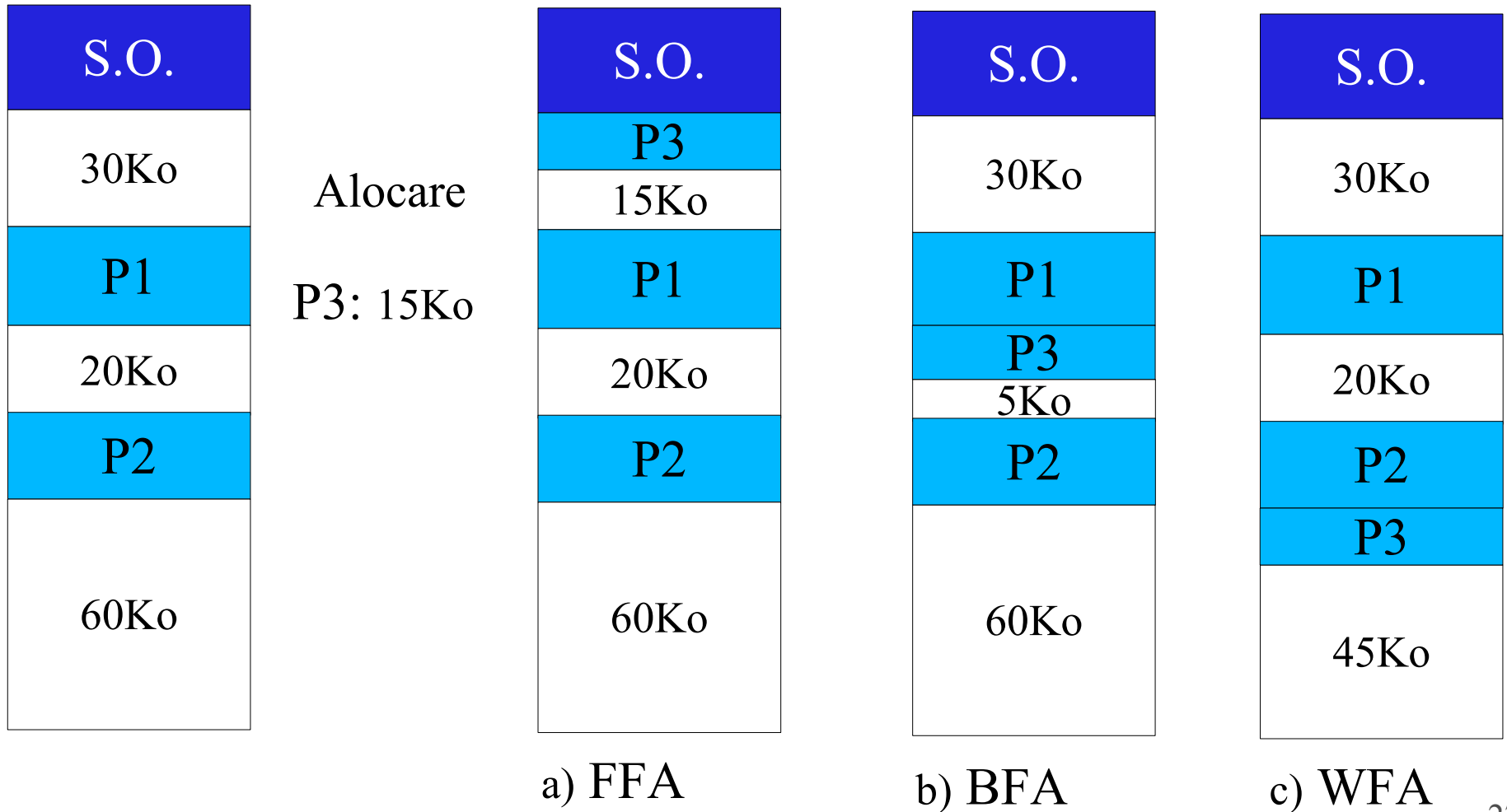
- este tot o schemă de alocare contiguă
- când se încarcă un proces în memorie, i se alocă exact spațiul de memorie necesar, din memoria *liberă* în acel moment, creându-se dinamic o partiție (de lungime variabilă)
- când se termină un proces, partiția ocupată de el devine spațiu liber (și se unifică cu eventualul spațiu liber adiacent)
- S.O.-ul trebuie să gestioneze o tabelă a partițiilor ocupate și o listă a spațiilor libere
- S.O.-ul folosește un algoritm de alocare, ce decide ce spațiu liber să fie alocat unui proces la încărcarea sa
- *alocarea absolută* vs. *alocarea relocabilă*
- utilizată tot la sistemele seriale (prima dată la IBM OS/360)

Scheme de alocare contiguă /8

- **Alocarea dinamică (în partiții variabile)** (cont.)
- algoritmi de alocare (de alegere a unui spațiu liber):
 - **FFA** (*first fit algorithm*): se parcurge lista spațiilor libere (ordonată crescător după adresa de început a spațiilor) și se alege *primul* spațiu de dimensiune suficientă (e.g. MINIX)
 - **BFA** (*best fit algorithm*): se parcurge lista spațiilor libere (ordonată crescător după dimensiunea spațiilor) și se alege *primul* spațiu de dimensiune suficientă (e.g. DOS)
Produce cel mai mic spațiu liber rest.
 - **WFA** (*worst fit algorithm*): se parcurge lista spațiilor libere (ordonată crescător după dimensiunea spațiilor) și se alege *ultimul* spațiu din listă
Produce cel mai mare spațiu liber rest.

Scheme de alocare contiguă /9

➤ Alocarea dinamică (în partiții variabile) (cont.)



Scheme de alocare contiguă /10

- **Alocarea dinamică (în partiții variabile) (cont.)**
 - în anumite scenarii oricare dintre cei 3 algoritmi este mai bun decât ceilalți; în general însă, FFA și BFA sunt mai buni decât WFA
 - problema: fenomenul de fragmentare a memoriei – în timp crește numărul spațiilor libere, iar dimensiunile acestora scad; se poate întâmpla ca un job să nu încapă în nici un spațiu liber, deși dimensiunea sa este mai mică decât suma spațiilor libere
 - Dezavantaje:
 - programele s-ar putea să nu încapă în spațiile libere
 - neutilizarea completă a memoriei: pot rămâne spații libere, nefolosite
 - fenomenul de fragmentare (externă) a memoriei

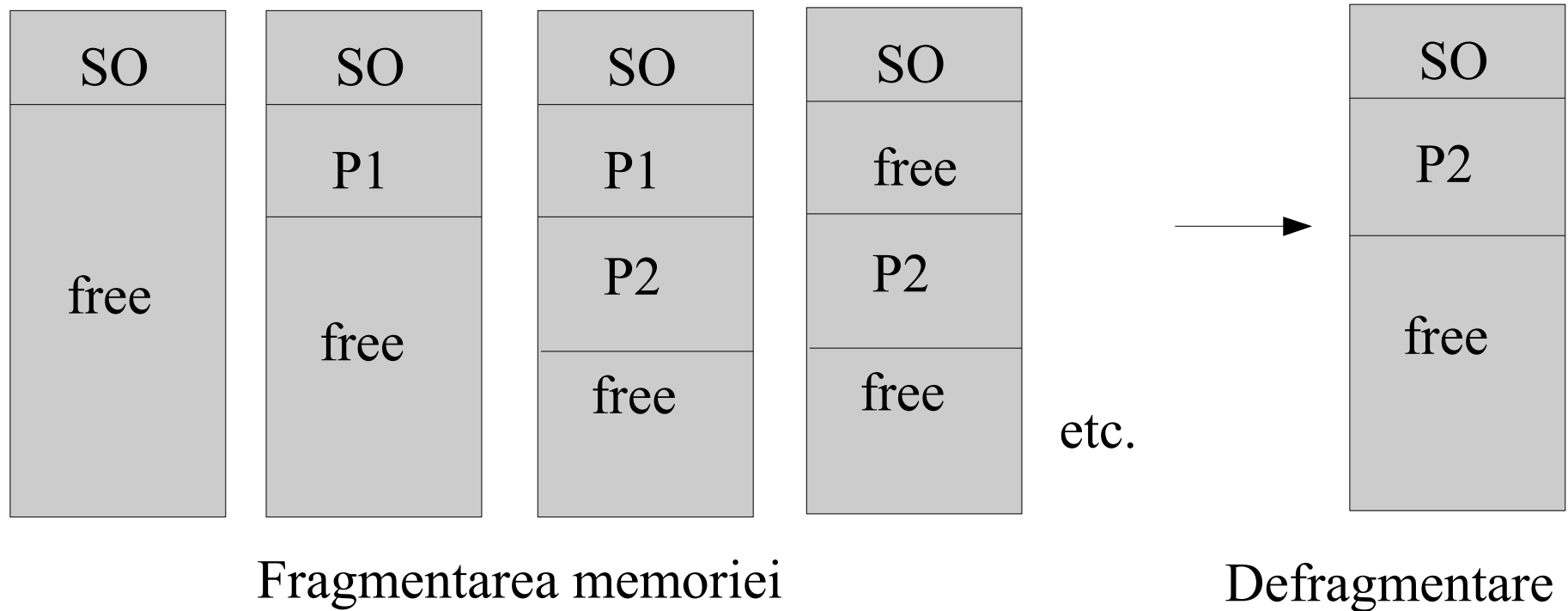
Scheme de alocare contiguă /11

➤ **Alocarea dinamică (în partiții variabile) (cont.)**

- Ce se întâmplă când un proces nu are spațiu liber în care să se încarce? S.O.-ul poate lua următoarele decizii:
 - procesul așteaptă până când se va elibera o cantitate suficientă de memorie
 - efectuarea unei operații de *compactare (relocare)* a memoriei (i.e., o defragmentare a memoriei principale) – se deplasează partițiile ocupate pentru a se unifica (total sau parțial) spațiile libere
- Compactarea rezolvă problema fragmentării memoriei, dar se poate aplica numai pentru *alocarea relocabilă* (i.e., când “legarea” adreselor este dinamică, la momentul execuției)

Scheme de alocare contigua /12

➤ Alocarea dinamică (în partiții variabile) (cont.)



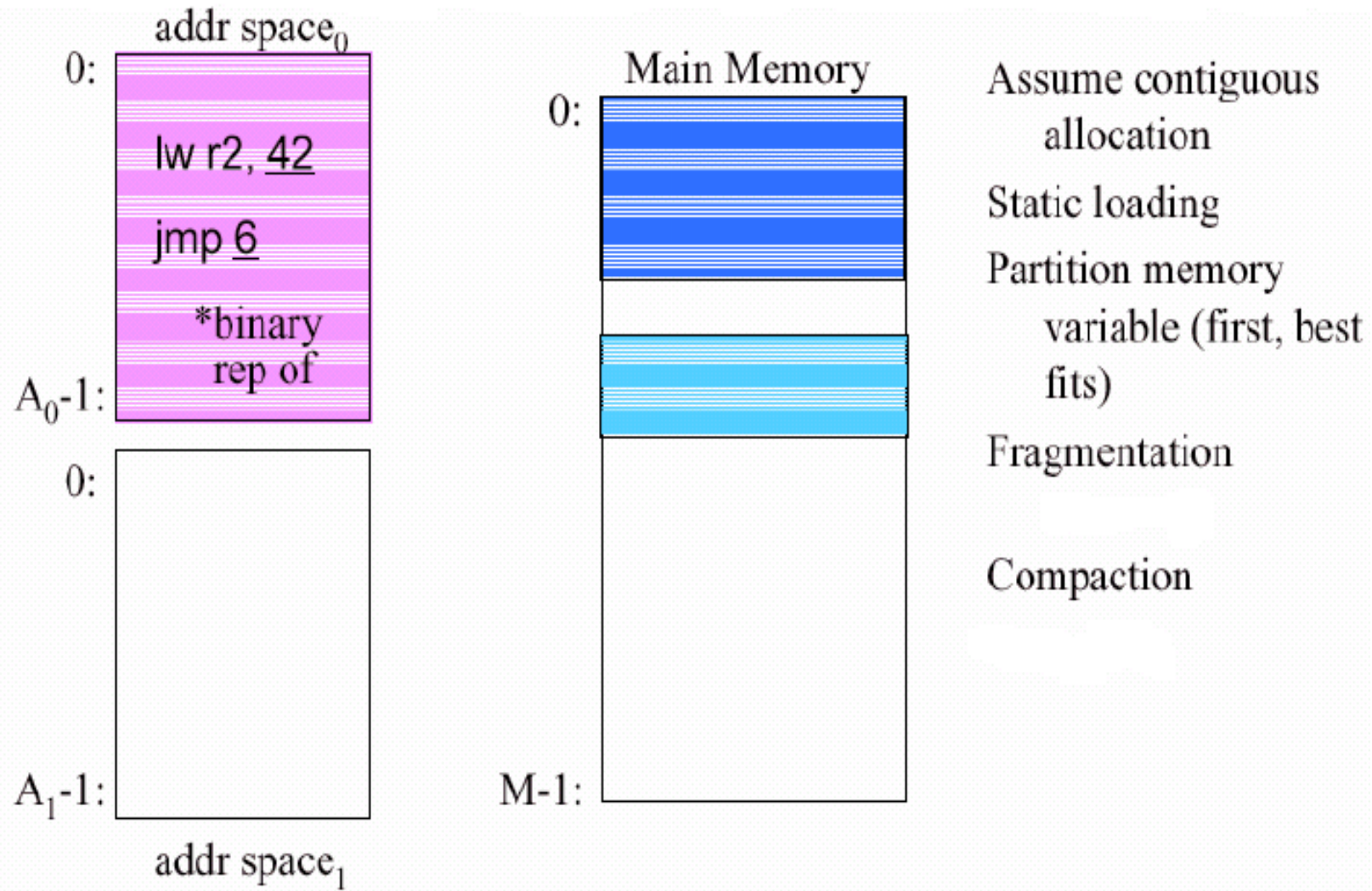
Scheme de alocare contiguă /13

➤ **Alocarea dinamică (în partiții variabile) (cont.)**

- Problema cu perifericele I/O: mecanismului DMA nu-i permite compactarea. Soluții: fie nu se permite compactarea în timpul operațiilor I/O, fie se folosesc buffere în spațiul de adrese al S.O. pentru operațiile I/O
- Altă problemă: cum decid când să fac compactare?
- Dezavantaj: *overhead*-ul implicat de compactare (i.e. de deplasarea unor zone voluminoase de memorie)
- Între alocarea cu partiții fixe și cea dinamică (cu partiții variabile) nu există practic diferențe hardware; ambele sunt realizate prin intermediul unor rutine specializate, din cadrul S.O.-ului

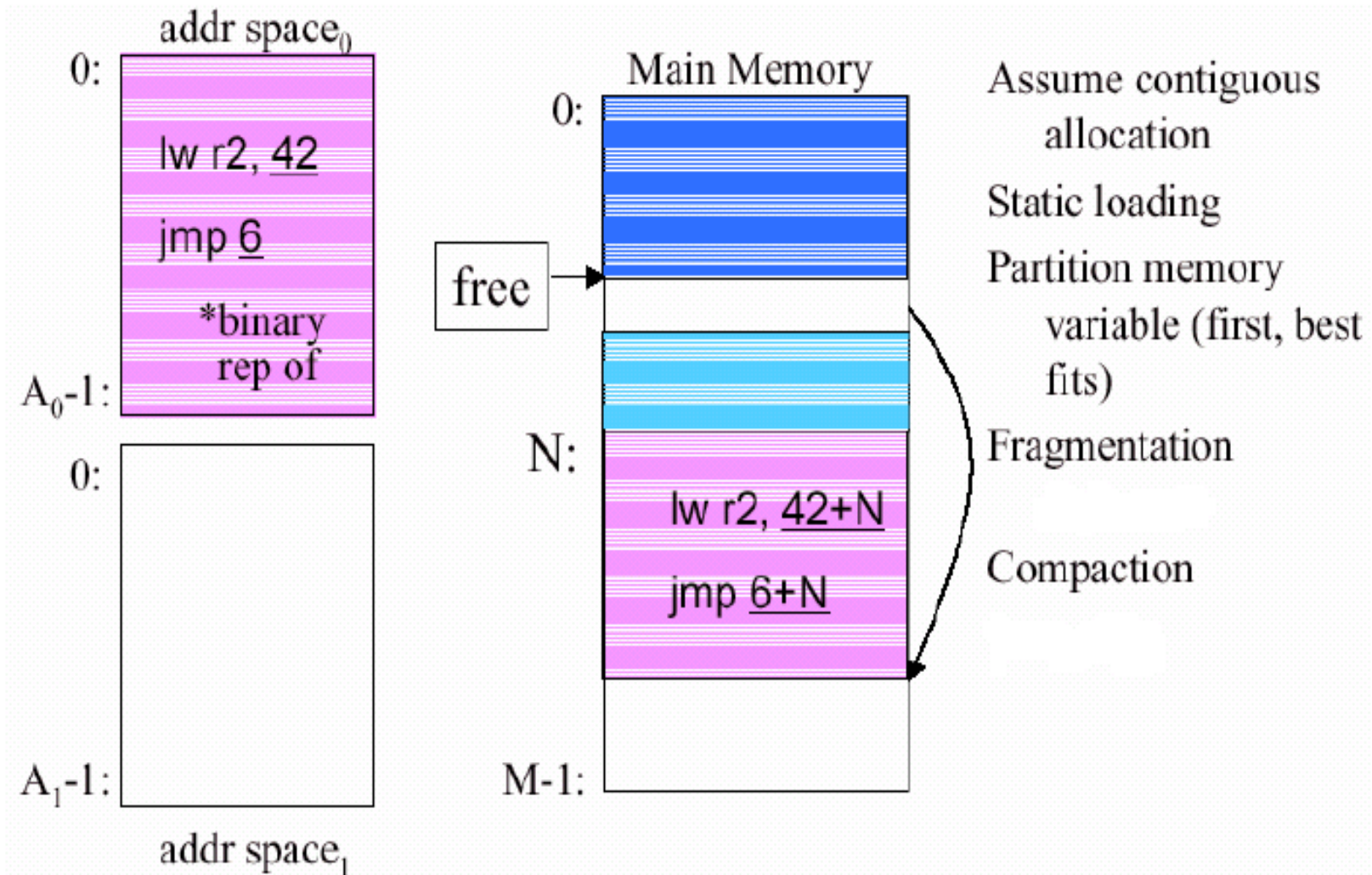
Scheme de alocare contiguă /14

➤ Exemplu de alocare dinamică



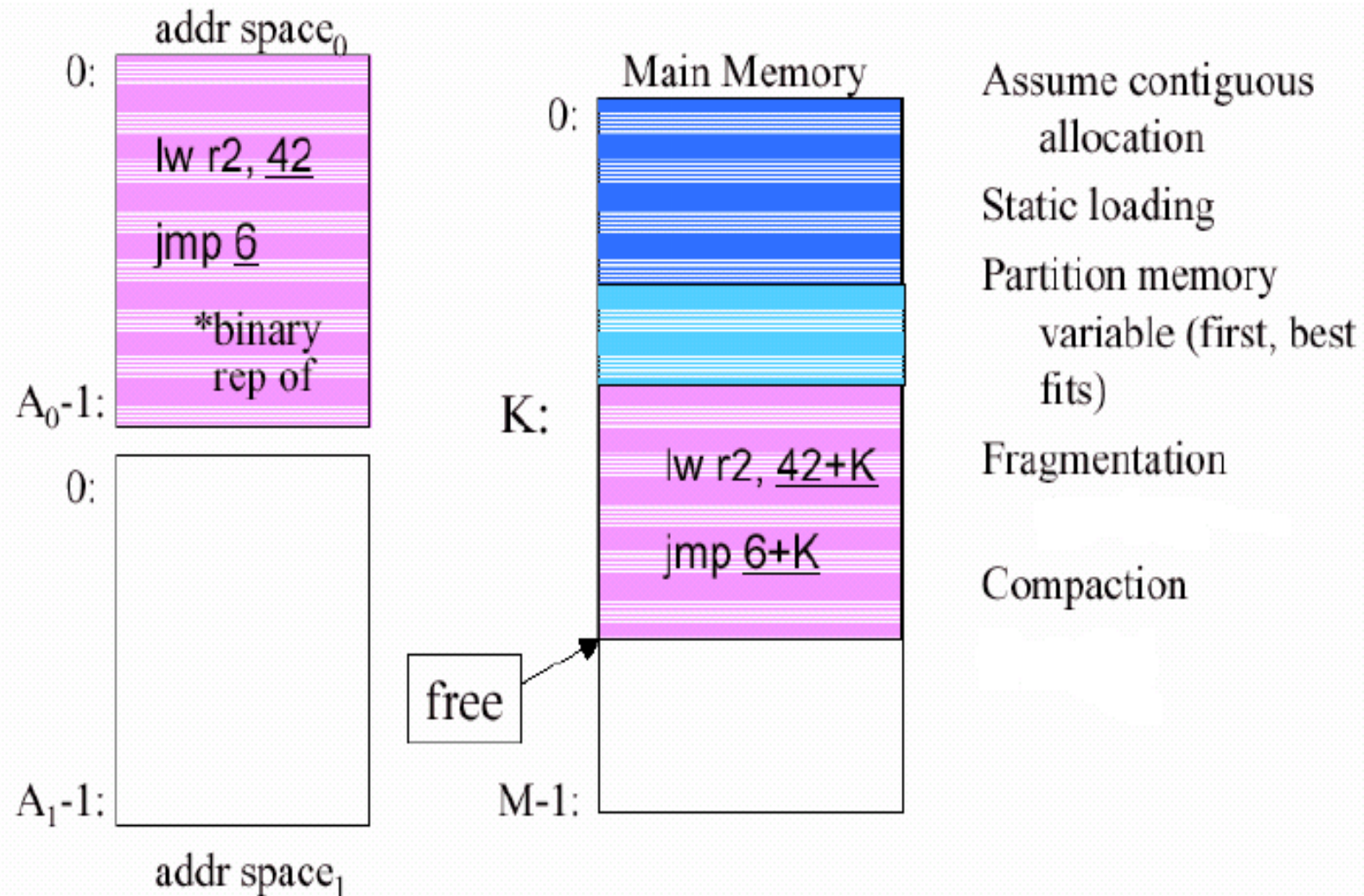
Scheme de alocare contiguă /15

➤ Exemplu de alocare dinamică (cont.)



Scheme de alocare contiguă /16

► Exemplu de alocare dinamică (cont.)



Scheme de alocare contiguă /17

➤ **Alocarea prin *swapping***

- după SC seriale au apărut SC cu *swapping*, denumite și SC *rollout-rollin*, sau SC cu *time-sharing*
- ideea: un proces aflat în starea de așteptare se evacuează temporar pe disc, eliberând astfel memoria principală ocupată
- reluarea execuției procesului se face prin reîncărcarea sa de pe disc în memorie
- e.g. S.O.-ul CTSS (Compatible Time-Sharing System)
- Avantaje:
 - se simulează o memorie mai mare decât cea fizică existentă
 - se îmbunătățește folosirea resurselor SC (CPU + memorie)

Scheme de alocare contiguă /18

➤ **Alocarea prin *swapping*** (cont.)

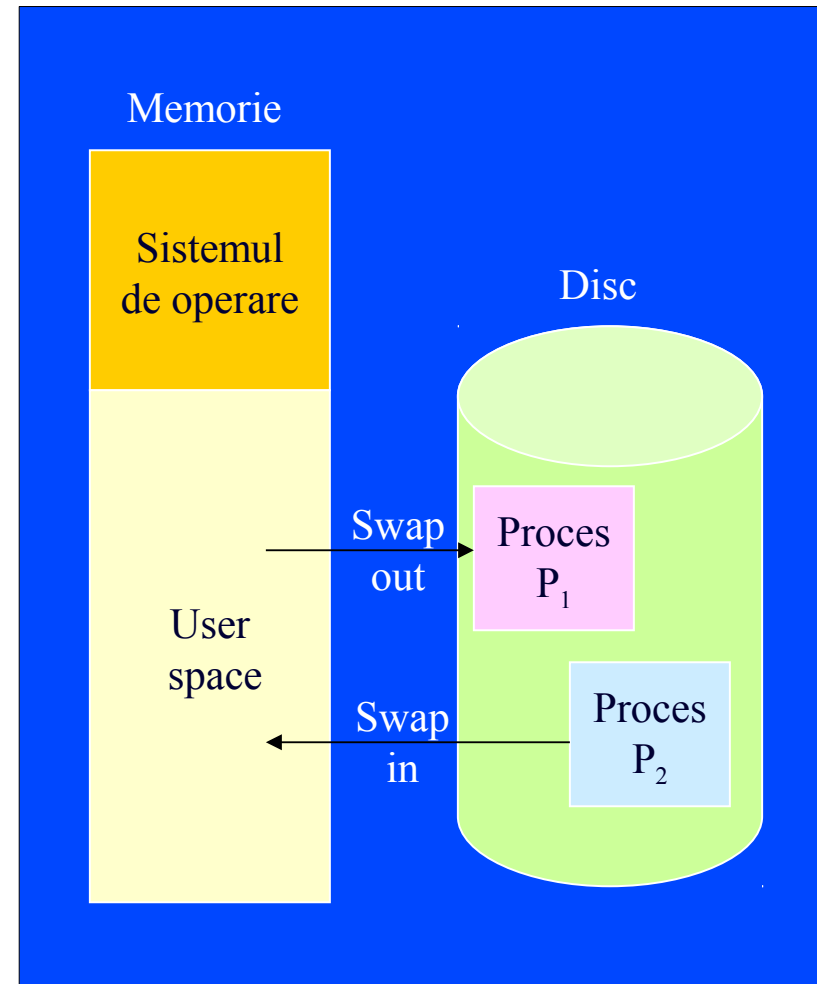
- SC seriale cu *time-sharing* foloseau pentru planificarea proceselor algoritmul RR; în acest caz evacuarea pe disc a jobului curent se face la expirarea cuantei sau la o cerere I/O, evacuare urmată de încărcarea altui proces de pe discul de swap în spațiul de memorie tocmai eliberat
- SC seriale denumite *rollout-rollin* foloseau pentru planificarea proceselor algoritmul pe bază de priorități; în acest caz evacuarea pe disc se face când începe (sau redevine *ready*) un proces mai prioritar sau la o cerere I/O

Scheme de alocare contiguă /19

➤ Alocarea prin *swapping*

– Dezavantaje:

- 1) transferul între memorie și disc este mare consumator de timp (deoarece viteza de acces a discului este cu câteva ordine de mărime mai mică decât cea a memoriei interne)
- 2) alocarea este încă contiguă
- 3) spațiul de adresare al unui proces nu poate depăși capacitatea memoriei interne



Memorie virtuală

➤ Mecanisme de memorie virtuală

- SC cu *memorie virtuală* au capacitatea de a adresa un spațiu de memorie mai mare decât memoria internă disponibilă
- ideea: *swapping*-ul pe disc al unor “bucăți” de memorie, introdusă de S.O.-ul ATLAS (1960, Univ. Manchester, U.K.)
- tehnici de virtualizare: paginarea și segmentarea
- Alocarea prin *swapping* vs. Memoria virtuală
 - *swapping*: mutarea proceselor în întregime afară din memorie pe disc (și invers) atunci când este nevoie
 - memorie virtuală: mutarea unor mici “bucăți” de memorie afară din memorie pe disc (și invers) atunci când este nevoie

- **Bibliografie obligatorie**

capitolele despre *gestiunea memoriei* din

- Silberschatz : “*Operating System Concepts*”

(cap.7, prima parte: §7.1–3, din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.3, prima parte: §3.1–2, din [MOS3])

- Introducere
 - Probleme
 - Ierarhii de memorie
 - Alocarea memoriei
 - Adrese de memorie logice vs. fizice
 - Scheme de alocare contigue
 - Memorie virtuală
- (va urma)
- Scheme de alocare necontigue

Întrebări ?

Sisteme de Operare

Administrarea memoriei – partea II

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

Alocarea memoriei (continuare)

- Scheme de alocare necontigue
 - Paginarea
 - Segmentarea
 - Segmentarea paginată

(va urma)

- Scheme de alocare cu memorie virtuală

Alocarea paginată /1

- **Paginarea** (paginare pură, fără *swapping*)
 - permite ca memoria alocată unui proces să fie necontiguă
 - **paginare hardware**
 - memoria fizică este împărțită în blocuri de lungime fixă, numite **cadre de pagină** (*frames*) (sau *pagini fizice*)
 - lungimea cadrelor de pagină este o putere a lui 2 și este o constantă a SC (e.g. pentru arhitecturile Intel este 4 Ko)
 - memoria logică a unui proces (zona de cod + zona de date) este împărțită în blocuri de lungime fixă (egală cu lungimea cadrelor de pagină), numite **pagini** (*pages*) (sau *pagini virtuale*)

➤ **Paginarea (cont.)**

– **paginare hardware**

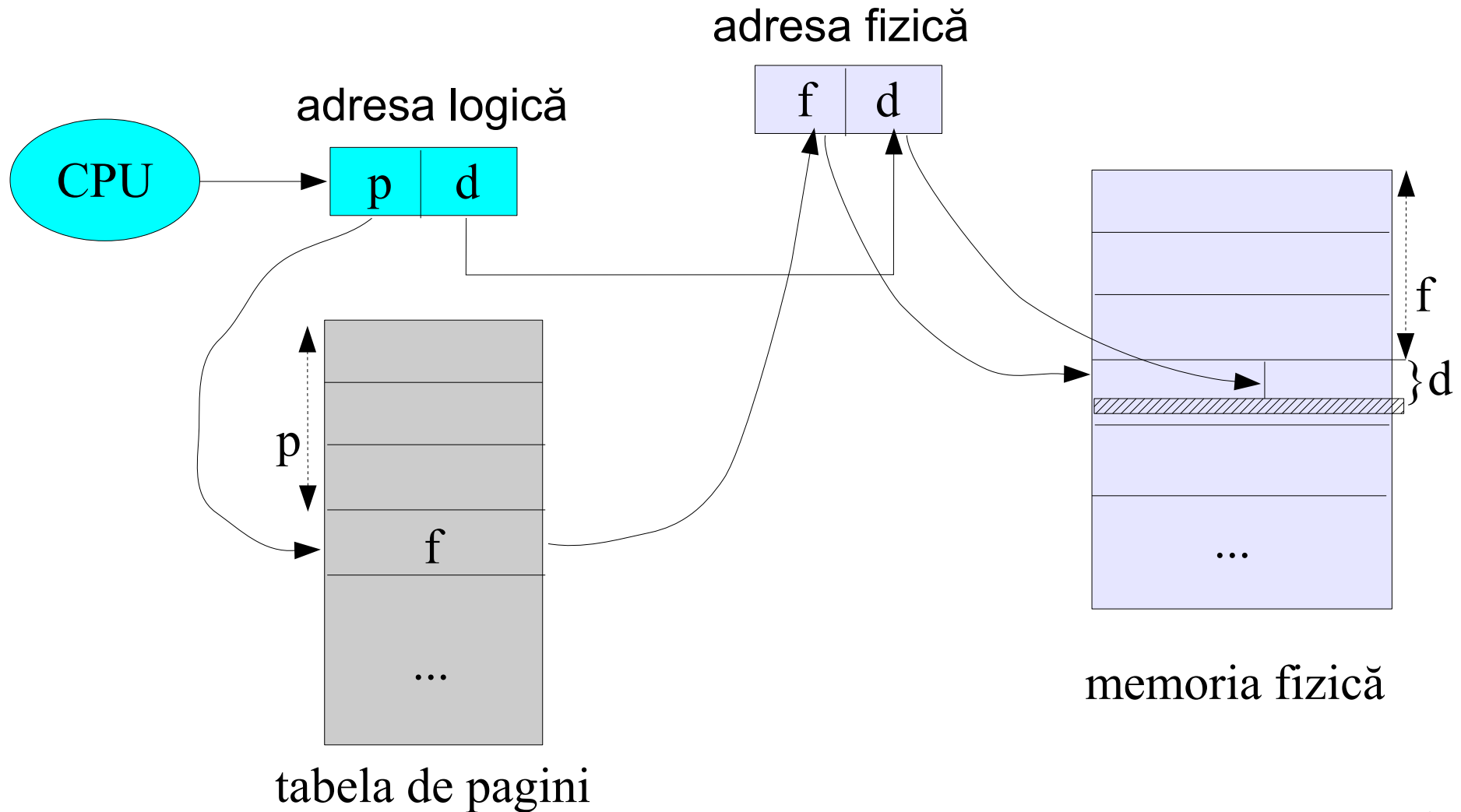
- pentru execuția procesului, paginile sale trebuie încărcate în orice cadre de pagină libere (nu neapărat contiguu!)
- S.O. păstrează evidența cadrelor de pagină libere
- pentru a rula un program ce necesită N pagini, este nevoie să se găsească și să se mapeze N cadre libere
- Avantaj: datorită alocării necontigue, se elimină fragmentarea *externă*, i.e. nu rămân zone de memorie nealocate ce nu pot fi folosite
- Apare însă fragmentarea *internă* a memoriei – poate rămâne spațiu nefolosit, dar alocat proceselor (deoarece dimensiunea procesului nu este un multiplu exact de lungimea paginilor)

➤ **Paginarea (cont.)**

- Adresele logice sunt formate din 2 componente:
numărul de pagină (p) și deplasamentul în pagină (d)
- Adresele fizice sunt formate din 2 componente:
numărul de cadru (f) și deplasamentul în cadru (d)
- Pentru mapare (i.e. translatarea adreselor logice în adrese fizice) se folosește o *tabelă de pagini* per proces, care păstrează pentru fiecare pagină, adresa de bază a cadrului asociat ei (în care este încărcată pagina respectivă)
- Mai precis, pentru translatare se folosește numărul de pagină drept index în tabela de pagini, conform schemei urmatoare

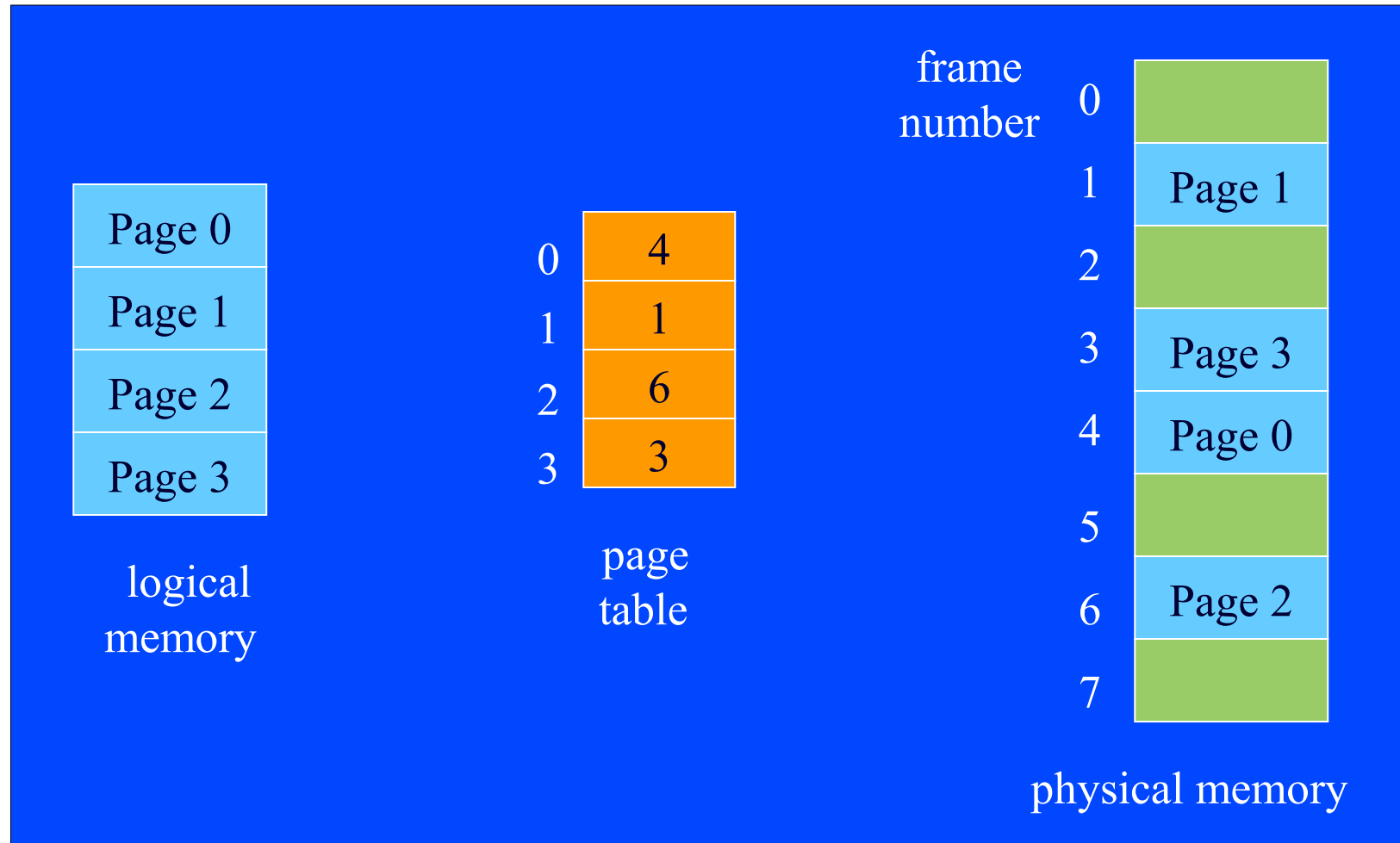
Alocarea paginată /4

Translatarea adresei logice în adresă fizică:



Alocarea paginată /5

Exemplu:

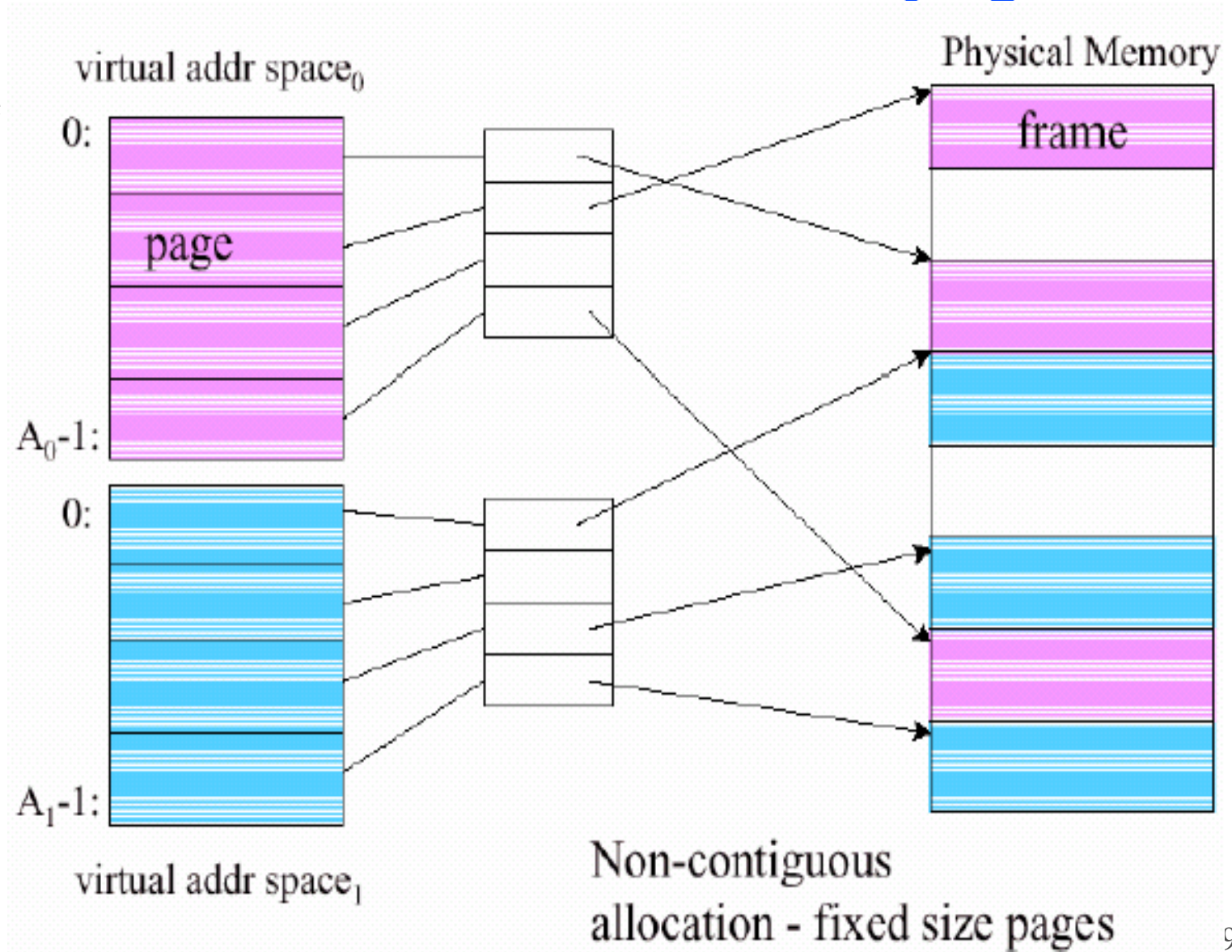


➤ **Paginarea (cont.)**

- Alt avantaj: suportă partajarea memoriei – două procese pot “vedea” aceeași zonă de memorie încercând paginile lor virtuale în același cadru fizic
- **Cod reentrant** = cod nemodificabil (poate fi partajat)
- Dezavantaj: programul trebuie, încă, să fie rezident în întregime în memoria principală
 - Înlăturarea acestui neajuns: *paginarea la cerere* (folosind tehnica de swapping, la nivel de pagină)
- Alt dezavantaj: fiecare acces de memorie presupune un acces suplimentar la tabela de pagini pentru calculul de adresă

Alocarea paginată /7

➤ Exemplu de alocare paginată (la cerere)



Alocarea paginată /8

- **Implementări pentru tabelele de pagini**
 - Păstrarea tabelelor de pagini se face în:
regiștri hardware sau memoria principală
 - Soluția cu regiștrii hardware este mult mai rapidă, dar devine prea costisitoare pe măsură ce crește dimensiunea tabelului de pagini;
în plus, accesul regiștrilor hardware este privilegiat
 - Problema soluției cu memoria principală: accese multiple la memorie pentru a accesa o dată din memorie (întâi trebuie accesată tabela de pagini, pentru aflarea adresei fizice asociate adresei logice dorite, și abia apoi se accesează memoria fizică la adresa aflată pe baza translatării)

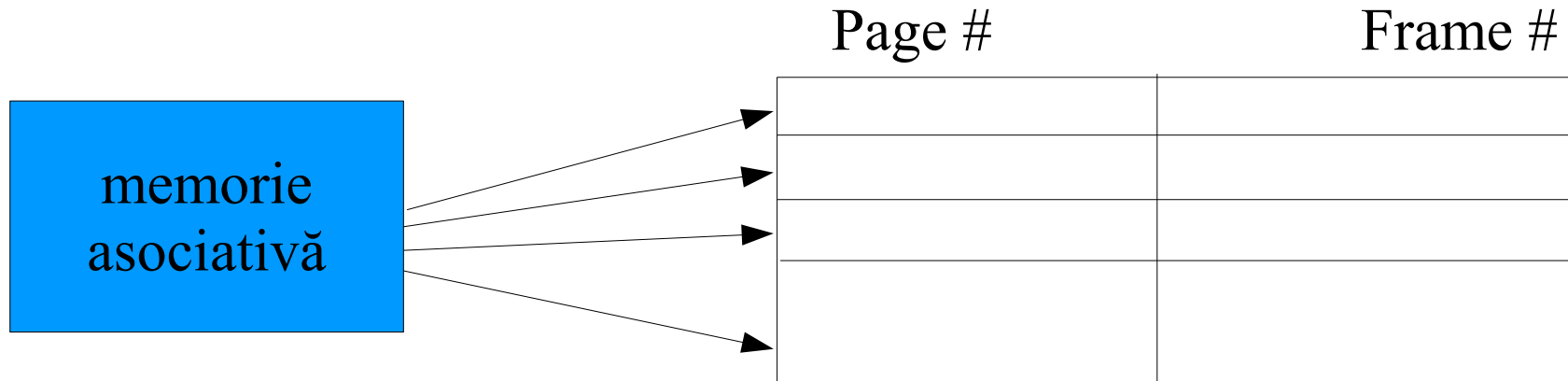
➤ Tabela de pagini în memorie

- O tabelă de pagini per proces
- Un registru, **PTBR** (*page table base register*), conține adresa de bază (de început) în memorie a tablei de pagini (e.g. registrul CR3 în cazul CPU x86)
- Un alt registru, **PTLR** (*page table length register*), indică dimensiunea tablei de pagini
- Problemă: fiecare acces la date sau instrucțiuni “costă” *două* accese de memorie
- Soluția: **TLB** (*translation look-aside buffer*)

Alocarea paginată /10

➤ **Traslation Look-aside Buffer (TLB)**

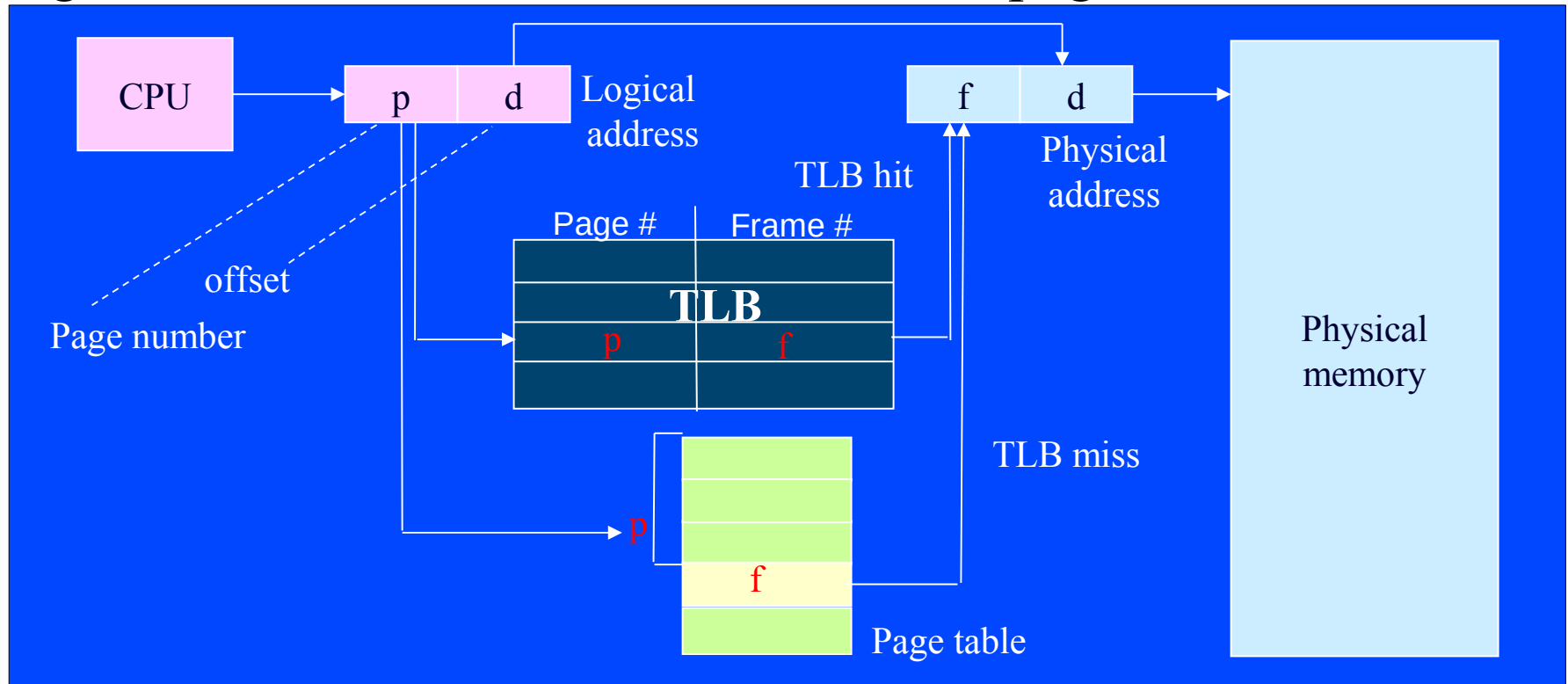
- Este o memorie specială de dimensiune mică, numită *memorie asociativă* (hardware special, scump)
- Calitatea ei fundamentală este *adresarea prin conținut*, și nu prin adresă: găsește locația care are un conținut specificat, căutând simultan (i.e., *în paralel*) în toate locațiile ei



Alocarea paginată /11

➤ Translation Look-aside Buffer (TLB) (cont.)

- TLB are rol de *cache* pentru tabela de pagini din memorie: când se încearcă aflarea numărului de cadru asociat unui număr de pagină virtuală, aceasta este căutată mai întâi în TLB, iar dacă nu-i găsită în TLB, este căutată în tabela de pagini din memorie



Alocarea paginată /12

➤ **Traslation Look-aside Buffer (TLB) (cont.)**

- Performanța adusă de folosirea TLB este dată de:
- **Hit_ratio** = procentajul de găsiți în TLB a paginii căutate
- *Timpul efectiv de acces:*

$$\text{EAT} = 1 * \text{Memory_AT} + \left(\text{Hit_ratio} * \text{TLB_AT} + \right. \\ \left. + (1 - \text{Hit_ratio}) * (\text{TLB_AT} + \text{Memory_AT}) \right)$$

- Valori uzuale: **Memory_AT** >> **TLB_AT** și **Hit_ratio** ≈ 90-95% și ca urmare **EAT** este foarte apropiat de **1 * Memory_AT**
- În tabela de pagini, paginile mai au asociați și alți biți de informație folosiți pentru **protecția memoriei** - restricții de acces (ReadOnly, ReadWrite, Execute, etc.)

➤ **Tabele de pagini pe nivele multiple**

- Tabelele de pagini pot avea dimensiuni mari
- Presupunem un spațiu de adresare de 2^{32} octeți (cca. 4 Go)
- Dacă cadrul de pagină are 4 Ko, atunci numărul de intrări în tabela de pagini este de cca. **un milion!**
- O tehnică utilizată pentru reducerea dimensiunii tablei constă în utilizarea unei **tabele de pagini pe nivele multiple**

Alocarea paginată /14

➤ Tabele de pagini pe nivele multiple - exemplu:

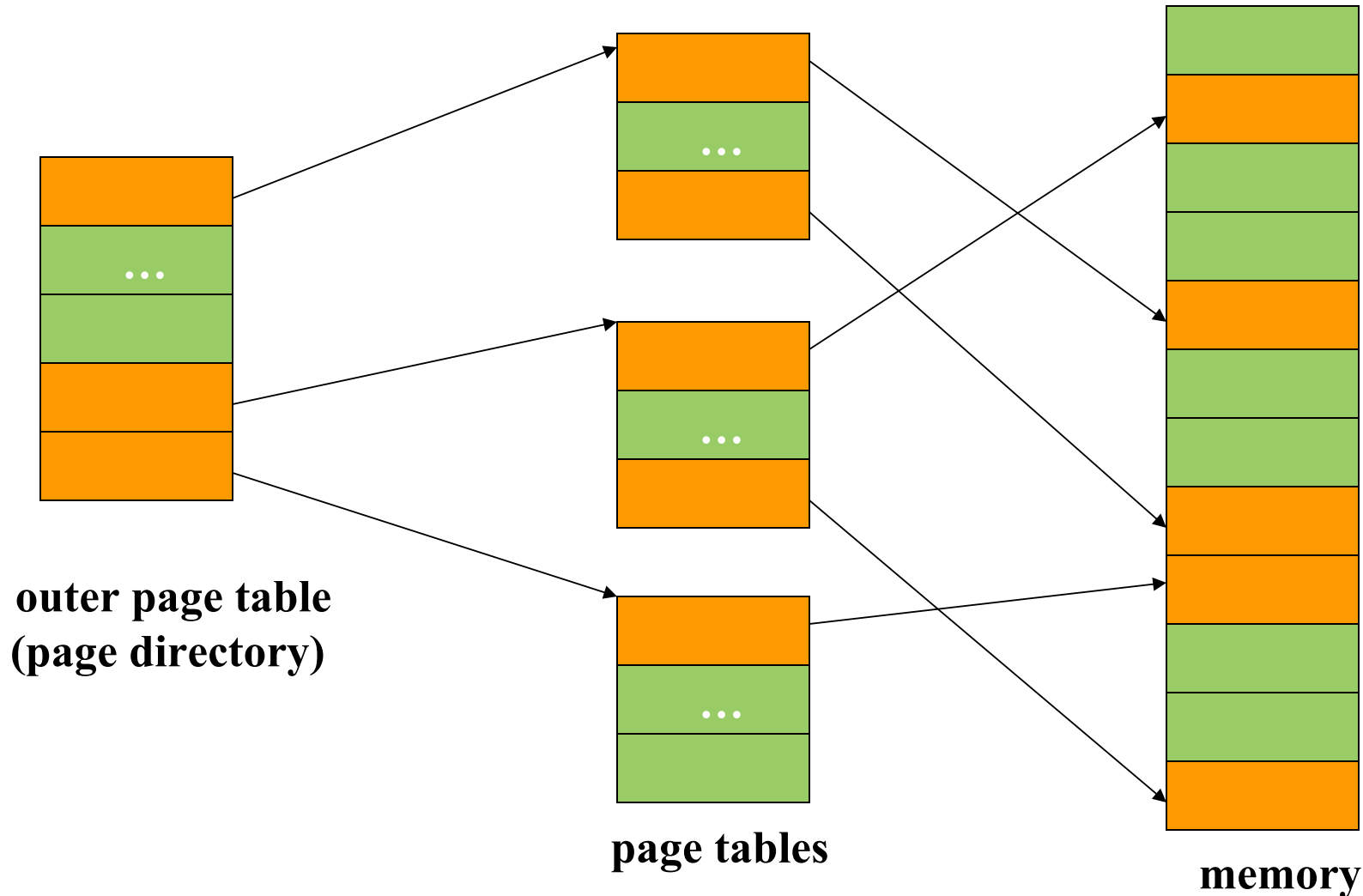
- Spațiul de adrese: 2^{32} octeți, dimensiunea paginii: 4096 octeți
- Un milion de intrări în tabela de pagini
- O adresă logică arată astfel:

numărul de pagină (pe 20 biți)	deplasamentul (pe 12 biți)
-----------------------------------	-------------------------------

- Partiționăm tabela de pagini în 4 “bucăți” (secțiuni), fiecare secțiune având 256K intrări
- Alocăm numai câte secțiuni sunt necesare, nu pe toate 4 !
- Acum o adresă logică arată astfel:

nr. secțiune (pe 2 biți)	numărul de pagină (pe 18 biți)	deplasamentul (pe 12 biți)
-----------------------------	-----------------------------------	-------------------------------

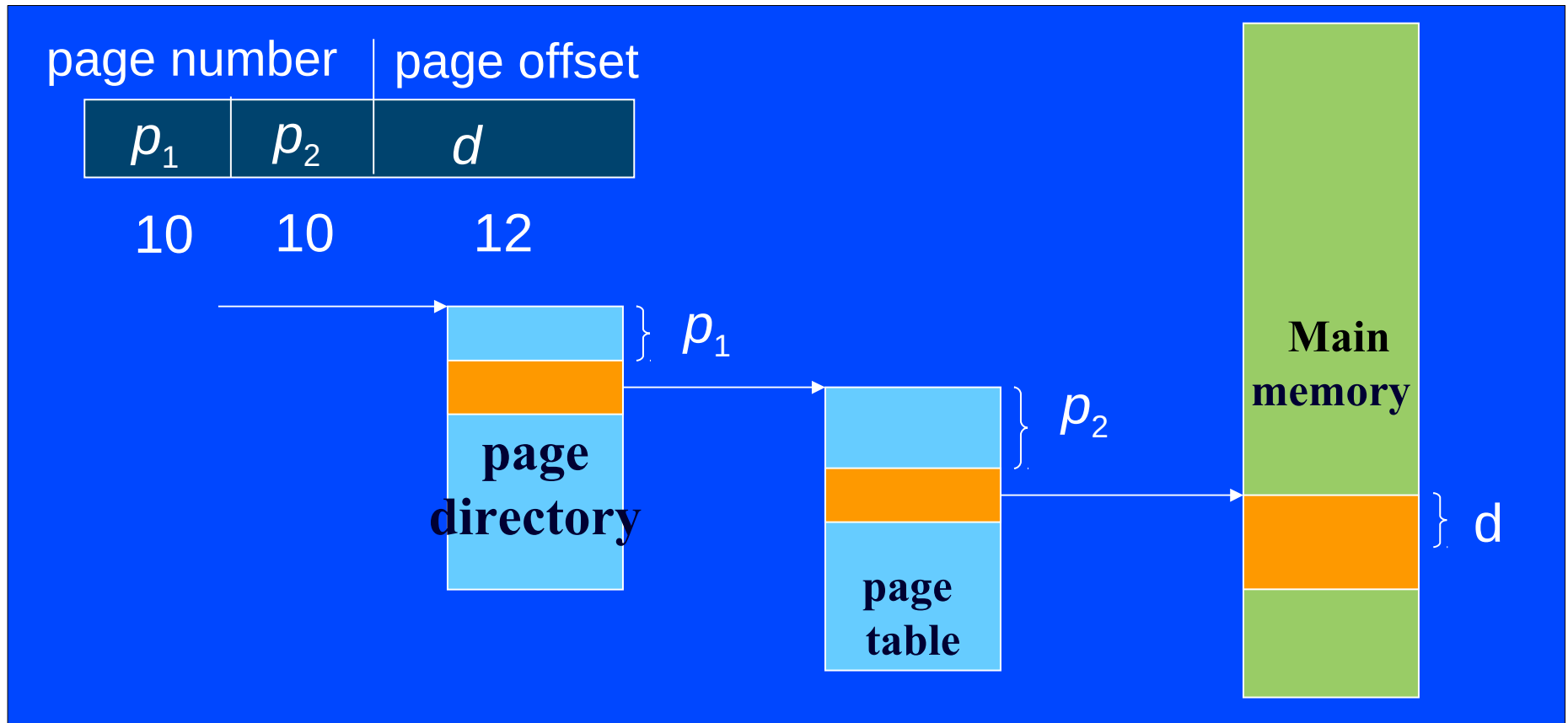
➤ Tabele de pagini pe nivele multiple - exemplu:



Alocarea paginată /16

➤ Tabele de pagini pe nivele multiple (cont.)

Translatarea adreselor pentru un CPU pe 32biți cu 2 nivele de paginare:



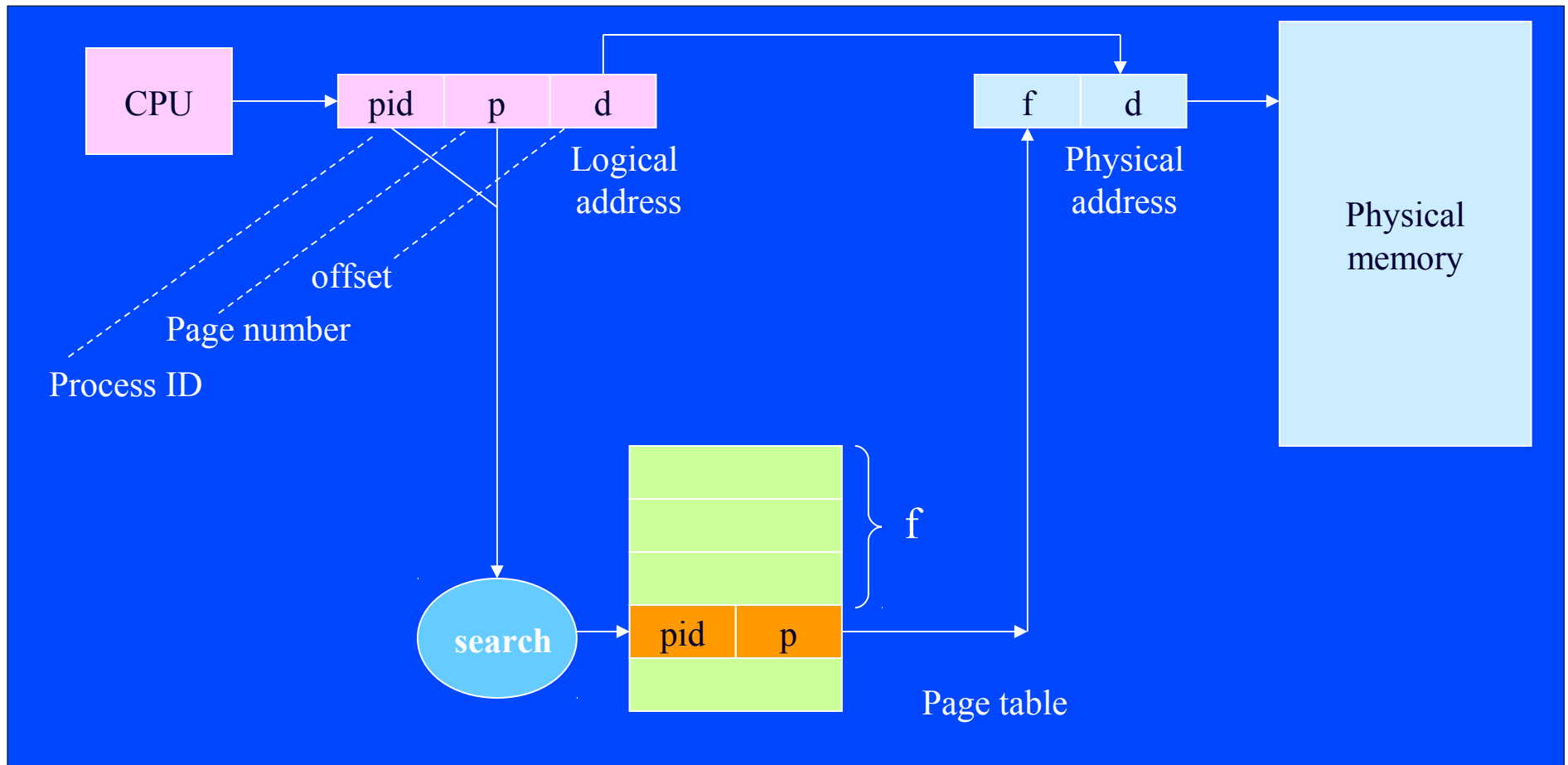
➤ **Tabele de pagini inverse**

(utilizate de sistemele IBM RT, IBM AS/400, HP-UX)

- O altă modalitate de reducere a dimensiunii tabelelor de pagini stocate în memorie
- În loc de a avea o intrare în tabelă pentru fiecare pagină virtuală, avem câte o intrare pentru fiecare pagină fizică, ce conține o pereche de forma: (PID, număr pagină virtuală)
- Când se translatează o adresă virtuală, se produce o căutare în tabela inversă de pagini a numărului de pagină virtuală dorit și se returnează numărul paginii fizice asociate, reprezentat de indexul intrării găsite

Alocarea paginată /18

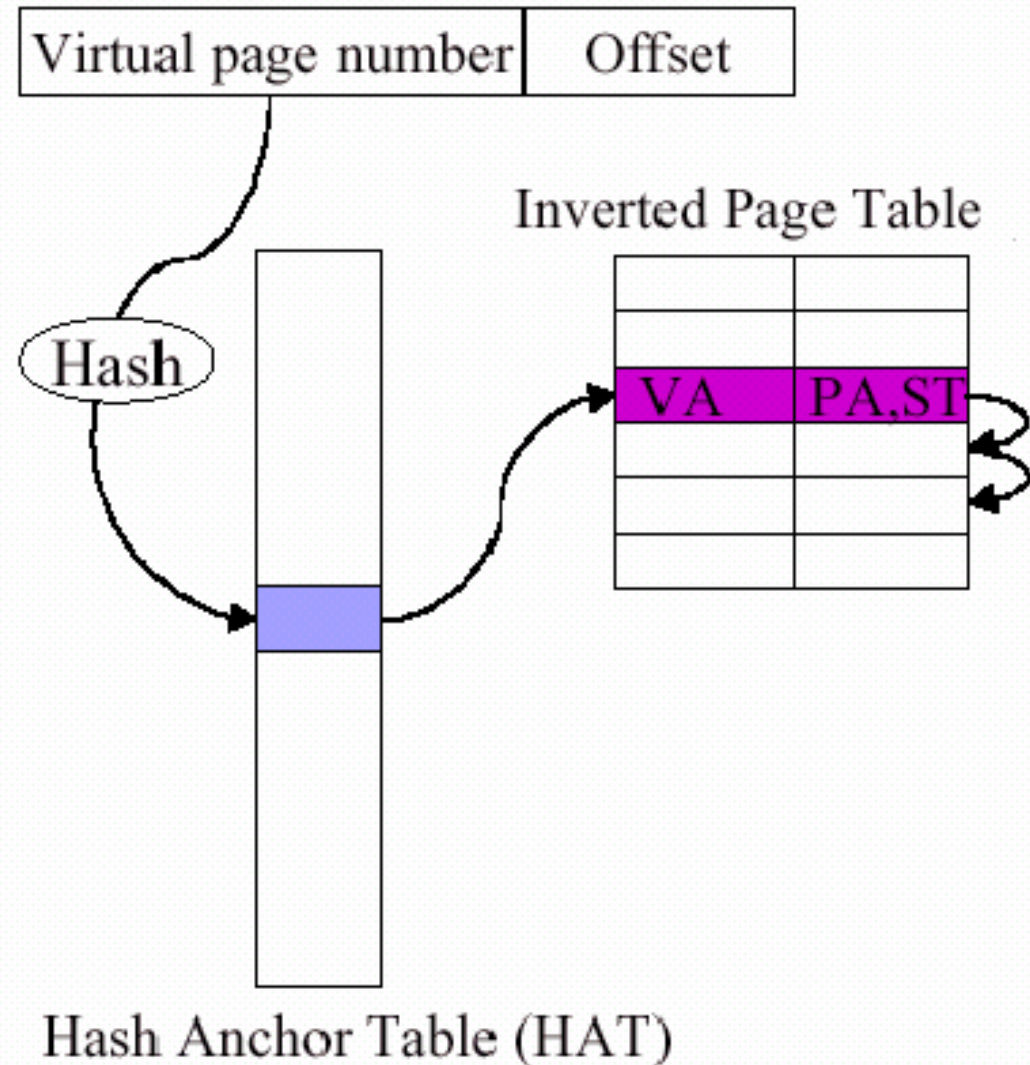
➤ Tabele de pagini inverse (cont.)



Alocarea paginată /19

➤ Tabele de pagini inverse – dezavantaje:

- Căutarea este înceatăă (pentru că trebuie căutată întreaga tabelă)
 - Soluție: utilizarea tehnicii de *hashing*
- Prezintă dificultăți în suportul memoriei partajate (deoarece permite o singură adresă virtuală pentru o pagină fizică)



Segmentarea /1

- **Segmentarea** = “paginare” în segmente (i.e. pagini de dimensiuni variabile)
- Segmentarea reprezintă o vedere a memoriei d.p.d.v. al utilizatorului – o mulțime de “bucăți” de memorie de diverse dimensiuni:
 - programul principal
 - procedură
 - funcție
 - stivă
 - vector / matrice

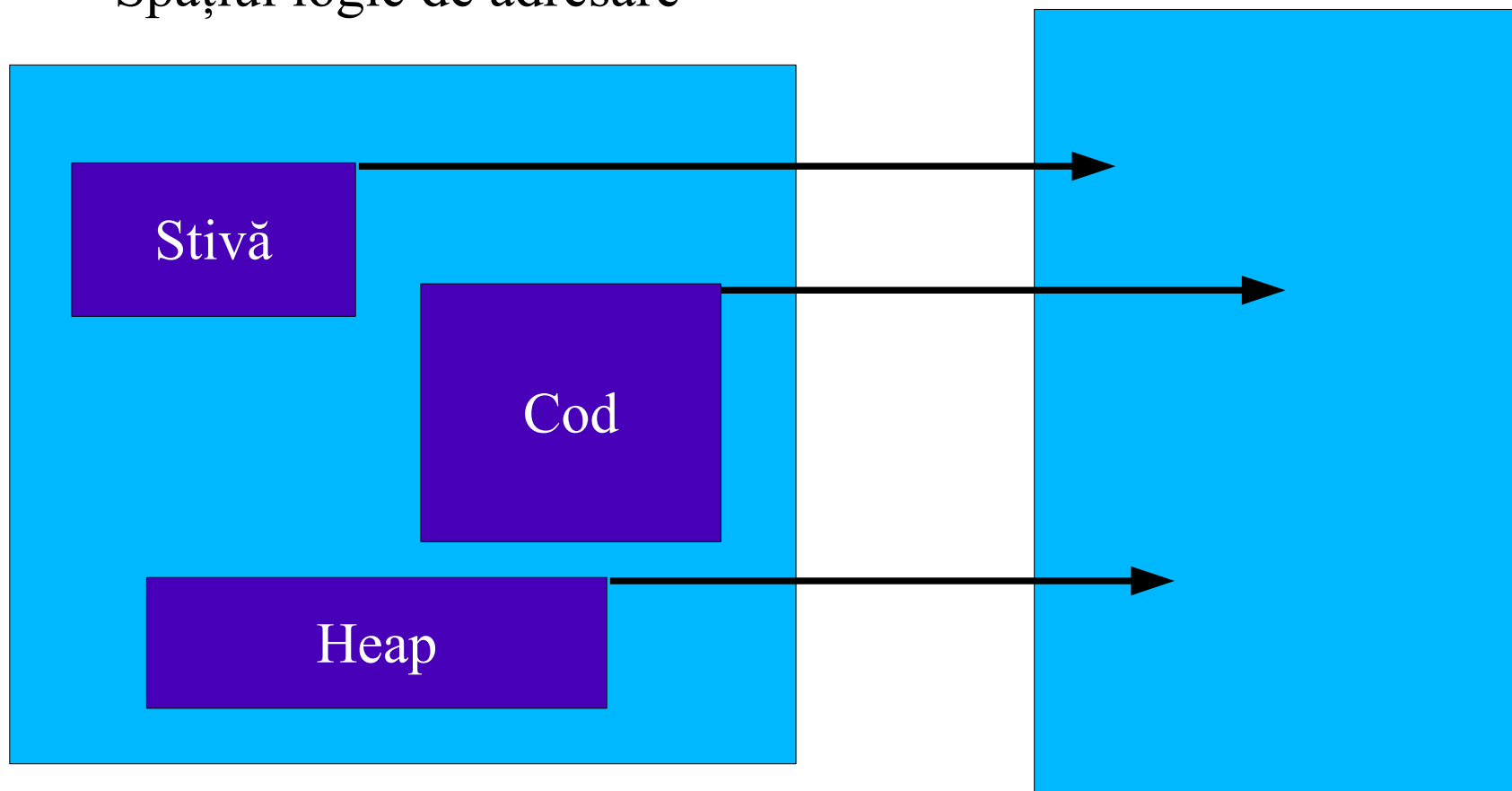
Segmentarea /2

- Vederea utilizatorului asupra memoriei nu este aceeași cu organizarea memoriei fizice a SC
- Fiecare din modulele și elementele de date ale programului (proceduri, funcții, stive, vectori, ...) este referit prin nume, fără ca utilizatorul să fie interesat ce adrese în memorie ocupă aceste elemente
- Spre deosebire, la paginare se practică o împărțire după organizarea memoriei fizice, transparentă pentru utilizator

Segmentarea /3

Spațiul logic de adresare

Memoria reală



➤ **Segmentarea (cont.)**

- Este o schemă de administrare a memoriei care suportă vederea utilizatorului asupra memoriei – programul este divizat în mai multe unități logice
- Spațiul logic de adresare al programului constă dintr-o colecție de **segmente** (câte un segment pentru fiecare unitate logică din program)
- Fiecare segment are un nume și o dimensiune
- Adresele logice specifică atât numele segmentului, cât și deplasamentul în cadrul segmentului

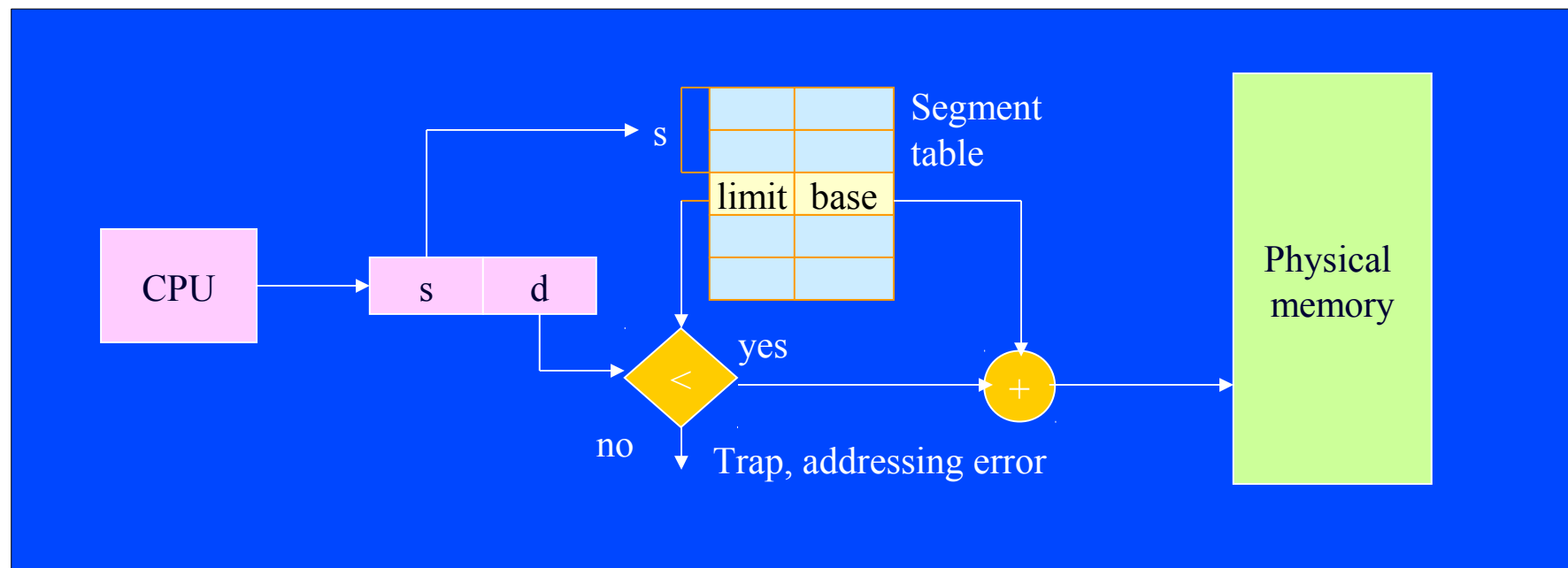
➤ Segmentarea (cont.)

- Programatorul specifică fiecare adresă prin două componente: numele segmentului și deplasamentul, i.e. $\text{adresa virtuală} = (\text{segment}, \text{deplasament})$
- Deci programatorul vede spațiul virtual de adresare al programului ca un spațiu 2-dimensional, nu ca un spațiu 1-dimensional ca la celelalte metode
- Fiecare proces are o *tabelă de segmente*, ce conține adresa de început a segmentului în memoria fizică

➤ Segmentarea (cont.)

– Calculul adresei fizice se face ca la paginare:

$\text{adresa fizică} = \text{adresa segmentului} + \text{deplasamentul}$



➤ Implementarea segmentării

- CPU generează adrese virtuale: (nume segment, deplasament)
Intel x86: segmentele **CODE**, **DATA**, **STACK**
- Se caută în tabela de segmente adresa de început în memorie a segmentului, iar apoi se adună deplasamentul pentru a obține locația dorită din memoria fizică
- Ca și la paginare, se folosesc regiștri hardware care păstrează translatarea segmentelor în adrese reale
Intel x86: registrul **CS** – adresa segmentului de cod, **DS** – adresa segmentului de date, **SS** – adresa segmentului de stivă, **ES** – adresa segmentului de date suplimentar (extra-segmentul)
- Probabil nu toate translatarea pot fi păstrate în regiștri, dacă există un număr mare de segmente

➤ Segmentarea - avantaje:

- Divizarea logică a spațiului virtual: toate porțiunile unui segment au probabil același înțeles semantic
- Memoria reală este împărțită în fragmente logice
- Partajarea memoriei este simplă: mai multe segmente care au aceeași adresă fizică; în particular, partajarea *codului reentrant*
- Protecția memoriei – fiecare segment primește anumite drepturi de acces, trecute în tabela segmentelor; la fiecare calcul de adresă se pot face și astfel de verificări
- Alte lucruri, precum gestiunea limitelor vectorilor se pot face mai simplu prin această metodă de alocare a memoriei

➤ Segmentarea - dezavantaje:

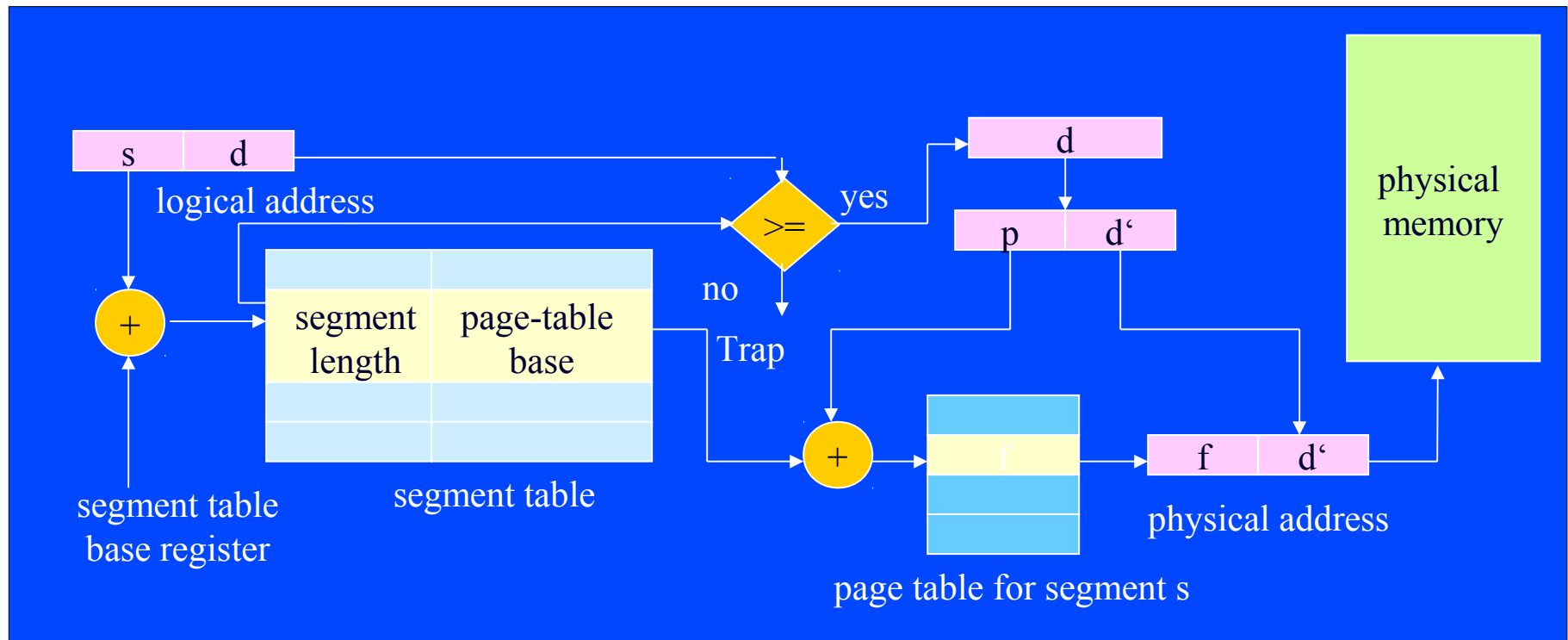
- Segmentele trebuie să încapă în memorie, deci nu pot avea lungimi mai mari decât dimensiunea memoriei fizice
- Necesitatea ca fiecare segment să fie alocat într-o porțiune contiguă a memoriei fizice, plus dimensiunile variabile ale segmentelor, fac posibilă apariția *fragmentării externe* a memoriei
 - Pentru a înlătura fragmentarea se poate utiliza *compactarea* (defragmentarea memoriei)
 - O altă soluție: segmentarea paginată

➤ Segmentarea paginată

- Introdusă de S.O.-ul MULTICS ('65-'69, AT&T+GE+MIT)
- Fiecare segment este împărțit în pagini
- Astfel se elimină două dezavantaje ale segmentării pure: alocarea contiguă și fragmentarea externă
- Fiecare proces are o *tabelă de segmente* (care conține referințe către tabelele de pagini asociate segmentelor), iar fiecare segment are o *tabelă de (mapare a) paginilor* (în cadre de pagină din memoria fizică)
- **adresa virtuală = (segment, pagină, deplasament)**
- **adresa fizică = (cadru de pagină, deplasament)**

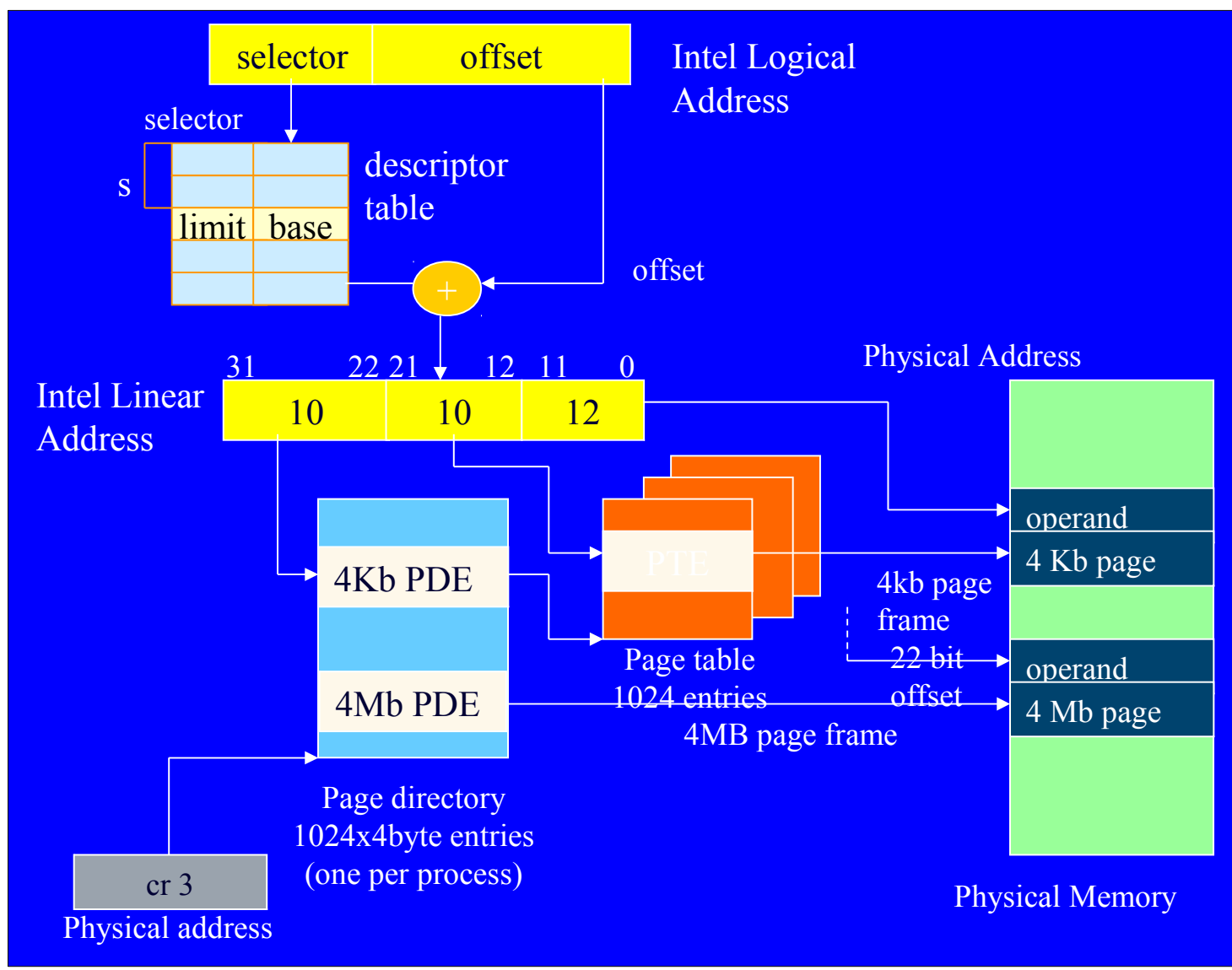
Segmentarea /11

- **Segmentarea paginată (cont.)**
 - Folosită pe calculatorul GE 645 (cu S.O.-ul MULTICS)
 - Adrese logice = Nr. segment: 18 biți + deplasament: 16 biți
 - Utiliza un număr relativ mic de segmente de 64 Ko



➤ Segmentarea paginată (cont.)

Segmentarea /12



µP-ul Intel 386
permite prin hardware
administrarea
memoriei folosind
segmentarea cu
paginare pe 2 nivele.

Memorie virtuală

➤ Memoria virtuală

- Conceptul de memorie virtuală
- Principiul localității

(va urma)

- Paginarea la cerere
- Algoritmi de înlocuire a paginilor
- Fenomenul de *trashing*
- Segmentarea la cerere

Memorie virtuală /1

- **Memoria virtuală:** păstrează separarea spațiului de adresare logic (virtual) de cel fizic (real), introdusă de tehnicile de paginare și de segmentare pure, dar ...
- Acum, o parte din spațiul logic de adrese poate fi rezident, în fapt, pe disc (în situația în care memoria fizică este prea mică pentru a putea cuprinde în întregime spațiile logice de adrese ale tuturor proceselor din sistem)

➤Memoria virtuală

- SC cu *memorie virtuală* au capacitatea de a adresa un spațiu de memorie mai mare decât memoria internă disponibilă
- Ideea: *swapping*-ul pe disc al unor “bucăți” de memorie (tehnică introdusă de S.O. ATLAS – 1960, Univ. Manchester, U.K.)
- Avantaje:
 - Spațiul logic de adrese poate fi mult mai mare decât memoria fizică disponibilă
 - Este nevoie ca doar o parte a programului să fie rezident în memoria principală pentru a putea fi executat (deci nu mai este nevoie să fie prezent întregul program în memorie)

➤Memoria virtuală

- Cum să o implementăm?
- Este nevoie ca “bucăți” de memorie să fie *swap*-ate, la cerere, din memorie pe disc și invers
- Implementare – tehnici de virtualizare folosite:
paginarea la cerere și **segmentarea la cerere** (e.g. OS/2)
- Presupune mutarea, controlată de sistem, în susul și în josul ierarhiei de memorii (RAM ↔ disc)
- Poate fi privită ca o automatizare a *overlay*-urilor (nu mai este sarcina programatorului de aplicații, ci a sistemului):
S.O.-ul încearcă să determine în mod dinamic care părți încărcate anterior pot fi înlocuite de părți necesare acum

➤ **Memoria virtuală**

- Metoda funcționează doar datorită “localității” referințelor la memorie
- Este adesea strâns legată de tehnica paginării / segmentării, deoarece necesită alocare necontiguă, tabelă de mapare, etc.:
paginarea la cerere / segmentarea la cerere
- Observație esențială: numai un subset din codul și, respectiv, datele unui program sunt necesare la un moment arbitrar de timp. Poate S.O.-ul să prezică care va fi acel subset la un moment următor, doar din observația comportamentului programului la momentele trecute de timp ?
→ Principiul localității (principiu euristic, desprins din practică)

Principiul localității /1

- **Principiul localității (vecinătății)** ('68 P.J. Denning)
- Două tipuri de localizare a secvențelor de accesare a memoriei:
 - **Localitate temporală** – tendința de a accesa în viitorul apropiat locații accesate deja în istoria recentă (e.g. instrucțiuni repetitive, variabile utilizate frecvent)
 - **Localitate spațială** – tendința de a accesa în viitorul apropiat locații apropiate de alte locații accesate deja în istoria recentă (e.g. instrucțiuni secvențiale, structuri de date contigue – vectori, înregistrări, etc.)
- Se justifică astfel mutarea în “bucăți” mai mari pt. swapping

Principiul localității /2

➤ Principiul localității

– Datorită lui s-a impus programarea structurată (în locul programării cu goto), precum și structurarea datelor

– *Morala*: dacă o matrice este introdusă în memorie pe linii (coloane), este bine să fie prelucrată tot pe linii (resp. pe coloane)

localitate bună:

```
for (i = 0; i++; i < n)
    for (j = 0; j++; j < m)
        A[i, j] = B[i, j]
```

localitate rea:

```
for (j = 0; j++; j < m)
    for (i = 0; i++; i < n)
        A[i, j] = B[i, j]
```

Assume:
arrays laid
out in rows

A[0,0]	A[0,1]
A[1,0]	A[1,1]
A[2,0]	A[2,1]

→

A[0,0]	A[0,1]	A[1,0]	A[1,1]	A[2,0]	A[2,1]
--------	--------	--------	--------	--------	--------

Bad locality is a contributing factor in *Thrashing* (page faulting behavior dominates).

Va urma:

- **Tehnici de implementare**
 - Paginarea cu încărcare la cerere
(= paginarea combinată cu tehnica de *swapping*)
 - Segmentarea cu încărcare la cerere
(= segmentarea combinată cu tehnica de *swapping*)
- **Algoritmi de *swapping* a paginilor/segmentelor**

- **Bibliografie obligatorie**

capitolele despre *gestiunea memoriei* din

- Silberschatz : “*Operating System Concepts*”

(cap.7, ultima parte: §7.4–7, din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.3, §3.3 și §3.7, din [MOS3])

Alocarea memoriei (continuare)

- Scheme de alocare necontigue
 - Paginarea
 - Segmentarea
 - Segmentarea paginată

(va urma)

- Scheme de alocare cu memorie virtuală

Întrebări ?

Sisteme de Operare

Administrarea memoriei – partea III

Cristian Vidrașcu

<http://www.info.uaic.ro/~vidrascu>

- Memoria virtuală (continuare)
 - Paginarea la cerere
 - MMU
 - Algoritmi de înlocuire a paginilor
 - Fenomenul de *trashing*
 - Segmentarea la cerere
- Concluzii legate de tehnicile de administrare a memoriei
- Metode de viitor pentru administrarea memoriei
- Administrarea memoriei în Linux și Windows

➤ **Paginarea (cu încărcare) la cerere**

= paginarea combinată cu tehnica de *swapping*

- **Ideea:** aducerea paginilor de pe disc în memorie doar în momentul când sunt referite (“când e nevoie de ele”)
- Astfel se elimină restricția ca programul să fie prezent în întregime în memorie pentru a putea fi executat
- Motive de eliminare a acestei restricții:
 - programele conțin zone de cod ce tratează cazuri mutual-exclusive (în funcție de datele de intrare), sau zone care se execută la momente de timp mutual-exclusive
 - unele programe conțin zone de date f. mari (e.g. compilatoarele)
 - programele au multe rutine de eroare, ce sunt apelate doar dacă apar erori

➤ Paginarea la cerere (cont.)

– Avantaje:

- programul este prezent doar parțial în memorie
- mai puține operații I/O decât la transferul proceselor în întregime (de la schema de administrare cu *swapping* pur)
- mai puțină memorie necesară la un moment dat
- mai multe procese la un moment dat (crește gradul de multi-programare)
- nu mai e nevoie de tehnica de programare a *overlay*-urilor, efortul programatorului fiind preluat de S.O.

– Dezavantaje:

- complexitatea hard și soft a acestui mecanism de gestiune a memoriei

➤ **Paginarea la cerere (cont.)**

- Când este referită o pagină, trebuie verificat dacă:
 - Este o **referință** validă?
Cu alte cuvinte, procesul are dreptul de a accesa
acea adresă?
 - Pagina referită este în memorie?
Dacă nu, caută un cadru de pagină liber și încarcă
pagina de pe disc în el.

Paginarea la cerere /4

➤ Paginarea la cerere (cont.)

- Pentru a păstra evidența paginilor aflate în memorie, se asociază, pentru fiecare pagină, un bit **validă sau invalidă** în *tabela de mapare a paginilor*

Pagina	Numărul cadrului	Bit (in)validă
1	300	1
2	85	0

➤ **Erori de pagină**

- Semnificația bitului: 1 – pagina este prezentă în memorie; 0 – pagina nu este în memorie
- **Eroare de pagină** (*page fault*): atunci când se încearcă accesarea unei pagini marcate ca invalidă (bitul este 0); pagina nu este în memorie și va trebui adusă de pe disc
- Se generează o **întrerupere de pagină** (PFI=*page fault interrupt*), de tip sincron, care este transparentă pentru utilizator și are o prioritate superioară celorlalte întreruperi; ea va întrerupe execuția programului și S.O.-ul va executa handler-ul asociat acestei întreruperi

➤ **Erori de pagină (cont.)**

– Rutina de tratare a întreruperii de pagină execută:

- Se examinează adresa solicitată pentru a vedea dacă este o adresă permisă; dacă nu este, procesul va fi terminat anormal
- Se caută un cadru de pagină liber în memoria principală și se solicită o operație I/O care va aduce pagina de pe disc în cadrul liber găsit; pentru aceasta se mai folosește și o *tabelă de mapare pe disc*, ce conține adresa de pe disc a fiecărei pagini virtuale
- După încărcarea paginii, se actualizează *tabela de mapare a paginilor* pentru a indica faptul că pagina este validă
- Apoi este reluată execuția procesului – instrucțiunea oprită este restartată (toată această procedură este transparentă pt. proces)

➤ **Erori de pagină (cont.)**

- Este posibil să apară *mai multe* erori de pagină la execuția unei singure instrucțiuni – cazul extrem: codul instrucțiunii “călare” pe 2 pagini, cu 2 operanzi fiecare “călare” pe 2 pagini, cu adresare indirectă/indexată – pot apare 3 întreruperi PFI
- Inconvenientul acestei metode: *overhead*-ul implicat în cazul erorilor de pagină (necesită accese la disc, plus folosirea CPU pentru ajustarea tabelelor)
- O memorie fizică mică și multe procese în sistem, ori o localitate proastă a datelor și/sau codului, poate conduce la un număr f. mare de întreruperi PFI, deci la apariția fenomenului de *trashing* (i.e. fenomenul de sufocare a SC-ului cu rezolvarea erorilor de pagină, în loc de a-și folosi resursele pentru execuția job-urilor utile)

➤ **Paginarea la cerere (cont.)**

- O altă problemă: la o eroare de pagină, ce facem dacă nu găsim nici un cadru fizic liber?
- Soluția: sacrificarea unei pagini din memorie – mutarea ei pe disc pentru a elibera spațiu în memorie pt. pagina ce se încarcă
- Cum alegem victima pentru *page swapping*?
Există mai multe criterii de alegere, mai mulți algoritmi pe care-i vom discuta puțin mai târziu
- Ideea: folosirea unui algoritm de înlocuire a paginilor care **să minimizeze numărul de erori de pagină** (i.e., de întreruperi PFI)
- În plus, se poate preveni supra-alocarea de memorie unui anumit proces – rutina de tratare a PFI poate decide să facă niște înlocuiri chiar dacă mai sunt cadre fizice libere

➤ **Paginarea la cerere (cont.)**

- O altă problemă: pe lângă bitul de pagină (in)validă, mai este nevoie de un bit de “tranzit” care să indice starea de pagină în curs de încărcare de pe disc în memorie
- Motivul: transferul de pe disc durează f. mult, timp în care CPU execută alt proces, și este posibil ca acesta să aleagă ca victimă pentru sacrificiu tocmai pagina în curs de încărcare
- Regulă: când aleg victima, evit paginile aflate în starea de tranzit
- *Observație*: tabela de mapare a paginilor unui proces este păstrată și ea în memoria acelui proces; pagina ce conține tabela nu trebuie evacuată din memorie de către alg. de *page swapping*, i.e. ea este “încuiată” pe toată durata execuției procesului, pe când paginile în tranzit sunt “încuiate” doar temporar, pe durata încărcării

Paginarea la cerere /10

➤ **Paginarea la cerere (cont.)**

- O altă optimizare: fiecărei pagini i se asociază în tabela de mapare a paginilor un al 3-lea bit, bitul *dirty*, care determină dacă pagina a fost vreodată modificată de la ultima încărcare de pe disc
- Inițial, la încărcare, bitul *dirty* este pus pe zero, iar apoi orice scriere în acea pagină îl setează pe 1 (se face $\text{bit} := \text{bit} \text{ or } 1$)
- Rostul acestui bit *dirty* – optimizarea operațiilor I/O: dacă pagina nu a fost modificată de la ultima încărcare de pe disc, atunci ea nu mai trebuie transferată pe disc atunci când este aleasă drept victimă de către algoritmul de *page swapping*

Paginarea la cerere /11

- **Concluzii – întrebări legate de paginarea la cerere:**
 - Cum împiedicăm utilizatorii să acceseze datele protejate?
 - drepturi de acces specificate la nivel de pagină
 - Dacă o pagină este prezentă în memorie, cum o găsim?
 - tabela de mapare a paginilor (în memoria fizică)
 - Dacă o pagină nu este prezentă în memorie, cum o găsim?
 - tabela de mapare (a paginilor) pe disc
 - Când este adusă o pagină în memorie?
 - politici de încărcare – la cerere (atunci când este nevoie de ea)
 - Dacă o pagină este adusă în memorie, unde o punem?
 - politici de plasare
 - Dacă o pagină este evacuată din memorie, unde o punem?
 - Cum decidem care pagini să fie evacuate din memorie?
 - politici de înlocuire – algoritmi de *page swapping*

Paginarea la cerere /12

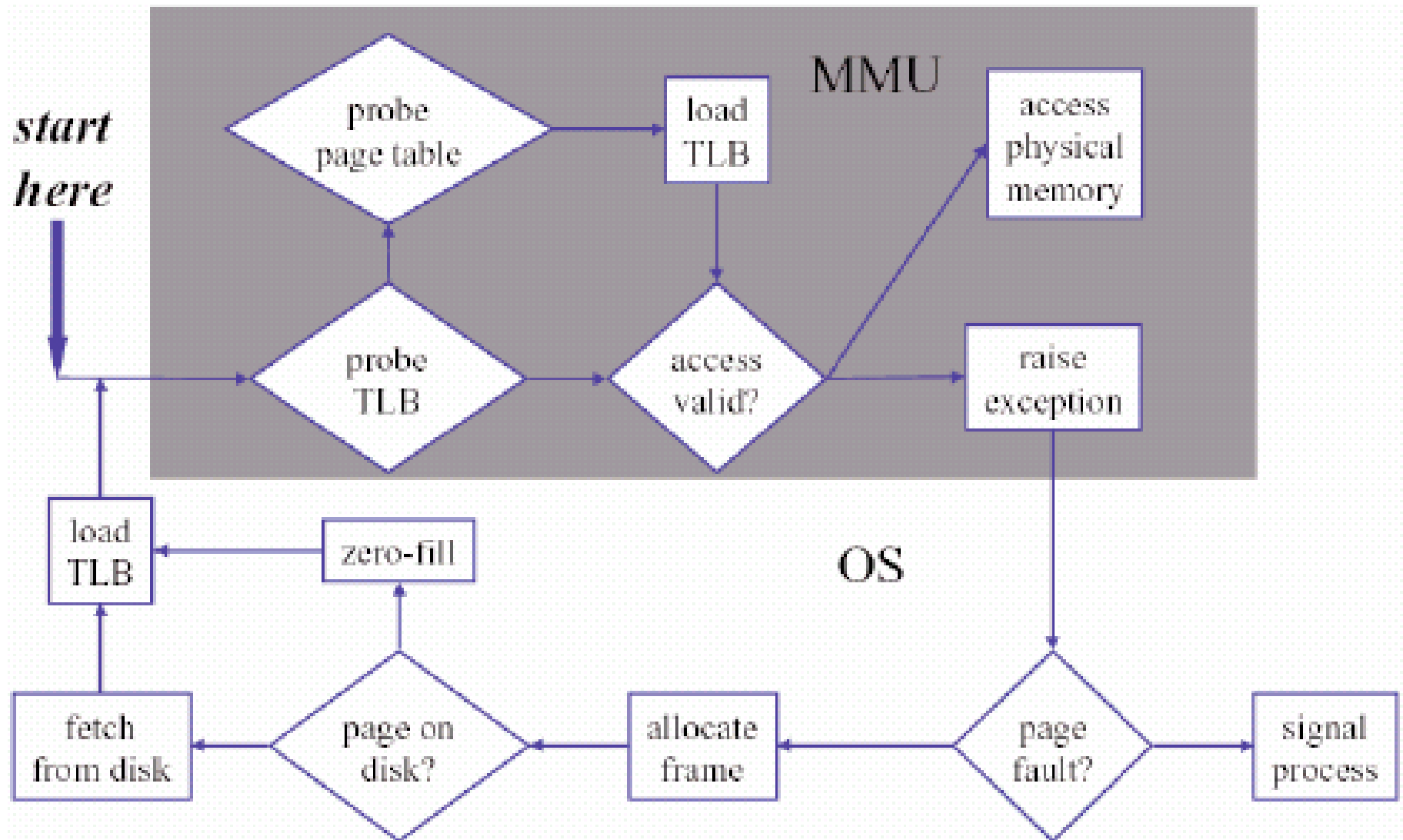
- **Mecanisme – pentru suportul paginării la cerere:**
 - **Suportul hardware** – pe lângă translatarea dinamică a adreselor necesară pentru suportul paginării sau segmentării (e.g. tabelele de mapare):
 - Mecanism de generare a erorilor de pagină (i.e. PFI) în situația accesării paginilor absente din memorie
 - Instrucțiuni restartabile
 - **Suportul software:**
 - Structuri de date pentru suportul politicilor de înlocuire, încărcare și plasare a paginilor în memorie
 - Structuri de date pentru localizarea în memoria secundară (i.e., pe disc) a paginii dorite

- **MMU (= Memory Management Unit)**
 - Input: adrese virtuale
 - Output: adresele fizice asociate, sau situații excepționale – de violare a accesului (o excepție, ce întrerupe procesorul)
 - Situații de violare a accesului:
 - pagină absentă din memorie (excepție PFI)
 - acces în mod utilizator vs. mod kernel
 - lipsa dreptului de read a memoriei
 - lipsa dreptului de write a memoriei
 - lipsa dreptului de execute a memoriei (pt. paginile de cod)

➤ MMU (cont.)

- S.O.-ul controlează operarea MMU pentru a selecta:
 1. Submulțimea de posibile adrese virtuale ce sunt valide pentru fiecare proces (i.e., spațiul virtual de adrese al procesului)
 2. Traslatările (mapările) fizice pentru acele adrese virtuale
 3. Modurile de acces permis la acele adrese virtuale (read/write/execute)
 4. Setul specific de translatări valabil la un moment dat (este necesar un *context-switch* rapid de la un spațiu de adrese la altul)
- MMU finalizează un acces la memorie numai dacă S.O.-ul spune că “referința e OK” (altfel, MMU produce o excepție)

➤ **MMU – finalizarea unui acces la memorie:**



Alg. de page swapping /1

- **Algoritmi de înlocuire a paginilor**
 - Alg. de înlocuire a unei pagini neutilizate recent: NRU
 - Alg. de înlocuire în ordinea încărcării: FIFO
 - Alg. de înlocuire a celei mai puțin utilizate recent pagini: LRU
 - Aproximări LRU pentru implementări
 - Alte abordări
 - Implementări reale

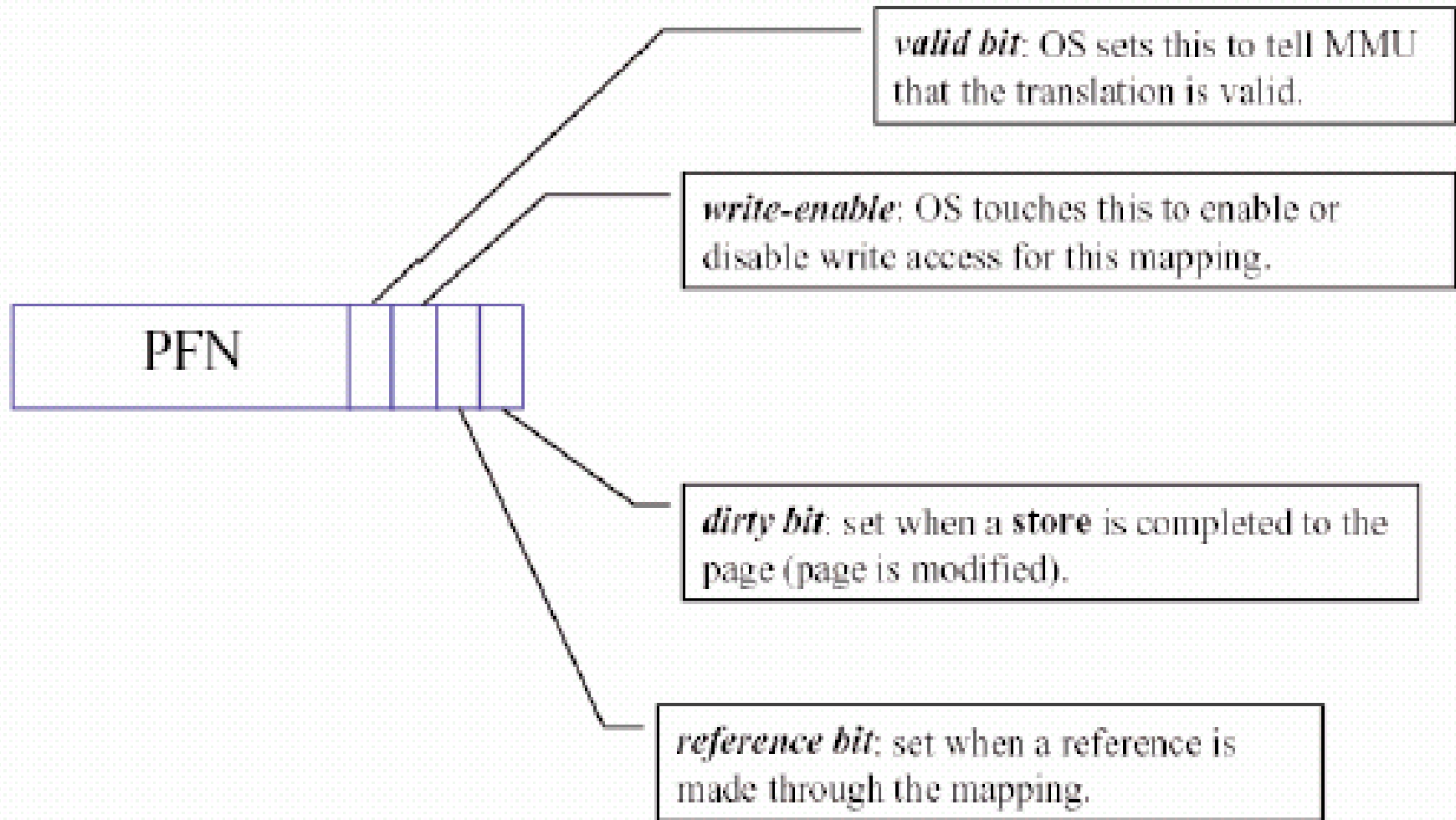
Alg. de page swapping /2

➤ Politica de înlocuire a paginilor

- Când nu mai sunt disponibile cadre de pagină libere, S.O.-ul trebuie să înlocuiască o pagină (**victimă**), înlăturând-o din memorie pentru a rămâne rezidentă doar pe disc (**depozitul pentru copia de siguranță**) și scriindu-i conținutul înapoi pe disc dacă fusese modificată după încărcarea în memorie (**pagina dirty**)
- **Algoritmul de înlocuire** – are drept scop alegerea celei mai bune victime, metrica utilizată de obicei pentru “cea mai bună” fiind reducerea ratei erorilor de pagină (i.e., se urmărește minimizarea numărului de întreruperi PFI)

Alg. de page swapping /3

➤ Intrările în tabela de mapare a paginilor



Alg. de page swapping /4

- **Problema *caching*-ului paginilor** (politica de înlocuire)
 - Fiecare thread/proces “produce” un flux de referințe la paginile virtuale din propriul spațiu de adresare
 - Vom modela execuția ca o **secvență de referințe la pagini**:
e.g. “1,2,3,1,2,3,4,2,3,5,...”
 - S.O. încearcă să minimizeze numărul de erori de pagină produse
 - **Mulțimea de lucru** (*working set*) = mulțimea de pagini folosite efectiv de fiecare proces, se modifică în timp relativ încet
 - S.O. încearcă să aranjeze mulțimea de pagini rezidente în memorie a fiecărui proces activ astfel încât să aproximeze cât mai bine mulțimea sa de lucru
 - Factorul determinant pt. succes este politica de înlocuire folosită
 - Aceasta determină **mulțimea de pagini rezidente** în memorie

Alg. de page swapping /5

➤ Algoritmul optim (inutilizabil)

- Răspunsul optim la întrebarea **care** pagină o aleg drept victimă? este simplu, dar imposibil de realizat:
se alege acea pagină care va fi solicitată cel mai târziu în viitor
- Inutilizabil deoarece răspunsul nu poate fi prevăzut în avans – evoluția unui program nu este previzibilă, ea depinde de datele concrete asupra cărora operează
- Au existat încercări, dar nu erau practice:
'66 L.A. Belady – un model de evidență statistică prin care se poate prevedea *cu o oarecare probabilitate* care este pagina ce va fi solicitată cel mai târziu

➤ Algoritmi practici

- Metode folosite: NRU, FIFO, LRU și alte variante
- Observație: indiferent de alg. de decizie folosit, trebuie avut în vedere faptul că, pentru implementare, gestiunea unei structuri de date care să permită decizia, trebuie făcută *la fiecare acces la memorie*
- Prin urmare, nu este permisă nici măcar gestiunea unei liste liniare simplu înlănțuite!, ci este nevoie de mecanisme mult mai ușor de gestionat
- De multe ori aceste metode se implementează prin hardware, făcându-se uneori anumite compromisuri

Alg. de page swapping /7

➤ Algoritmul NRU (= Not Recently Used)

- Ideea: înlocuirea unei pagini care nu a fost utilizată recent
- Fiecare pagină fizică are asociați 2 biți, folosiți pentru a decide ce pagină se va evacua
- Bitul de *referire* este resetat (pus pe 0) la încărcarea paginii și este setat (pus pe 1) la fiecare acces (referire) la pagină
- Periodic (de obicei la 20ms) se face operația de *clearing bits*, prin care biții de referire ai tuturor paginilor sunt resetati
- Al doilea bit este bitul de *modificare* (bitul *dirty*), ce este resetat la încărcarea paginii și setat la scrierea în pagină

Alg. de page swapping /8

➤ Algoritmul NRU

- Paginile se împart la fiecare moment în patru clase, pe baza acestor biți:
 - Clasa 0 – biții=(0,0) : pagini nereferite și nemodificate
 - Clasa 1 – biții=(0,1) : pagini nereferite (de la ultimul *clearing*), dar modificate (de la încărcarea lor)
 - Clasa 2 – biții=(1,0) : pagini referite, dar nemodificate
 - Clasa 3 – biții=(1,1) : pagini referite și modificate
- Pagina “victimă” se alege din prima clasă nevidă (se caută întâi în clasa 0, apoi în clasa 1, ș.a.m.d.); dacă pagina aleasă este din clasa 1 sau 3, ea va fi salvată pe disc înainte de a fi înlocuită
- Avantaje: alg. f. simplu; nu-i optimal, dar e eficient în practică

Alg. de page swapping /9

➤ Algoritmul FIFO (= First In First Out)

- Ideea: înlocuirea în ordinea încărcării paginilor
- Implementarea este simplă: se gestionează o listă FIFO cu paginile rezidente în memorie; actualizarea ei se face *la fiecare încărcare de pagină* (nu la fiecare acces la memorie!)
- Alegerea “victimei”: prima pagină din listă (i.e. se va evacua cea mai veche pagină din memorie)
- Bineînțeles, și aici se folosește bitul de *modificare* (bitul *dirty*), pentru a ști dacă pagina aleasă trebuie salvată pe disc înainte de a fi înlocuită

Alg. de page swapping /10

➤ Algoritmul FIFO

- Exemplu: un program cu 5 pagini virtuale, cu secvența de referire “1,2,3,4,1,2,5,1,2,3,4,5”
memoria fizică: 3 cadre de pagină

Lista FIFO:

Pagina solicitată	1	2	3	4	1	2	5	1	2	3	4	5	
Pagina cea mai recentă	–	1	2	3	4	1	2	5	5	5	3	4	4
	–	–	1	2	3	4	1	2	2	2	5	3	3
Pagina cea mai veche	–	–	–	1	2	3	4	1	1	1	2	5	5

↓

timp →

Înlocuire de pagină (PFI): Da Da Da Da Da Da Da Nu Nu Da Da Nu

Rata erorilor de pagină: $PFI_ratio = 9/12 = 75\%$

Alg. de page swapping /11

➤ Algoritmul FIFO

- Exemplu: același program cu 5 pagini virtuale, cu aceeași secvență de referire “1,2,3,4,1,2,5,1,2,3,4,5” dar memoria fizică: 4 cadre de pagină

Lista FIFO:

Pagina solicitată	1	2	3	4	1	2	5	1	2	3	4	5	 timp
Pagina cea mai recentă	—	1	2	3	4	4	5	1	2	3	4	5	
	—	—	1	2	3	3	4	5	1	2	3	4	
	—	—	—	1	2	2	3	4	5	1	2	3	
Pagina cea mai veche	—	—	—	—	1	1	1	2	3	4	5	1	

Înlocuire de pagină (PFI): Da Da Da Da Nu Nu Da Da Da Da Da Da

Rata erorilor de pagină: $PFI_ratio = 10/12 = 83,33\%$

Alg. de page swapping /12

➤ Algoritmul FIFO

- *Anomalia lui Belady*: deși bunul simț ne spune că șansa ca o pagină să fie înlocuită *scade* pe măsură ce *crește* numărul de cadre fizice, totuși nu este așa în cazul alg. FIFO, după cum rezultă și din exemplul anterior
- Alt dezavantaj: (spre deosebire de LRU) algoritmul FIFO dezavantajează paginile “esențiale” (i.e. utilizate frecvent), ce vor fi în mod periodic evacuate pe disc și reîncărcate curând după evacuare

Alg. de page swapping /13

➤ Algoritmul LRU (= Least Recently Used)

- Ideea: înlocuirea paginii cel mai puțin folosite în ultimul timp
- Cum? Se va folosi localitatea temporală/spațială a programului pentru a înlocui pagina cel mai puțin utilizată recent
[principiul localității: o pagină ce a fost accesată des (rar) de către ultimele instrucțiuni, probabil va fi accesată des (rar) și în continuare]
- Pentru implementare este nevoie să se țină evidența utilizărilor paginilor, adică să ordonăm paginile după timpul celei mai recente referințe la ele – necesită deci gestiunea informației *la fiecare acces la memorie* și hardware f. costisitor
- Teoretic, gestionarea informației referitoare la utilizarea paginilor se poate face cu o coadă FIFO, cu observația că la accesarea unei pagini, ea este scoasă din coadă și mutată în vârful cozii, dar nu-i eficient practic

Alg. de page swapping /14

➤ Algoritmul LRU

- Exemplu: același program cu 5 pagini virtuale, cu aceeași secvență de referire “1,2,3,4,1,2,5,1,2,3,4,5”
memoria fizică: 3 cadre de pagină

Coadă LRU:

Pagina solicitată	1	2	3	4	1	2	5	1	2	3	4	5	 timp	
Pagina cu cel mai recent acces	—	1	2	3	4	1	2	5	1	2	3	4		5
	—	—	1	2	3	4	1	2	5	1	2	3		4
Pagina cu cel mai vechi acces	—	—	—	1	2	3	4	1	2	5	1	2		3
Înlocuire de pagină (PFI):	Da	Da	Da	Da	Da	Da	Da	Nu	Nu	Da	Da	Da		

Rata erorilor de pagină: $PFI_ratio = 10/12 = 83,33\%$

Alg. de page swapping /15

➤ Algoritmul LRU

- Exemplu: același program cu 5 pagini virtuale, cu aceeași secvență de referire “1,2,3,4,1,2,5,1,2,3,4,5” dar memoria fizică: 4 cadre de pagină

	Coadă LRU:												
Pagina solicitată	1	2	3	4	1	2	5	1	2	3	4	5	
Pagina cu cel mai recent acces	—	1	2	3	4	1	2	5	1	2	3	4	5
	—	—	1	2	3	4	1	2	5	1	2	3	4
	—	—	—	1	2	3	4	1	2	5	1	2	3
Pagina cu cel mai vechi acces	—	—	—	—	1	2	3	4	4	4	5	1	2
													→ timp

Înlocuire de pagină (PFI): Da Da Da Da Nu Nu Da Nu Nu Da Da Da

Rata erorilor de pagină: PFI_ratio = $8/12 = 66\%$

Alg. de page swapping /16

➤ Algoritmul LRU

- Modalități de implementare a alg. LRU:
 - LRU cu contor de accese
 - LRU cu stivă
 - LRU cu matrice de referințe
- Multe SC folosesc pentru paginare o *aproximare* a LRU
- Avantaje:
 - Nu prezintă *anomalia lui Belady* (se poate demonstra matematic că alg. LRU cu stivă are proprietatea de monotonie – nu funcționează mai rău dacă crește numărul de cadre de pagină ale memoriei fizice)
 - spre deosebire de alg. FIFO, alg. LRU avantajează paginile “esențiale” (utilizate frecvent), ce vor fi păstrate în memorie

Alg. de page swapping /17

➤ LRU cu contor de accese

- Se implementează hard. CPU are un registru (de obicei pe 64 biți) numit *contor* cu rol de ceas logic – este incrementat la fiecare instrucțiune, sau acces la memorie
- În tabela de pagini există un câmp rezervat pentru a păstra valoarea acestui contor – la fiecare acces la o pagină, contorul este memorat în spațiul corespunzător acelei pagini
- Cu ajutorul acestor valori salvate, putem ști dacă o pagină a fost sau nu utilizată de la ultimul *clear*-ing al biților de referință, precum și în ce ordine (aproximativă) au fost accesate paginile
- Alegerea “victimei” pentru evacuare constă în căutarea în tabela de pagini a paginii cu cea mai mică valoare a contorului

Alg. de page swapping /18

➤ **LRU cu stivă**

- Se folosește o stivă cu numerele de pagină
- Când este referită o pagină, ea este scoasă din stivă (doar dacă exista deja în stivă) și apoi este pusă în vârful stivei
- În orice moment, pagina de la baza stivei este pagina cea mai puțin utilizată recent
- Alegerea “victimei” pentru evacuare este simplă – pagina de la baza stivei, nu presupune o căutare, dar în schimb la fiecare acces este nevoie de căutarea paginii dorite în stivă pentru a o muta în vârful stivei

Alg. de page swapping /19

➤ LRU cu matrice de referințe

- Se folosește o matrice binară (cu 0 și 1) de dimensiune $n*n$, unde n este numărul de pagini fizice ale memoriei SC-ului
- Inițial toate elementele matricii se pun pe 0
- În momentul când se referă pagina k , se pune mai întâi 1 peste tot pe linia k , iar apoi se pune 0 peste tot pe coloana k
- În orice moment numărul de cifre 1 de pe o linie l ne arată ordinea de referire a paginii l – ultima pagină referită va avea $n-1$ cifre de 1, penultima $n-2$, ș.a.m.d.
- Alegerea “victimei” pentru evacuare se face căutând linia din matrice cu cele mai puține cifre de 1 (cu cele mai multe zero-uri)
- Implementarea matricii de referințe se face prin hard

Alg. de page swapping /20

➤ **Algoritmul LFU (=Least Frequently Used)**

- Ideea: înlocuirea paginii cel mai puțin folosite (per total, nu doar în ultimul timp)
- Alg. actualizează la fiecare acces de pagină un contor de accese păstrat în tabela de pagini (contor ce nu este resetat periodic ca la LRU cu contor de accese)
- “Victima”: se alege pagina cu cel mai mic contor
- Dezavantaj: dacă o pagină este utilizată f. des în faza inițială a unui proces, iar apoi nu mai este utilizată deloc, ea rămâne totuși în memorie pentru că are o valoare f. mare a contorului

➤ **Algoritmul MFU (=Most Frequently Used)**

- dualul alg. LFU; se alege pagina cu cel mai mare contor

Alg. de page swapping /21

➤ **Alte abordări:**

- NRU combinat cu FIFO
 - Ideea: se aplică întâi NRU, iar la nivelul fiecărei clase se aplică FIFO
- Metoda celei de-a doua șanse (e.g. Mach OS)
 - Ideea: se aplică FIFO cu următoarea modificare: dacă pagina cea mai veche are bitul de referință pus pe 1, atunci i se mai dă o șansă – bitul este pus pe 0 și pagina este mutată la sfârșitul listei (astfel devine cea mai recentă pagină); căutarea se reia cu noua listă, până se găsește o pagină cu bitul pus pe 0 – acea pagină este selectată pentru înlocuire
- Algoritmi de înlocuire a paginilor bazați pe prioritatea proceselor
- Algoritmi de înlocuire cu pagini de dimensiuni variabile

Alg. de page swapping /22

➤ Implementări reale: **paged daemon**

- Majoritatea S.O.-urilor au unul sau mai multe procese de sistem responsabile cu implementarea politicii de înlocuire a paginilor pentru memoria virtuală
 - Un **demon** (*daemon*) este un proces de sistem autonom care execută periodic o anumită sarcină de administrare a SC
- Demonul de paginare pregătește sistemul pentru evacuarea de pagini înainte ca să apară nevoia de evacuare
 - Demonul se “trezește” când cantitatea de memorie liberă devine mică
 - “Curăță” paginile *dirty* prin scrierea lor pe disc – *prewrite* sau *pageout*
 - Administrează liste ordonate cu candidații pentru evacuare
 - Decide cât de multă memorie să aloce pentru memoria virtuală

➤ Fenomenul de *trashing*

- **Trashing** = fenomenul de “sufocare” a sistemului atunci când comportamentul swapping devine excesiv
(sistemul este ocupat predominant cu tratarea erorilor de pagină, în loc de a-și folosi timpul pentru execuția job-urilor utile)
- Paginarea la cerere funcționează datorită localității (temporale și spațiale) manifestate de programe, dar atunci când cerințele minime de memorie nu pot fi satisfăcute, apar numeroase operații de swapping a paginilor
- Aceasta duce la o proastă utilizare a CPU și la f. multe operații I/O → S.O.-ul poate încerca să îmbunătățească utilizarea CPU prin *mărirea* numărului de procese

➤ Fenomenul de *trashing*

- **Mulțimile de lucru** (*working sets*) – sunt o modalitate utilă pentru controlul fenomenului de *trashing*
- O *mulțime de lucru* este mulțimea de pagini folosite de un proces la un moment dat (luând în calcul localitatea)
- Fenomenul de *trashing* va apare dacă suma dimensiunilor mulțimilor de lucru ale tuturor proceselor din sistem depășește limita impusă de cantitatea de memorie fizică din acel sistem
- În caz de nevoie, se pot suspenda unele procese pentru a se coborâ sub această limită

➤ Alte mecanisme de memorie virtuală

– Segmentarea (cu încărcare) la cerere

= Segmentarea combinată cu tehnica de *swapping*

- **Ideea:** aducerea segmentelor de pe disc în memorie doar în momentul când sunt referite (“când e nevoie de ele”)
- Ridică probleme asemănătoare cu cele de la paginarea la cerere

– Segmentarea paginată (cu încărcare) la cerere

= Segmentarea paginată combinată cu tehnica de *swapping*

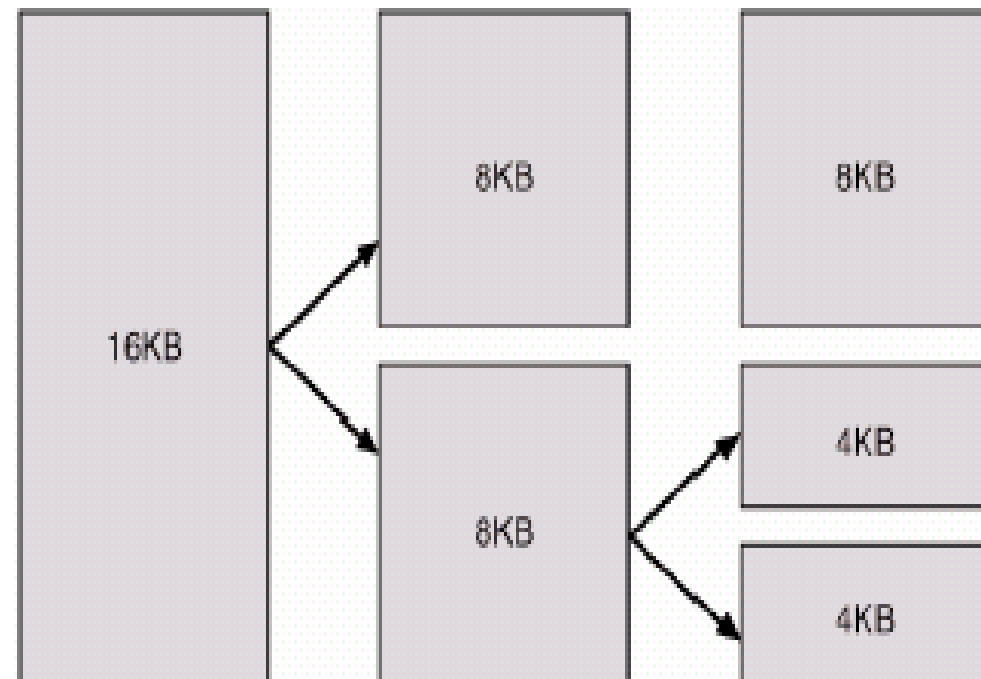
➤ Planificarea schimburilor cu memoria

- **Întrebările gestiunii memoriei:** indiferent de metoda de alocare folosită, S.O.-ul trebuie să rezolve o serie de probleme cum ar fi:
 - **Cât?** – problema cantității de memorie alocată:
 - Alocare statică – toată memoria la început (e.g. alocarea pe partiții)
 - Alocare dinamică – doar necesarul curent (e.g. paginarea la cerere)
 - **Unde?** – problema locului liber pe care va fi plasat programul (apare la, e.g., alocarea cu partiții variabile) → Politici de plasare
 - **Când?** – problema momentului de încărcare a paginilor în cadre fizice (apare la alocarea cu paginare) → Politici de încărcare (*fetch*)
 - problema momentului de compactare (apare la, e.g., alocarea cu partiții variabile, sau la segmentarea nepaginată)
 - **Care?** – problema alegerii victimei la încărcarea paginilor (apare la, e.g., alocarea cu paginare la cerere) → Politici de înlocuire

➤ Planificarea schimburilor cu memoria (cont.)

– Politici de schimb utilizate de gestiunea memoriei:

- **Politici de plasare** – utilizate de S.O. și programe (malloc/free)
 - alg. FFA, BFA, WFA, alocarea prin camarazi (buddy-system) :
- **Politici de încărcare (*fetch*)**
 - încărcarea la începutul execuției
 - încărcarea la cerere (pe parcursul execuției) , cu varianta:
încărcarea în avans (pe baza principiului localității)
- **Politici de înlocuire (*swapping*)**
 - alg. NRU, FIFO, LRU, etc.



➤ Planificarea schimburilor cu memoria (cont.)

– Tehnici de alocare utilizate:

	Alocare contiguă	Alocare necontiguă	Tip alocare
Memorie reală	Alocarea unică Alocarea cu partiții - fixe - variabile	Paginarea (pură) Segmentarea (pură) Segmentarea paginată (pură)	
Memorie virtuală	Alocarea cu swapping	Paginarea la cerere Segmentarea la cerere Segmentarea paginată la cerere	
Tip memorie			

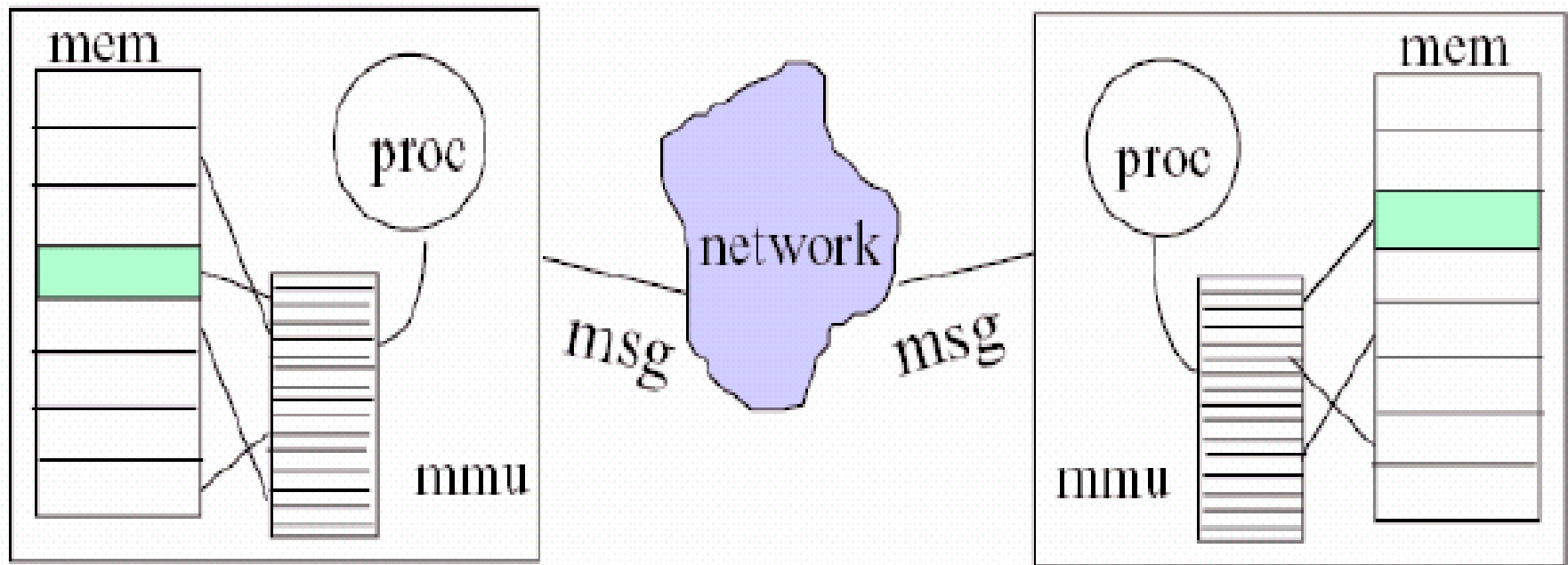
- **SC evoluează**

- Apar noi arhitecturi hardware / noi vederi ale “ierarhiei” de memorii

- **Memorie la distanță**

- **NUMA** (Non-Uniform Memory Access)
- **DSM** (Distributed Shared Memory)
- **GMS** (Global Memory Systems)
 - Implementată pe Digital UNIX si FreeBSD ('98)

- **Distributed Shared Memory (DSM)**
 - Permite utilizarea modelului de programare cu memorie partajată (spațiu de adresare partajat) într-un sistem distribuit (procesoare dotate doar cu memorie locală și o rețea de comunicație)



Administrarea memoriei în Linux /1

- **Planificarea memoriei**

- Subsistemul de administrare a memoriei fizice din Linux are ca sarcină alocarea și eliberarea paginilor (fizice), a grupurilor de pagini și a blocurilor mici de memorie
- Linux-ul are mecanisme suplimentare pentru gestionarea memoriei virtuale, i.e. a memoriei mapate în spațiile de adrese logice ale proceselor aflate în curs de execuție în sistem

Administrarea memoriei în Linux /2

- **Administrarea memoriei fizice**

- Alocatorul de pagini alocă și eliberează toate paginile fizice; poate alocă la cerere intervale contigue de pagini fizice (i.e. grupuri contigue de pagini)
- Politica de plasare utilizată e *alocarea cu camarazi*
 - Fiecare regiune de memorie alocabilă este asociată cu o regiune parteneră adiacentă
 - Ori de câte ori două regiuni partenere alocate sunt eliberate, ele sunt combinate într-o singură regiune liberă, dublă
 - Dacă o cerere mică de memorie nu poate fi satisfăcută prin alocarea unei regiuni libere mici, o regiune liberă mai mare se împarte în 2 regiuni partenere pentru a satisface cererea

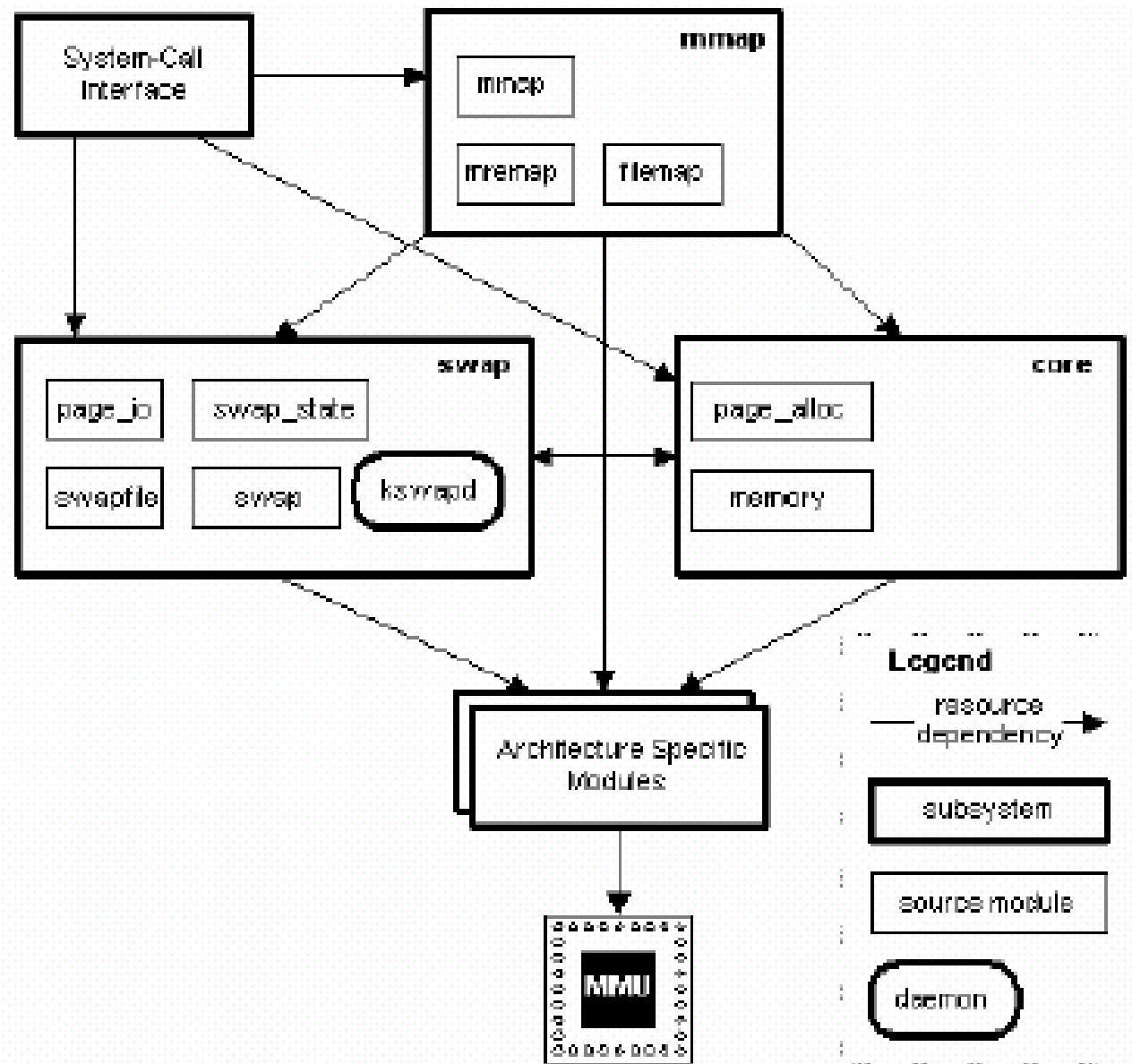
Administrarea memoriei în Linux /3

- **Administrarea memoriei virtuale**

- Nucleul Linux rezervă o zonă de lungime constantă, dependentă de arhitectura SC, din spațiul virtual de adresare al fiecărui proces, pt. propriul său uz intern
- Această zonă rezervată de nucleu conține 2 regiuni:
 - O zonă statică ce conține o tabelă de pagini cu referințe la fiecare pagină fizică disponibilă în sistem, astfel încât să existe o translație simplă de la adrese fizice la adrese virtuale atunci când se rulează codul nucleului
 - Restul secțiunii rezervate nu este rezervată pentru un scop anume; intrările sale din tabela paginilor pot fi modificate pentru a indica spre orice alte zone din memorie

Administrarea memoriei în Linux /4

- Administrarea memoriei



Administrarea memoriei în Windows /1

- **Administrarea memoriei** – Windows NT/2k/XP/etc.
 - Administratorul de memorie lucrează cu procese, nu threaduri
 - Spațiul de adresare virtual este paginat cu încărcare la cerere, cu pagini de dimensiune fixă – 4 KB (max 64 KB pt. Itanium)
 - S.O.-ul poate folosi și pagini mari (cu dimensiunea de 4 MB) pentru a reduce spațiul ocupat de tabelele de paginare
 - Fiecare pagină virtuală poate fi: liberă, rezervată, sau angajată (*committed*)
 - Paginile libere și cele rezervate au pagini *shadow* pe disc, iar referirile (accesul) la ele provoacă întotdeauna eroare de pagină
 - Se pot folosi maxim 16 fișiere de swap
 - Segmentarea nu este suportată
 - Tabelele de paginare sunt pe 2 nivele (la Windows pe 32 biți)

Administrarea memoriei în Windows /2

- **Administrarea memoriei (cont.)**

- Adresele virtuale sunt pe 32 de biți (→ spațiu virtual de 4 GB)
- Vârful (primii 64 KB) și baza (ultimii 64 KB) spațiului de adrese virtual al fiecărui proces, în mod normal nu sunt mapate
- Începând de la 64 KB de la baza spațiului și până aproape de jumătate (2 GB) este zona utilizator privată – conține codul și datele programului
- Ultima porțiune din jumătatea de jos conține date de sistem (contoare și *timer*-e) partajate read-only de toți utilizatorii
- Jumătatea de sus (2 GB) a spațiului virtual conține S.O.-ul, inclusiv codul său, zona de date și diverse regiuni (*pools*), paginate și nepaginate (contigue), utilizate pentru obiectele sistemului; această porțiune nu poate fi scrisă și, în cea mai mare parte, nici citită, de către procesele utilizator

- **Bibliografie obligatorie**

capitolele despre *gestiunea memoriei* din

- Silberschatz : “*Operating System Concepts*”

(cap.8, prima parte: §8.1–6, din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.3, §3.4-6, din [MOS3])

Suplimentar: capitolele studii de caz (despre Unix/Linux și respectiv Windows NT) din cele două cărți de mai sus

Sumar

- Memoria virtuală
 - Principiul localității
 - Paginarea la cerere
 - MMU
 - Algoritmi de înlocuire a paginilor
 - Fenomenul de *trashing*
 - Segmentarea la cerere
- Concluzii
- Metode de viitor pentru administrarea memoriei
- Administrarea memoriei în Linux și Windows

Întrebări ?

Sisteme de Operare

Administrarea informațiilor : Sisteme de fișiere

Cristian Vidrașcu

<https://profs.info.uaic.ro/~vidrascu>

- Conceptul de fișier
- Interfața sistemului de fișiere
- Implementarea sistemului de fișiere

Conceptul de fișier /1

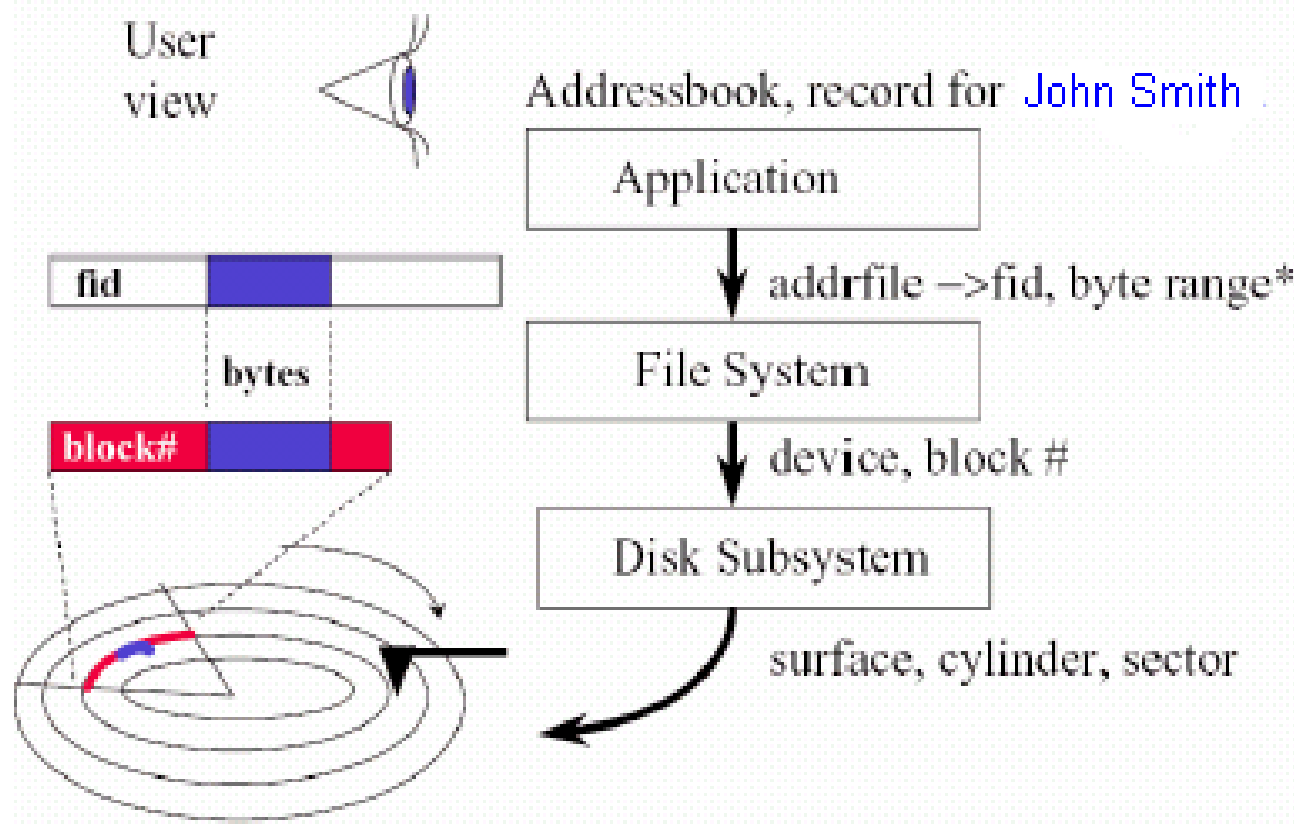
- **Fișier:** zonă de stocare contiguă d.p.d.v. logic (nu neapărat contiguă și d.p.d.v. fizic)
- **Conținutul unui fișier:**
 - date (numerice, de tip caracter, binare)
 - program
- **Structura unui fișier:**
 - nestructurat (secvență de octeți)
 - structură simplă de tip înregistrare (linii/înregistrări, de lungime fixă sau variabilă)
 - structură complexă (e.g. document formatat, fișier executabil cu încărcare relocabilă)
 - Cine decide structura (i.e. modul de interpretare a conținutului) ?
sistemul de operare (Windows, OS/2) vs. **programul** (UNIX)

Conceptul de fișier /2

- **Rolul fișierelor:**

- Persistență – date cu viață lungă, pentru posteritate
 - medii de stocare nevolatile
 - nume cu înțeles semantic (d.p.d.v. al utilizatorului)

- **Abstracții fișier**



Atributele fișierului (metadata)

- **Nume** – singura informație păstrată în formă inteligibilă uman
- **Tip** – necesar pentru sistemele ce suportă diverse tipuri
- **Locație** – pointer la locația fișierului pe perifericul de stocare
- **Dimensiune** – dimensiunea curentă a fișierului
- **Protecție** – controlează cine poate citi, scrie, executa, ... fișierul
- **Timp, dată și identificatorul de utilizator** – informații pentru protecție, securitate și monitorizarea utilizării
- Informațiile despre fișiere sunt păstrate în structura de directoare, ce este menținută pe disc

Operații asupra fișierelor

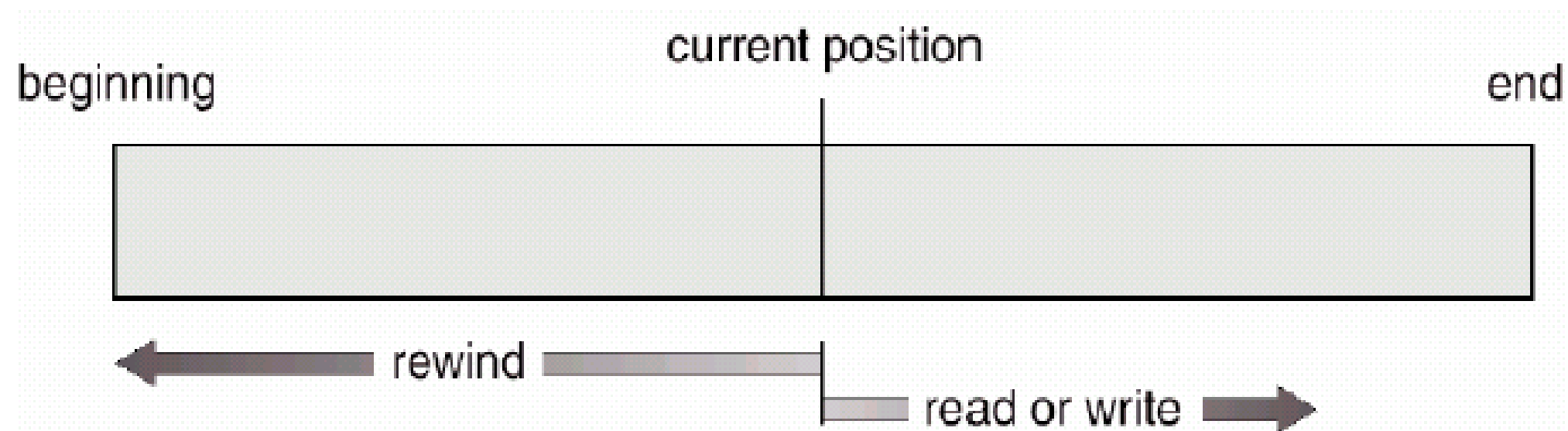
- Creare – `creat()`
- Citire – `read()`
- Scriere – `write()`
- Repoziționare în fișier (căutare în fișier) – `lseek()`
- Ștergere – `unlink()`
- Trunchiere – `trunc()`
- Deschidere (căutarea în structura de directoare de pe disc a intrării fișierului și copierea conținutului intrării în memorie) – `open()`
- Închidere (mutarea conținutului intrării fișierului din memorie în structura de directoare de pe disc) – `close()`

Accesul la fișiere /1

- **Acces secvențial** – acces de la început spre sfârșit, posibilitate de întoarcere la început (rewind) pentru reluarea parcurgerii secvențiale, se pot face citiri și scrieri, dar nu ambele simultan în timpul aceleiași deschideri – *metafora benzii magnetice*
- **Acces direct** (acces aleatoriu) – acces după numărul (adresa) înregistrării, se pot face citiri și scrieri în orice combinații dorite – *metafora discului de vinyl*
- **Acces indexat** – acces direct prin conținut, folosind diverse tehnici (fișiere de index, fișiere multilistă, *hashing*, B-arbori, ș.a.)

Accesul la fișiere /2

- Acces secvențial



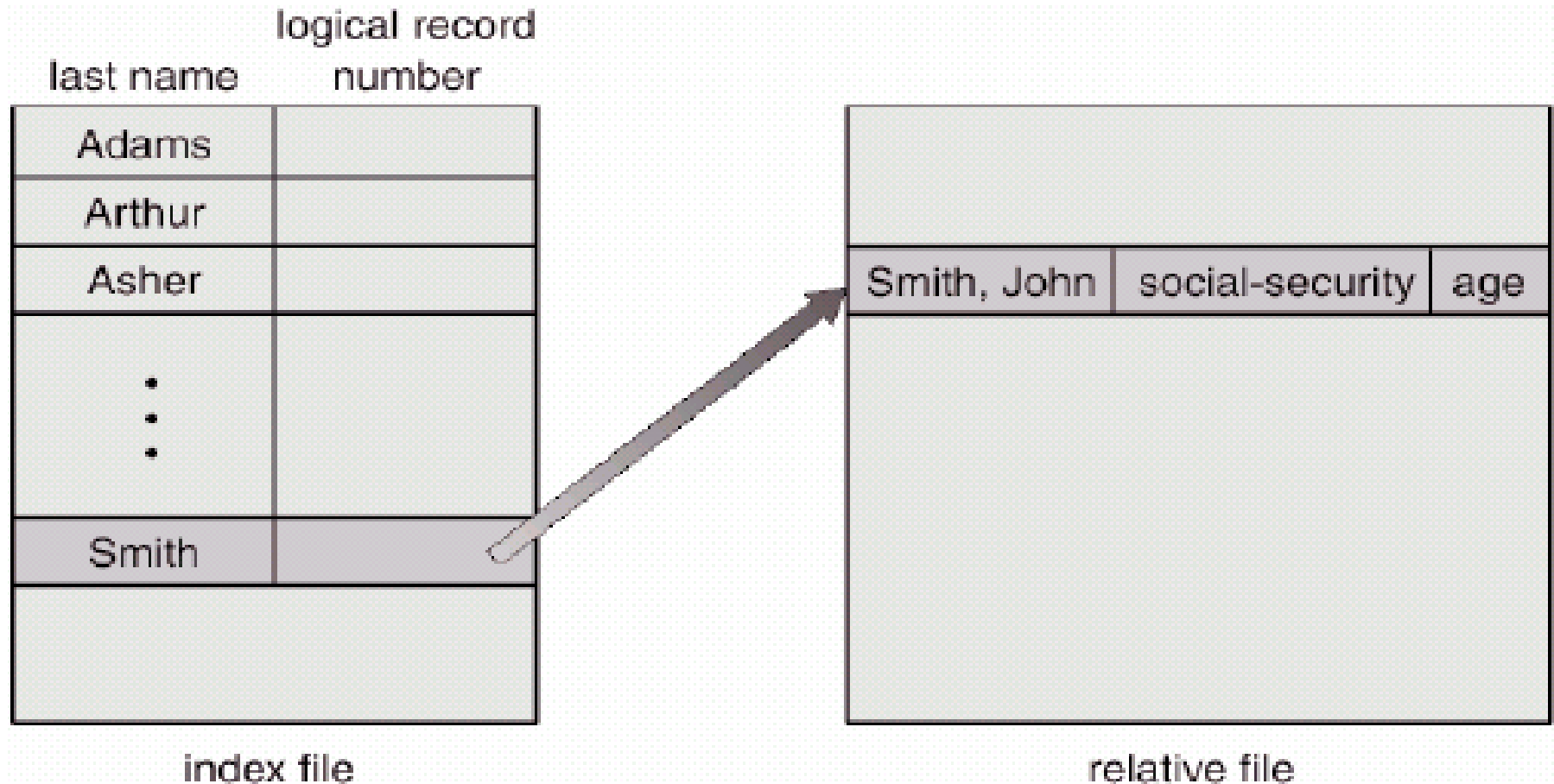
Accesul la fișiere /3

- Simularea accesului secvențial pe un fișier cu acces direct

acces secvențial	implementarea cu acces direct
reset	<code>cp = 0;</code>
read next	<code>read cp;</code> <code>cp = cp + 1;</code>
write next	<code>write cp;</code> <code>cp = cp + 1;</code>

Accesul la fișiere /4

- Exemplu de acces indexat – fișiere index și relative



Clasificări ale fișierelor

- Clasificarea după posibilitatea de tipărire (afișare) ASCII:
 - fișiere text
 - fișiere binare
- Clasificarea după suportul pe care sunt rezidente:
 - fișiere pe disc magnetic (HD, FD, ZIP, ...) sau tambur magnetic
 - fișiere pe bandă magnetică sau casetă magnetică
 - fișiere pe suport optic (CD, DVD) sau memorii NAND (stick USB, SD card, ...)
 - fișiere pe imprimantă sau plotter
 - fișiere pe cartele perforate sau bandă de hârtie perforată
 - fișiere pe ecran
 - fișiere tastatură
- Suportul influențează operațiunile și modurile de acces posibile e.g. doar suportul disc/tambur acceptă accesul direct, celelalte suporturi acceptă doar accesul secvențial; fișierele pe cartele sau tastatură acceptă doar citiri, iar cele pe imprimantă, plotter sau

Interfața sistemului de fișiere

- Structura de directoare
- Funcții
- Organizare
- Partajare
- Montare
- Protecție

Structura de directoare /1

- **Director** – colecție de noduri ce conțin informații despre toate fișierele
- Atât fișierele, cât și structura de directoare sunt păstrate pe disc
- Copiile de siguranță ale acestora sunt păstrate (de obicei) pe benzi, CD-ROM-uri, ș.a.
- La început: director unic (e.g. CP/M, SIRIS – '60), iar apoi director pe 2 nivele (e.g. RSX – '70-'80, un S.O. în timp real pentru DEC PDP-11)
- În prezent: directoare cu structura **arborescentă** sau cu structură de **graf aciclic**

Structura de directoare /2

- Cum este organizată structura de directoare?
 - Listă înlănțuită
 - Vector sortat
 - Tabelă hash

(a se vedea la Implementarea sistemului de fișiere)

Sisteme de fișiere/1

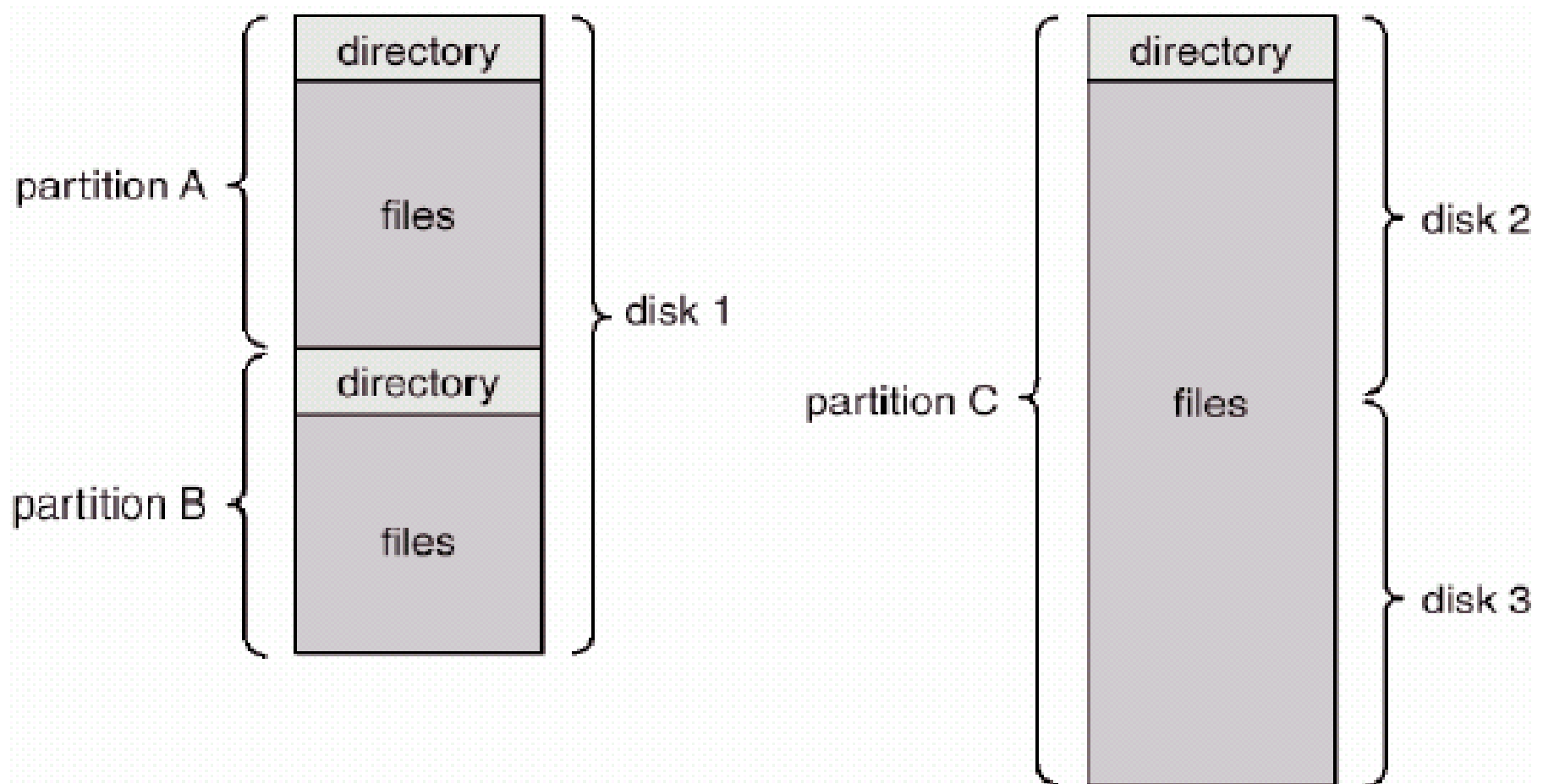
- Definiție: un **sistem de fișiere** este o colecție oarecare de fișiere, împreună cu structura de directoare în care sunt acestea organizate
- Există mai multe **tipuri** de sisteme de fișiere, e.g. *NTFS* (specific Windows) sau *ext2fs/ext3fs/ext4fs*, *btrfs*, *zfs*, ș.a. (specifice Linux / UNIX), ce diferă prin structurile de date folosite pentru stocarea lor și prin modul de implementare a operațiilor asupra fișierelor și directoarelor
- Structurile de date utilizate pentru stocarea unui sistem de fișiere folosesc conceptul de **cluster**, drept unitate de măsură pentru alocarea spațiului necesar memorării unui fișier, la nivelul *file-system*-ului
Cu alte cuvinte, dimensiunea unui fișier (i.e. informația utilă stocată în el) este un atribut exprimat în octeți, însă spațiul ocupat de conținutul fișierului pe disc este un multiplu de clustere.
- *Dimensiunea unui cluster*: o putere de forma 2^n , cu $n=0,1,2,\dots$, în termeni de **sectoare** (sau *blocuri-disc*, i.e. unitatea de stocare la nivelul *discului*)

Sisteme de fișiere/2

- Pentru persistență, sistemele de fișiere se păstrează pe ***dispozitive de stocare nevolatilă*** (e.g. disc, stick USB, CD, etc.), în entități numite **volume**.
- Un **volum** poate fi (mai multe detalii – în cursul următor):
 - o ***partiție a unui disc*** (schema clasică, MBR/GPT, ce permite stocarea mai multor volume /disc)
 - un ***disc întreg*** (e.g. stick USB, CD/DVD, etc.)
 - un ***ansamblu de mai multe discuri*** (ansamblu ce stochează un singur sistem de fișiere, prin tehnici precum RAID)
- **Formatarea logică** a unui volum = “crearea sistemului de fișiere” rezident pe acel volum, prin inițializarea (pe disc) a structurilor de date specifice tipului acelui sistem de fișiere; operația are ca parametri: tipul sistemului de fișiere, numele volumului (i.e., o etichetă) și dimensiunea clusterului (în termeni de sectoare)

Sisteme de fișiere/3

- Un exemplu de organizare a sistemului de fișiere



Funcțiile sistemului de fișiere

- (Subsistemul de directoare) Mapează numele de fișiere în identificatori de fișiere prin primitiva **open** (sau **creat**). Creează structurile de date din nucleu
- Menține structura de nume (**unlink**, **mkdir**, **rmdir**)
- Determină plasarea fișierelor (și a metadatelor) pe disc, în termeni de cluster. Gestionează alocarea clusterelor (1 cluster = 2^n sectoare consecutive). Gestionează clusterelor eronate (i.e., cele ce conțin *bad sectors*)
- Gestionează apelurile de sistem **read** și **write**
- Inițiază operațiile I/O pentru transferul blocurilor dinspre disc spre RAM, respectiv dinspre RAM spre disc
- Gestionează tamponanele *cache* pentru disc

Informații într-un director de periferice

- Nume
- Tip
- Adresă
- Lungimea curentă
- Lungimea maximă
- Data ultimului acces (pentru arhivare)
- Data ultimei actualizări (pentru *dump*)
- Identificatorul proprietarului
- Informații pentru protecție

Operații asupra directoarelor

- Căutarea unui fișier
- Crearea unui fișier
- Ștergerea unui fișier
- Redenumirea unui fișier
- Listarea unui director
- Traversarea sistemului de fișiere
- Salvarea/restaurarea unui sistem de fișiere

Organizarea directoarelor

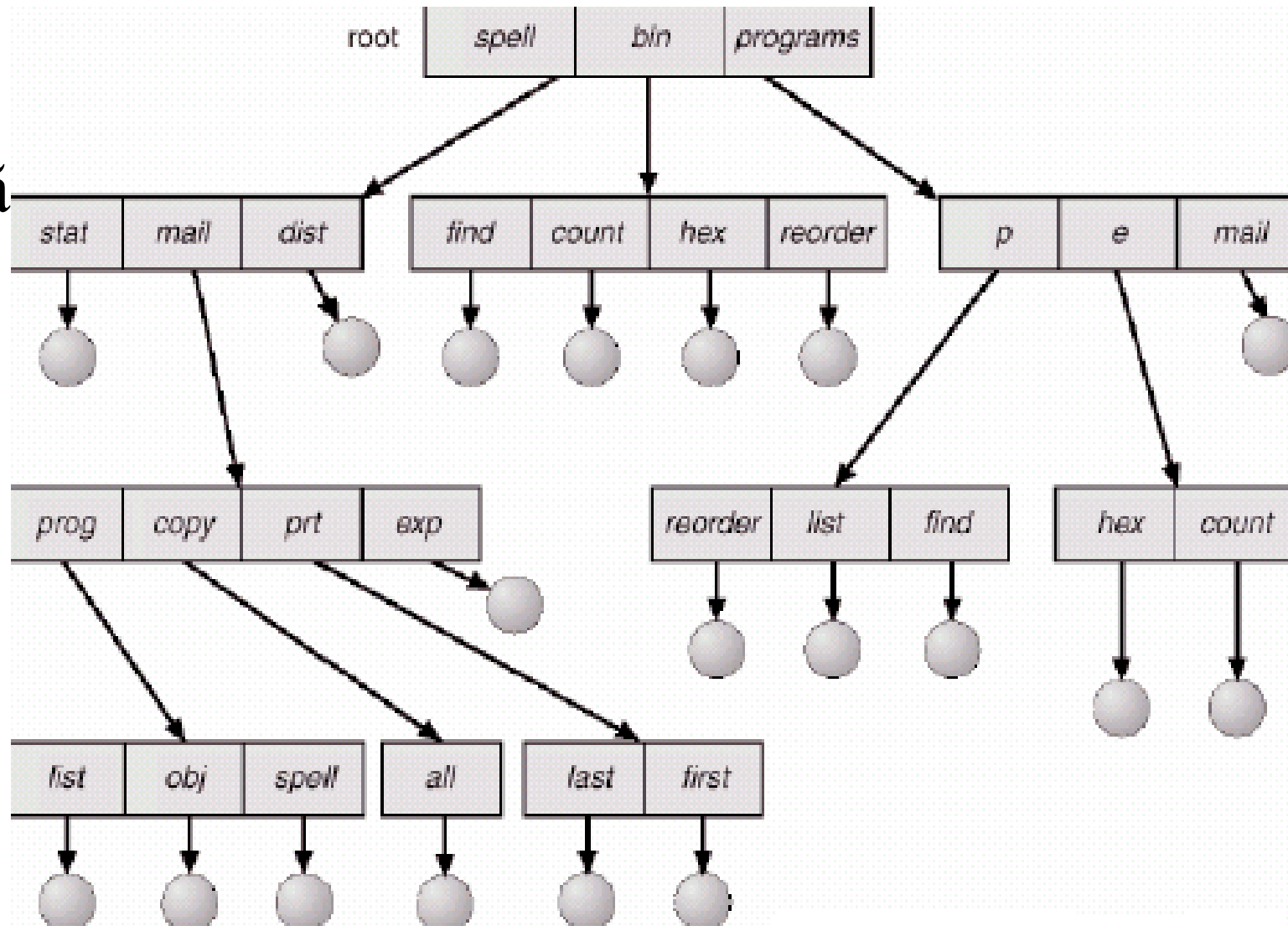
- Organizarea (logică a) directoarelor, pentru a obține:
 - **Eficiență** – localizarea rapidă a unui fișier
 - **Nume** – numele sunt utile pentru utilizatori
 - Doi utilizatori pot avea același nume pentru fișiere diferite
 - Același fișier poate avea mai multe nume diferite
 - **Grupare** – gruparea logică a fișierelor după proprietăți (e.g. toate programele C, toate jocurile, etc.)

Numirea fișierelor

- Scopuri:
 - Pentru a identifica fișierele (la deschidere, SO-ul mapează numele de fișiere în obiecte de tip fișier utilizate intern pentru accesul la fișiere)
 - Pentru a permite utilizatorilor să-și aleagă propriile nume de fișiere fără probleme de conflict de nume
 - Ușurința de utilizare: nume scurte, grupări

Sisteme de fișiere /1

- Directoare cu structură arborescentă

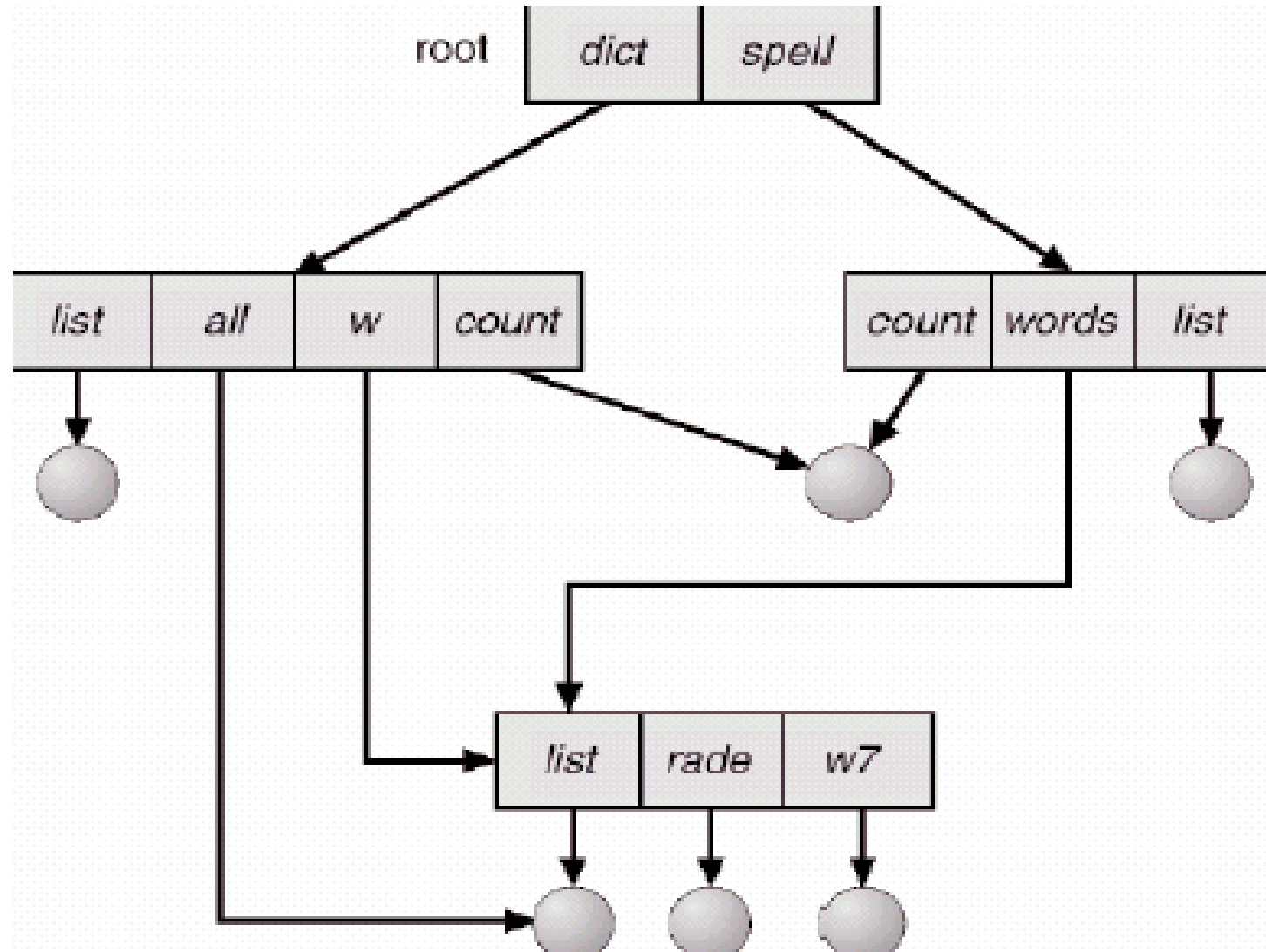


Sisteme de fişiere /2

- Directoare cu structură arborescentă
 - Căutare eficientă
 - Posibilităţi de grupare a fişierelor
 - Căi de nume **absolute** şi **relative**
 - Crearea unui fişier nou se face în directorul curent
 - Crearea unui subdirector nou se face în directorul curent

Sisteme de fișiere /3

- Directoare cu structură de graf aciclic



Sisteme de fișiere /4

- Directoare cu structură de graf aciclic
 - Pot exista subdirectoare și fișiere partajate
 - Pot avea două nume diferite
 - *linking* hard/soft – `link()`, `symlink()`
 - **Problemă:** ștergerea unui fișier partajat
 - **Soluții:**
 - Back-pointers, pentru a putea șterge toate referințele
 - Problemă: înregistrări cu lungime variabilă
 - Soluția cu contor de referințe păstrat în intrarea fișierului

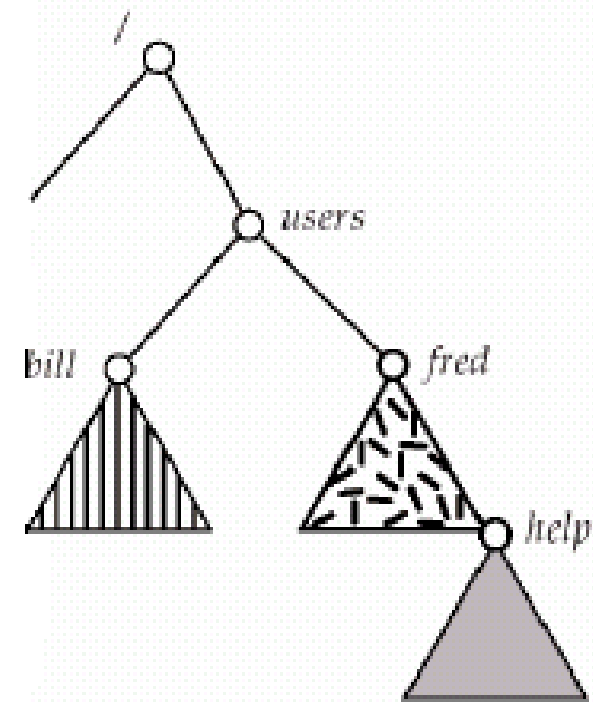
Sisteme de fișiere /5

- Directoare cu structură de graf general
 - **Problemă:** cum garantăm inexistența circuitelor?
 - **Soluții:**
 - Permitted *link*-uri doar către fișiere, nu și către subdirectoare
 - Garbage collection
 - De fiecare dată când se adaugă un nou *link*, să se utilizeze un algoritm de detecție a circuitelor pentru a decide dacă să se permită acea adăugare sau nu

- Un sistem de fișiere trebuie să fie **montat** înainte de a putea fi accesat
- Un sistem de fișiere nemontat se montează la un **punct de montare** (un director de montare)
- Auto-montarea (la introducerea unei dischete, CD, stick USB, ș.a.)
- În UNIX/Linux: `mount()`, `umount()`

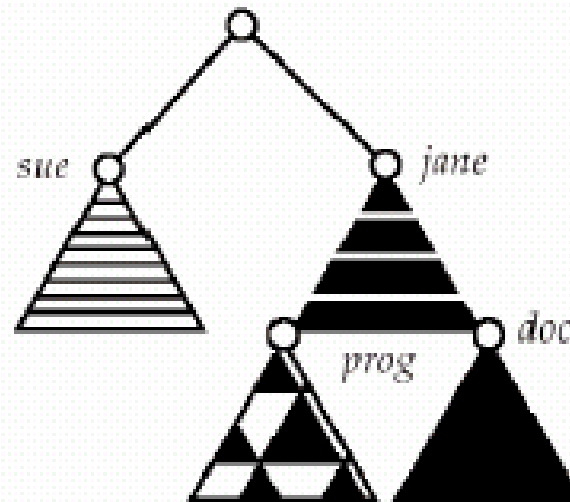
Montarea /2

- Exemplu:



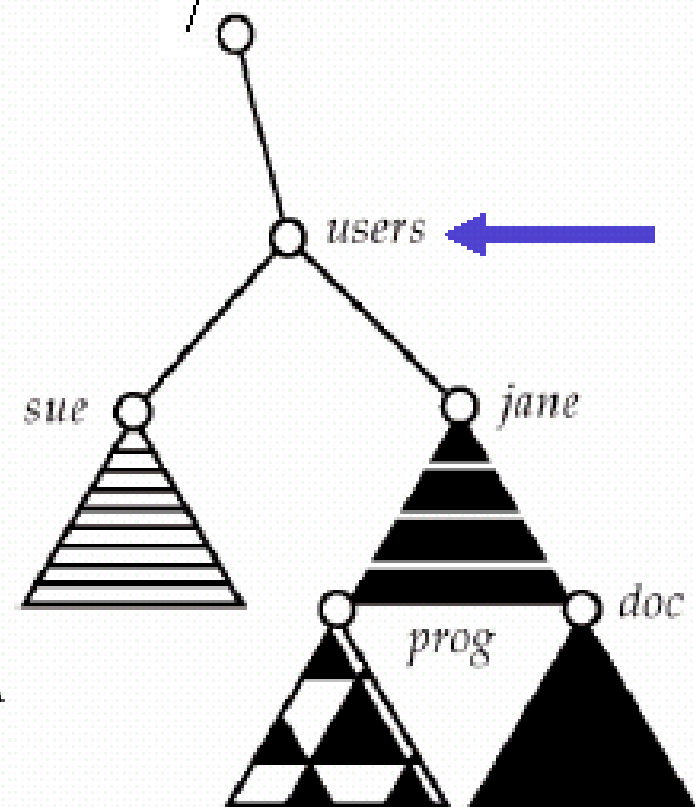
(a)

(a) Sistemul existent



(b)

(b) Partiție nemontată



Punctul de montare

Partajarea fișierelor

- Partajarea fișierelor în sisteme multi-utilizator este o facilitate de dorit
- Partajarea se poate face printr-o schemă de protecție
- În sisteme distribuite, fișierele pot fi partajate prin intermediul rețelei
- Metode:
 - **NFS** (Networked File System), ce utilizează RPC – SUN
 - **Partajarea fișierelor** din Windows 9x
 - **Active Directory**-ul din Windows Server 200x
 - **NIS+** (fostul **Yellow Pages**), folosit de UNIX/Linux

Protecția fișierelor

- Proprietarul/creatorul unui fișier trebuie să poată controla:
 - ce operații pot fi făcute asupra fișierului
 - și de către cine
- Drepturi de acces:
 - Read
 - Write
 - Execute
 - Append (adăugare de articole noi la sfârșitul fișierului)
 - Delete
 - List

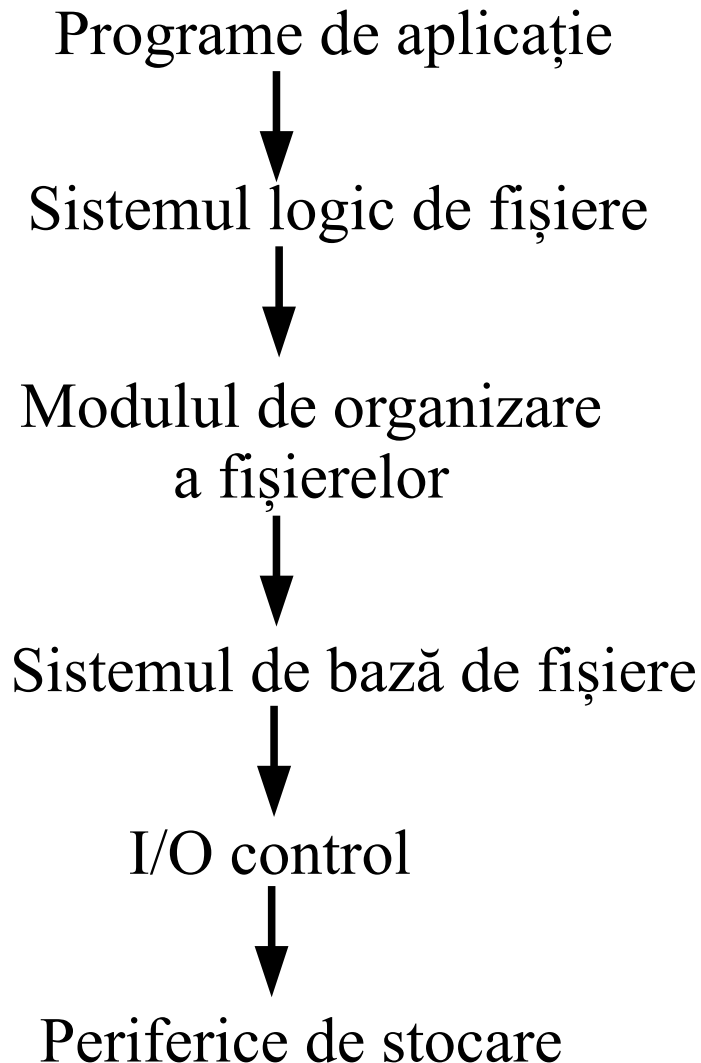
Implementarea sistemului de fișiere

- Structura sistemului de fișiere
- Implementarea fișierelor și directoarelor
- Metode de alocare
- Gestiunea spațiului liber
- Eficiența și performanța
- Recuperare
- Sisteme de fișiere jurnalizate
- Networked File System (NFS)
- Sisteme de fișiere “*next-generation*”

Structura sistemului de fișiere

- Structura fișierului
 - Unitatea logică de stocare
 - Colecție de informații înrudite
- Sistemul de fișiere se păstrează pe memoria secundară (perifericele de stocare – discuri: HD, FD, CD, ș.a.)
- Sistem de fișiere organizat pe nivele
- Blocul de control al fișierului – **FCB** (*File Control Block*) – structura de stocare pe disc ce conține anumite informații despre un fișier
- Deschiderea unui fișier pune la dispoziție un *handle* de referință la FCB, utilizat pentru accesele la fișier

Sistem de fișiere pe nivele

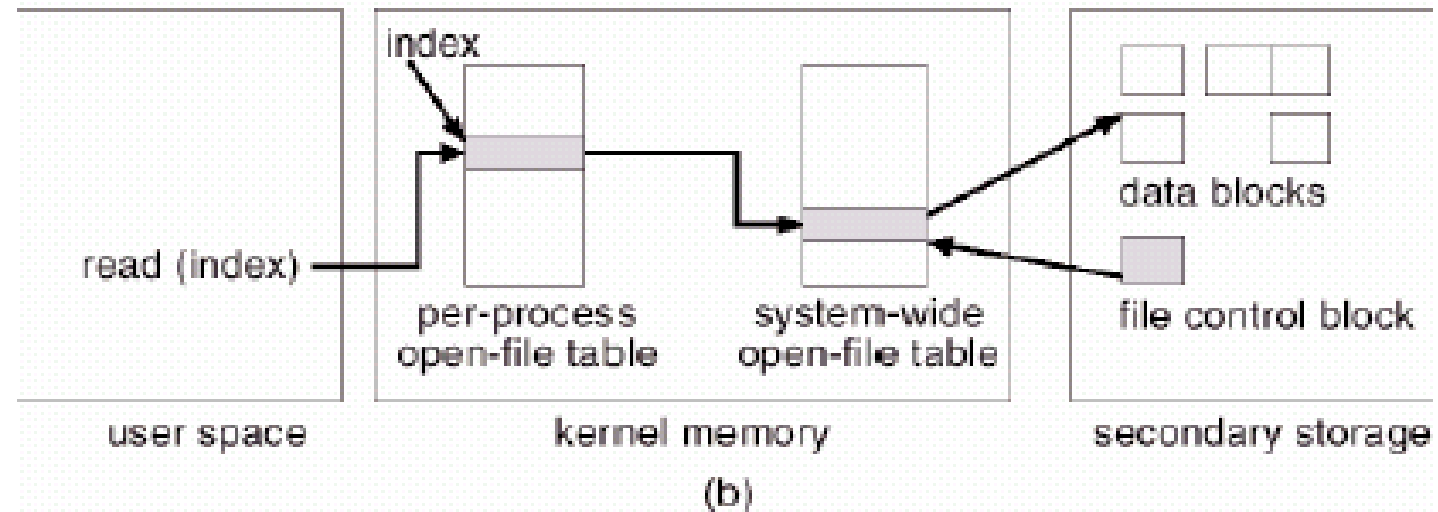
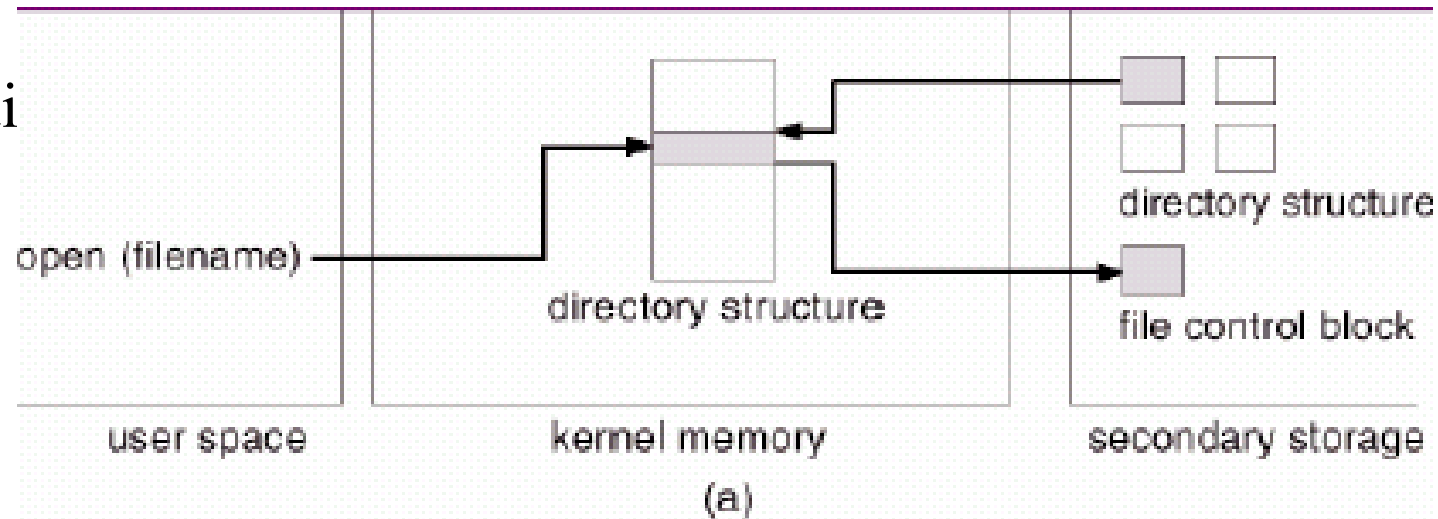


permisiuni de acces la fișier
data (creării, actualizării, accesării)
proprietar, grup, ACL
dimensiunea fișierului
blocurile de date

FCB-ul unui fișier

Structuri interne

Structurile sistemului
de fişiere necesare,
furnizate de S.O.-uri
(a) deschiderea
(b) citirea

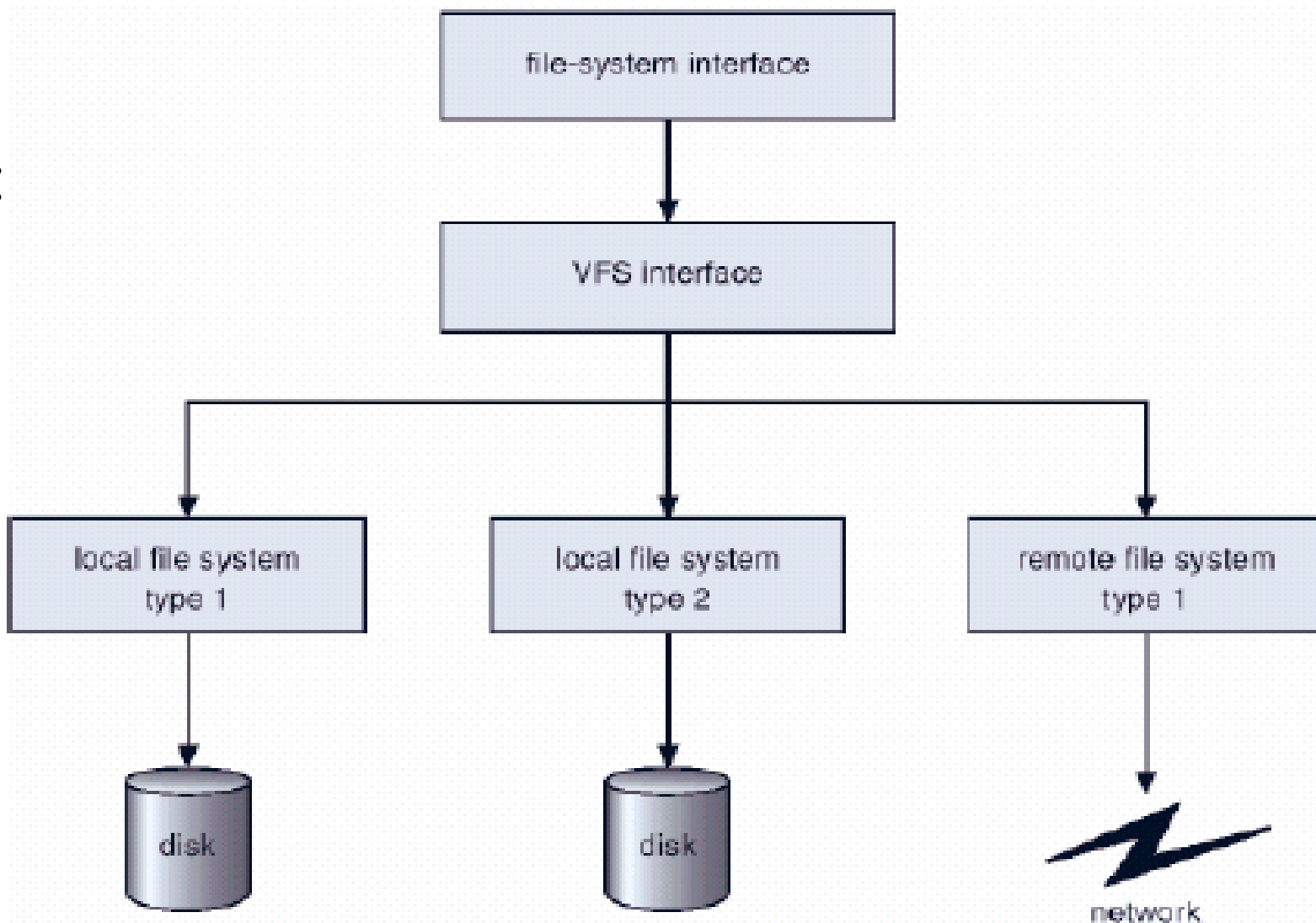


Sisteme de fișiere virtuale /1

- VFS furnizează o modalitate orientată-obiect de implementare a sistemelor de fișiere
- VFS permite utilizarea aceleiași interfețe a apelurilor de sistem (API-ul) pentru diferite tipuri de sisteme de fișiere (e.g. ext2fs, ext3fs, vfat, fat16, hpfs, ntfs, nfs, ...)
- Utilizat în nucleul Linux

Sisteme de fişiere virtuale /2

- Schema unui VFS:



Implementarea directoarelor

- Listă liniară de nume de fișiere cu pointeri către blocurile de date
 - simplu de programat
 - consumator de timp la execuție
- Tabelă hash – listă liniară cu structură de date hash
 - micșorează timpul de căutare în director
 - coliziuni – situații în care două nume de fișiere au prin funcția hash aceeași locație
 - dimensiune fixată

Metode de alocare /1

- Metoda de alocare se referă la modul în care blocurile disc sunt alocate pentru fișiere
 - **Alocare contiguă**
 - **Alocare înlănțuită**
 - **Alocare indexată**

- **Alocarea contiguă**

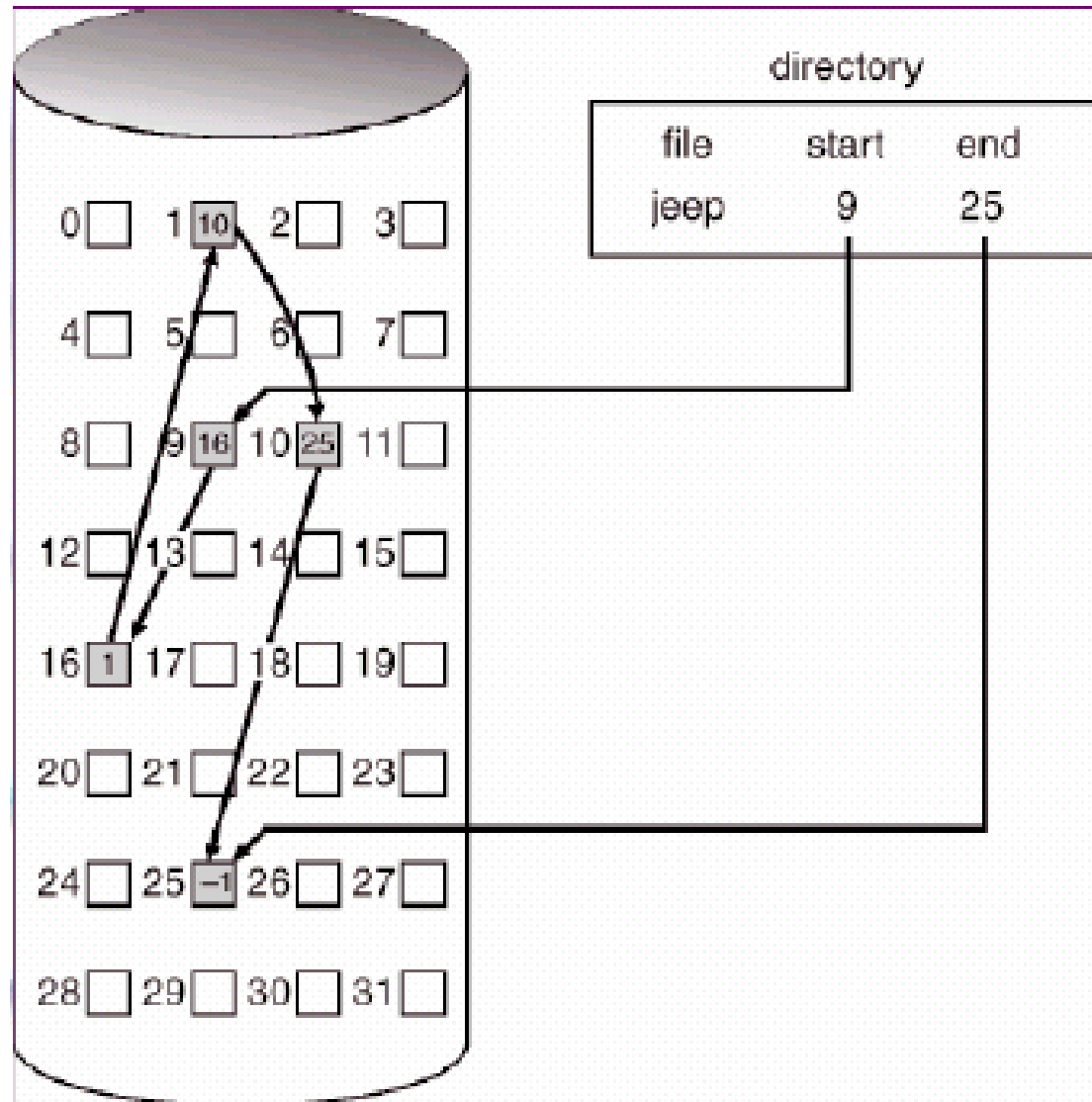
- Fiecare fișier ocupă o mulțime contiguă de blocuri pe disc
- Simplă – sunt necesare doar locația de start (numărul blocului de început) și lungimea (numărul de blocuri ocupate)
- Acces aleatoriu (direct)
- Risipă de spațiu (problema alocării dinamice a spațiului de stocare)
- Fișierele nu pot crește în dimensiune
- Alocarea bazată pe extindere – [Veritas File System](#)

- **Alocarea înlănțuită**

- Fiecare fișier este o listă liniară simplu înlănțuită de blocuri disc; fiecare bloc conține o parte din datele fișierului și o **referință** (un pointer) către următorul bloc
- Blocurile pot fi “împrăștiate” oriunde pe disc (nu mai sunt într-o zonă contiguă)
- Blocuri adiționale sunt alocate pe măsură ce fișierul crește în dimensiune
- Accesul secvențial nu ridică nici o problemă, în schimb accesul direct nu se poate face eficient
- Coruperea lanțului de pointeri provoacă probleme majore
- Tabela de alocare a fișierelor (**FAT**) – alocare utilizată de MS-DOS, Windows 9x și OS/2 <3.0

Metode de alocare /4

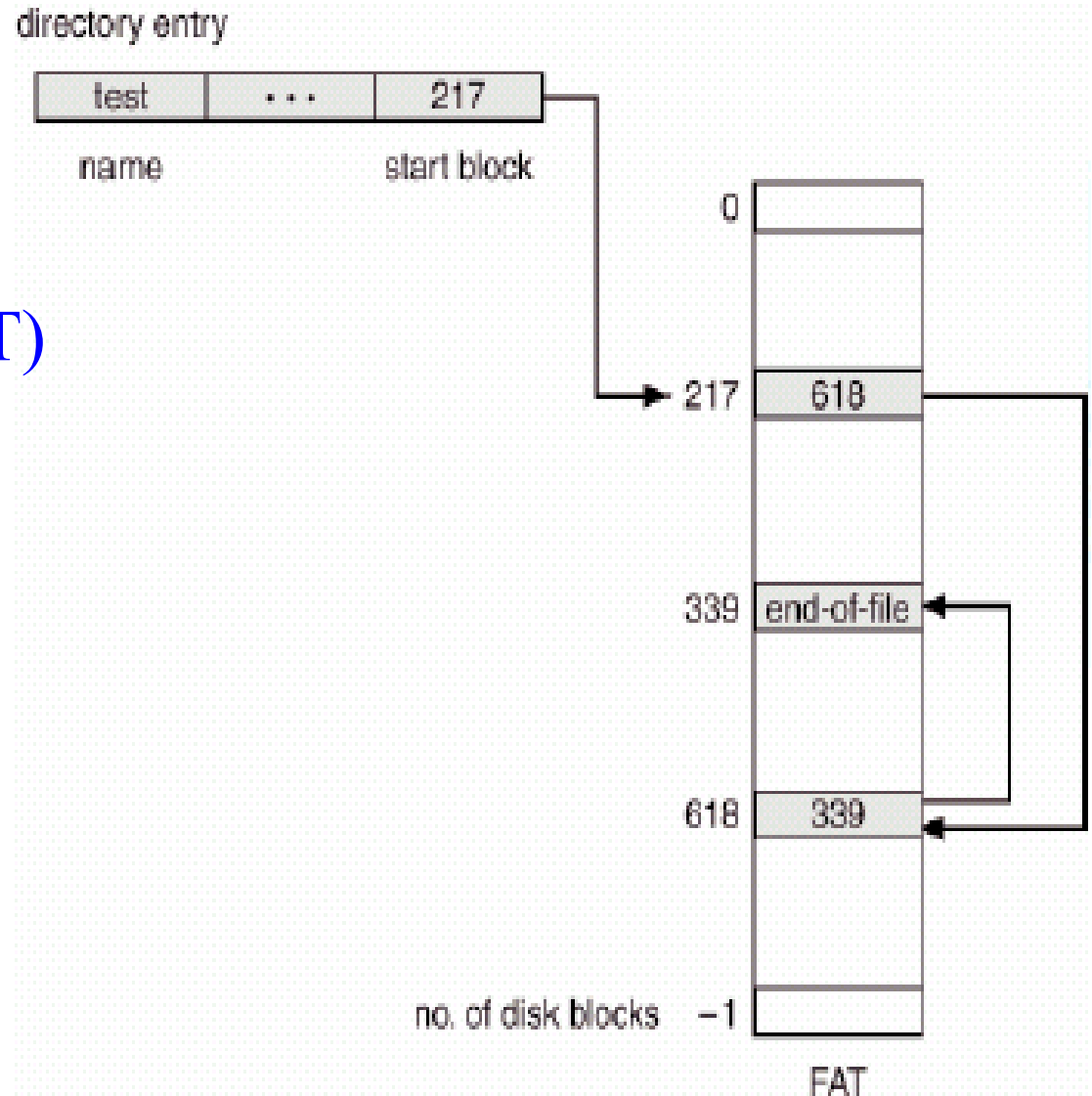
- Alocarea înlănțuită



Metode de alocare /5

- **Alocarea înlănțuită**

File Allocation Table (FAT)

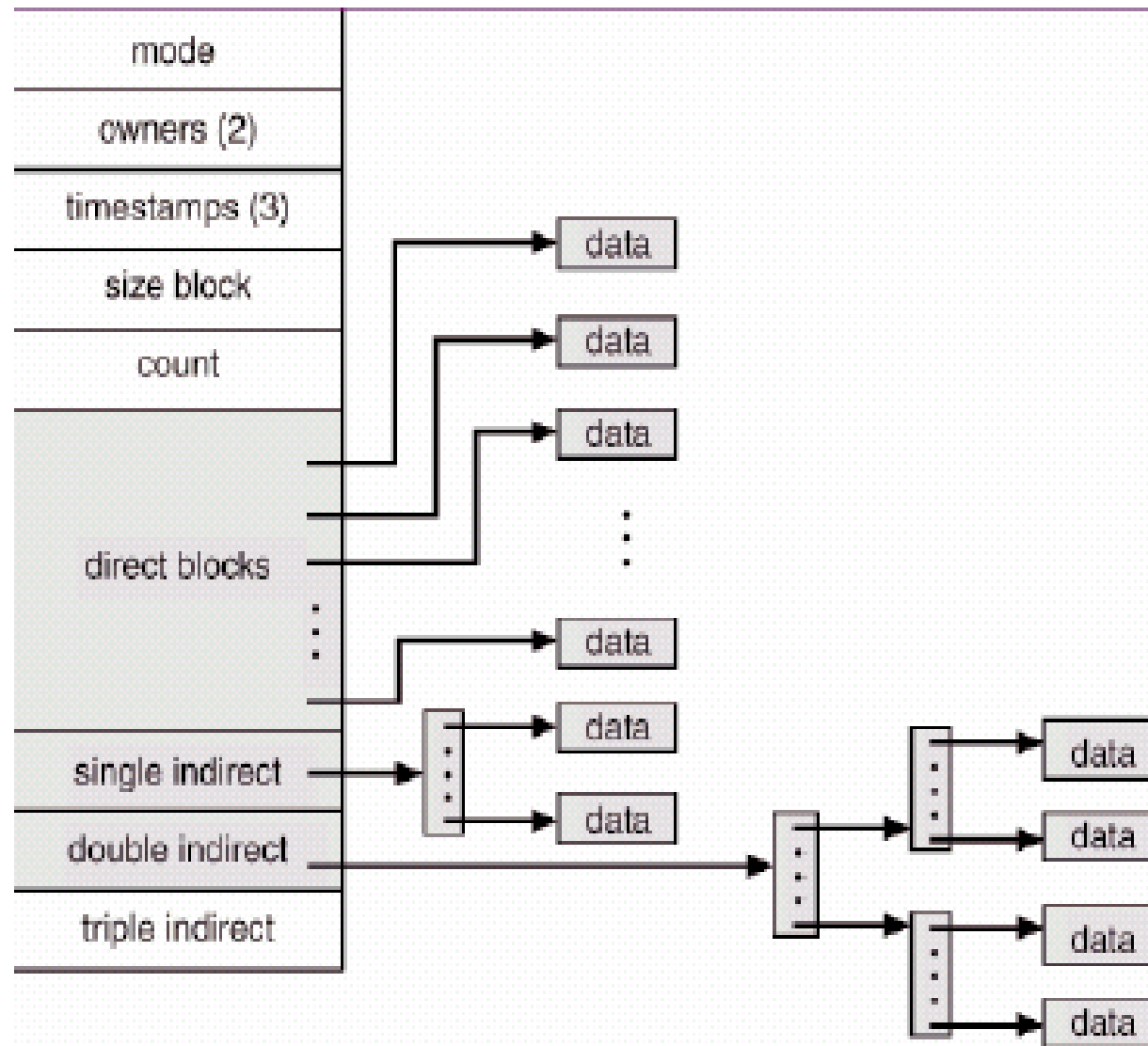


- **Alocarea indexată**

- Pune laolaltă toți pointerii într-un bloc de index (alocarea indexată grupează referințele împreună și le asociază cu un fișier particular)
- Se utilizează o tabelă de indecși
- Acces aleatoriu (direct)
- Acces dinamic fără fragmentare externă (optimizat pentru situația în care în sistemul de fișiere sunt multe fișiere mici), dar prezintă *overhead*-ul implicat de accesul suplimentar la blocul de index
- **i-noduri** – alocare utilizată de UNIX/Linux, Mac OS X

- **Alocarea indexată**

i-noduri UNIX

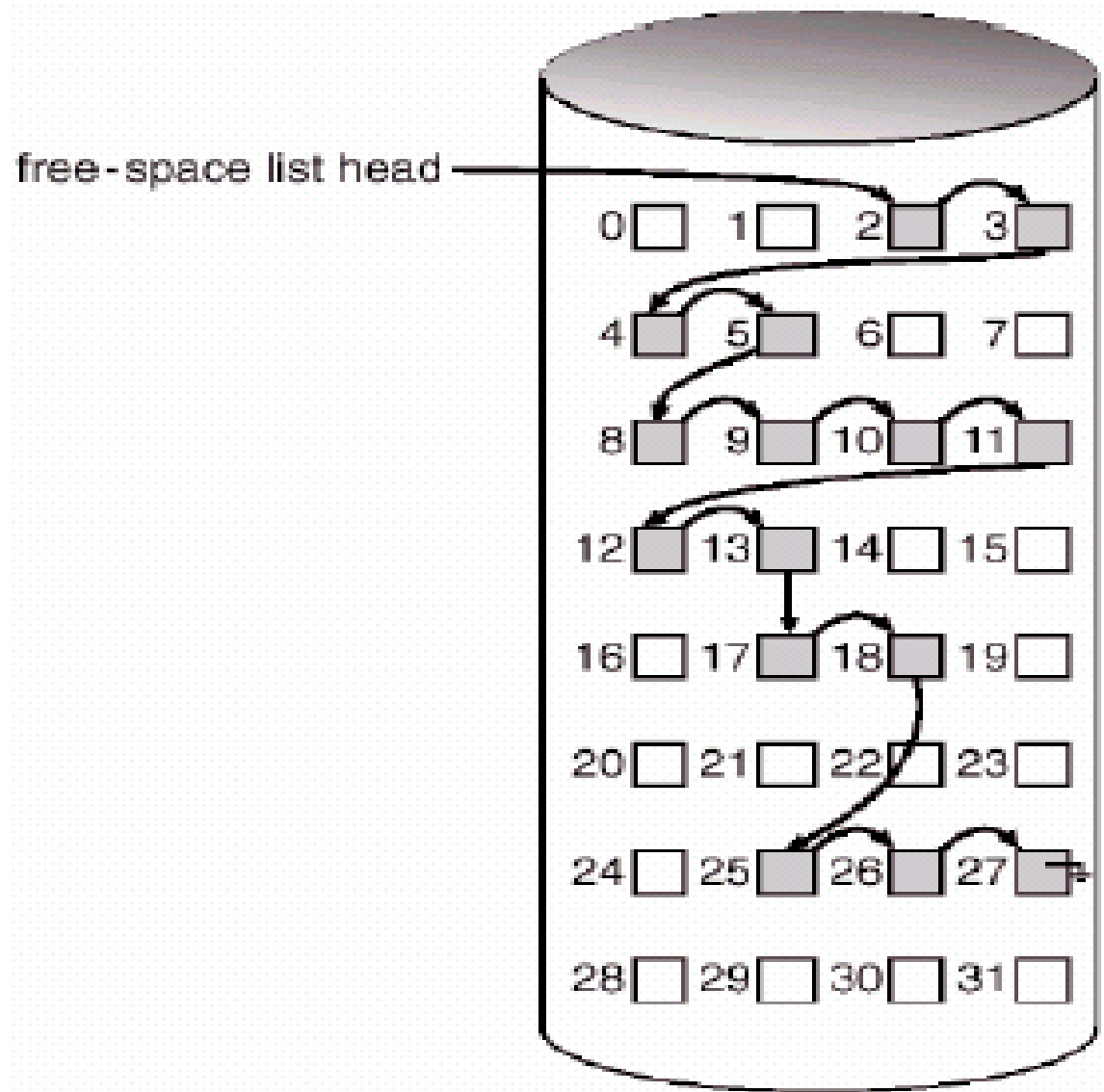


Gestiunea spațiului liber /1

- **Metode folosite pentru evidența blocurilor libere:**
 - **Vectorul de biți**
 - Instrucțiuni mașină pentru găsirea primului bit setat
 - Pentru eficiență, este nevoie de păstrarea vectorului de biți în memorie, nu pe disc
 - Exemplu: Mac OS
 - **Lista înlănțuită** : exemplu pe următorul slide
 - **Gruparea** : exemplu peste două slide-uri
 - Păstrarea unei liste cu zonele libere contigue – perechi de forma (numărul blocului liber de început, dimensiune)

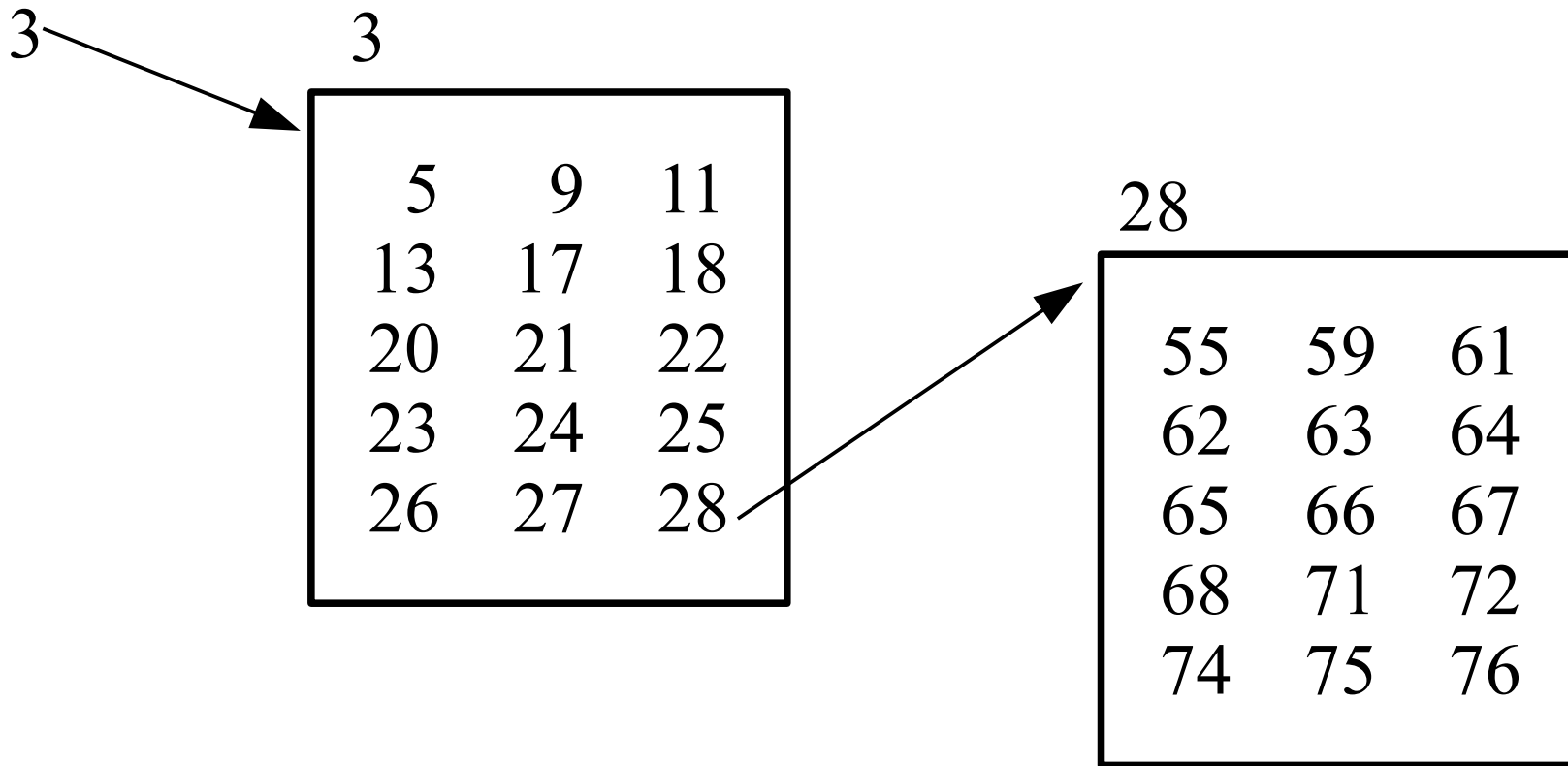
Gestiunea spațiului liber /2

- Lista înlanțuită a spațiului liber pe disc



Gestiunea spațiului liber /3

- Gruparea spațiului liber pe disc



Eficiența și performanța /1

- **Eficiența depinde de:**

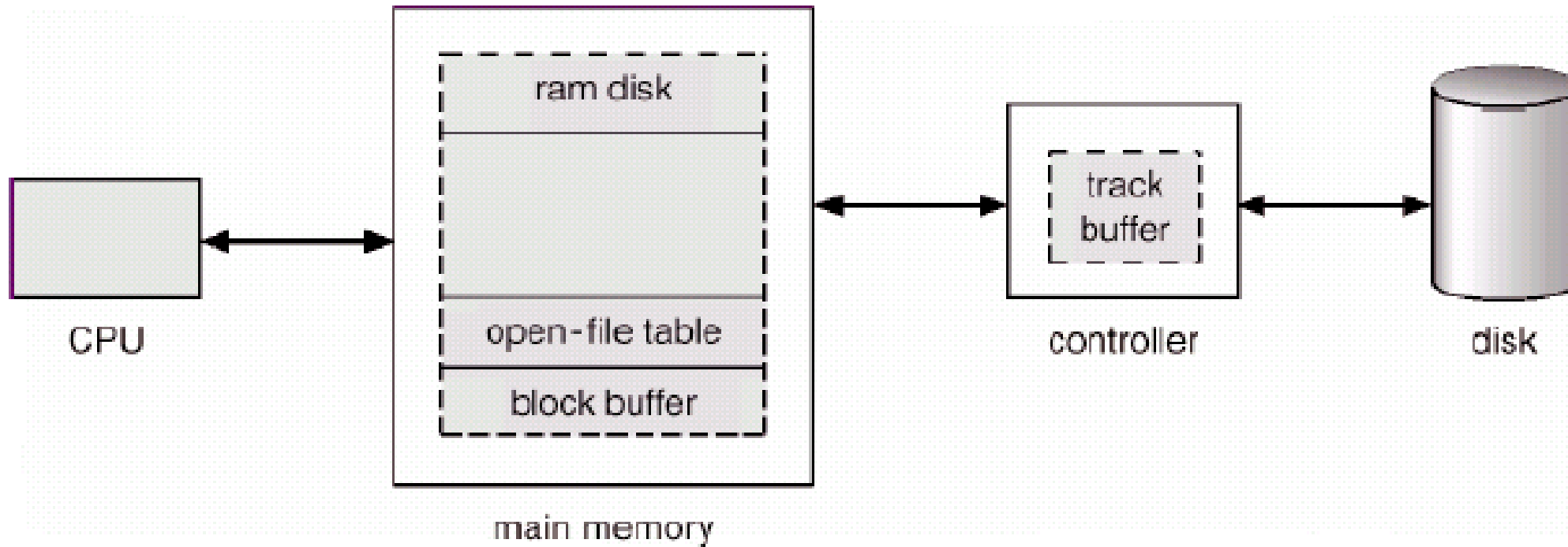
- Algoritmii de alocare a discului și pentru directoare
- Tipurile de date păstrate în intrarea fișierului din director

- **Performanța**

- **Disk cache** – o secțiune separată a memoriei principale utilizată ca memorie *cache* a blocurilor de disc utilizate frecvent
- **Free-behind** și **read-ahead** – tehnici de optimizare a accesului secvențial
- Dedicarea unei secțiuni a memoriei principale drept **disk virtual** (numit uneori și **RAM disk**)

Eficiența și performanța /2

- Localizarea *cache*-ului de disc



Recuperare

- Verificări de consistență – compară datele din structura de directoare cu blocurile de date de pe disc și încearcă să remedieze inconsistențele găsite; dezavantaj: poate dura mult timp. Unealta *consistency checker*: fsck (în UNIX) sau chkdsk (în Windows)
- Folosirea unor utilitare de sistem pentru **backup**-ul datelor de pe disc pe un alt periferic de stocare (disc extern, CD/DVD, bandă magnetică, ș.a.), operație ce trebuie realizată periodic!
+ Recuperarea unor fișiere “pierdute” / unui disc “pierdut” prin **restaurarea** datelor de pe un backup realizat anterior!
- O altă soluție interesantă este furnizată de sistemele de fișiere **jurnalizate**, respectiv, mai nou, de sistemele de fișiere “*next-generation*”

Sisteme de fișiere jurnalizate

- Sistemele de fișiere **jurnalizate** înregistrează fiecare actualizare a sistemului de fișiere ca pe o **tranzacție**
- Toate tranzacțiile sunt scrise într-un **jurnal** (*log*). O tranzacție este considerată **comisă** atunci când este scrisă în jurnal. Totuși, se poate ca sistemul de fișiere să nu fi fost încă actualizat
- Tranzacțiile din jurnal sunt operate asincron în sistemul de fișiere. După ce sistemul de fișiere este modificat, tranzacția este ștearsă din jurnal
- Dacă sistemul “crapă”, toate tranzacțiile rămase în jurnal trebuie să fie operate după repornirea sistemului
- Exemple: **NTFS** (Windows) sau *ext3fs/ext4fs* (Linux)

Networked File System

- **NFS** = o implementare a firmei Sun Microsystems (în S.O.-urile Solaris, SunOS) și o specificație a unui sistem software pentru accesarea fișierelor de la distanță în rețele LAN (sau WAN)
- Stațiile de lucru interconectate prin intermediul rețelei sunt privite ca o mulțime de mașini independente, cu sisteme de fișiere independente, ce permit partajarea fișierelor între aceste sisteme de fișiere, într-o manieră transparentă pentru utilizator (folosind protocoalele de montare, RPC, UDP/IP, modelul client-server)
- Specificațiile NFS sunt independente de hardware, sistem de operare și tehnologia de rețea

Sisteme de fișiere “*next-generation*”

- Sistemele de fișiere “*next-generation*” încearcă să ofere *toleranță la tipuri de erori* ce nu sunt prevenite de tehnologiile anterioare: sistemele de fișiere jurnalizate (ce oferă protecție doar la nivelul structurii *file-system*-ului, nu și pentru conținutul fișierelor) sau tehnicile RAID (ce oferă protecție doar în anumite situații, dar nu și contra “bit rot”)
- “**Bit-rot**” = “the silent corruption of data on disk; one at a time, from time to time, a random bit here or there gets flipped.”
Referință: <https://arstechnica.com/information-technology/2014/01/bitrot-and-atomic-cows-inside-next-gen-filesystems/>
- Exemple: **ZFS** (Solaris), **btrfs** (Linux) sau, mai nou, **ReFS** (Windows)
- Protecția contra “bit-rot”: ZFS stochează *checksums* pentru toate datele și metadatele unui volum, și astfel poate detecta (și corecta) coruperea acestora
- Alte facilități: operații *COW* atomice, *snapshots*, *built-in handling of disk failures and redundancy*, *integration of RAID functionality*, etc.

- **Bibliografie obligatorie**

capitolele despre *sisteme de fişiere* din

- Silberschatz : “*Operating System Concepts*”

(cap.9 şi cap.10 din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.4 din [MOS3])

Sumar

- Conceptul de fișier
- Interfața sistemului de fișiere
- Implementarea sistemului de fișiere

Întrebări ?

Sisteme de Operare

Administrarea perifericelor de stocare

Cristian Vidrașcu

<https://profs.info.uaic.ro/~vidrascu>

- Introducere
- Memoria secundară (discul)
 - Structura
 - Planificarea
 - Administrarea
- Gestiunea spațiului de swap
- Construirea unui disc mai bun
- Administrarea memoriei terțiare

- Clasificarea perifericelor (d.p.d.v. funcțional)
 - **Periferice de intrare/ieșire** – pentru schimbul de informații cu mediul extern
(e.g. tastatură, ecran, imprimantă, cititor de cartele, ș.a.)
 - **Periferice de stocare** – pentru păstrarea nevolatilă (i.e. permanentă) a informațiilor
(e.g. disc magnetic, bandă magnetică, dischetă, CD/DVD, ș.a.)

Alte clasificări ale perifericelor /1

- Clasificarea perifericelor d.p.d.v. al modului de operare (de servire a cererilor)
 - **Periferice dedicate** – pot servi un singur proces la un moment dat
(modelul tipic: imprimanta)
 - **Periferice partajabile** – pot servi mai multe procese simultan, în sensul concurenței aparente
(modelul tipic: discul magnetic)

Alte clasificări ale perifericelor /2

- Clasificarea perifericelor d.p.d.v. al modului de transfer și de memorare a informației

– Periferice bloc

- memorează informațiile în *blocuri* de lungime fixă, fiecare cu adresa sa
- blocul este unitatea de transfer între periferic și memoria internă
- fiecare bloc poate fi citit sau scris independent de celelalte blocuri
- blocul conține informații propriu-zise și mai poate conține, eventual, informații de control de paritate, pentru verificarea corectitudinii informației memorate
- e.g. disc magnetic, bandă magnetică, CD/DVD, memorii flash, ș.a.

– Periferice caracter

Alte clasificări ale perifericelor /3

- Clasificarea perifericelor d.p.d.v. al modului de transfer și de memorare a informației (cont.)
 - **Perifere bloc**
 - **Perifere caracter**
 - furnizează/acceptă un *flux* de octeți fără nici o structură de bloc
 - octeții din flux nu sunt adresabili
 - fiecare octet este disponibil ca și *character curent* numai până la apariția următorului octet
 - e.g. tastatură, mouse, ecran, imprimantă, difuzor, ș.a.

Periferice de stocare

- Clasificarea perifericelor de stocare după variația timpului de acces (t_{ij} = timpul de acces de la informația i la informația j)
 - **Periferice cu acces secvențial** – timpul de acces t_{ij} are variații foarte mari (modelul tipic: banda magnetică)
 - **Periferice cu acces complet direct** – timpul de acces $t_{ij}=k$, este constant (modelul tipic: memoriile bidimensionale obișnuite – RAM)
 - **Periferice cu acces direct** – timpul de acces t_{ij} are variații foarte mici (modelul tipic: discul magnetic)

Structura discului /1

- Discurile magnetice sunt adresate ca o matrice 1-dimensională (un vector) foarte mare de **blocuri logice**, unde blocul logic este cea mai mică unitate de informații ce se poate transfera (între disc și memorie)

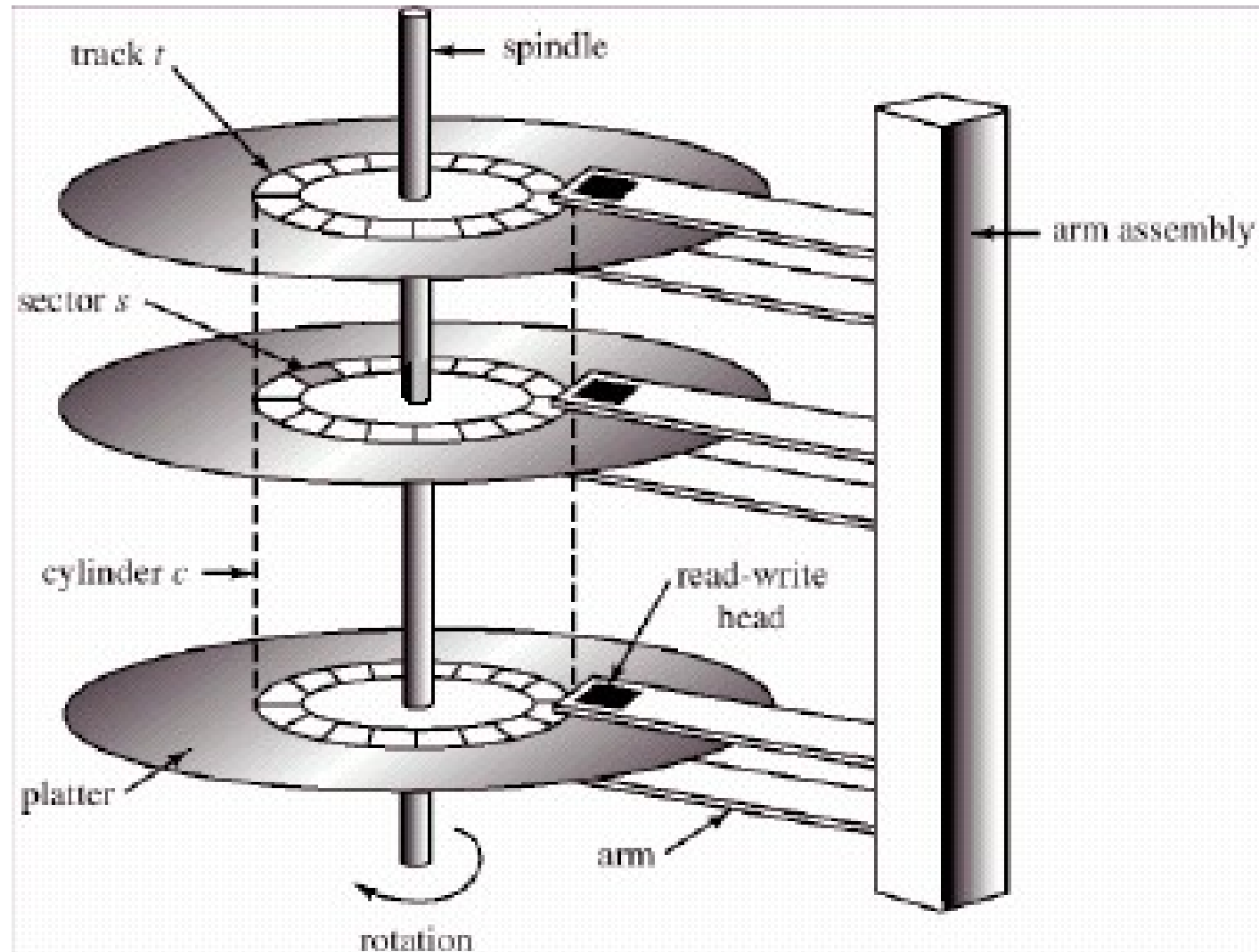
Indexul unui bloc în acest vector este adresa sa LBA.

- Matricea 1-dimensională de blocuri logice este mapată secvențial pe sectoarele discului

Adresa LBA se convertește în adresa fizică (C,H,S), de către BIOS (în trecut) sau firmware-ul discului (în prezent).

Structura discului /2

- Organizarea discului
 - platane
 - piste
 - cilindri
 - sectoare
 - data



Viteza discului

- **Timpul de căutare** (*Seek time*)
 - timpul necesar pentru mișcarea (mecanică a) capului de citire-scriere până la pista specificată
- **Latența de rotație** (*rotational Latency*)
 - timpul de așteptare necesar pentru ca sectorul specificat să ajungă prin rotație sub capul de citire-scriere
- **Timpul de transfer** (*Transfer time*)
 - timpul necesar pentru a citi datele de pe sectorul specificat
- **Timpul total de acces** = $S + L + T$

Probleme

- **Planificarea accesului la disc**
 - ideea este de a reorganiza cererile de acces la disc pentru a minimiza timpii de căutare (*seek*-urile)
- **Plasarea datelor pe disc (*Layout*)**
 - un mod de plasare care să minimizeze *overhead*-ul operațiilor cu discul
- **Construirea unui **disc mai bun** (sau a unui substituent pentru discurile actuale)**
 - exemplu: RAID

Planificarea discului /1

- Sistemul de operare este responsabil pentru utilizarea eficientă a hardware-ului
 - pentru discurile hard, aceasta înseamnă a avea un timp de acces rapid și o lățime de bandă mare
- Lățimea de bandă a discului (*disk bandwidth*) este numărul total de octeți transferați, împărțit la timpul total scurs între prima cerere de serviciu și sfârșitul ultimului transfer solicitat

Planificarea discului /2

- Timpul de acces are două componente majore:
 - *seek time* – timpul necesar discului pentru a muta capul pe cilindrul ce conține sectorul dorit
 - *rotational latency* – timpul adițional de așteptare pentru ca discul să rotească sectorul dorit sub capul de citire-scriere
 - cea de-a treia componentă, *timpul efectiv de transfer*, este o constantă specifică perifericului respectiv
- Planificarea discului urmărește **minimizarea timpului de căutare**, eventual și a latenței de rotație
- Ideea: schimbarea ordinii de servire a cererilor venite de la procesele concurente (cu păstrarea ordinii cererilor fiecărui proces)

Planificarea discului /3

- Algoritmi de planificare (a acceselor la disc)
 - FCFS (First Come, First Served)
 - SSTF (Shortest Seek Time First)
 - SCAN
 - C-SCAN (Circular SCAN)
 - LOOK
 - C-LOOK (Circular LOOK)

Planificarea discului /4

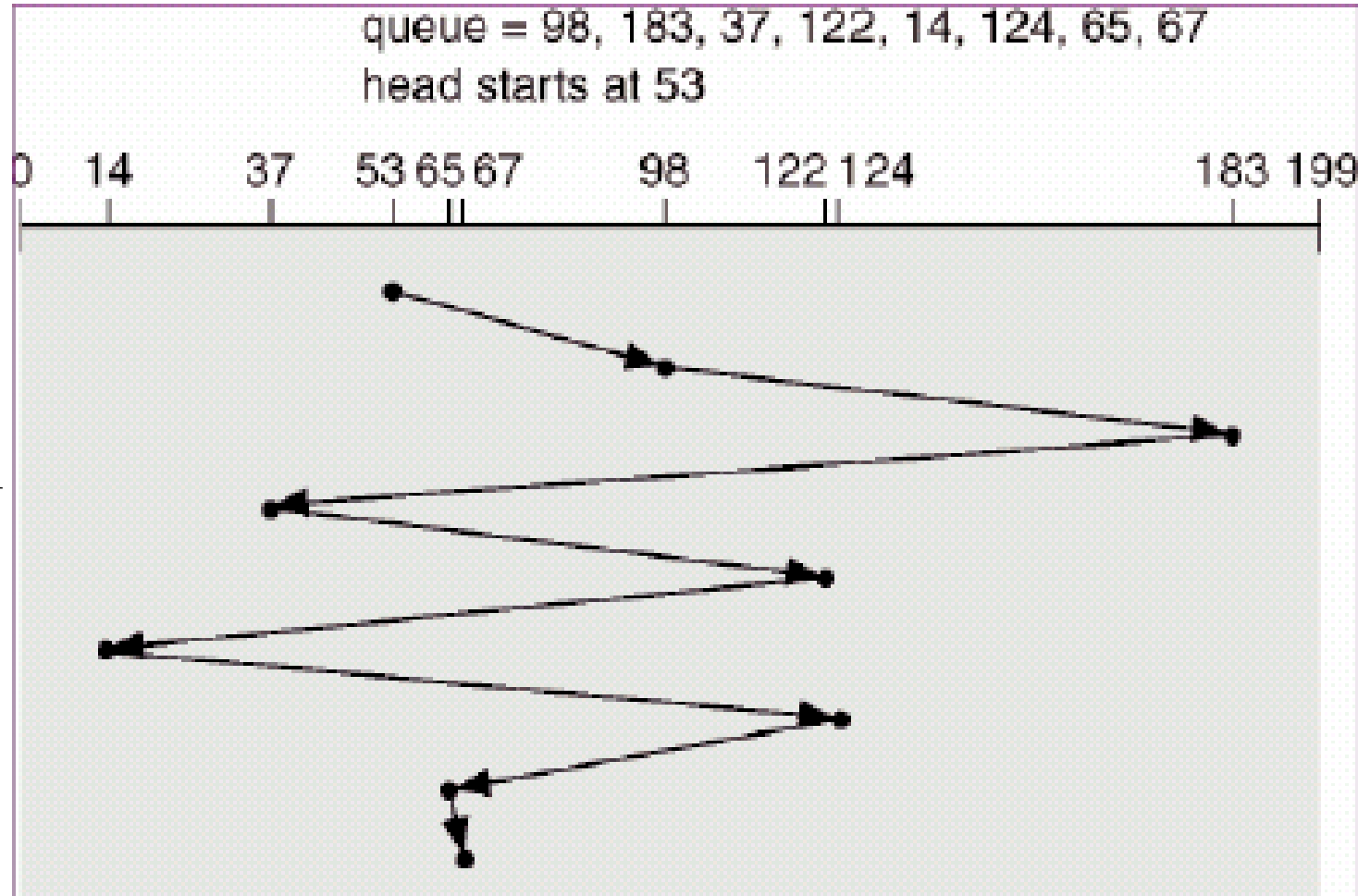
- Pentru exemplificările următoare ale algoritmilor de planificare, considerăm următorul scenariu:
 - un disc cu 200 cilindri (numerotați cu 0-199)
 - coada cererilor de acces (doar cilindrul ce conține sectorul dorit): 98, 183, 37, 122, 14, 124, 65, 67 (fiecare cerere provenind de la un proces distinct)
 - poziția inițială a furcii cu capetele de citire-scriere a discului: cilindrul 53

Planificarea discului /5

- **FCFS** (First Come, First Served)

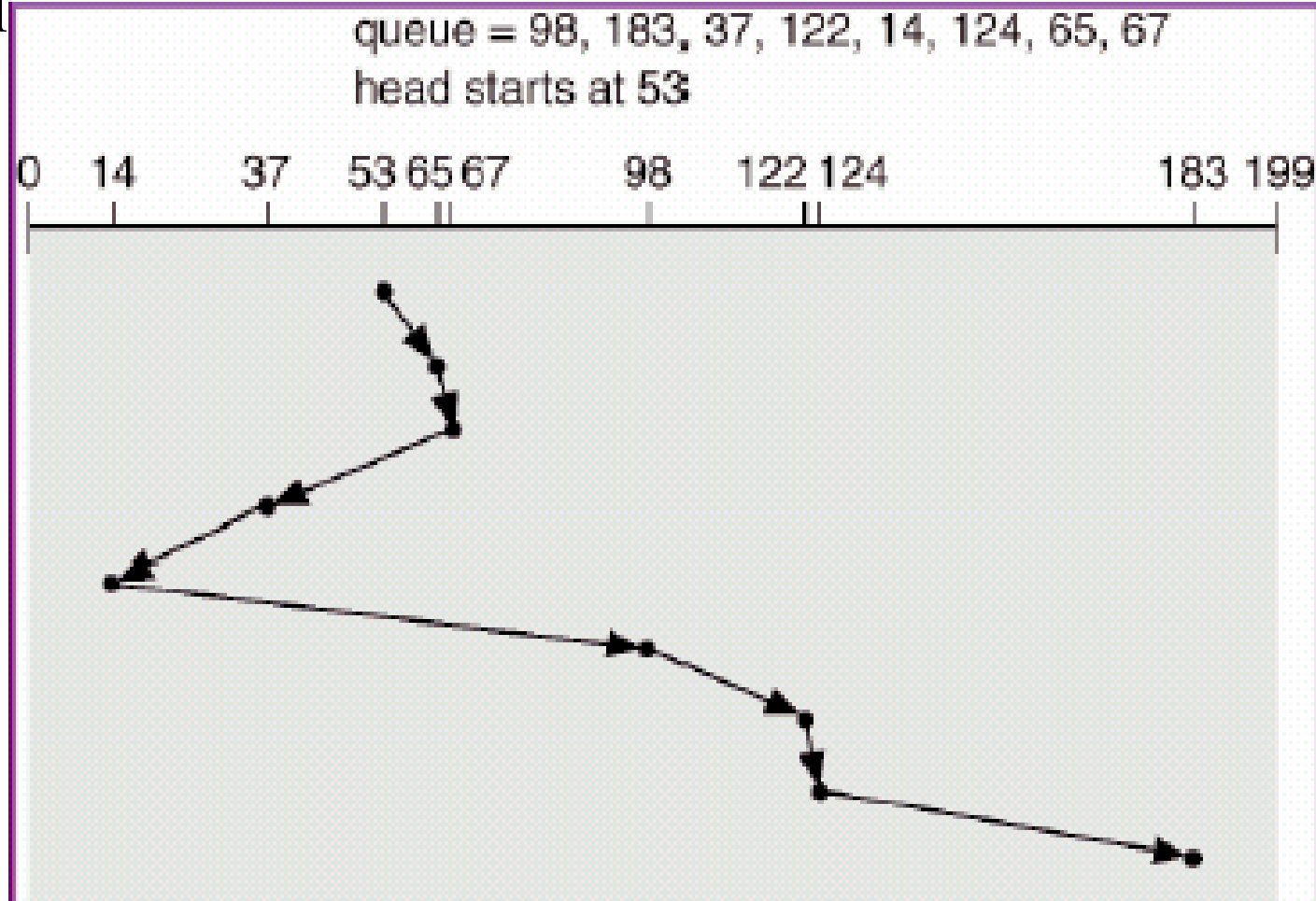
- Alg.: cererile sunt servite în ordinea sosirii
- Figura arată o mișcare totală a capului discului de 640 cilindri

Ipoteză de lucru:
coada statică
(i.e. **toate** cererile au ajuns în coadă la momentul $t=0$, în ordinea specificată)



Planificarea discului /6

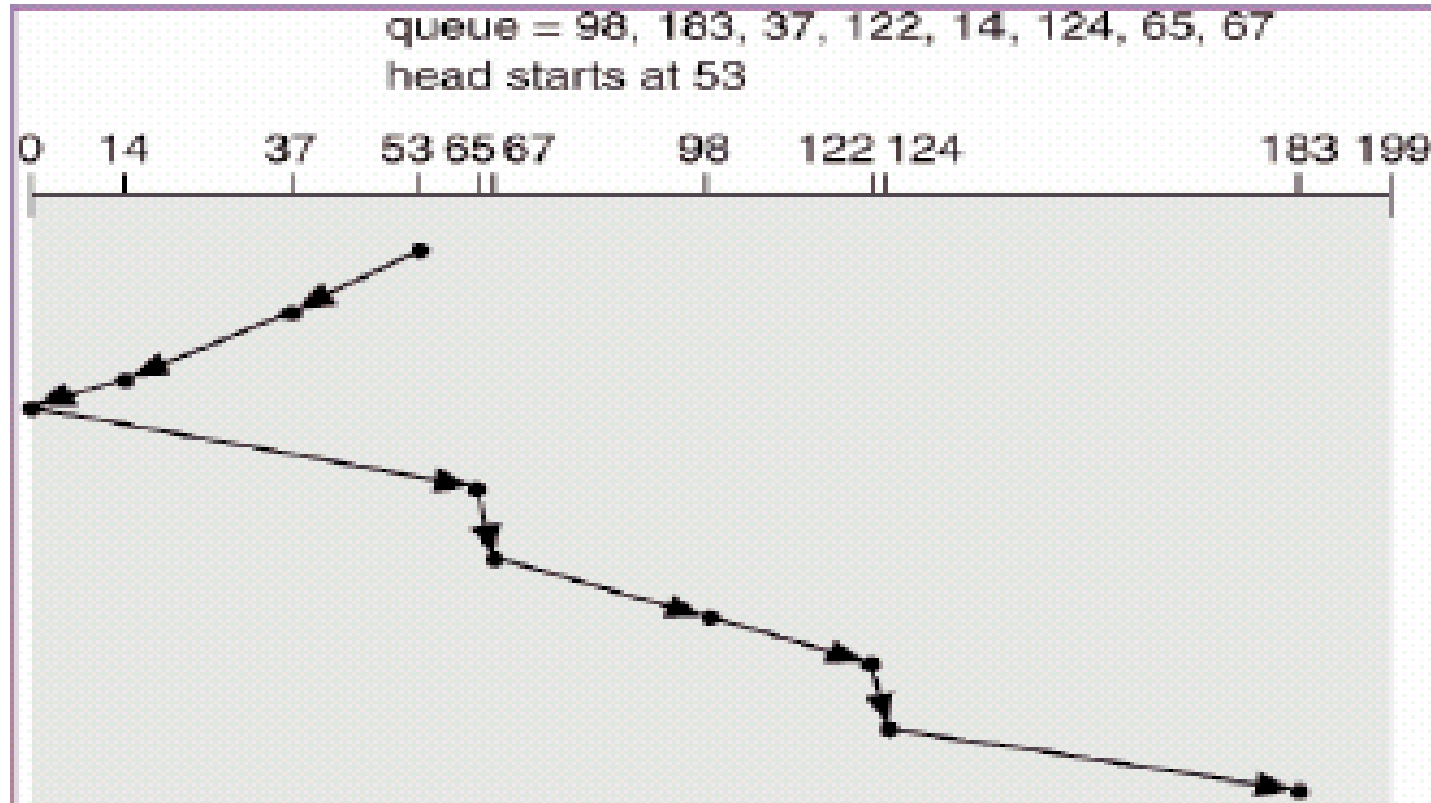
- **SSTF** (Shortest Seek Time First)
 - Alg.: se alege cererea cu timp de căutare minim de la poziția curentă a capului
 - Figura arată o mișcare totală a capului discului de 236 cilindri
 - Este mai eficient decât FCFS, dar nu este echitabil (poate favoriza fenomenul de *starvation*: întârzierea servirii unor cereri)



Planificarea discului /7

- **SCAN** (algoritmul elevator)
 - Alg.: brațul cu capetele R/W începe la un capăt al discului și se deplasează spre celălalt capăt, rezolvând și cererile pe parcurs, iar când ajunge la celălalt capăt, se întoarce înapoi, continuând servirea cererilor

- Figura arată o mișcare totală a capului discului de 236 cilindri (în ipoteza că sensul inițial de deplasare era în jos. Altfel, mișcarea totală este de 331 cilindri).



Planificarea discului /8

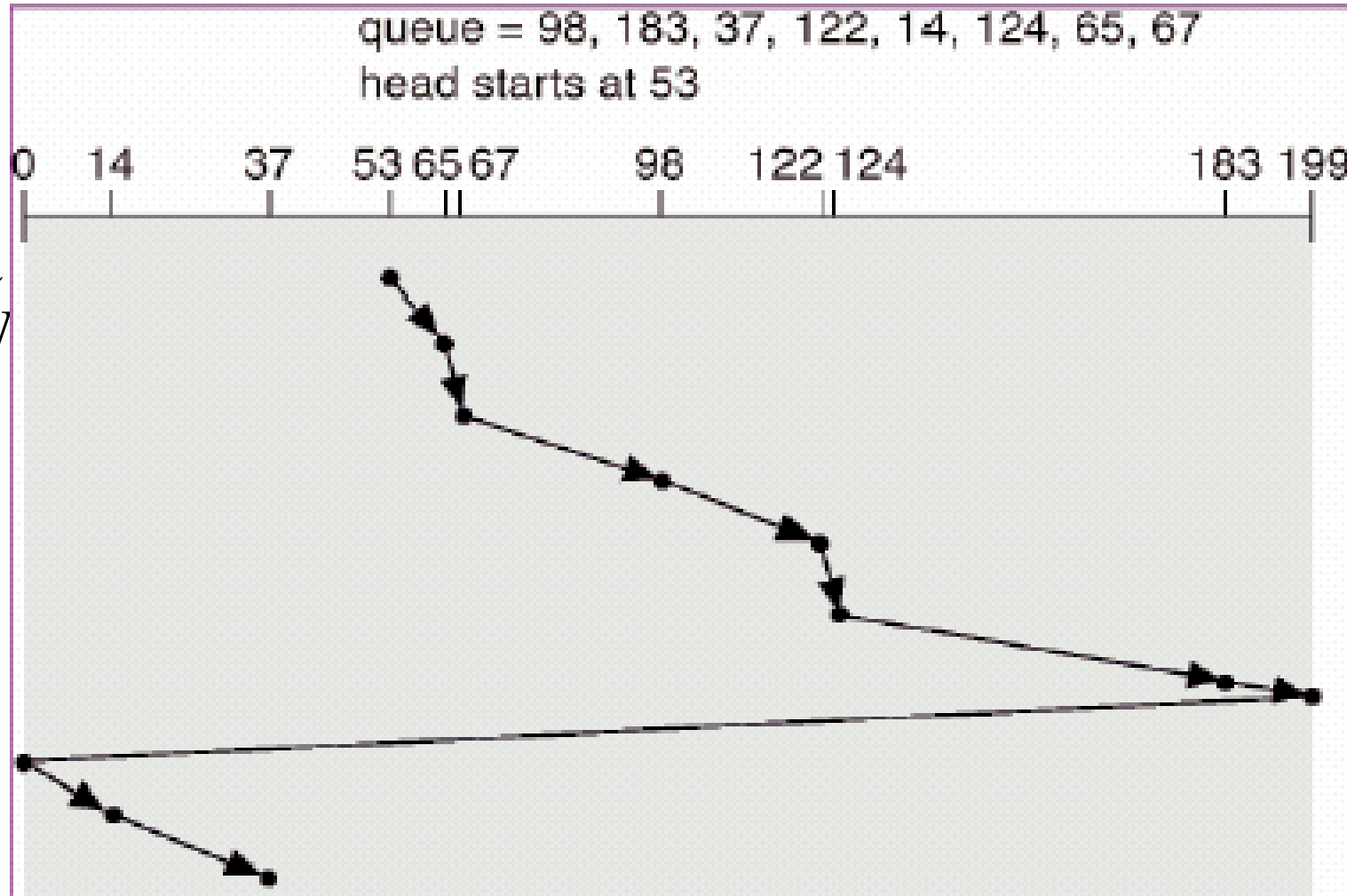
- **C-SCAN** (Circular SCAN)
 - Alg.: brațul cu capetele R/W începe la capătul 0 al discului și se deplasează spre celălalt capăt, rezolvând și cererile pe parcurs, iar când ajunge la celălalt capăt, se întoarce imediat (*foarte rapid*) înapoi la începutul discului, fără să servească nici o cerere pe drumul de întoarcere, și apoi reia lucrul
 - Practic, acest algoritm tratează cilindrii discului ca o listă circulară care “conectează” ultimul cilindru cu primul cilindru
 - Avantaj: furnizează un timp de așteptare (a rezolvării cererii) mai uniform decât algoritmul SCAN (la care este posibil ca o cerere să aștepte două parcurgeri ale discului până când este servită)

Planificarea discului /9

- **C-SCAN** (Circular SCAN)

- Exemplu

- Figura arată o mișcare totală a capului R/W a discului de 382 cilindri



Planificarea discului /10

- **LOOK și C-LOOK** (Circular LOOK)

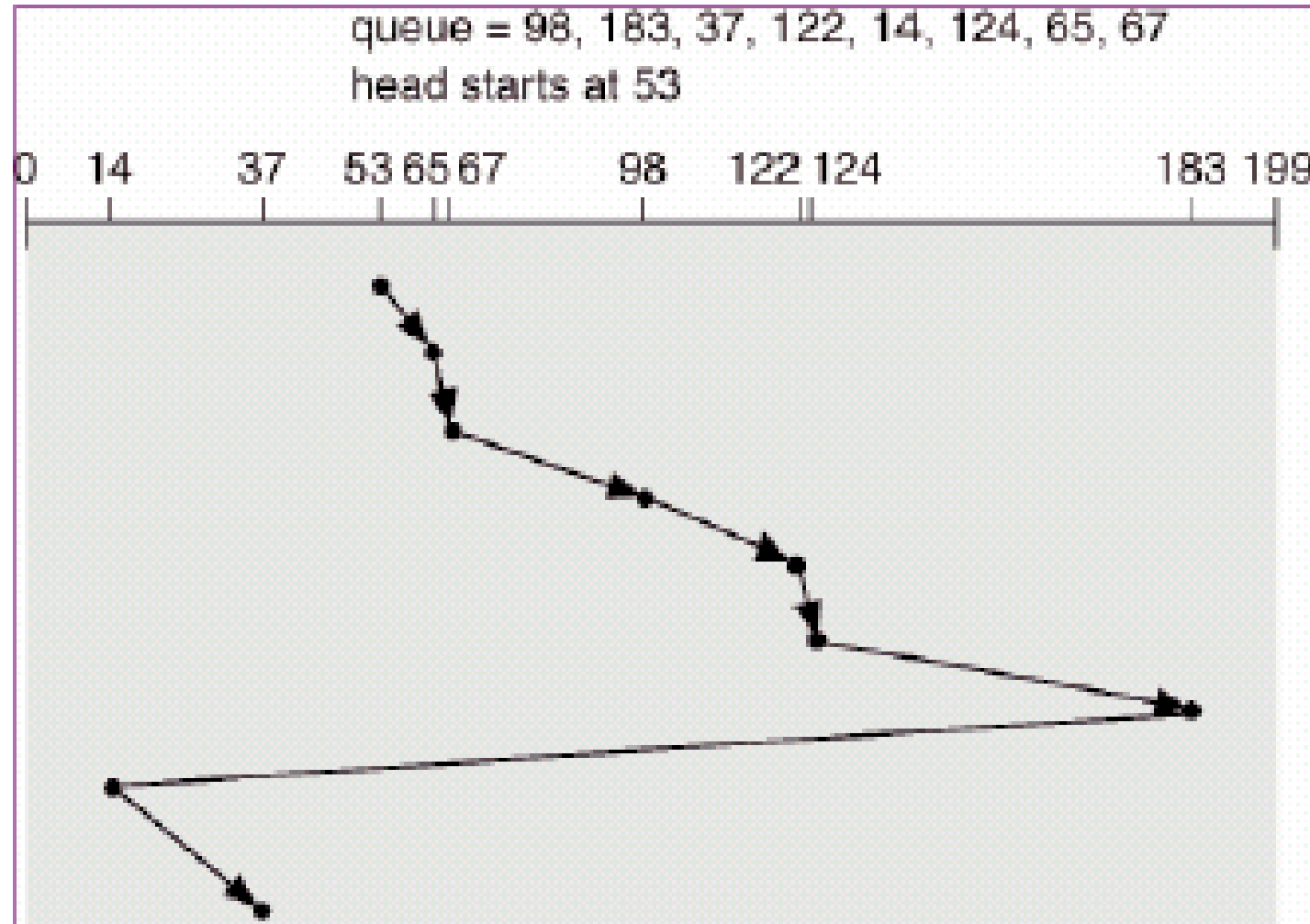
- Alg.: brațul cu capetele R/W se mișcă la fel ca la algoritmi SCAN și respectiv C-SCAN, cu deosebirea că aici se mișcă doar până la ultima cerere în fiecare sens, după care inversează direcția de deplasare imediat, fără să meargă mai întâi până la capătul discului
- Practic, LOOK/C-LOOK reprezintă o optimizare a algoritmilor SCAN/C-SCAN

Planificarea discului /11

- **LOOK și C-LOOK** (Circular LOOK)

- Exemplu

- Figura arată o mișcare totală a capului discului de 322 cilindri pentru C-LOOK (pentru LOOK ar fi 276 cilindri)



Planificarea discului /12

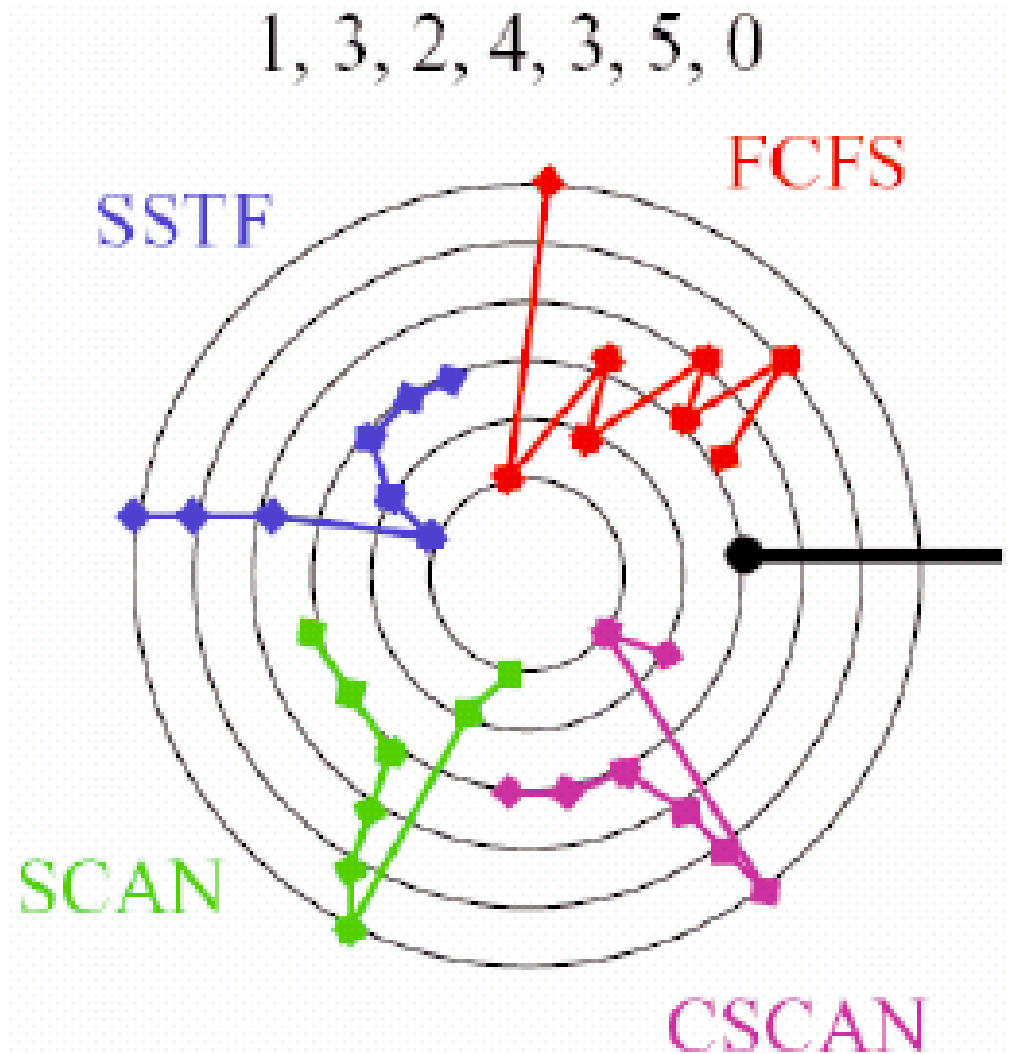
- **Selectarea unui alg. de planificare a discului**
 - SSTF este utilizat frecvent (dezavantaj: pericol de înfometare a unor cereri de acces la disc)
 - SCAN și C-SCAN se comportă mai bine pentru sisteme care au o încărcare mare a discului (i.e. multe operații I/O)
 - Algoritmul de planificare a discului este indicat să fie scris ca un modul separat al sistemului de operare, pentru a permite o înlocuire ușoară a sa cu un alt algoritm dacă se consideră necesar
 - Fie SSTF, fie LOOK este o alegere rezonabilă pentru algoritmul implicit de planificare a discului

Planificarea discului /13

- **Observații**

- Performanța depinde de numărul și tipurile cererilor de acces
- Cererile de acces pot fi influențate de metoda de alocare a fișierelor utilizată

Alt exemplu:



- **Observații** (cont.)

- De exemplu, la alg. SSTF cele mai favorizate d.p.d.v. al timpului de acces vor fi pistele din mijloc; ca urmare se pot alocă pe aceste piste fișierele cu frecvență ridicată de utilizare, sau structura de directoare (deoarece este folosită frecvent)
- La fel, la o recompactare (defragmentare) a discului, se pot alocă în aceste zone favorizate fișierele pentru care s-a constatat că au un grad de folosire mai ridicat

Planificarea discului /15

• Observații (cont.)

- în exemplele anterioare am simplificat expunerea, presupunând coada ca fiind statică (i.e. **toate** cererile ajungeau în coadă la momentul $t=0$)
- în realitate, coada este dinamică (i.e. **noi** cereri ajung în coadă pe parcursul trecerii timpului); iată un exemplu în acest sens:

Coada de cereri: 24, 45, 85 (toate ajung la $t=0$) ; 120, 97, 61 (toate la $t=100$ ms)

Se dau: *seek time* = direct proporțional cu distanța (2 ms/traversarea a 2 cilindri consecutivi);
latența de rotație: în medie, 1 ms (pentru toate cererile); *timpul de transfer* = neglijabil.

Inițial, capul R/W este la cilindrul 50, iar sensul de deplasare este spre cilindrul cu nr. maxim.

Ordinea de servire a celor 6 cereri, folosind algoritmul LOOK, este:

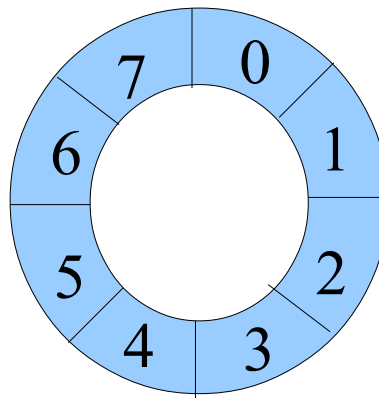
- 1) la momentul $t=0$, capul R/W este poziționat la cilindrul 50;
- 2) la momentul $t=70$ ms, capul ajunge la cilindrul 85, iar după 1ms începe servirea cererii respective;
- 3) la momentul $t=151$ ms, capul ajunge la cilindrul 45, iar după 1ms începe servirea cererii respective;
- 4) la momentul $t=202$ ms, capul ajunge la cilindrul 20, iar după 1ms începe servirea cererii respective;
- 5) la momentul $t=285$ ms, capul ajunge la cilindrul 61, iar după 1ms începe servirea cererii respective;
- 6) la momentul $t=358$ ms, capul ajunge la cilindrul 97, iar după 1ms începe servirea cererii respective;
- 7) la momentul $t=405$ ms, capul ajunge la cilindrul 120, iar după 1ms începe servirea cererii respective.

Planificarea discului /16

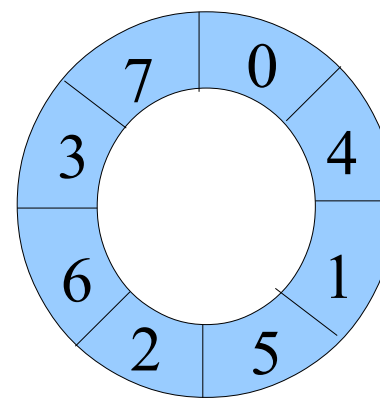
- **Reducerea așteptării rotației**

- În S.O.-urile mai pretențioase se urmărește și minimizarea latenței de rotație
- Ideea: reordonarea servirilor cererilor existente la un moment dat pentru același cilindru – alg. **SLTF (Shortest Latency Time First)**: *cel ce așteaptă cel mai puțin va fi servit primul*
- Altă metodă (statică): numerotarea *întreșută* a sectoarelor în cadrul unei piste

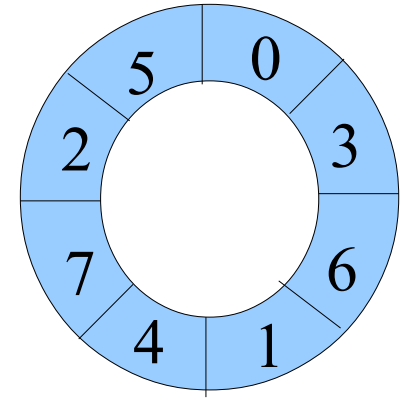
- a) numerotare normală
- b) numerotare întreșută cu factorul 1
- c) numerotare întreșută cu factorul 2



a)



b)



c)

Administrarea discului /1

- **Formatarea fizică** (formatarea *low-level*)
 - Înseamnă operația de împărțire a discului în sectoare pe care controlerul de disc le poate citi și scrie
 - Notă: în ultimii 30 de ani dimensiunea sectoarelor a fost de 512 B, dar recent au apărut discurile cu sectoare de 4 KB
- Pentru a stoca fișiere pe disc, S.O.-ul trebuie să-și înregistreze propriile structuri de date pe disc
 - **Partiționarea** discului într-unul sau mai multe grupuri de cilindri, numite partiții
 - **Formatarea logică** a unei partiții = “crearea sistemului de fișiere” rezident pe acea partiție
- Blocul de boot folosit pentru inițializarea sistemului de calcul:
 - *Bootstrap*-ul este păstrat în memoria ROM
 - programul *bootstrap loader* este păstrat pe disc în **blocul de boot**

Administrarea discului /2

- **Schema de partiționare MBR**

- este standardul de partiționare folosit de PC-uri (BIOS)
- un disc partiționat după această schemă va conține:
 - prima pistă este zonă rezervată (63 sectoare la discurile cu 512B/sector)
 - maxim 4 **partiții primare**, din care una poate fi **partiție extinsă** (i.e. poate conține un număr oarecare de **partiții logice**)
 - primul sector din cadrul primei piste se numește **sectorul MBR** (= Master Boot Record) și reprezintă blocul de boot, ce conține următoarele informații:
 - primii 446 octeți: conțin programul *bootstrap loader*
 - următorii 64 octeți: conțin tabela de partiții (cu informații despre poziția pe disc a celor 4 partiții primare și tipul sistemelor de fișiere stocate)
 - ultimii 2 octeți conțin întotdeauna valoarea 55AA (cu rol de semnătură)
 - restul de 62 sectoare din cadrul primei piste sunt rezervate (pentru programe de bootstrap mai mari de 446 octeți, e.g. **grub stage2**)

Administrarea discului /3

- Alocarea blocurilor pe disc poate adresa ambele probleme, a timpului de căutare și a latenței de rotație
- Alocarea blocurilor urmărește scopuri competitive:
 - costul alocării
 - lățimea de bandă pentru transferul unor volume mari de date
 - eficiența operațiilor cu directoare
- **Scop:** reducerea mișcării brațului (cu capetele R/W ale) discului și a *overhead*-ului datorat *seek*-urilor
 - metrica utilizată: lățimea de bandă utilizată
- Metode de gestiune a blocurilor bad (e.g. [sector sparing](#))

Administrarea discului /4

- O abordare posibilă pentru alocarea blocurilor:
 - Grupurile de cilindri utilizate de FFS (Fast File System)
 - FFS definește grupurile de cilindri drept unitatea de localitate a discului și factorizează localitatea în posibilități de alegere pentru alocare
 - **Strategia:** plasarea blocurilor de date “înrudite” în același grup de cilindri ori de câte ori acest lucru este posibil

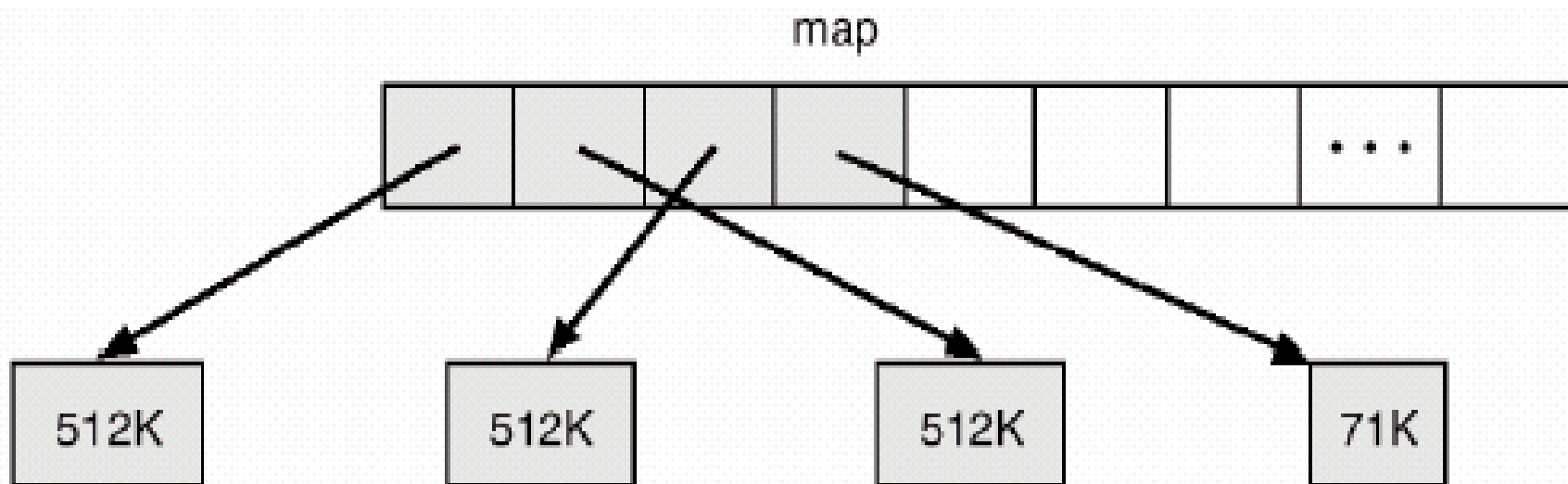
Gestiunea spațiului de swap /1

- **Spațiul de swap**
 - Memoria virtuală utilizează spațiu pe disc drept o extensie a memoriei principale
- Spațiul de swap poate fi localizat:
 - într-un fișier (sau mai multe) din sistemul normal de fișiere (e.g. Windows)
 - pe o partiție (sau un disc) separată (e.g. UNIX/Linux)

Gestiunea spațiului de swap /2

- **Exemplu:**

- UNIX BSD 4.3 îi alocă spațiu de swap la începutul execuției procesului
- spațiul alocat conține:
segmentul de text (programul) și **segmentul de date**



Construirea unui disc mai bun /1

- **De ce?**

- “Mai bun” a însemnat de obicei o densitate mai mare pentru producătorii de discuri – discurile mai mari sunt mai bune
- *I/O bottleneck* – discrepanța de viteză cauzată de faptul că procesoarele devin mai rapide mult mai repede decât discurile
- O idee este de a folosi paralelismul mai multor discuri
 - **Împrăștierea datelor** (*data striping*) pe mai multe discuri
 - Probleme de siguranță a păstrării datelor – introducerea tehnicilor de **redundanță** a datelor

Construirea unui disc mai bun /2

- **Soluție**

- **RAID (Redundant Array of Independent Disks)**

- Discurile multiple asigură **siguranța** păstrării datelor prin **redundanța** datelor
- Discurile RAID se clasifică în 7 nivele RAID
- *Striping*-ul utilizează un grup de discuri ca o singură unitate de stocare
- Schemele RAID îmbunătățesc performanța și siguranța sistemului de stocare prin stocarea redundantă a datelor
 - prin tehnica oglindirii (*mirroring* sau *shadowing*) se păstrează un duplicat al fiecărui disc
 - tehnica *block interleaved parity* folosește mult mai puțină informație pentru redundanță

Un disc mai bun /3

- **Nivelele RAID**



(a) RAID 0: non-redundant striping



(b) RAID 1: mirrored disks



(c) RAID 2: memory-style error-correcting codes



(d) RAID 3: bit-interleaved Parity



(e) RAID 4: block-interleaved parity



(f) RAID 5: block-interleaved distributed parity



(g) RAID 6: P + Q redundancy

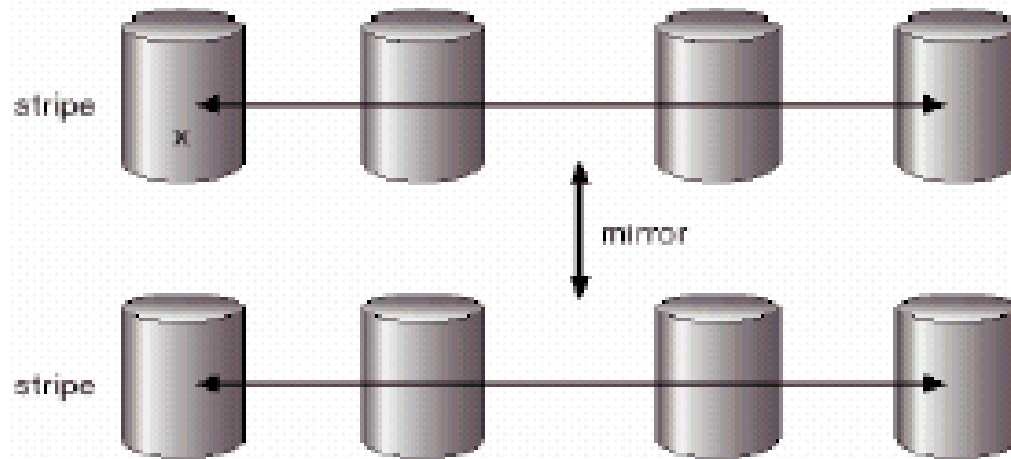
Construirea unui disc mai bun /4

- **Nivelele RAID**

- Nivelul 0: fără redundanță, doar *striping*
- Nivelul 1: discuri oglindite
- Nivelul 2: coduri Hamming corectoare de erori
- Nivelul 3: un disc de paritate la fiecare grup, *bit-interleaved*
- Nivelul 4: citiri/scrieri independente, *block-interleaved*
- Nivelul 5: datele/informația de paritate sunt împrăștiate pe toate discurile (mărește accesul concurent)
- Nivelul 6: rezistă la mai mult de o singură eroare de disc

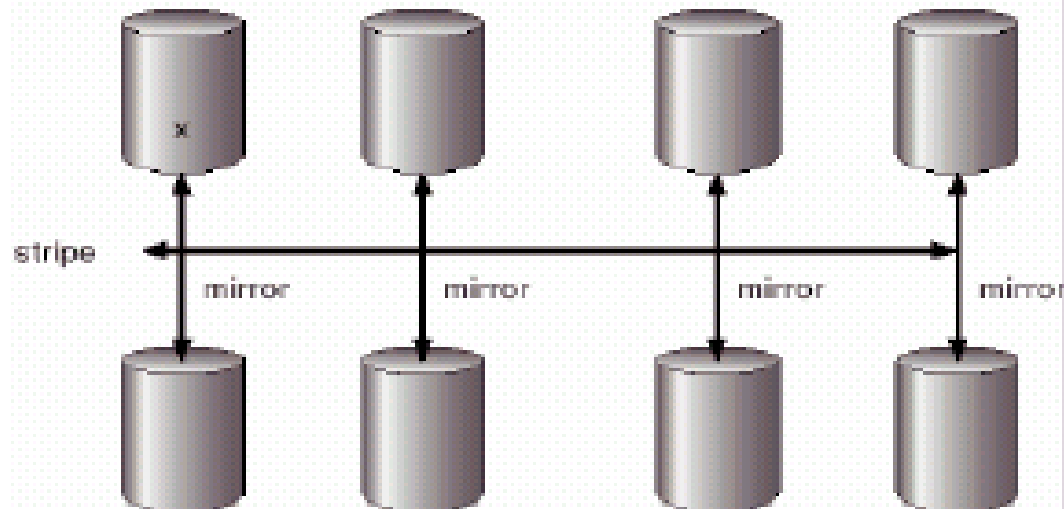
Construirea unui disc mai bun /5

- **RAID (0+1)**



a) RAID 0 + 1 with a single disk failure

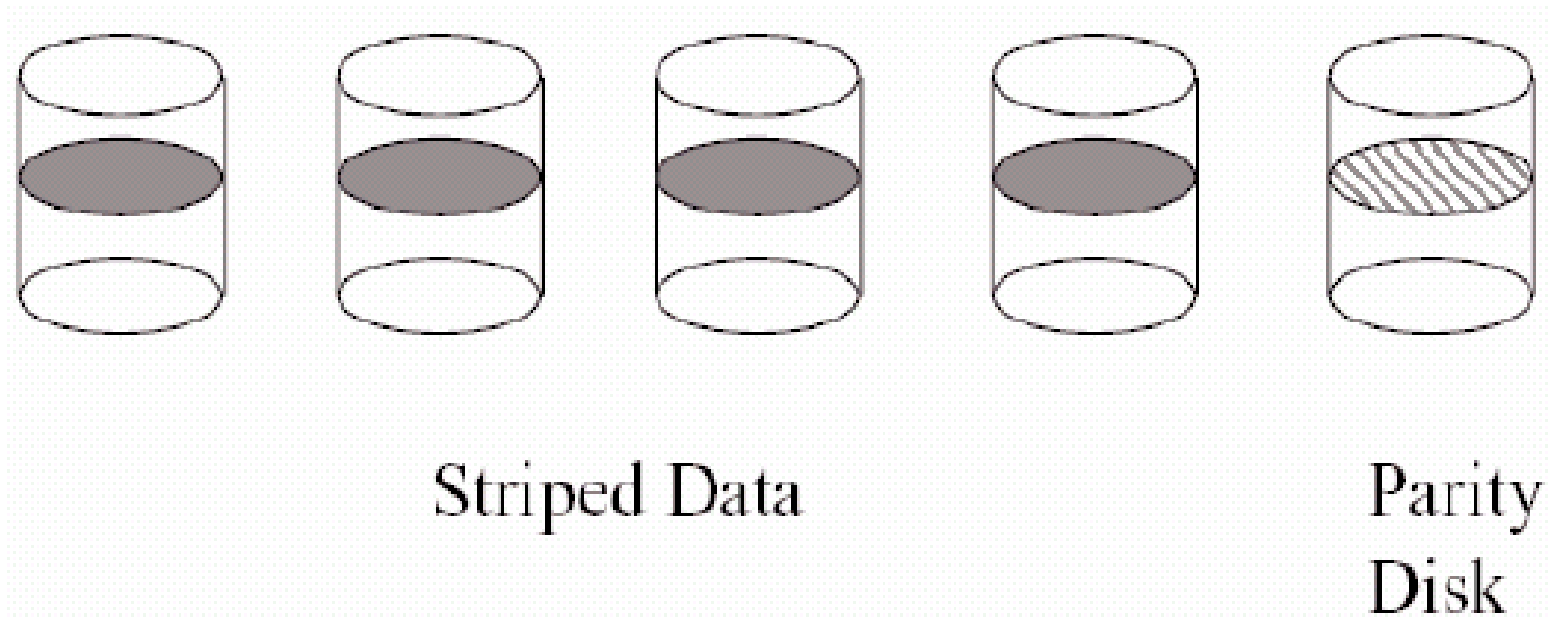
- **RAID (1+0)**



b) RAID 1 + 0 with a single disk failure

Construirea unui disc mai bun /6

- **RAID – nivelele 2 și 3**



Construirea unui disc mai bun /7

- **RAID : Paritatea**

- paritate bit/block-interleaved pentru toleranță la erori
- exemplu: stocarea valorii binare 1010



- Tolerează eroarea unui singur disc, întrucât eroarea este fie pe discul de paritate, fie pe unul dintre discurile de date
- Citește data, iar în caz de eroare, citește informația de paritate și corectează data
- Scrierea necesită în plus și actualizarea informației de paritate

Construirea unui disc mai bun /8

- **RAID – implementare hardware:**

- controlerul RAID este o placă hardware (fie discretă, fie este integrată în northbridge, la unele modele de plăci de bază)
- plus un set de discuri, cu caracteristici specifice nivelului RAID dorit

- **RAID – implementare software:**

- controlerul RAID este implementat prin software
- plus un set de discuri, cu caracteristici specifice nivelului RAID dorit

Exemple de implementări software:

- un *device* virtual, la nivelul nucleului SO: md (Linux), softraid (OpenBSD)
- some logical *volume managers*: LVM (Linux), discuri dinamice create cu unealta Disk Management (Windows), sau tehnologia Storage Spaces (Win8/Win10)
- o componentă la nivelul *file-system*-ului: ZFS, btrfs (vezi cursul precedent)
- un nivel/o aplicație deasupra *file-system*-ului: SnapRAID, FlexRAID, etc.

Implementări la nivel de firmware+drivere specializate: e.g. Intel Matrix RAID

Referință: <https://en.wikipedia.org/wiki/RAID#Implementations>

Administrarea memoriei terțiare /1

- Caracteristica definitorie a memoriei terțiare: **costul scăzut**
- În general, memoria terțiară este construită folosind medii de stocare de tip *removable* (e.g. dischete, CD/DVD-uri, memorii flash)
- Sunt disponibile și alte tipuri ...

Administrarea memoriei terțiare /2

- **Discuri *removable*:**
 - **Discul floppy** (discheta) – un disc subțire și flexibil, acoperit cu un strat de material magnetic și închis într-o carcasă protectivă de plastic
 - **Discul mageto-optic** – înregistrează datele pe un platan rigid acoperit cu un strat de material magnetic
 - **Discul optic** – nu utilizează magnetismul; se folosesc materiale speciale ce sunt modificate de raza laser
- **Medii WORM** (Write Once, Read Many times)
 - Exemple: **CD-uri**, **DVD-uri**
- **Benzi magnetice**
- **Memorii flash** (tehnologie NAND)
 - Exemple: **stick-uri USB**, **disk-uri SSD**, **card-uri de memorie** (în diverse formate: **CF**, **SD**, **MMC**, ș.a.)

- **Bibliografie obligatorie**

capitolele despre *administrarea perifericelor de stocare* din

- Silberschatz : “*Operating System Concepts*”

(cap.11 din [OSCE8])

sau

- Tanenbaum : “*Modern Operating Systems*”

(cap.5, §5.4 din [MOS3])

Sumar

- Introducere
- Memoria secundară (discul)
 - Structura
 - Planificarea
 - Administrarea
- Gestiunea spațiului de swap
- Construirea unui disc mai bun
- Administrarea memoriei terțiare

Întrebări ?

Sisteme distribuite

Clasificarea lui Flynn (1966) (1)

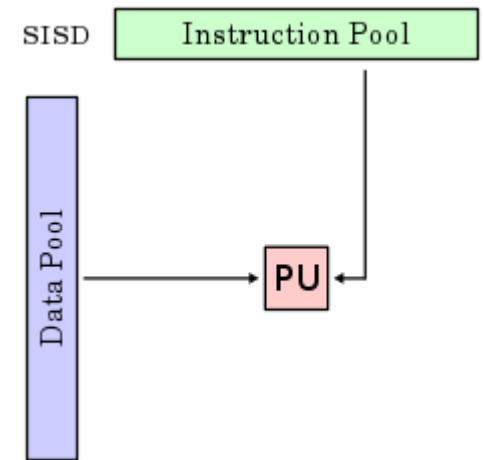
- procesorul - 2 părți
 - unitatea de control - decide ce operații trebuie realizate
 - unitatea de execuție - realizează efectiv operațiile
- o unitate de control poate comanda mai multe unități de execuție

Clasificarea lui Flynn (2)

- SISD
 - Single Instruction, Single Data
- SIMD
 - Single Instruction, Multiple Data
- MISD
 - Multiple Instruction, Single Data
- MIMD
 - Multiple Instruction, Multiple Data

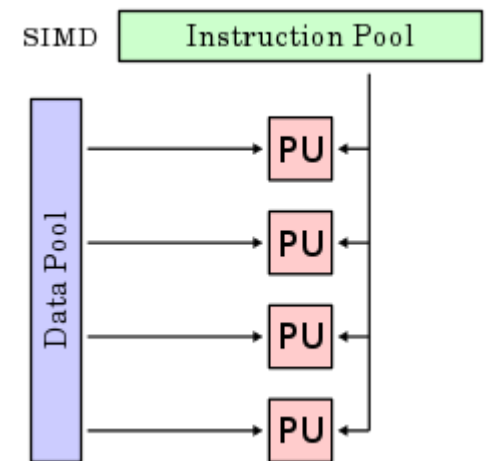
SISD

- o singură unitate de control
- o singură unitate de execuție
- arhitectura von Neumann
- complet secvențială
- o instrucțiune începe după terminarea celei anterioare

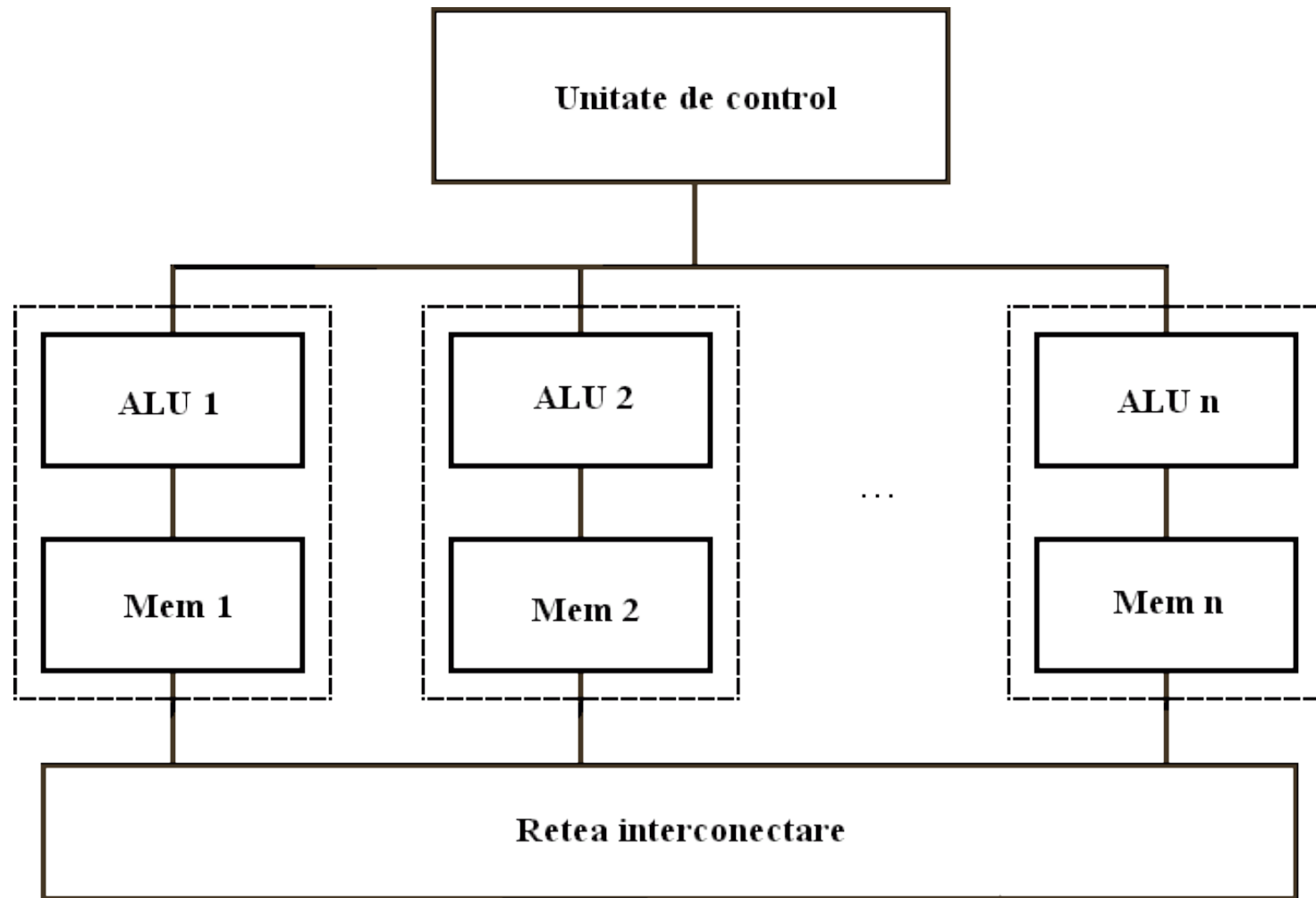


SIMD (1)

- o singură unitate de control, mai multe unități de execuție
- aceeași instrucțiune aplicată în paralel pe date diferite
- implementări
 - calculatoare vectoriale
 - calculatoare matriciale - operații de nivel mai înalt



SIMD (2)

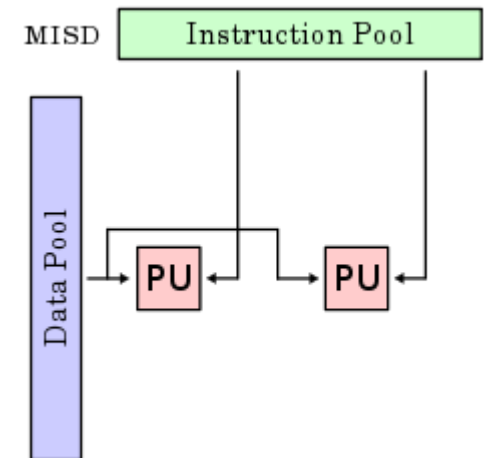


SIMD (3)

- nu toate prelucrările se pot face în paralel
- practic - combinații SISD-SIMD
 - Intel: MMX, SSE
- problemă - instrucțiuni condiționale
 - fluxuri de execuție diferite
 - unitățile care au date incorecte - dezactivate
 - pierdere de performanță

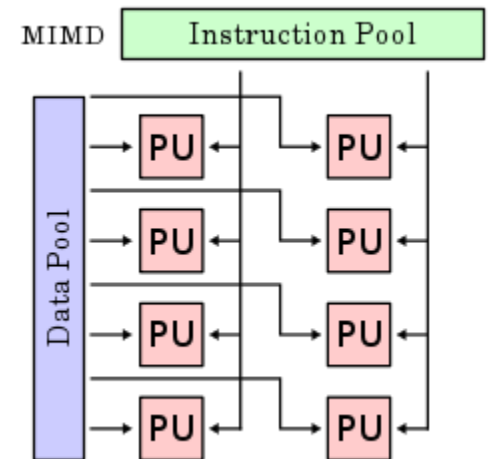
MISD

- mai multe unități de control, o singură unitate de execuție
- prelucrări diferite pe aceeași dată
 - în paralel
- implementare - ???
- sisteme pipeline
 - arhitecturi superscalare
- sisteme redundante

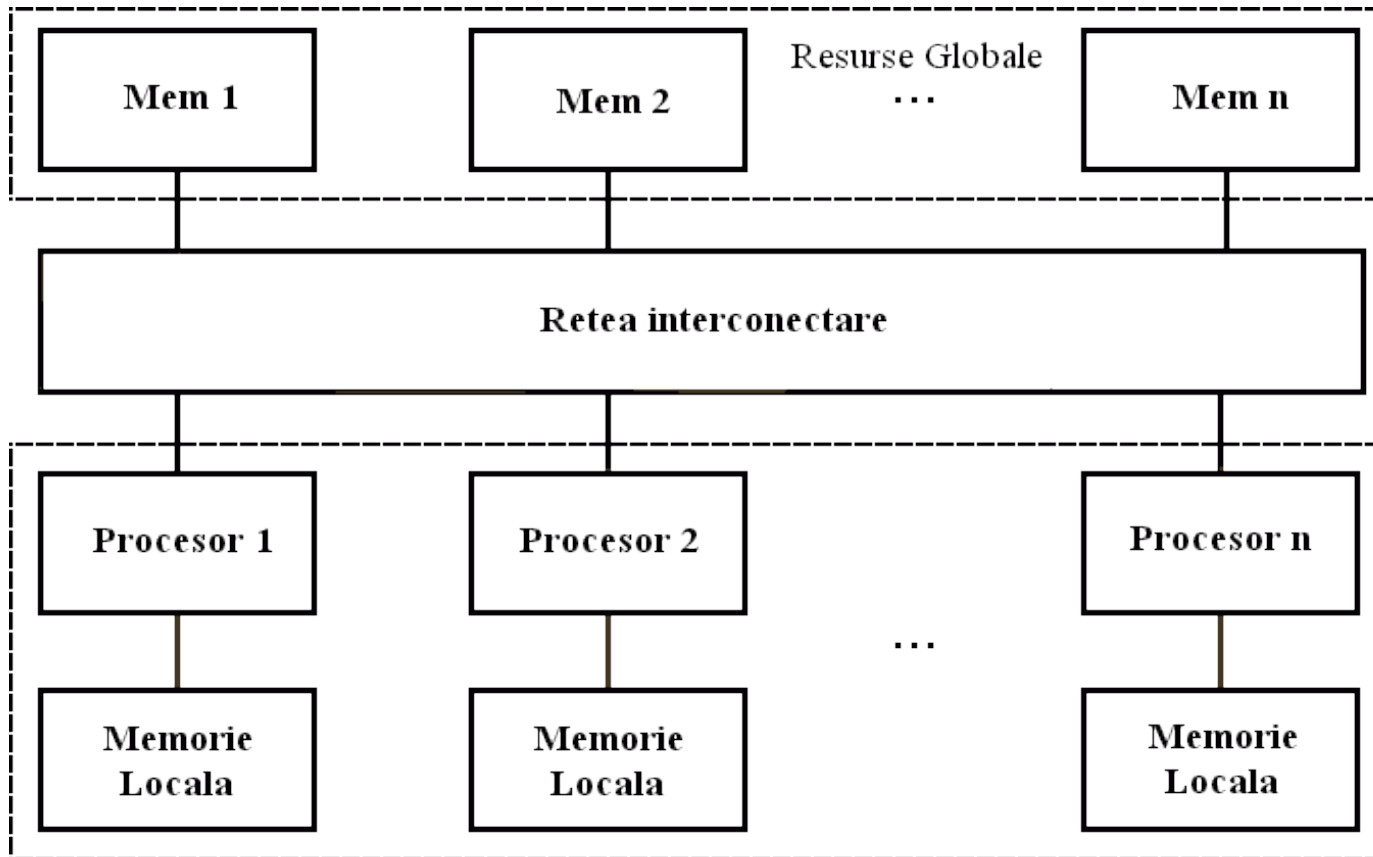


MIMD (1)

- mai multe unități de control,
mai multe unități de execuție
 - ca la SISD, dar multiplicat
- mai multe fluxuri de execuție
 - independente
 - cooperative
- sistemele distribuite sunt
MIMD



MIMD (2)



MIMD - clasificare

- pot avea memorie comună sau nu
 - sisteme cu memorie comună - multiprocesoare
 - spațiu unic de adrese pentru toate procesoarele
 - în general - sisteme paralele
 - sisteme fără memorie comună - multicalculatoare
 - fiecare procesor are propriul spațiu de adrese
 - în general - sisteme distribuite
- in functie de arhitectura rețelei de interconectare:
 - bus vs. switched

Interconectare

- sisteme puternic conectate (*tightly coupled*)
 - transmitere rapidă a mesajelor între unitățile de execuție
 - rata de transfer - ridicată
- sisteme slab conectate (*loosely coupled*)
 - invers
- se aplică atât pentru software, cât și pentru hardware

Tipuri de sisteme distribuite

- diverse combinații posibile privind modurile de conectare hardware/software
- sistemele distribuite sunt în mod normal slab conectate în hardware
- variante
 - sisteme de operare de rețea
 - sisteme distribuite "adevărate"

Sisteme de operare de rețea (1)

- software slab conectat, rulând peste hardware slab conectat
- calculatoare (stații de lucru) independente, legate în rețea
- fiecare are propriul sistem de operare
 - același pe toate stațiile
 - sau
 - diferit între stații

Sisteme de operare de rețea (2)

- de pe o stație se pot efectua acțiuni pe alte stații
 - execuția unor comenzi
 - transfer de fișiere
- mecanisme de comunicare mai evoluate
 - ex.: servere de fișiere
- trebuie definit un cadru general
 - i.e. protocoale pentru mesajele schimbate între stații

Sisteme distribuite "adevărate" (1)

- software puternic conectat, rulând peste hardware slab conectat
- utilizatorii văd sistemul distribuit ca pe un singur calculator clasic (*single-system image*)
 - ce înseamnă utilizator?
 - aplicațiile văd resursele în mod unitar, indiferent pe care mașini rulează
 - utilizatori finali, dar și programatori

Sisteme distribuite "adevărate" (2)

Caracteristici:

- mecanism unitar de comunicare interproces
- management uniform al proceselor
 - nu sunt acceptabile politici de planificare diferite pe mașini diferite
- sistem de fișiere unitar
- concluzie: pe toate mașinile rulează același nucleu

Sisteme paralele

- software puternic conectat, rulând peste hardware puternic conectat
- hardware-ul consta într-un singur calculator multiprocesor (i.e. cu mai multe CPU-uri fizice sau, mai nou, mai multe core-uri)
- software: e.g. orice sistem de operare cu time-sharing pentru SMP-uri (UNIX, Windows, etc.)

Cerințe de proiectare

- sistemele distribuite au (și) cerințe diferite de cele secvențiale
- când proiectăm un sistem distribuit, ce aspecte trebuie să avem în vedere?

Transparența

- d.p.d.v. al utilizatorului final
 - la lansarea unei comenzi, nu ne interesează pe câte (și care) mașini va rula
- d.p.d.v. al programatorului
 - interfața apelurilor sistem ascunde structura distribuită a sistemului

Aspecte ale transparenței (1)

- transparența localizării
 - resursele hardware și software se pot afla oriunde în sistem
 - pot fi accesate în același mod de pe orice mașină
 - exemple de resurse: procesoare, fișiere, periferice

Aspecte ale transparenței (2)

- transparența la mutare
 - duce mai departe transparența localizării
 - după mutarea unei resurse, aceasta poate fi în continuare accesată folosind același nume ca înainte
 - importantă pentru mutarea resurselor între servere

Aspecte ale transparenței (3)

- transparența replicării
 - sistemul de operare poate realiza copii ale resurselor în mod invizibil pentru utilizator
 - exemplu: mai multe servere de fișiere, care lucrează împreună
 - fiecare server memorează o parte dintre fișiere
 - când apare o cerere, un server poate decide să facă o copie a unui fișier de pe alt server
 - scopul: pentru performanța (accese viitoare)

Aspecte ale transparenței (4)

- transparența concurenței
 - se referă la accesul concurent la resurse comune de către mai mulți utilizatori
 - utilizatorii nu sunt conștienți de existența concurenței
 - accesele solicitate în paralel sunt secvențializate de către sistemul de operare

Aspecte ale transparenței (5)

- transparența paralelismului
 - scop: aplicațiile sunt scrise ca pentru un sistem uniprocessor (secvențial), dar la compilare si/sau executie profită de capacitatea de calcul paralel a sistemului
 - numită în calculul paralel: paralelism implicit
 - dificil de atins în stadiul actual

Fiabilitatea

- sistem cu multe unități de calcul
 - foarte probabil ca unele componente să se defecteze
 - disponibilitatea - sistemul să aibă timpi cât mai scurți în care este nefuncțional
 - toleranța la defectări - sistemul să fie capabil să lucreze și în condițiile în care unele componente sunt defecte

Performanța

- motivul pentru care există sisteme paralele și distribuite
- nu este justificat economic să construim asemenea sisteme pentru sarcini care pot fi rezolvate și de sisteme mai simple
- principalul factor de limitare a performanței în sistemele distribuite - comunicarea
 - hardware și software
 - *fine-grained parallelism* vs. *coarse-grained parallelism*

Scalabilitatea

- cum crește performanța unui sistem când se adaugă unități de calcul (procesoare, calculatoare) suplimentare?
- de preferat - liniar
- nu e ușor de obținut
 - unele aplicații nu se pretează la execuție în paralel
 - comunicarea între unități de calcul - gâtuire
 - componente sau date centralizate - gâtuire
 - algoritmi centralizați vs. distribuiți